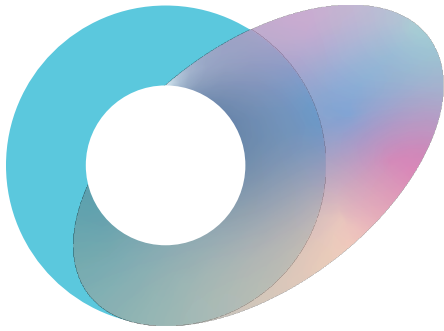




Outlier Playground

[Daily Sparks](#)[Leaderboard](#)[Challenges](#)
[Chat](#)
Starred



[6.PDIPM DLO 주-쌍대 문제](#)



[7. PDIPM DLO pilot code](#)



[8-2 하이브리드 DFN 방법론](#)



[8-1 DFN 구축 방법론 제안](#)



[0. 일인 창업 아이디어 브레인스토밍](#)



[1. PDIPM 한계 평형 해석 이해](#)



[2. PDIPM과 DLO 차이점](#)



[3. PDIPM 3D 압반 해석 진행](#)



[4. PDIPM PoC 검증용 터미 데이터 생성](#)



[5. PDIPM DLO 압반블록](#)

Today



[비용 분석을 위한 Q' 활용방안](#)

Last 7 Days



[RQD 검증 연구계획 요약](#)



[Chi-Square vs T-Test](#)



[Gemini Code Assist 설정](#)



[RQD 검증 연구 계획](#)



[터널공학 압반분류 Q2](#)



[터널공학 압반분류 Q3](#)

Last 30 Days

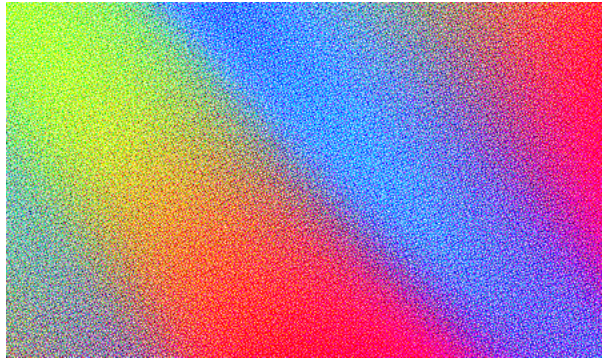
[터널공학 압반분류 Q](#)



[강화학습 강의준비](#)

✧
[강화학습 이해 심화](#)

✧
[AI 에이전트 구축 계획](#)



✧
[Mac Mini 클러스터 구축 안내](#)

✧

[글로벌 탄소 규제 대응 #2](#)


✧
[글로벌 탄소 규제 대응 #1](#)

✧
[MCMC 개념 탐구](#)

✧
[블록 안정성 분석](#)

✧
[6.PDIPM DLO 주-쌍대 문제](#)

✧

[7. PDIPM DLO pilot code](#)





[트랜스포머 강의 준비](#)



[Infra RYL 2010 정보](#)



[8-2 하이브리드 DFN 방법론](#)



[8-1 DFN 구축 방법론 제안](#)



[MOE Architecture 탐구](#)



[0. 일인 창업 아이디어 브레인스토밍](#)



[구조방정식 모델링 소개](#)

[의료 합병증 조연 요청](#)

Earlier



[테이블 형태 GUI 설계](#)



[1. PDIPM 한계 평형 해석 이해](#)



[2. PDIPM과 DLO 차이점](#)



[3. PDIPM 3D 압반 해석 진행](#)



[4. PDIPM PoC 검증용 더미 데이터 생성](#)



[5. PDIPM DLO 압반블록](#)



[폭발물 발파의 SPH 적용](#)



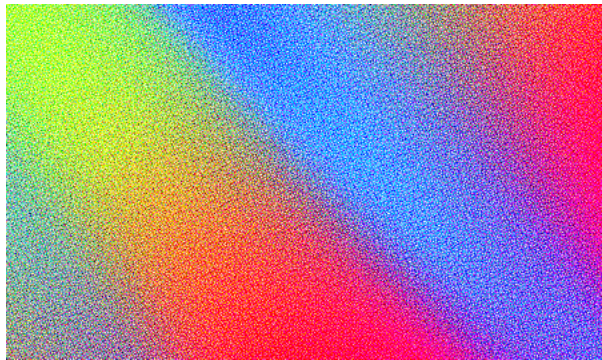
[인사말 전달](#)



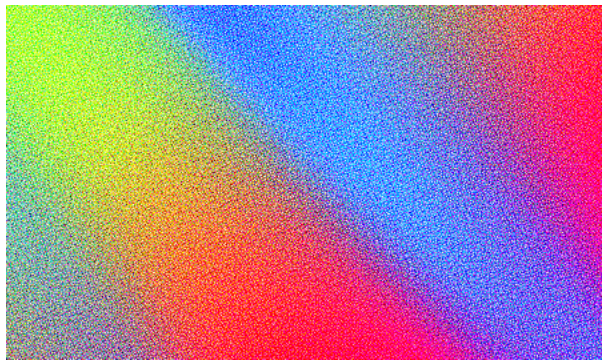
[Diffusion Lecture Preparation](#)



[R2와 Adjusted R2 비교](#)



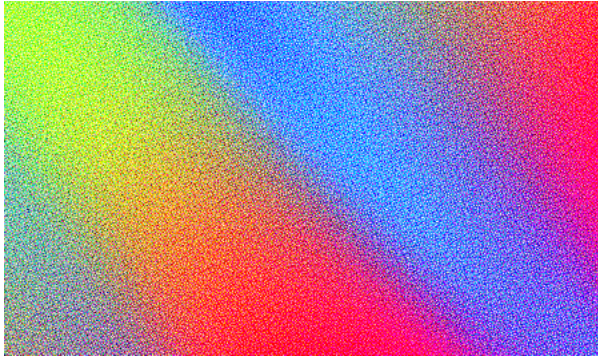
[Clang 17 CPP-23 지원 여부](#)



[이텐션 모델의 중요성](#)



[Bayesian Ridge Explanation](#)



[바이오 그라우트 실험 필요성](#)

2180

2

m

방이름 RQD 검증 연구계획 2 -> 새로운 창의 맥락 요약:

최종 연구 목적:

- 지하 캐번 건설 전 수평시추공 Q'를 이용해 굴착 진행/보류를 판단하는 접근이 실제로 도움이 되는지 검증하고자 함.
- 핵심은 “시추공 Q'와 굴진면 Q'가 1:1로 일치하는가”가 아니라, “시추공 Q' 기반 사전 판단이 false-safe / false-alarm을 줄이고 baseline보다 나은 decision value를 가지는가”임.
- 궁극 질문:
수평시추공 Q'는 굴진면 암반 상태 및 굴착 가능성을 조건부로 신뢰성 있게 예측하는 데 사용할 수 있는가?

용어:

- Face condition = 실제 굴진면 조건
- Borehole condition = 시추공 기반 사전 판단
- False-safe = 시추공은 good/굴착 가능이라고 했지만 face는 bad/부적합인 경우
- False-alarm = 시추공은 bad/부적합이라고 했지만 face는 good/굴착 가능인 경우
- Decision usefulness는 accuracy보다 false-safe, false-alarm, expected cost, baseline comparison이 중요함.

기존 구현 구조:

- 폴더 구조:
 - src/core
 - src/cli
 - src/export
 - src/gui
- 주요 파일:
 - src/core/dfn_generator.py
 - src/core/joint_models.py
 - src/core/domain.py

- src/core/rqd_calculator.py
- src/core/tunnel.py
- src/core/comparison.py
- src/core/hypothesis_test.py
- src/core/decision_test.py
- src/cli/batch_runner.py
- src/cli/multicase_batch_runner.py
- src/cli/run_multicase_decision.py

DFN 생성:

- DiscreteFractureNetwork에서 절리군별로 disk fracture 생성.
- JointSetDefinition:
 - mean_dip
 - mean_dip_dir
 - fisher_kappa
 - size_alpha
 - size_r_min
 - size_r_max
 - density_type
 - P32
 - mean_spacing
 - Jr_mean/std
 - Ja_mean/std
- density_type == 'P32'이면 mean_spacing은 코드상 사용되지 않음.
- 절리 개수:


```
if density_type == 'P32':
    n_expected = P32 * domain_volume / mean_area
else:
    n_expected = domain_volume / (mean_spacing * mean_area)
```
- 이번 연구에서는 P32 기준으로 밀도 조절.

Q' 계산:

- $Q' = (RQD / J_n) * (J_r / J_a)$
- J_n 은 현재 절리군 수 기반으로 DomainJointConfig.Jn에서 결정:


```
n_sets=2이면 Jn=4
n_sets=3이면 Jn=9
```
- $Q = Q' * (J_w / SRF)$

현재 sampling:

- Tunnel.sample_borehole():
 - x방향 borehole line sampling
 - direction = [1,0,0]
 - face_x 주변 borehole_window 범위를 샘플링
 - 각 segment마다 RQD, Jr, Ja, Q, Q' 계산
- Tunnel.sample_face():
 - 터널 face 원형 단면에서 다방향 scanline 기반 RQD 계산
 - face_joints에서 Jr/Ja 평균
 - face Q' 계산

기존 보정 모델:

Tunnel.py에 다음 practice correction이 있었음:

- BH 쪽:
 - MECHANICAL_BREAK_PROB
 - MICRO_FRACTURE_MULTIPLIER
 - CORE_RECOVERY_DECAY
 - JR_SCALE_FACTOR_BH = 1.10
 - JA_UNDERESTIMATE_BH = 0.85
- Face 쪽:
 - JR_SCALE_FACTOR_FACE = 0.85
 - JA_SCALE_FACTOR_FACE = 1.05
 - FACE_CONSERVATISM = 0.95
 - face RQD practical = $0.6 * \text{rqd_mean} + 0.4 * \text{rqd_min_dir}$, 이후 0.95 곱함
- 이 보정은 구조적으로 BH Q'를 높이고 face Q'를 낮추는 방향임.

DecisionUsefulnessTester:

- 새로 src/core/decision_test.py를 추가했음.
- 입력: face_rows
- 핵심 컬럼:
 - Qp_face_mean
 - Qp_borehole_mean
 - Q_face_mean
 - Q_borehole_mean
 - face_fracture_density
 - orientation_bias_gap
- 기능:
 - threshold 기준 분류
 - confusion matrix
 - false_safe_rate
 - false_alarm_rate
 - PPV/NPV
 - balanced_accuracy
 - expected_cost
 - always_excavate / always_reject / majority baseline 비교
 - threshold sweep
 - density/orientation quantile regime별 분석

MultiCaseBatchRunner:

- 기존 BatchRunner는 1 case × many seeds 구조였음.
- 새로운 src/cli/multicase_batch_runner.py 추가:
 - 여러 YAML case × 여러 seeds × 여러 face positions 실행
 - 결과 병합
 - all_cases_face_rows.csv 저장
- 실행 스크립트:
 - src/cli/run_multicase_decision.py
- 실행:


```
python -m src.cli.run_multicase_decision
```

실험한 6개 YAML case:

1. granite_good_dense.yaml
2. granite_medium.yaml
3. granite_sparse.yaml
4. granite_dense_unfavorable.yaml
5. granite_sparse_unfavorable.yaml

6. granite_sparse_favorable.yaml

케이스 의미:

- granite_good_dense:
 - 좋은 조건 대조군
 - 밀도는 중~높지만 J_r/J_a 가 양호하고 방향도 불리하지 않음
- granite_medium:
 - face Q'가 threshold 근처에 오도록 의도한 중간 조건
- granite_sparse:
 - 희소 절리망 효과 확인용
- granite_dense_unfavorable:
 - 절리는 많지만 borehole 방향에 불리한 방향성
- granite_sparse_unfavorable:
 - 위험 조건. 희소 + borehole에 불리한 방향성
- granite_sparse_favorable:
 - sparse_unfavorable의 대조군. 희소하지만 borehole에 유리한 방향성

방향성 기준:

- borehole direction = x direction = [1,0,0]
- 절리 법선 n과 borehole direction d의 $|n \cdot d|$ 가 크면 borehole이 절리를 잘 만남.
- favorable orientation 예:
mean_dip: 80
mean_dip_dir: 90 또는 270
- unfavorable orientation 예:
mean_dip: 80
mean_dip_dir: 0 또는 180
- kappa가 높을수록 방향성이 강함:
20 = moderate
50 = high
70 = very high

practice correction mode 실험:

- practice correction이 켜진 상태에서 6 cases × 10 seeds × 8 faces = 480 face rows 실행.
결과:
Face Q' min/p10/p25/p50/p75/p90/max:
8.17 / 8.77 / 9.59 / 11.96 / 14.72 / 30.99 / 33.51
BH Q':
9.99 / 11.89 / 12.49 / 16.33 / 20.17 / 42.96 / 53.66
- H1:
mean Qp diff = +5.1468
 $p \approx 1.29e-103$
mean abs log10 error = 0.1233
- H5 sparse + unfavorable:
overestimation rate = 1.0
mean diff = +2.864
 $p \approx 4.08e-22$
- Decision threshold=10:
prior face good rate = 0.660
TP=317, FP=false-safe=162, FN=0, TN=1
false_safe_rate=0.994
balanced_accuracy=0.503

expected_cost=3.375
always excavate cost=3.396
majority cost=3.396

- 해석:
practice mode에서는 BH Q'가 face Q'를 체계적으로 과대평가.
threshold=10에서 bad face 대부분을 good으로 판단.
시추공 Q' 전략은 always-excavate와 거의 동일한 수준.
false-safe 위험이 큼.
그러나 이 결과는 practice correction 자체가 BH를 좋게, face를 나쁘게 만드는 구조의 영향일 가능성이 큼.

correction_mode flag 추가:

- Tunnel에 correction_mode를 추가하기로 함.
- mode:
 - "practice": 기존 보정 전부 사용
 - "neutral_strength": Jr/Ja/face conservatism 제거, 나머지는 유지
 - "pure": 모든 artificial correction 제거
- 수정 위치:
 - src/core/tunnel.py:
Tunnel.init(self, domain, correction_mode="practice")
helper:
_use_strength_scaling()
_use_drilling_damage()
_use_micro_fractures()
_use_face_conservatism()
_use_practical_face_rqd()
 - sample_borehole:
mechanical breaks, micro fractures, core recovery, BH Jr/Ja scaling을 mode에 따라 on/off
 - sample_face:
face Jr/Ja scaling, face conservatism, face practical RQD를 mode에 따라 on/off
 - _multi_direction_face_rqd:
face micro fractures를 mode에 따라 on/off
 - src/cli/batch_runner.py:
BatchConfig에 correction_mode 추가
Tunnel(domain, correction_mode=self.config.correction_mode)
 - src/cli/multicase_batch_runner.py:
MultiCaseBatchConfig에 correction_mode 추가
BatchConfig(... correction_mode=self.config.correction_mode)

pure mode 결과:

- 보정하지 않는 pure mode로 같은 6 cases × 10 seeds × 8 faces 실행.
- 결과:
Face Q':
11.02 / 11.82 / 12.47 / 15.56 / 19.27 / 40.90 / 44.01
BH Q':
10.30 / 12.00 / 13.00 / 15.44 / 19.65 / 40.93 / 50.54
- H1:
mean diff = +0.181
p = 0.029
mean abs log10 error = 0.026
- H5 sparse + unfavorable:
overestimation rate = 0.510
mean diff = -0.047
p = 0.602

- Decision threshold=40:
prior face good rate=0.135
TP=50, FP=9, FN=15, TN=406
false_safe_rate=0.022
false_alarm_rate=0.231
balanced_accuracy=0.874
expected_cost=0.281
majority cost=0.406
- Data-driven threshold sweep:
threshold≈15.56에서 prior good=0.5, accuracy=1.0, false_safe=0, false_alarm=0.
- 해석:
보정 제거 후 BH Q'와 face Q'의 차이가 거의 사라짐.
sparse+unfavorable overestimation 가설도 사라짐.
따라서 practice mode에서 보였던 체계적 과대평가와 false-safe 위험은 순수 DFN sampling geometry라기보다 보정/practice model 때문일 가능성이 큼.

현재 중요한 의문:

- pure mode에서 BH Q'와 face Q'가 잘 맞는 이유가 정말 대표성 때문인지, 아니면 둘의 계산 방식이 너무 유사하기 때문인지 의혹이 있음.
- 현재 pure mode에서도:
 - Jn은 동일
 - Jr/Ja는 같은 DFN 절리에서 평균
 - RQD도 둘 다 DFN intersection 기반 line scan
 - face RQD도 multi-direction scanline의 평균
 - face scanline에 x-direction, 즉 borehole direction이 포함됨
- 따라서 pure mode의 높은 일치가 “진짜 독립적 검증”이 아닐 수 있음.

다음 단계로 제안된 estimator separation test:

1. Face Q' estimator를 여러 개 저장:
 - Qp_face_from_rqd_mean
 - Qp_face_from_rqd_min_dir
 - Qp_face_from_rqd_practical
 - Q_face_from_rqd_mean
 - Q_face_from_rqd_min_dir
 - Q_face_from_rqd_practical
2. 수정 위치:
src/core/tunnel.py 의 sample_face()에서
현재:
Q_mean = QCalculator.calc_Q(rqd_practical, ...)
Qp_mean = QCalculator.calc_Qprime(rqd_practical, ...)
주변에 추가:
Qp_from_rqd_mean = QCalculator.calc_Qprime(rqd_mean, jn_val, jr_mean, ja_mean)
Qp_from_rqd_min_dir = QCalculator.calc_Qprime(rqd_min_dir, jn_val, jr_mean, ja_mean)
Qp_from_rqd_practical = QCalculator.calc_Qprime(rqd_practical, jn_val, jr_mean, ja_mean)
Q_from_rqd_mean = QCalculator.calc_Q(rqd_mean, jn_val, jr_mean, ja_mean, jw_mean, srf_mean)
Q_from_rqd_min_dir = QCalculator.calc_Q(rqd_min_dir, jn_val, jr_mean, ja_mean, jw_mean, srf_mean)
Q_from_rqd_practical = QCalculator.calc_Q(rqd_practical, jn_val, jr_mean, ja_mean, jw_mean, srf_mean)
return dict에 위 값들을 _Qp_from_rqd_mean 등으로 추가.
3. src/core/comparison.py:
compare_at_face()의 comparison dict에 추가:
"Qp_face_from_rqd_mean": face.get("_Qp_from_rqd_mean", 0)

- ```

"Qp_face_from_rqd_min_dir": face.get("_Qp_from_rqd_min_dir", 0)
"Qp_face_from_rqd_practical": face.get("_Qp_from_rqd_practical", 0)
"Q_face_from_rqd_mean": face.get("_Q_from_rqd_mean", 0)
"Q_face_from_rqd_min_dir": face.get("_Q_from_rqd_min_dir", 0)
"Q_face_from_rqd_practical": face.get("_Q_from_rqd_practical", 0)

```
- src/core/comparison.py:  
 comparisons\_to\_face\_rows() row dict에도 동일 컬럼 추가:  

```

"Qp_face_from_rqd_mean": c.get("Qp_face_from_rqd_mean")
"Qp_face_from_rqd_min_dir": c.get("Qp_face_from_rqd_min_dir")
"Qp_face_from_rqd_practical": c.get("Qp_face_from_rqd_practical")
"Q_face_from_rqd_mean": c.get("Q_face_from_rqd_mean")
"Q_face_from_rqd_min_dir": c.get("Q_face_from_rqd_min_dir")
"Q_face_from_rqd_practical": c.get("Q_face_from_rqd_practical")

```
  - face estimator comparison 함수:  

```

bh = Qp_borehole_mean
face estimators:
Qp_face_mean
Qp_face_from_rqd_mean
Qp_face_from_rqd_min_dir
Qp_face_from_rqd_practical

```

 각 estimator 별:  

```

mean_diff = mean(bh - face)
mean_abs_log10_error = mean(abs(log10(bh/face)))
corr = corr(bh, face)

```

 비교.  
 만약 BH가 Qp\_face\_from\_rqd\_mean과만 잘 맞고 min\_dir/practical과는 안 맞으면 계산 방식 유사성 의혹이 큼.
  - 더 강한 테스트:  
 face scanline에서 borehole과 같은 x-direction scan을 제거해야 함.  
 현재 \_multi\_direction\_face\_rqd()에:  

```
scan_directions.append(np.array([1.0, 0.0, 0.0]))
```

 가 있음.  
 이것이 borehole direction과 동일.  
 pure test에서는 이걸 빼고 비교해야 함.

제안 수정:

- \_multi\_direction\_face\_rqd() signature:  
 def \_multi\_direction\_face\_rqd(self, face\_x, n\_scanlines=8, scan\_length=10.0, include\_axial\_direction=True):
- 기존:  

```
scan_directions.append(np.array([1.0, 0.0, 0.0]))
```
- 수정:  

```
if include_axial_direction:
 scan_directions.append(np.array([1.0, 0.0, 0.0]))
```
- sample\_face()에서 호출할 때:  

```
include_axial = self.correction_mode != "pure"
face_rqd_values = self._multi_direction_face_rqd(
 face_x, n_scanlines, scan_length,
 include_axial_direction=include_axial
)
```

 또는 더 명시적으로 Tunnel에 include\_face\_axial\_scan flag를 추가하는 것이 좋음.

가장 중요한 다음 테스트:

A. pure + face axial scan included

B. pure + face axial scan removed

C. BH vs face\_rqd\_mean / min\_dir / practical 각각 비교

D. 가능하면 나중에 Jv 기반 face RQD도 추가:

$RQD_{Jv} = 115 - 3.3 * Jv$

단, Jv 구현은 다음 단계.

현재까지 가장 중요한 결론:

- practice correction이 켜진 경우:  
BH Q'는 face Q'를 체계적으로 과대평가하고, threshold=10에서 false-safe 위험이 매우 큼.
- pure mode:  
BH Q'와 face Q' 차이는 거의 사라짐.
- 따라서 기존 false-safe 문제는 순수 DFN sampling bias라기보다 시추공/굴진면에 다르게 적용된 practice correction 또는 산정 방식의 차이에서 비롯되었을 가능성이 큼.
- 그러나 pure mode의 좋은 일치가 계산 방식 유사성 때문일 수 있으므로 estimator separation test와 face axial scan 제거 테스트가 필요함.

새 창에서 우선 하고 싶은 일:

1. Tunnel/sample\_face/comparison에 face Q' estimator 다중 저장 코드 추가.
2. face axial scan 제거 flag 추가.
3. pure mode에서 axial included vs removed를 비교.
4. BH Q'와 여러 face estimator 간 mean\_diff, abs\_log\_error, corr, decision metrics 비교.



Thought Process

좋습니다. 지금 새 창에서 바로 할 일은 \*\*\*"face estimator를 분리해서 저장하고, face RQD 산정에서 borehole과 같은 x-방향 scan을 제거할 수 있게 만든 뒤, pure mode에서 included/removed를 비교"\*\*\*하는 것입니다.

아래처럼 구현하면 됩니다.

---

## 1. Tunnel에 include\_face\_axial\_scan flag 추가

src/core/tunnel.py

### 1-1. Tunnel.\_\_init\_\_ 수정

기존이 대략 이런 형태라면:

```
python
class Tunnel:
 def __init__(self, domain, correction_mode="practice"):
 self.domain = domain
 self.correction_mode = correction_mode
```

다음처럼 바꿉니다.

```
python
class Tunnel:
 def __init__(
 self,
 domain,
 correction_mode="practice",
 include_face_axial_scan=True,
):
 self.domain = domain
 self.correction_mode = correction_mode
 self.include_face_axial_scan = include_face_axial_scan
```

핵심은 `correction_mode == "pure"`와 axial scan 제거를 자동으로 묵지 않는 것입니다.

즉, 아래 두 조건을 독립적으로 실험할 수 있어야 합니다.

```
text
pure + axial included
pure + axial removed
```

따라서 `include_face_axial_scan`은 명시 flag로 두는 게 좋습니다.

---

## 2. `_multi_direction_face_rqd()`에서 axial direction 선택화

**src/core/tunnel.py**

기존 함수가 대략 다음 구조일 가능성이 큽니다.

```
python
def _multi_direction_face_rqd(self, face_x, n_scanlines=8, scan_length=10.0):
 scan_directions = []

 scan_directions.append(np.array([1.0, 0.0, 0.0]))

 ...
```

다음처럼 수정합니다.

```
python
def _multi_direction_face_rqd(
 self,
 face_x,
 n_scanlines=8,
 scan_length=10.0,
 include_axial_direction=True,
):
 scan_directions = []
```

```

Axial scan: same direction as borehole, i.e. x-direction
if include_axial_direction:
 scan_directions.append(np.array([1.0, 0.0, 0.0]))

기존 transverse / diagonal / other face-related directions는 그대로 유지
예:
scan_directions.append(np.array([0.0, 1.0, 0.0]))
scan_directions.append(np.array([0.0, 0.0, 1.0]))
scan_directions.append(...)

```

그리고 함수 끝부분에 안전장치를 하나 넣는 것이 좋습니다.

```

python
if len(scan_directions) == 0:
 raise ValueError(
 "No face scan directions defined. "
 "If include_axial_direction=False, at least one non-axial scan direction "
 "is required."
)

```

이렇게 해야 axial direction을 제거했을 때 실수로 scan direction이 하나도 남지 않는 상황을 바로 잡을 수 있습니다.

### 3. `sample_face()`에서 `axial flag` 전달

`src/core/tunnel.py`

기존에 다음처럼 호출하고 있을 것입니다.

```

python
face_rqd_values = self._multi_direction_face_rqd(
 face_x,
 n_scanlines=n_scanlines,
 scan_length=scan_length,
)

```

다음처럼 바꿉니다.

```

python
face_rqd_values = self._multi_direction_face_rqd(
 face_x,
 n_scanlines=n_scanlines,
 scan_length=scan_length,
 include_axial_direction=self.include_face_axial_scan,
)

```

이제 같은 `pure mode`에서도 다음 두 실험을 분리할 수 있습니다.

python

```
Tunnel(domain, correction_mode="pure", include_face_axial_scan=True)
Tunnel(domain, correction_mode="pure", include_face_axial_scan=False)
```

#### 4. `sample_face()`에 face Q' estimator 다중 저장 추가

##### 핵심 목적

현재 `Qp_face_mean` 하나만 있으면 그것이 어떤 RQD estimator에 기반한 것인지 모호합니다.

따라서 face 쪽에서 적어도 아래 값을 모두 저장해야 합니다.

```
text
Qp_face_from_rqd_mean
Qp_face_from_rqd_min_dir
Qp_face_from_rqd_practical

Q_face_from_rqd_mean
Q_face_from_rqd_min_dir
Q_face_from_rqd_practical
```

##### `src/core/tunnel.py`의 `sample_face()` 내부

현재 내부에 아마 다음 값들이 있을 것입니다.

```
python
rqd_mean = ...
rqd_min_dir = ...
rqd_practical = ...

jr_mean = ...
ja_mean = ...
jw_mean = ...
srf_mean = ...
jn_val = ...
```

그 주변에 다음을 추가합니다.

```
python
Qp_from_rqd_mean = QCalculator.calc_Qprime(
 rqd_mean,
 jn_val,
 jr_mean,
 ja_mean,
)

Qp_from_rqd_min_dir = QCalculator.calc_Qprime(
 rqd_min_dir,
 jn_val,
 jr_mean,
 ja_mean,
```



```

)

Qp_from_rqd_practical = QCalculator.calc_Qprime(
 rqd_practical,
 jn_val,
 jr_mean,
 ja_mean,
)

Q_from_rqd_mean = QCalculator.calc_Q(
 rqd_mean,
 jn_val,
 jr_mean,
 ja_mean,
 jw_mean,
 srf_mean,
)

Q_from_rqd_min_dir = QCalculator.calc_Q(
 rqd_min_dir,
 jn_val,
 jr_mean,
 ja_mean,
 jw_mean,
 srf_mean,
)

Q_from_rqd_practical = QCalculator.calc_Q(
 rqd_practical,
 jn_val,
 jr_mean,
 ja_mean,
 jw_mean,
 srf_mean,
)

```

그리고 기존 return dict에 다음 값을 추가합니다.

```

python
return {
 ...

 "_Qp_from_rqd_mean": Qp_from_rqd_mean,
 "_Qp_from_rqd_min_dir": Qp_from_rqd_min_dir,
 "_Qp_from_rqd_practical": Qp_from_rqd_practical,

 "_Q_from_rqd_mean": Q_from_rqd_mean,
 "_Q_from_rqd_min_dir": Q_from_rqd_min_dir,
 "_Q_from_rqd_practical": Q_from_rqd_practical,

 "_face_include_axial_scan": self.include_face_axial_scan,
}

```

가능하면 RQD 자체도 같이 저장하는 것을 추천합니다.

```
python
return {
 ...

 "_face_rqd_mean": rqd_mean,
 "_face_rqd_min_dir": rqd_min_dir,
 "_face_rqd_practical": rqd_practical,
}
```

이것이 있으면 나중에 Q' 차이가 RQD 때문인지 Jr/Ja 때문인지 분리하기 쉽습니다.

---

## 5. comparison.py에 새 컬럼 추가

### 5-1. compare\_at\_face() 수정

src/core/comparison.py

기존 comparison dict에 다음을 추가합니다.

```
python
comparison = {
 ...

 "Qp_face_from_rqd_mean": face.get("_Qp_from_rqd_mean", None),
 "Qp_face_from_rqd_min_dir": face.get("_Qp_from_rqd_min_dir", None),
 "Qp_face_from_rqd_practical": face.get("_Qp_from_rqd_practical", None),

 "Q_face_from_rqd_mean": face.get("_Q_from_rqd_mean", None),
 "Q_face_from_rqd_min_dir": face.get("_Q_from_rqd_min_dir", None),
 "Q_face_from_rqd_practical": face.get("_Q_from_rqd_practical", None),

 "face_rqd_mean": face.get("_face_rqd_mean", None),
 "face_rqd_min_dir": face.get("_face_rqd_min_dir", None),
 "face_rqd_practical": face.get("_face_rqd_practical", None),

 "face_include_axial_scan": face.get("_face_include_axial_scan", None),
}
```

0보다는 None을 추천합니다.

이유는 값이 없을 때 0으로 들어가면 나중에 log error 계산에서 가짜 outlier가 됩니다.

---

### 5-2. comparisons\_to\_face\_rows() 수정

row dict에도 동일하게 추가합니다.

```
python
row = {
 ...

 "Qp_face_from_rqd_mean": c.get("Qp_face_from_rqd_mean"),
 "Qp_face_from_rqd_min_dir": c.get("Qp_face_from_rqd_min_dir"),
 "Qp_face_from_rqd_practical": c.get("Qp_face_from_rqd_practical"),

 "Q_face_from_rqd_mean": c.get("Q_face_from_rqd_mean"),
 "Q_face_from_rqd_min_dir": c.get("Q_face_from_rqd_min_dir"),
 "Q_face_from_rqd_practical": c.get("Q_face_from_rqd_practical"),

 "face_rqd_mean": c.get("face_rqd_mean"),
 "face_rqd_min_dir": c.get("face_rqd_min_dir"),
 "face_rqd_practical": c.get("face_rqd_practical"),

 "face_include_axial_scan": c.get("face_include_axial_scan"),
}
```

## 6. Batch config에 `include_face_axial_scan` 추가

`src/cli/batch_runner.py`

BatchConfig에 추가합니다.

```
python
@dataclass
class BatchConfig:
 ...
 correction_mode: str = "practice"
 include_face_axial_scan: bool = True
```

그리고 Tunnel 생성 부분을 다음처럼 수정합니다.

```
python
tunnel = Tunnel(
 domain,
 correction_mode=self.config.correction_mode,
 include_face_axial_scan=self.config.include_face_axial_scan,
)
```

`src/cli/multicase_batch_runner.py`

MultiCaseBatchConfig에도 추가합니다.

```
python
@dataclass
class MultiCaseBatchConfig:
```

```
...
correction_mode: str = "practice"
include_face_axial_scan: bool = True
```

그리고 내부에서 BatchConfig 만들 때 전달합니다.

```
python
batch_config = BatchConfig(
 ...
 correction_mode=self.config.correction_mode,
 include_face_axial_scan=self.config.include_face_axial_scan,
)
```

## 7. run\_multicase\_decision.py에서 A/B 실험 가능하게 만들기

### 단순한 방법

src/cli/run\_multicase\_decision.py에서 두 번 실행할 수 있게 합니다.

예:

```
python
def main():
 # A. pure + axial included
 config_included = MultiCaseBatchConfig(
 ...
 correction_mode="pure",
 include_face_axial_scan=True,
 output_dir="outputs/multicase_pure_axial_included",
)

 runner_included = MultiCaseBatchRunner(config_included)
 runner_included.run()

 # B. pure + axial removed
 config_removed = MultiCaseBatchConfig(
 ...
 correction_mode="pure",
 include_face_axial_scan=False,
 output_dir="outputs/multicase_pure_axial_removed",
)

 runner_removed = MultiCaseBatchRunner(config_removed)
 runner_removed.run()
```

더 깔끔하게 하려면 CLI argument로 받는 것이 좋습니다.

예:

```
bash
```

```
python -m src.cli.run_multicase_decision --correction-mode pure --include-face-axial-scan true
python -m src.cli.run_multicase_decision --correction-mode pure --include-face-axial-scan false
```

하지만 지금은 우선 실험 재현이 목적이므로 hard-coded 두 번 실행도 충분합니다.

---

## 8. Face estimator 비교 함수 추가

새 파일로 분리하는 것을 추천합니다.

예:

```
text
src/core/estimator_comparison.py
```

또는 기존 decision\_test.py에 넣어도 됩니다.

---

### src/core/estimator\_comparison.py

```
python
import numpy as np
import pandas as pd

def _safe_corr(x, y):
 x = np.asarray(x, dtype=float)
 y = np.asarray(y, dtype=float)

 mask = np.isfinite(x) & np.isfinite(y)

 if mask.sum() < 3:
 return np.nan

 x = x[mask]
 y = y[mask]

 if np.std(x) == 0 or np.std(y) == 0:
 return np.nan

 return float(np.corrcoef(x, y)[0, 1])

def compare_bh_to_face_estimators(
 face_rows,
 bh_col="Qp_borehole_mean",
 face_estimator_cols=None,
 threshold=40.0,
```

```

cost_false_safe=10.0,
cost_false_alarm=1.0,
eps=1e-9,
):
 """
 Compare borehole Q' against multiple face Q' estimators.

 false-safe:
 borehole predicts good/excavatable, but face is bad.

 false-alarm:
 borehole predicts bad/not excavatable, but face is good.
 """

 if face_estimator_cols is None:
 face_estimator_cols = [
 "Qp_face_mean",
 "Qp_face_from_rq_d_mean",
 "Qp_face_from_rq_d_min_dir",
 "Qp_face_from_rq_d_practical",
]

 df = pd.DataFrame(face_rows).copy()

 results = []

 for face_col in face_estimator_cols:
 if face_col not in df.columns:
 continue

 if bh_col not in df.columns:
 raise KeyError(f"Missing borehole column: {bh_col}")

 sub = df[[bh_col, face_col]].copy()
 sub = sub.replace([np.inf, -np.inf], np.nan).dropna()

 sub = sub[(sub[bh_col] > 0) & (sub[face_col] > 0)]

 if len(sub) == 0:
 results.append({
 "face_estimator": face_col,
 "n": 0,
 "mean_diff": np.nan,
 "median_diff": np.nan,
 "mean_abs_log10_error": np.nan,
 "corr": np.nan,
 "accuracy": np.nan,
 "balanced_accuracy": np.nan,
 "false_safe_rate": np.nan,
 "false_alarm_rate": np.nan,
 "expected_cost": np.nan,
 })
 continue

```

```

bh = sub[bh_col].astype(float).values
face = sub[face_col].astype(float).values

diff = bh - face
abs_log10_error = np.abs(np.log10((bh + eps) / (face + eps)))

Actual condition from face
actual_good = face >= threshold

Prediction from borehole
pred_good = bh >= threshold

TP = int(np.sum(pred_good & actual_good))
FP = int(np.sum(pred_good & ~actual_good)) # false-safe
FN = int(np.sum(~pred_good & actual_good)) # false-alarm
TN = int(np.sum(~pred_good & ~actual_good))

n_good = TP + FN
n_bad = FP + TN

sensitivity = TP / n_good if n_good > 0 else np.nan
specificity = TN / n_bad if n_bad > 0 else np.nan

balanced_accuracy = np.nanmean([sensitivity, specificity])

accuracy = (TP + TN) / len(sub)

false_safe_rate = FP / n_bad if n_bad > 0 else np.nan
false_alarm_rate = FN / n_good if n_good > 0 else np.nan

expected_cost = (
 cost_false_safe * FP + cost_false_alarm * FN
) / len(sub)

results.append({
 "face_estimator": face_col,
 "n": len(sub),

 "bh_mean": float(np.mean(bh)),
 "face_mean": float(np.mean(face)),

 "mean_diff": float(np.mean(diff)),
 "median_diff": float(np.median(diff)),
 "mean_abs_diff": float(np.mean(np.abs(diff))),
 "mean_abs_log10_error": float(np.mean(abs_log10_error)),
 "median_abs_log10_error": float(np.median(abs_log10_error)),
 "corr": _safe_corr(bh, face),

 "threshold": threshold,
 "prior_face_good_rate": float(np.mean(actual_good)),

 "TP": TP,
 "FP_false_safe": FP,
 "FN_false_alarm": FN,

```

```

 "TN": TN,

 "accuracy": float(accuracy),
 "balanced_accuracy": float(balanced_accuracy),
 "false_safe_rate": float(false_safe_rate),
 "false_alarm_rate": float(false_alarm_rate),
 "expected_cost": float(expected_cost),
 })

 return pd.DataFrame(results)

```

## 9. Axial included vs removed 비교용 스크립트

예를 들어 다음 파일을 추가합니다.

```

text
src/cli/analyze_estimator_separation.py

```

### 코드 예시

```

python
import pandas as pd

from src.core.estimator_comparison import compare_bh_to_face_estimators

def main():
 included_csv = "outputs/multicase_pure_axial_included/all_cases_face_rows.csv"
 removed_csv = "outputs/multicase_pure_axial_removed/all_cases_face_rows.csv"

 threshold = 40.0

 df_included = pd.read_csv(included_csv)
 df_removed = pd.read_csv(removed_csv)

 face_cols = [
 "Qp_face_mean",
 "Qp_face_from_rqd_mean",
 "Qp_face_from_rqd_min_dir",
 "Qp_face_from_rqd_practical",
]

 res_included = compare_bh_to_face_estimators(
 df_included,
 face_estimator_cols=face_cols,
 threshold=threshold,
)
 res_included["experiment"] = "pure_axial_included"

 res_removed = compare_bh_to_face_estimators(
 df_removed,
 face_estimator_cols=face_cols,
 threshold=threshold,
)

```



```

)
res_removed["experiment"] = "pure_axial_removed"

out = pd.concat([res_included, res_removed], ignore_index=True)

out = out[
 [
 "experiment",
 "face_estimator",
 "n",
 "threshold",
 "prior_face_good_rate",
 "bh_mean",
 "face_mean",
 "mean_diff",
 "mean_abs_diff",
 "mean_abs_log10_error",
 "corr",
 "TP",
 "FP_false_safe",
 "FN_false_alarm",
 "TN",
 "accuracy",
 "balanced_accuracy",
 "false_safe_rate",
 "false_alarm_rate",
 "expected_cost",
]
]

print(out.to_string(index=False))

out.to_csv(
 "outputs/estimator_separation_summary.csv",
 index=False,
)

if __name__ == "__main__":
 main()

```

실행:

```

bash
python -m src.cli.analyze_estimator_separation

```

## 10. 이번 테스트에서 봐야 할 핵심 패턴

결과 해석은 다음 기준으로 하면 됩니다.

### Case 1. Axial 제거 후에도 BH와 face estimator가 잘 맞음

예:

```
text
pure + axial included:
 mean_abs_log10_error ≈ 0.02~0.03
 corr high

pure + axial removed:
 mean_abs_log10_error ≈ 0.02~0.04
 corr high
```

이 경우에는 다음 해석이 가능합니다.

```
text
BH Q'와 face Q'의 일치는 단순히 x-direction scan 공유 때문만은 아니다.
동일 DFN 내에서 수평시추공 Q'가 face condition을 어느 정도 대표하고 있을 가능성이 있다.
```

다만 여전히  $J_r/J_a, J_n$ 이 공유되어 있으므로 완전 독립 검증은 아닙니다.

---

### Case 2. Axial 제거 후 일치도가 크게 나빠짐

예:

```
text
pure + axial included:
 mean_abs_log10_error ≈ 0.026
 corr very high

pure + axial removed:
 mean_abs_log10_error 증가
 corr 감소
 false-safe 또는 false-alarm 증가
```

이 경우에는 다음 해석이 강합니다.

```
text
pure mode의 높은 일치는 face scanline에 borehole과 동일한 x-direction 정보가 포함되어 있었기
때문일 수 있다.
즉, 기존 pure mode 결과는 독립적인 face validation이라기보다 estimator overlap 효과를
포함한다.
```

이 결과가 나오면 이후 face estimator는 axial scan을 제거한 값 또는  $J_v$  기반 값을 주력으로 삼는 것이 좋습니다.

---

### Case 3. BH가 Qp\_face\_from\_rqd\_mean과만 잘 맞고 min\_dir, practical과는 안 맞음

예:

```
text
BH vs Qp_face_from_rqd_mean:
 error low, corr high

BH vs Qp_face_from_rqd_min_dir:
 error high, corr lower

BH vs Qp_face_from_rqd_practical:
 error high, decision metrics worse
```

이 경우 핵심 결론은 다음입니다.

```
text
BH Q'는 평균적 line-scan RQD와는 잘 맞지만, 불리한 방향 또는 conservative face condition을
대표하지 못한다.
```

즉, “굴진면 평균 조건 예측”에는 유용할 수 있지만, “시공 리스크 판단”에는 별도 보수적 estimator가 필요하다는 결론으로 갈 수 있습니다.

---

### Case 4. min\_dir 기준에서 false-safe가 증가함

이 경우 매우 중요합니다.

예:

```
text
BH vs Qp_face_from_rqd_mean:
 false_safe_rate low

BH vs Qp_face_from_rqd_min_dir:
 false_safe_rate high
```

해석:

```
text
수평시추공 Q'는 평균 face quality는 잘 예측하지만, 최악 방향 또는 국부적으로 불리한 굴진면 조건을
놓칠 수 있다.
```

이건 논문/연구 관점에서 매우 좋은 결과입니다.

왜냐하면 단순히 “맞는다/안 맞는다”가 아니라,

text

어떤 face condition 정의에 대해서 borehole Q'가 decision-useful한가?

를 구분할 수 있기 때문입니다.

---

## 11. Threshold sweep 관련 주의점

이전 pure mode에서 threshold  $\approx 15.56$ 에서 accuracy = 1.0이 나왔다는 점은 흥미롭지만, 그대로 강한 결론으로 쓰면 위험합니다.

왜냐하면 그 threshold는 같은 데이터에서 찾은 값이므로 data leakage 가능성이 있습니다.

따라서 다음처럼 구분하는 것이 좋습니다.

text

1. Fixed engineering threshold:  
예:  $Q' = 10, 40$  등 사전에 정한 값
2. Data-driven threshold:  
같은 데이터에서 sweep하여 찾은 최적 threshold
3. Cross-validated threshold:  
일부 case/seed에서 threshold를 정하고, 나머지 case/seed에서 평가

현재 단계에서는 우선 1번과 2번을 같이 보고, 나중에 3번으로 확장하면 됩니다.

---

## 12. 추천 실행 순서

지금은 다음 순서로 진행하는 것이 가장 좋습니다.

---

### Step A. 코드 수정

수정 파일:

text

```
src/core/tunnel.py
src/core/comparison.py
src/cli/batch_runner.py
src/cli/multicase_batch_runner.py
src/cli/run_multicase_decision.py
```

추가 파일:

```
text
src/core/estimator_comparison.py
src/cli/analyze_estimator_separation.py
```

---

### Step B. pure + axial included 실행

```
bash
python -m src.cli.run_multicase_decision
```

설정:

```
python
correction_mode="pure"
include_face_axial_scan=True
output_dir="outputs/multicase_pure_axial_included"
```

---

### Step C. pure + axial removed 실행

```
bash
python -m src.cli.run_multicase_decision
```

설정:

```
python
correction_mode="pure"
include_face_axial_scan=False
output_dir="outputs/multicase_pure_axial_removed"
```

---

### Step D. estimator separation 분석

```
bash
python -m src.cli.analyze_estimator_separation
```

출력:

```
text
outputs/estimator_separation_summary.csv
```

---

## 13. 이번 단계의 핵심 판정 질문

이번 테스트가 답해야 하는 질문은 이것입니다.

```
text
pure mode에서 BH Q'와 face Q'가 잘 맞았던 이유는
정말 수평시추공이 face condition을 대표했기 때문인가,
아니면 face estimator가 borehole과 유사한 line-scan 구조를 공유했기 때문인가?
```

따라서 결과표에서는 특히 아래 네 개를 비교하면 됩니다.

text

1. pure + axial included + Qp\_face\_from\_rqd\_mean
2. pure + axial removed + Qp\_face\_from\_rqd\_mean
3. pure + axial included + Qp\_face\_from\_rqd\_min\_dir
4. pure + axial removed + Qp\_face\_from\_rqd\_min\_dir

이 네 조합에서 `mean_abs_log10_error`, `corr`, `false_safe_rate`, `expected_cost`가 어떻게 변하는지가 핵심입니다.

GPT-5.5

고마워 수고 많이 했고. 내일 또 하자고.



천만에요. 오늘 정리 아주 잘 됐습니다.

내일은 바로 이어서:

1. Tunnel에 `include_face_axial_scan` 추가
2. `sample_face()`에서 `face Q' estimator` 여러 개 저장
3. `comparison.py` 컬럼 확장
4. `pure + axial included / removed` 비교 실행
5. estimator별 `mean_diff`, `abs_log_error`, `corr`, `decision metrics` 비교

이 순서로 가면 될 것 같습니다.

오늘 핵심 결론은 명확합니다:

- **practice mode**의 **false-safe** 문제는 상당 부분 **artificial correction** 구조에서 온 것 같음
- **pure mode**의 좋은 일치는 아직 독립 검증이라 보기 어려움
- 그래서 내일 할 일은 **estimator separation + axial scan 제거 테스트**입니다.

수고 많으셨습니다. 내일 이어서 하면 됩니다.

GPT-5.5

`rqd_calculator.py` : """

RQD 계산기

- 직접법 (Deere 정의)
  - 이론식 (Priest-Hudson, 1976)
  - 방향별 RQD (시추공/막장면 전용)
- """

```
import numpy as np
from typing import List, Tuple, Dict, Optional
```

```
class RQDCalculator:
```

```
 """RQD 계산 (정적 메서드 모음)"""
```

```
 text
```

```
 @staticmethod
```

```
 def calc_direct(intersection_distances: List[float],
 total_length: float,
 threshold: float = 0.1) -> float:
```

```
 """
```

```
 Deere 직접 정의:
```

```
 $RQD = \sum(\text{코어길이} \geq \text{threshold}) / \text{total_length} \times 100$
```

```
 """
```

```
 if total_length <= 0:
```

```
 return 0.0
```

```
 if len(intersection_distances) == 0:
```

```
 return 100.0
```

```
 points = [0.0] + sorted(intersection_distances) + [total_length]
```

```
 sum_intact = 0.0
```

```
 for i in range(len(points) - 1):
```

```
 spacing = points[i + 1] - points[i]
```

```
 if spacing >= threshold:
```

```
 sum_intact += spacing
```

```
 return np.clip((sum_intact / total_length) * 100.0, 0, 100)
```

```
 @staticmethod
```

```
 def calc_theoretical(linear_frequency: float,
 threshold: float = 0.1) -> float:
```

```
 """
```

```
 Priest & Hudson (1976):
```

```
 $RQD = 100 \times \exp(-\lambda t) \times (\lambda t + 1)$
```

```
 """
```

```
 lt = linear_frequency * threshold
```

```
 return np.clip(100.0 * np.exp(-lt) * (lt + 1.0), 0, 100)
```

```
 @staticmethod
```

```
 def calc_apparent_frequency(joint_normal: np.ndarray,
 true_spacing: float,
 scanline_direction: np.ndarray) -> float:
```

```
 """
```

```
 겉보기 선형빈도:
```

```
 $\lambda_{app} = |\cos \alpha| / \text{spacing}$
```

```
 α = 법선과 스캔라인의 각도
```

```
 """
```

```
 cos_alpha = abs(np.dot(
 joint_normal / np.linalg.norm(joint_normal),
 scanline_direction / np.linalg.norm(scanline_direction)
))
```

```
 return cos_alpha / true_spacing
```

class DirectionalRQDCalculator:

"""시추공(1D) 및 막장면(2D) 방향별 RQD 계산"""

text

```
def __init__(self, dfn):
 self.dfn = dfn
```

```
def rqd_along_borehole(self,
 origin: np.ndarray,
 direction: np.ndarray,
 length: float,
 window: float = 1.0,
 step: float = 1.0
) -> Tuple[np.ndarray, np.ndarray]:
 """이동 윈도우 방식 시추공 RQD"""
 direction = direction / np.linalg.norm(direction)
 all_ix = self.dfn.get_joints_intersecting_line(origin, direction, length)
 t_values = np.array([t for t, _ in all_ix])

 positions = np.arange(0, length - window + step, step)
 rqd_values = np.zeros(len(positions))

 for idx, pos in enumerate(positions):
 mask = (t_values >= pos) & (t_values < pos + window)
 window_t = (t_values[mask] - pos).tolist()
 rqd_values[idx] = RQDCalculator.calc_direct(window_t, window)

 return positions, rqd_values

def rqd_at_face(self,
 face_x: float,
 y_range: Tuple[float, float],
 z_range: Tuple[float, float],
 grid_spacing: float = 1.0,
 scan_length: float = 3.0
) -> Dict:
 """막장면(2D)에서의 RQD 분포 (다방향 스캔)"""
 scan_dirs = [
 np.array([1, 0, 0]),
 np.array([0, 1, 0]),
 np.array([0, 0, 1]),
 np.array([0, 1, 1]) / np.sqrt(2),
 np.array([0, 1, -1]) / np.sqrt(2),
]

 y_pts = np.arange(y_range[0], y_range[1], grid_spacing)
 z_pts = np.arange(z_range[0], z_range[1], grid_spacing)
 rqd_map = np.zeros((len(y_pts), len(z_pts)))
 rqd_by_dir = np.zeros((len(y_pts), len(z_pts), len(scan_dirs)))

 for iy, y in enumerate(y_pts):
 for iz, z in enumerate(z_pts):
 pt = np.array([face_x, y, z])
```



```

 for idir, d in enumerate(scan_dirs):
 origin = pt - (scan_length / 2) * d
 ix = self.dfn.get_joints_intersecting_line(origin, d, scan_length)
 t_vals = [t for t, _ in ix]
 rqd_by_dir[iy, iz, idir] = RQDCalculator.calc_direct(t_vals,
scan_length)
 rqd_map[iy, iz] = np.mean(rqd_by_dir[iy, iz, :])

 return {
 'y_points': y_pts, 'z_points': z_pts,
 'rqd_map': rqd_map, 'rqd_by_direction': rqd_by_dir,
 'rqd_mean': float(np.mean(rqd_map)),
 'rqd_std': float(np.std(rqd_map)),
 'face_x': face_x,
 }

```

<- 이걸 rqd calculator 인데 여기서 사용한 스캔라인법을 정리해주고 혹시 팜스트롬의 Jv 식을 활용한 방법이 사용되었는지 확인해줘



네. 올려주신 `rqd_calculator.py` 기준으로 보면, 여기서는 직접 선형 scanline 기반 RQD 계산법과 Priest-Hudson 이론식은 구현되어 있지만, 팜스트롬 Palmström의 Jv 기반 RQD 식은 사용되지 않았습니다.

정리하면 다음과 같습니다.

## 1. 이 파일에서 사용한 RQD 계산 방식

이 파일에는 크게 세 가지 RQD 관련 기능이 있습니다.

text

1. 직접법 RQD
2. Priest-Hudson 이론식 RQD
3. 시추공/막장면 방향별 scanline RQD

## 2. 직접법: `RQDCalculator.calc_direct()`

python

```
def calc_direct(intersection_distances, total_length, threshold=0.1)
```

이 함수는 Deere의 기본 RQD 정의를 그대로 사용합니다.

공식은 다음과입니다.

text

$$RQD = \frac{\text{절리 사이 intact segment 중 길이} \geq 0.1 \text{ m 인 구간들의 합}}{\text{전체 길이}} \times 100$$

코드 흐름은 다음과 같습니다.

```
python
points = [0.0] + sorted(intersection_distances) + [total_length]
```

즉, scanline 시작점과 끝점 사이에 절리 교차 위치들을 넣습니다.

예를 들어 scanline 길이가 3 m이고 절리 교차점이 다음과 같다면:

```
text
0.4 m, 0.8 m, 2.0 m
```

points는 다음이 됩니다.

```
text
0.0, 0.4, 0.8, 2.0, 3.0
```

그 다음 각 intact segment 길이를 계산합니다.

```
text
0.0 ~ 0.4 = 0.4 m
0.4 ~ 0.8 = 0.4 m
0.8 ~ 2.0 = 1.2 m
2.0 ~ 3.0 = 1.0 m
```

각 segment가 threshold 이상이면 RQD에 포함합니다.

기본 threshold는 다음입니다.

```
python
threshold = 0.1
```

즉, 10 cm 이상인 코어 조각만 RQD에 포함합니다.

따라서 이 함수는 시추공 또는 scanline 위에서 절리 교차점들을 직접 찾고, 그 사이 간격으로 RQD를 계산하는 방식입니다.

---

### 3. Priest-Hudson 이론식: `calc_theoretical()`

```
python
def calc_theoretical(linear_frequency, threshold=0.1)
```

여기서는 Priest & Hudson, 1976 이론식이 사용됩니다.

코드상 공식은 다음입니다.

```
python
lt = linear_frequency * threshold
RQD = 100.0 * np.exp(-lt) * (lt + 1.0)
```

즉,

```
text
RQD = 100 * exp(-λt) * (λt + 1)
```

여기서:

```
text
λ = linear_frequency
t = threshold = 기본 0.1 m
```

입니다.

이 방식은 실제 절리 교차점을 하나하나 이용하는 직접법이 아니라, 선형 절리 빈도  $\lambda$ 를 이용해서 확률론적으로 RQD를 추정하는 방식입니다.

하지만 이론식 함수는 이 파일에 존재할 뿐, 아래 `rqd_along_borehole()`이나 `rqd_at_face()`에서는 직접 사용되지 않습니다.

즉, 이 파일의 실제 방향별 RQD 계산은 대부분 `calc_direct()` 기반입니다.

---

#### 4. 겉보기 선형빈도 계산: `calc_apparent_frequency()`

```
python
def calc_apparent_frequency(joint_normal, true_spacing, scanline_direction)
```

이 함수는 절리 법선과 scanline 방향의 각도를 이용해서 apparent linear frequency를 계산합니다.

코드 공식은 다음입니다.

```
python
cos_alpha = abs(np.dot(joint_normal_unit, scanline_direction_unit))
lambda_app = cos_alpha / true_spacing
```

즉,

```
text
λ_app = |cos α| / spacing
```

여기서:

```
text
```

$\alpha$  = 절리 법선과 scanline 방향 사이의 각도

입니다.

해석은 다음과 같습니다.

- 절리 법선 방향과 scanline 방향이 평행하면  $|\cos \alpha|$ 가 큼.
- 그러면 scanline이 절리를 자주 교차함.
- 절리 법선 방향과 scanline 방향이 직교하면  $|\cos \alpha|$ 가 작음.
- 그러면 scanline이 절리를 거의 만나지 않음.

이 함수는 방향성 효과를 설명할 수 있는 함수입니다.

다만 현재 올려주신 코드 안에서는 이 함수도 `rqd_along_borehole()`이나 `rqd_at_face()`에서 직접 호출되지는 않습니다.

---

## 5. 시추공 RQD scanline 방식: `rqd_along_borehole()`

```
python
def rqd_along_borehole(
 origin,
 direction,
 length,
 window=1.0,
 step=1.0
)
```

이 함수는 시추공 방향을 따라 이동 윈도우 방식으로 RQD를 계산합니다.

### 5-1. 전체 시추공과 절리의 교차점 탐색

```
python
all_ix = self.dfn.get_joints_intersecting_line(origin, direction, length)
t_values = np.array([t for t, _ in all_ix])
```

여기서 `dfn.get_joints_intersecting_line()`은 주어진 선분과 DFN 절리 disk들이 만나는 지점을 찾습니다.

`t`는 scanline 시작점으로부터의 거리입니다.

즉, 시추공을 하나의 1D line으로 보고, 그 line이 절리들과 만나는 위치를 구합니다.

---

### 5-2. 이동 윈도우 생성

```
python
```

```
positions = np.arange(0, length - window + step, step)
```

예를 들어:

```
text
length = 10 m
window = 1 m
step = 1 m
```

이면 positions는 대략 다음과 같습니다.

```
text
0, 1, 2, 3, 4, 5, 6, 7, 8, 9
```

각 위치에서 1 m 구간 RQD를 계산합니다.

---

### 5-3. 각 window 안의 절리 교차점만 추출

```
python
mask = (t_values >= pos) & (t_values < pos + window)
window_t = (t_values[mask] - pos).tolist()
```

예를 들어 pos = 4이고 window = 1이면, 4~5 m 사이의 절리 교차점만 골라냅니다.

그 후 window 시작점을 0으로 다시 맞추습니다.

```
text
절리 교차점이 전체 시추공 기준 4.2, 4.8 m라면
window 기준으로는 0.2, 0.8 m
```

---

### 5-4. 직접법 RQD 계산

```
python
rqd_values[idx] = RQDCalculator.calc_direct(window_t, window)
```

따라서 시추공 RQD는 다음 구조입니다.

```
text
시추공 line 설정
→ DFN 절리와 line 교차점 계산
→ 일정 길이 window로 나눔
→ 각 window 안의 절리 간격 계산
→ 0.1 m 이상 intact segment 합산
→ RQD 산정
```

즉, 진짜 **scanline** 직접법입니다.

---

## 6. 막장면 RQD scanline 방식: `rqd_at_face()`

```
python
def rqd_at_face(
 face_x,
 y_range,
 z_range,
 grid_spacing=1.0,
 scan_length=3.0
)
```

이 함수는 막장면, 즉 `x = face_x` 평면 위에 격자점을 만들고, 각 격자점에서 여러 방향 scanline을 쏘아서 RQD를 계산합니다.

---

### 6-1. 사용된 scan direction

코드에서는 총 5개 방향을 사용합니다.

```
python
scan_dirs = [
 np.array([1, 0, 0]),
 np.array([0, 1, 0]),
 np.array([0, 0, 1]),
 np.array([0, 1, 1]) / np.sqrt(2),
 np.array([0, 1, -1]) / np.sqrt(2),
]
```

이를 해석하면 다음과 같습니다.

| 방향 벡터                        | 의미                      |
|------------------------------|-------------------------|
| <code>[1, 0, 0]</code>       | x방향, 즉 터널 축방향 또는 시추공 방향 |
| <code>[0, 1, 0]</code>       | y방향, 막장면 내 수평 방향        |
| <code>[0, 0, 1]</code>       | z방향, 막장면 내 수직 방향        |
| <code>[0, 1, 1] / √2</code>  | yz 평면 내 대각 방향           |
| <code>[0, 1, -1] / √2</code> | yz 평면 내 반대 대각 방향        |

중요한 점은 첫 번째 방향입니다.

```
python
np.array([1, 0, 0])
```

이 방향은 **borehole direction**과 동일한 x방향입니다.

따라서 이 `rqd_at_face()` 함수는 막장면 RQD를 계산하면서도, 하나의 `scanline`은 실제 막장면 평면 안의 방향이 아니라 **막장면을 관통하는 축방향 scan**을 포함하고 있습니다.

이게 이전에 이야기했던 의심점과 직접 연결됩니다.

text

```
face RQD 계산에 borehole과 같은 x-direction scan이 포함되어 있음
→ pure mode에서 BH Q'와 face Q'가 잘 맞는 이유가 estimator overlap 때문일 수 있음
```

---

## 6-2. 막장면 격자 생성

python

```
y_pts = np.arange(y_range[0], y_range[1], grid_spacing)
z_pts = np.arange(z_range[0], z_range[1], grid_spacing)
```

즉, `x = face_x`인 평면에서 `y-z` 격자를 만듭니다.

예를 들어:

text

```
y_range = (-5, 5)
z_range = (-5, 5)
grid_spacing = 1
```

이면 `y, z` 방향으로 1 m 간격 격자를 만들고, 각 격자점에서 RQD를 계산합니다.

---

## 6-3. 각 격자점에서 여러 방향 scanline 생성

python

```
pt = np.array([face_x, y, z])
```

각 격자점은 다음 좌표입니다.

text

```
(face_x, y, z)
```

그 후 각 방향 `d`에 대해 `scanline` 시작점을 다음처럼 잡습니다.

python

```
origin = pt - (scan_length / 2) * d
```

즉, `scanline`의 중심이 막장면 격자점 `pt`가 되도록 합니다.

예를 들어 `scan_length = 3.0`이면:

```
text
origin = pt - 1.5 d
end = pt + 1.5 d
```

따라서 각 격자점에서 길이 3 m짜리 scanline을 중심 정렬로 배치합니다.

#### 6-4. DFN 절리와 scanline 교차점 탐색

```
python
ix = self.dfn.get_joints_intersecting_line(origin, d, scan_length)
t_vals = [t for t, _ in ix]
```

각 scanline에 대해 절리 교차점을 찾습니다.

#### 6-5. 각 방향별 RQD 계산

```
python
rqd_by_dir[iy, iz, idir] = RQDCalculator.calc_direct(t_vals, scan_length)
```

즉, 각 격자점마다 5개 방향의 RQD가 생깁니다.

구조는 다음과 같습니다.

```
text
rqd_by_dir[y index, z index, direction index]
```

#### 6-6. 방향 평균 RQD 계산

```
python
rqd_map[iy, iz] = np.mean(rqd_by_dir[iy, iz, :])
```

한 격자점에서 5개 방향 RQD를 평균내서 그 격자점의 RQD로 저장합니다.

그 다음 전체 막장면 평균 RQD를 반환합니다.

```
python
'rqd_mean': float(np.mean(rqd_map))
```

따라서 이 함수의 막장면 RQD는 다음 구조입니다.

```
text
막장면 y-z 격자 생성
→ 각 격자점에서 5개 방향 scanline 생성
→ 각 scanline과 DFN 절리 교차점 계산
```



- 직접법 RQD 계산
- 방향 평균으로 격자점 RQD 산정
- 전체 격자점 평균으로 face RQD 산정

## 7. 이 코드에서 막장면 scanline의 핵심 특징

현재 `rqd_at_face()`는 엄밀히 말하면 다음 두 종류의 scan을 섞고 있습니다.

### 7-1. 막장면 내 scan

다음 네 방향은 y-z 평면, 즉 막장면 안에 놓인 scanline입니다.

```
python
np.array([0, 1, 0])
np.array([0, 0, 1])
np.array([0, 1, 1]) / np.sqrt(2)
np.array([0, 1, -1]) / np.sqrt(2)
```

이들은 `x = face_x` 평면 안에서 움직입니다.

### 7-2. 막장면 관통 scan / 축방향 scan

다음 방향은 x방향입니다.

```
python
np.array([1, 0, 0])
```

이 scanline은 막장면 평면 안이 아니라, 막장면을 앞뒤로 관통합니다.

즉,

```
text
pt = (face_x, y, z)
origin = (face_x - scan_length/2, y, z)
end = (face_x + scan_length/2, y, z)
```

이 됩니다.

이 방향은 현재 borehole 방향과 동일합니다.

따라서 이 방향을 face RQD에 포함하면, face RQD가 borehole RQD와 구조적으로 유사해질 가능성이 있습니다.

## 8. Palmström의 Jv 기반 RQD 식이 사용되었는가?

결론부터 말하면:

text

아니요. 이 `rqd_calculator.py`에서는 Palmström의  $J_v$  기반 RQD 식이 사용되지 않았습니다.

현재 코드에 있는 RQD 식은 다음 두 가지입니다.

python

```
RQDCalculator.calc_direct()
RQDCalculator.calc_theoretical()
```

각각 다음입니다.

text

Deere 직접법:

$$RQD = \sum (\text{intact piece length} \geq 0.1 \text{ m}) / \text{total\_length} \times 100$$

Priest-Hudson 이론식:

$$RQD = 100 \times \exp(-\lambda t) \times (\lambda t + 1)$$

하지만 Palmström 방식인 다음 형태는 없습니다.

text

$$RQD = 115 - 3.3 J_v$$

또는 일반적으로 쓰이는 clipping 포함 형태:

text

$$RQD = \max(0, \min(100, 115 - 3.3 J_v))$$

도 코드에 없습니다.

또한  $J_v$ 를 계산하는 부분도 없습니다.

Palmström 방식이라면 보통 다음 같은 개념이 필요합니다.

text

$J_v$  = 체적절리계수, volumetric joint count

절리군별 간격을 이용하면 개념적으로는:

text

$$J_v = \sum 1 / S_i$$

여기서  $S_i$ 는  $i$ 번째 절리군의 평균 간격입니다.

하지만 현재 코드에는 다음과 같은 함수가 없습니다.

```
python
calc_jv(...)
calc_rqd_from_jv(...)
```

따라서 이 파일은 **Palmström Jv** 기반 RQD 산정법을 아직 포함하지 않는다고 보면 됩니다.

---

## 9. 현재 코드의 RQD 방식과 Palmström Jv 방식의 차이

### 현재 코드 방식

현재 코드는 DFN에서 실제 절리 disk와 scanline의 교차점을 찾습니다.

즉:

```
text
line을 긋는다
→ 그 line이 절리와 어디서 만나는지 계산한다
→ 절리 사이 간격으로 RQD를 계산한다
```

이 방식은 방향성이 강합니다.

예를 들어 borehole 방향이 절리 법선과 평행하면 절리를 많이 만나고, RQD가 낮아질 수 있습니다.

반대로 borehole 방향이 절리면과 거의 평행하면 절리를 잘 만나지 않아 RQD가 높게 나올 수 있습니다.

즉, 현재 방식은:

```
text
directional RQD
scanline-based RQD
intersection-based RQD
```

입니다.

---

### Palmström Jv 방식

Palmström 방식은 선형 scanline 교차점이 아니라, 암반 내 절리의 체적 밀도 또는 절리군별 간격에서 RQD를 추정합니다.

개념적으로:

```
text
절리군별 spacing 또는 volumetric joint count 계산
→ Jv 산정
```

```
→ RQD = 115 - 3.3 Jv
```

입니다.

이 방식은 특정 scanline 방향 하나보다는 암반 블록화 정도, 체적 절리 발달 정도를 반영하는 성격이 강합니다.

따라서 현재 연구에서 Jv 기반 face RQD를 추가하면, borehole scanline RQD와 더 독립적인 face condition estimator가 될 수 있습니다.

---

## 10. 현재 코드 기준 요약

현재 `rqd_calculator.py`의 구조를 한 줄로 요약하면:

```
text
DFN 절리와 scanline의 실제 교차점을 이용해 직접법 RQD를 계산하는 코드이며,
막장면에서는 y-z 격자점마다 x, y, z, yz 대각 방향 scanline을 사용한다.
Palmström Jv 기반 RQD는 아직 구현되어 있지 않다.
```

특히 연구 관점에서 중요한 포인트는 이겁니다.

```
text
rqd_at_face()의 scan_dirs에 [1, 0, 0]이 포함되어 있다.
이 방향은 borehole direction과 동일하므로,
pure mode에서 BH Q'와 face Q'가 잘 맞는 이유를 검증하려면
이 axial scan을 제거한 face RQD를 별도로 계산해야 한다.
```

따라서 다음 단계에서 추가할 flag는 적절합니다.

```
python
include_axial_direction=True
```

그리고 pure estimator separation test에서는 반드시 다음 두 조건을 비교해야 합니다.

```
text
A. pure + face axial scan included
B. pure + face axial scan removed
```

---

## 11. Palmström Jv 방식 추가 시 예상 함수 형태

나중에 추가한다면 `RQDCalculator`에 이런 함수를 넣을 수 있습니다.

```
python
@staticmethod
def calc_from_jv(jv: float) -> float:
```

```

"""
Palmström-type RQD estimate:
RQD = 115 - 3.3 * Jv
clipped to [0, 100]
"""
return float(np.clip(115.0 - 3.3 * jv, 0.0, 100.0))

```

그리고 절리군 spacing 기반으로 Jv를 계산한다면:

```

python
@staticmethod
def calc_jv_from_spacings(spacings: List[float]) -> float:
 """
 Jv = $\sum 1 / S_i$
 """
 valid = [s for s in spacings if s is not None and s > 0]
 if len(valid) == 0:
 return 0.0
 return float(sum(1.0 / s for s in valid))

```

다만 지금 연구에서는 `density_type == 'P32'`를 주로 쓰고 있으므로, 단순히 `mean_spacing`에서 Jv를 계산하면 P32 기반 DFN 생성과 일관성이 떨어질 수 있습니다.

따라서 Jv 추가는 두 가지 중 하나로 가는 게 좋습니다.

```

text
1. YAML에 mean_spacing을 명시하고 Jv = $\sum 1/S_i$ 로 계산
2. 생성된 DFN에서 face 주변 control volume을 잡고 실제 절리 trace/volume density로 Jv를 추정

```

연구의 독립성 측면에서는 2번이 더 좋지만, 구현은 1번이 훨씬 쉽습니다.

GPT-5.5  
Show Claude Haiku 4.5's response

일단 코드의 수정방향을 예기해줄게... 스캔라인법이 많이 바뀌어야 해. 스캔라인법은 말그대로 굴착면에 대해서만 할 수 있고, 따라서 굴진면 방향으로 한다는 것은 전방 암반의 내부를 꿰뚫어 보겠다는 건데 당연히 불가능하지. 말도안되는 예기야. 그리고 스캔라인을 여러방향에 대해서 한다는 것은 현실적인 실제 터널 공사 환경에서 거의 불가능해. 바쁜 스케줄상으로 그렇게 많은 시간을 투자할 수 없거든 그래서 보통은 터널 굴진면을 수평으로 가로지르는 하나의 선분에 몇개의 절리가 교차하는지 정도만 찾아서 하는 정도야. 5개를 평균 낸다고 하면 학계, 산업계 모두에게서 비난을 받을 거야. 알겠지...그래서 스캔라인은 굴진면의 중앙부를 수평으로 가로지르는 선분 하나만 사용하는 것으로 해주고, Hudson 방법 역시 x 방향, 즉 굴진 방향으로 하는 작업은 동일한 이유로 코드를 제거해야해. 마지막으로 Jv를 이용하는 방법이 추가되어야 하고. 요청이 좀 많지? 어떻게 하면 좋은지 예기해주고...이걸 vscode의 google code assist 채팅창에 붙여 넣으면 작동을 할까?



No response provided  
GPT-5.5

일단 코드의 수정방향을 예기해줄게... 스캔라인법이 많이 바뀌어야 해. 스캔라인법은 말그대로 굴착면에 대해서만 할 수 있고, 따라서 굴진면 방향으로 한다는 것은 전방 암반의 내부를 꿰뚫어 보겠다는 건데 당연히 불가능하지. 말도안되는 예기야. 그리고 스캔라인을 여러방향에 대해서 한다는 것은 현실적인 실제 터널 공사 환경에서 거의 불가능해. 바쁜 스케줄상으로 그렇게 많은 시간을 투자할 수 없거든 그래서 보통은 터널 굴진면을 수평으로 가로지르는 하나의 선분에 몇개의 절리가 교차하는지 정도만 찾아서 하는 정도야. 5개를 평균 낸다고 하면 학계, 산업계 모두에게서 비난을 받을 거야. 알겠지...그래서 스캔라인은 굴진면의 중앙부를 수평으로 가로지르는 선분 하나만 사용하는 것으로 해주고, Hudson 방법 역시 x 방향, 즉 굴진 방향으로 하는 작업은 동일한 이유로 코드를 제거해야해. 마지막으로 Jv를 이용하는 방법이 추가되어야 하고. 요청이 좀 많지? 아직 코딩 작업은 하지말고 어떻게 하면 좋은지 플랜을 예기해줘~



맞습니다. 이 수정 방향이 훨씬 연구적으로 안전합니다.

지금 말씀하신 내용은 단순 코드 수정이 아니라 **face condition 정의 자체를 현실적인 터널 현장 관찰 방식으로 재정의하는 것**입니다.

핵심은 이것입니다.

굴진면에서의 scanline은 노출된 굴착면 위에서만 가능해야 하며, 굴진 방향 x축으로 scanline을 넣는 것은 “전방 암반 내부를 보는 것”이므로 현장적으로 불가능하고 연구적으로도 방어하기 어렵다.

따라서 지금까지의 multi-direction face RQD는 폐기하거나 legacy로 내려야 합니다.

---

## 1. 전체 수정 방향 요약

기존 face RQD 방식:

```
text
face_x 평면 위 격자 생성
각 격자점에서 5방향 scanline
[1,0,0] x방향 포함
방향별 RQD 평균
```

이 방식의 문제:

- ```
text
```
1. x방향 scanline은 굴진면 내부를 관통하므로 실제 관찰 불가능
 2. 5방향 scanline 평균은 현장 터널 공정상 비현실적
 3. borehole 방향과 face estimator가 겹쳐서 pure mode 결과가 과도하게 좋아질 수 있음
 4. 학계/산업계 관점에서 “이런 face RQD를 실제로 어떻게 측정했나?”라는 비판 가능성이 큼

새 방향:

text

1. face scanline은 굴착면 위에서만 수행
2. 굴진면 중앙부를 수평으로 가로지르는 단일 선분만 사용
3. 방향은 y방향, 즉 $[0, 1, 0]$
4. 위치는 face_x에서 $z = \text{tunnel center}$, $y = -R \sim +R$
5. x방향 face scanline 제거
6. multi-direction 평균 제거
7. 별도 face estimator로 Palmström Jv 기반 RQD 추가

이게 훨씬 좋습니다.

2. face scanline의 새 정의

터널 축을 x방향, 굴진면을 $x = \text{face_x}$ 평면이라고 하면, 굴진면은 y-z 평면입니다.

따라서 현실적인 중앙 수평 scanline은:

```
python
origin = np.array([face_x, -tunnel_radius, 0.0])
direction = np.array([0.0, 1.0, 0.0])
length = 2.0 * tunnel_radius
```

입니다.

즉:

text

```
x = face_x 고정
z = 0 고정
y = -R 에서 +R까지
```

이 선분 하나만 사용합니다.

이 방식은 실제 현장에서 다음과 같이 설명 가능합니다.

굴진면 중앙부를 수평으로 가로지르는 scanline을 설정하고, 해당 선분과 교차하는 절리 trace 수 및 간격을 이용해 face RQD를 산정하였다.

이 설명은 방어 가능합니다.

3. 기존 `rqd_at_face()`는 어떻게 처리할까?

현재 `DirectionalRQDCalculator.rqd_at_face()`는 구조적으로 문제가 있습니다.

```
python
scan_dirs = [
    np.array([1, 0, 0]),
    np.array([0, 1, 0]),
    np.array([0, 0, 1]),
    np.array([0, 1, 1]) / np.sqrt(2),
    np.array([0, 1, -1]) / np.sqrt(2),
]
```

여기서 `[1,0,0]`은 반드시 제거해야 하고, 사실 나머지 여러 방향 평균도 제거하는 게 좋습니다.

권장 방향은 두 가지입니다.

선택지 A: 기존 함수 삭제 또는 사용 중단

기존 `rqd_at_face()`를 더 이상 사용하지 않고 새 함수를 만듭니다.

예:

```
python
rqd_at_face_horizontal_scanline()
```

이게 가장 명확합니다.

장점:

```
text
기존 multi-direction face RQD와 새 realistic face RQD가 섞이지 않음
```

선택지 B: 기존 함수는 **legacy**로 남김

기존 함수를 남기되 이름과 주석을 바꿉니다.

예:

```
python
rqd_at_face_multidirectional_legacy()
```

그리고 새 함수는 별도로 만듭니다.

```
python
rqd_at_face_horizontal_scanline()
```


이 방식이 더 안전합니다.

이유는 기존 실험 결과를 재현할 수도 있고, 논문에서는 다음처럼 설명할 수 있기 때문입니다.

text

초기 검토에서는 multi-direction synthetic scanline을 사용했으나,
현장 적용성 검토 후 실제 분석에서는 중앙 수평 단일 scanline 방식으로 변경하였다.

개인적으로는 **선택지 B**를 추천합니다.

코드 추적성과 기존 결과 비교에 좋습니다.

4. 새 face scanline 함수의 역할

새 함수는 다음 값을 반환하면 됩니다.

text

```
face_x  
scanline_origin  
scanline_direction  
scanline_length  
intersection_positions  
n_intersections  
fracture_frequency  
rqd_direct  
rqd_hudson_from_face_scanline_frequency, optional  
method = "single_horizontal_face_scanline"
```

여기서 핵심 RQD는 `rqd_direct`입니다.

text

RQD_direct = scanline 위 절리 교차점 간격으로 Deere 방식 계산

그리고 optional로 Priest-Hudson 식을 쓸 수 있습니다.

다만 이때도 **x방향**이 아니라 **face** 위의 수평 scanline에서 얻은 frequency를 써야 합니다.

즉, 허용 가능한 Hudson 방식은 이것입니다.

text

λ = 중앙 수평 face scanline에서 관찰된 절리 선형빈도
 $RQD = 100 \exp(-\lambda t)(\lambda t + 1)$

허용하면 안 되는 방식은 이것입니다.

text

face condition을 평가한다고 하면서 x방향, 즉 굴진 방향으로 λ 를 계산

따라서 `calc_theoretical()` 함수 자체를 없앨 필요는 없습니다.
하지만 **face에서 x방향으로 Hudson RQD를 계산하는 경로는 제거해야** 합니다.

5. borehole RQD와 face RQD를 명확히 분리해야 함

여기서 중요한 구분이 있습니다.

Borehole RQD

수평시추공은 실제로 굴착 전 전방 암반을 x방향으로 시추한 것입니다.

따라서 borehole RQD는 x방향이어도 됩니다.

```
python
borehole direction = [1,0,0]
```

이건 실제 관측입니다.

Face RQD

굴진면 RQD는 굴착 후 노출된 면에서 관찰하는 것입니다.

따라서 face scanline은 x방향이면 안 됩니다.

```
python
face scanline direction = [0,1,0]
```

즉, 최종적으로는 다음 구조가 됩니다.

```
text
Borehole condition:
  x-direction horizontal borehole RQD

Face condition:
  single horizontal scanline on exposed face
  direction = y-direction
```

이렇게 해야 연구 질문이 명확해집니다.

굴착 전 수평시추공 x방향 Q'가, 굴착 후 실제 face 위에서 관찰되는 조건을 예측할 수 있는가?

이게 훨씬 좋은 구조입니다.

6. Tunnel.sample_face() 수정 방향

기존 sample_face()에서는 아마 이런 흐름이 있었을 겁니다.

```
text
multi-direction face RQD 계산
rqd_mean 계산
rqd_min_dir 계산
rqd_practical 계산
face Q' 계산
```

이제 이 구조는 바뀌야 합니다.

새 구조는 다음이 좋습니다.

```
text
1. 중앙 수평 face scanline RQD 계산
2. face에 노출된 절리들의 Jr/Ja 평균 계산
3. Qp_face_from_scanline 계산
4. Jv 기반 RQD 계산
5. Qp_face_from_jv 계산
6. 필요하면 face target estimator 선택
```

즉, face estimator는 최소한 두 개가 됩니다.

```
text
Qp_face_from_scanline
Qp_face_from_jv
```

그리고 선택적으로:

```
text
Qp_face_conservative = min(Qp_face_from_scanline, Qp_face_from_jv)
```

도 나중에 추가할 수 있습니다.

7. 기존 face estimator 이름도 바꾸는 게 좋음

기존 컬럼:

```
text
Qp_face_mean
Qp_face_from_rqd_mean
Qp_face_from_rqd_min_dir
Qp_face_from_rqd_practical
```

이제는 다소 부적절합니다.

새 컬럼은 이렇게 가는 게 좋습니다.

```
text
face_rqd_scanline_horizontal
face_scanline_n_intersections
face_scanline_fracture_frequency
Qp_face_from_scanline_horizontal

face_jv
face_rqd_from_jv
Qp_face_from_jv
```

그리고 기존 `Qp_face_mean`은 당분간 호환성 때문에 유지하되, 내부적으로는 새 기준을 넣는 것이 좋습니다.

예:

```
text
Qp_face_mean = Qp_face_from_scanline_horizontal
```

또는 config로 선택:

```
text
face_reference_estimator = "scanline_horizontal"
```

그러면 decision tester는 기존처럼 `Qp_face_mean`을 읽어도 작동합니다.

8. Palmström Jv 방식 추가 방향

Palmström 방식은 다음 식을 추가하면 됩니다.

```
text
RQD = 115 - 3.3 Jv
```

그리고 0~100으로 clipping합니다.

```
text
RQD_Jv = clip(115 - 3.3 Jv, 0, 100)
```

여기서 중요한 문제는 `Jv`를 어떻게 계산하느냐입니다.

가능한 방법은 세 가지입니다.

방법 1. YAML의 `mean_spacing` 기반 Jv

가장 단순한 방식입니다.

```
text
Jv =  $\sum 1 / S_i$ 
```

여기서 S_i 는 각 절리군의 평균 간격입니다.

장점:

```
text
구현이 쉽다
Palmström 정의와 설명이 직관적이다
```

단점:

```
text
현재 연구는 P32 기반으로 밀도를 조절하고 있음
density_type == "P32"이면 mean_spacing은 현재 코드상 사용되지 않음
face 위치별 변동성이 없다
```

따라서 이것만 쓰면 현재 DFN 실험 구조와 완전히 맞지는 않습니다.

방법 2. P32 기반 Jv proxy

현재 연구에서 밀도는 P32로 조절하고 있습니다.

이 경우 절리군별로 다음과 같은 근사값을 사용할 수 있습니다.

```
text
Jv_proxy  $\approx \sum P32_i$ 
```

이유는 이상적인 평행 절리군에서는 면적밀도 P32가 절리 간격의 역수와 비슷한 차원을 갖기 때문입니다.

```
text
P32  $\approx 1 / S$ 
```

따라서:

```
text
Jv  $\approx \sum P32_i$ 
```

장점:

```
text
현재 YAML 구조와 잘 맞음
```

P32 기반 실험에 바로 적용 가능

단점:

text

Palmström의 현장 Jv와 완전히 동일하다고 말하기는 어려움
"Jv proxy" 또는 "P32-derived Jv"라고 불러야 안전함
face 위치별 변동성이 없다

이 방식은 1차 구현으로 좋습니다.

방법 3. DFN local volume 기반 Jv

가장 연구적으로 좋은 방식입니다.

굴진면 주변에 작은 control volume 또는 slab를 잡습니다.

예:

text

$x = \text{face_x} \pm \Delta x/2$
y, z = 터널 단면 범위

그 안의 DFN 절리 발달 정도를 이용해서 local Jv를 계산합니다.

개념적으로:

text

$Jv_{\text{local}} \approx \text{local P32_total}$

또는 절리군별 local density를 구해서:

text

$Jv_{\text{local}} = \sum \text{local_P32_i}$

장점:

text

face 위치별 변동성이 생김
DFN 실험과 가장 잘 맞음
case/seed/face position별 차이를 반영 가능

단점:

text

구현이 더 복잡함

disk가 control volume과 부분적으로 겹칠 때 면적 계산이 까다로움

초기에는 근사 구현이 필요함

추천 순서

처음부터 local volume 기반 Jv를 완벽하게 구현하려고 하면 일이 커집니다.

그래서 단계적으로 가는 게 좋습니다.

text

1단계:

RQDCalculator.calc_from_jv(jv) 추가

2단계:

joint set 입력값 기반 Jv_global 추가

density_type == "P32"이면 $Jv = \sum P32_i$

density_type == "spacing"이면 $Jv = \sum 1/\text{mean_spacing}$

3단계:

face 주변 local DFN 기반 Jv_proxy 추가

4단계:

최종 분석에서는 scanline RQD와 Jv RQD를 둘 다 비교

9. 최종 face condition 후보

수정 후 face condition은 하나로 고정하기보다, 여러 estimator를 비교하는 게 좋습니다.

후보는 다음입니다.

9-1. Face scanline condition

text

Qp_face_from_scanline_horizontal

정의:

text

굴진면 중앙부 수평 scanline에서 직접 계산한 RQD 기반 Q'

의미:

text

현장에서 실제로 빠르게 측정 가능한 face Q'

9-2. Face Jv condition

text
Qp_face_from_jv

정의:

text
Jv 기반 RQD를 사용한 face Q'

의미:

text
절리군의 체적 발달 정도를 반영한 face Q'

9-3. Conservative face condition

text
Qp_face_conservative = min(
 Qp_face_from_scanline_horizontal,
 Qp_face_from_jv
)

의미:

text
scanline 또는 Jv 중 더 불리한 값을 face condition으로 사용

이건 decision test에서 유용합니다.

왜냐하면 굴착 가능성 판단에서는 평균적인 일치보다 false-safe를 줄이는 것이 중요하기 때문입니다.

10. 수정 대상 파일별 계획

10-1. `src/core/rqd_calculator.py`

수정 방향:

text
1. calc_direct 유지
2. calc_theoretical 유지하되, face x-direction에서 사용하지 않도록 함
3. calc_from_jv 추가

4. 기존 `rqd_at_face`는 legacy 처리
5. 새 `rqd_at_face_horizontal_scanline` 추가

새 함수 개념:

```
text
rqd_at_face_horizontal_scanline(
    face_x,
    y_min,
    y_max,
    z_center=0.0
)
```

또는 터널 반경을 받는 방식:

```
text
rqd_at_face_horizontal_scanline(
    face_x,
    tunnel_radius,
    y_center=0.0,
    z_center=0.0
)
```

반환값:

```
text
{
    "method": "single_horizontal_face_scanline",
    "face_x": face_x,
    "scanline_origin": ...,
    "scanline_direction": [0,1,0],
    "scanline_length": 2R,
    "intersection_positions": ...,
    "n_intersections": ...,
    "fracture_frequency": n / length,
    "rqd_direct": ...,
    "rqd_hudson": optional
}
```

10-2. `src/core/tunnel.py`

수정 방향:

- ```
text
```
1. `_multi_direction_face_rqd()`는 사용 중단 또는 legacy 처리
  2. `sample_face()`에서 중앙 수평 scanline RQD 사용
  3. face Q'를 scanline RQD 기반으로 계산
  4. Jv 기반 RQD와 Q'도 계산
  5. `return dict`에 새 컬럼 저장

새 return key 예:

```
text
"_face_rqd_scanline_horizontal"
"_face_scanline_n_intersections"
"_face_scanline_fracture_frequency"
"_Qp_face_from_scanline_horizontal"

"_face_jv"
"_face_rqd_from_jv"
"_Qp_face_from_jv"

"_Qp_face_conservative"
```

그리고 기존 호환용:

```
text
"_Qp_mean" = _Qp_face_from_scanline_horizontal
```

또는 config에 따라:

```
text
face_reference_estimator = "scanline_horizontal"
```

---

### 10-3. src/core/comparison.py

새 컬럼을 추가해야 합니다.

```
text
face_rqd_scanline_horizontal
face_scanline_n_intersections
face_scanline_fracture_frequency
Qp_face_from_scanline_horizontal

face_jv
face_rqd_from_jv
Qp_face_from_jv

Qp_face_conservative
```

그리고 기존 Qp\_face\_mean이 무엇을 의미하는지 명확히 해야 합니다.

추천:

```
text
Qp_face_mean = Qp_face_from_scanline_horizontal
```

이렇게 하면 기존 decision tester가 깨지지 않습니다.

---

#### 10-4. src/core/decision\_test.py

큰 수정은 필요 없을 수 있습니다.

다만 분석할 때 face target column을 선택할 수 있게 하면 좋습니다.

예:

```
python
DecisionUsefulnessTester(
 face_col="Qp_face_from_scanline_horizontal",
 bh_col="Qp_borehole_mean"
)
```

또는:

```
python
face_col="Qp_face_from_jv"
```

또는:

```
python
face_col="Qp_face_conservative"
```

이렇게 해야 다음 비교가 가능합니다.

```
text
BH vs face scanline
BH vs face Jv
BH vs conservative face
```

---

#### 11. 기존 연구 결과는 어떻게 해석해야 하나?

중요합니다.

기존 pure mode 결과:

```
text
BH Q'와 face Q'가 매우 잘 맞음
```

이 결과는 이제 이렇게 재해석해야 합니다.

```
text
기존 face estimator는 x-direction scanline과 multi-direction average를 포함했기 때문에,
실제 현장 face condition이라기보다 synthetic estimator에 가까웠다.
따라서 기존 pure mode의 높은 일치는 실제 face 검증 결과로 보기 어렵다.
```

이건 나쁜 게 아닙니다.

오히려 연구가 더 정교해지는 겁니다.

최종 연구 흐름은 이렇게 정리할 수 있습니다.

text

1. 초기 synthetic multi-direction face RQD에서는 BH와 face가 잘 일치했다.
2. 그러나 이 방식은 현장 적용성이 낮고, x-direction scan이 포함되어 독립성이 부족했다.
3. 따라서 실제 현장 관찰 가능한 중앙 수평 face scanline과 Jv 기반 estimator로 재정의했다.
4. 이후 borehole Q'의 decision usefulness를 다시 평가했다.

이 흐름은 논리적으로 매우 좋습니다.

---

## 12. 앞으로 실험 설계

수정 후 실험은 이렇게 가면 됩니다.

### Experiment 1. BH vs single horizontal face scanline

text

```
face target = Qp_face_from_scanline_horizontal
bh predictor = Qp_borehole_mean
```

확인할 것:

text

```
mean_diff
mean_abs_log10_error
corr
false-safe
false-alarm
expected_cost
baseline comparison
```

---

### Experiment 2. BH vs Jv-based face Q'

text

```
face target = Qp_face_from_jv
bh predictor = Qp_borehole_mean
```

확인할 것:

text

```
BH가 체적 절리 발달 정도를 얼마나 예측하는가?
```

---

### Experiment 3. BH vs conservative face Q'

```
text
face target = min(
 Qp_face_from_scanline_horizontal,
 Qp_face_from_jv
)
```

확인할 것:

```
text
false-safe가 증가하는가?
BH가 위험한 face condition을 놓치는가?
```

---

#### 13. 이 방향으로 바꾸면 연구 질문이 더 좋아짐

기존 질문은 약간 애매했습니다.

```
text
BH Q'와 face Q'가 일치하는가?
```

하지만 수정 후에는 훨씬 강한 질문이 됩니다.

```
text
굴착 전 수평시추공 Q'는,
실제 굴진면에서 현장적으로 관찰 가능한 중앙 수평 scanline Q' 또는 Jv 기반 Q'를
조건부로 예측할 수 있는가?
```

그리고 decision 관점에서는:

```
text
수평시추공 Q' 기반 사전 판단은
false-safe와 false-alarm을 줄이고,
always-excavate 또는 majority baseline보다 낮은 expected cost를 가지는가?
```

이 질문은 훨씬 방어 가능합니다.

---

#### 14. VSCode Google Code Assist에 한 번에 붙여넣으면 될까?

전체 내용을 한 번에 붙여넣으면 부분적으로는 작동할 수 있지만 추천하지 않습니다.

이유는 다음과 같습니다.

```
text
```

1. 수정 범위가 rqd\_calculator.py, tunnel.py, comparison.py, decision\_test.py 등 여러 파일에 걸침
2. 기존 함수 일부는 삭제가 아니라 legacy 처리해야 함
3. face scanline, Hudson, Jv, decision column이 서로 연결됨
4. 한 번에 요청하면 Code Assist가 맥락을 놓치거나 기존 호환성을 깨뜨릴 수 있음

따라서 Google Code Assist에는 **작은 단위로 나눠서 요청**하는 것이 좋습니다.

추천 순서는 다음입니다.

text

Prompt 1:

- rqd\_calculator.py 수정
- 기존 rqd\_at\_face는 legacy로 rename
- 새 rqd\_at\_face\_horizontal\_scanline 추가
- calc\_from\_jv 추가

Prompt 2:

- tunnel.py 수정
- sample\_face에서 새 horizontal scanline RQD 사용
- multi-direction face RQD 사용 중단
- Jv 기반 RQD/Q' 추가

Prompt 3:

- comparison.py 수정
- 새 face scanline/Jv 컬럼 export

Prompt 4:

- decision\_test.py 또는 분석 스크립트 수정
- face target column 선택 가능하게 변경

Prompt 5:

- run\_multicase\_decision.py 수정
- 새 realistic face estimator 기준으로 실행

## 15. Code Assist용 작업 지시문 형태

나중에 실제로 붙여넣을 때는 이런 식으로 짧고 명확하게 주는 게 좋습니다.

예:

text

Modify src/core/rqd\_calculator.py only.

Goal:

Replace the unrealistic multi-direction face RQD workflow with a field-realistic single horizontal scanline method.

Requirements:

1. Keep `RQDCalculator.calc_direct()` unchanged.
2. Keep `RQDCalculator.calc_theoretical()`, but do not use it for any x-direction face scan.
3. Add `RQDCalculator.calc_from_jv(jv)`, using `RQD = clip(115 - 3.3*jv, 0, 100)`.
4. Rename or mark `DirectionalRQDCalculator.rqd_at_face()` as legacy if possible.
5. Add `DirectionalRQDCalculator.rqd_at_face_horizontal_scanline()`.
6. The new face scanline must lie on the exposed face plane `x = face_x`.
7. Direction must be `[0, 1, 0]`.
8. The scanline should pass through the tunnel center, `z = 0`.
9. It must not include `[1, 0, 0]` or any multi-direction averaging.
10. Return `rqd_direct`, `n_intersections`, `fracture_frequency`, `intersection positions`, `scanline origin`, `direction`, and `length`.

Do not modify other files yet.

이렇게 한 파일씩 지시하면 성공률이 높습니다.

---

## 16. 결론

지금 말씀하신 수정 방향은 맞습니다.

특히 중요한 결정은 이 세 가지입니다.

text

1. face x-direction scanline 제거
2. multi-direction face scanline 평균 제거
3. 중앙 수평 단일 scanline + Jv 기반 RQD 추가

이렇게 바꾸면 연구가 훨씬 현실적이고 방어 가능해집니다.

다음 단계는 코딩 전에 먼저 다음 설계를 확정하는 것입니다.

text

- A. face scanline 길이 = 터널 직경 2R로 할지
- B. face scanline 위치 =  $z=0$  중앙선으로 고정할지
- C. Jv를 우선 P32 기반 proxy로 계산할지, local DFN 기반으로 계산할지
- D. decision test의 기본 face target을 scanline으로 할지 conservative로 할지

제 추천은 다음입니다.

text

- A. scanline length =  $2 * \text{tunnel\_radius}$
- B. scanline position =  $z\_center = 0$

```
C. 1차 Jv = P32-derived global Jv proxy
D. 기본 face target = Qp_face_from_scanline_horizontal
E. 보조 분석 = Qp_face_from_jv, Qp_face_conservative
```

이 순서로 가면 가장 안정적으로 진행할 수 있습니다.

GPT-5.5

1. 시추공으로 하는 방법, 2. Hudson lambda, 3. Scan line, 4. Jv 이렇게 4가지 방법을 비교할 수 있으면 좋겠어. 그런데 각각의 방법이 명확히 정의되고 현실성이 있어야 해.



좋습니다. 이 방향이 가장 깔끔합니다.

최종적으로는 **RQD/Q'를 산정하는 네 가지 방법**을 명확히 정의하고, 각각이 **어디에서 관측 가능한지, 현실적인지, 무엇과 비교해야 하는지**를 분리해야 합니다.

제가 보기에는 네 방법을 다음처럼 정의하면 좋습니다.

text

1. Borehole direct method  
= 굴착 전 수평시추공에서 직접 RQD 계산
2. Hudson lambda method  
= 관측 가능한 선형 절리빈도  $\lambda$ 로 Priest-Hudson RQD 계산
3. Face scanline method  
= 굴착 후 노출된 굴진면 중앙 수평 scanline에서 직접 RQD 계산
4. Jv method  
= 절리 체적계수 Jv 또는 그 proxy로 Palmström형 RQD 계산

핵심은 네 방법 모두 **\*\*\*현실적으로 관측 가능한 정보만 사용한다\*\*\***는 원칙을 지키는 것입니다.

## 1. 네 방법의 역할을 먼저 구분해야 함

중요한 점은 네 방법이 모두 같은 관측 조건을 갖는 것은 아니라는 점입니다.

크게 나누면 다음과 같습니다.

text

굴착 전 사용 가능:

1. Borehole direct method
2. Hudson lambda method, if  $\lambda$  is estimated from borehole



굴착 후 face에서 사용 가능:

3. Face scanline method

4. Jv method, if Jv is mapped from exposed face or inferred from joint sets

즉, 연구 질문은 이렇게 정리할 수 있습니다.

굴착 전 수평시추공 기반 방법들이, 굴착 후 실제 face 기반 방법들과 얼마나 잘 대응하는가?

따라서 비교 구조는 다음이 좋습니다.

text

Pre-excavation predictors:

- Qp\_borehole\_direct
- Qp\_borehole\_hudson\_lambda

Post-excavation face references:

- Qp\_face\_scanline
- Qp\_face\_jv

그리고 필요하면 네 방법 전체를 같은 표에서 비교합니다.

---

## 2. Method 1: Borehole direct method

### 정의

굴착 전 수평시추공을 터널 축방향으로 뚫고, 시추공 코어 또는 DFN상 시추공 교차점에서 직접 RQD를 계산하는 방법입니다.

text

direction = [1, 0, 0]

즉, 이 방법에서 x방향은 허용됩니다.

왜냐하면 이것은 굴진면에서 x방향으로 scanline을 넣는 것이 아니라, 실제로 굴착 전에 수평시추공을 뚫은 것이기 때문입니다.

### RQD 계산

Deere 직접법을 사용합니다.

text

$$RQD_{bh\_direct} = \frac{\sum(\text{length of intact core pieces} \geq 0.1 \text{ m})}{\text{borehole interval length}} \times 100$$

코드상으로는 현재의 `RQDCalculator.calc_direct()`를 사용하면 됩니다.

## Q' 계산

text

```
Q'_bh_direct = (RQD_bh_direct / Jn_bh) * (Jr_bh / Ja_bh)
```

## 현실성

이 방법은 현실성이 높습니다.

text

관측 대상: 굴착 전 수평시추공

관측 가능성: 가능

목적: 사전 예측

## 연구상 역할

이 방법은 최종 연구의 핵심 predictor입니다.

text

굴착 전 수평시추공 Q'가 실제 face condition을 예측할 수 있는가?

에서 바로 이 방법이 주 predictor가 됩니다.

---

## 3. Method 2: Hudson lambda method

여기서 말하는 Hudson 방법은 정확히는 **Priest-Hudson RQD** 식으로 이해하면 됩니다.

## 정의

선형 절리빈도  $\lambda$ 를 이용해 RQD를 계산합니다.

text

```
RQD = 100 * exp(-λt) * (λt + 1)
```

여기서:

text

$\lambda$  = number of discontinuities per unit length

t = threshold length = 0.1 m

즉:

text

```
RQD_Hudson = 100 * exp(-0.1λ) * (0.1λ + 1)
```

---

## 중요한 원칙

Hudson lambda 방법에서도  $\lambda$ 를 어디서 얻었는지가 반드시 명확해야 합니다.

가능한  $\lambda$ 는 두 가지입니다.

- text
1.  $\lambda_{bh}$   
수평시추공에서 얻은 선형 절리빈도
  2.  $\lambda_{face\_scanline}$   
굴진면 중앙 수평 scanline에서 얻은 선형 절리빈도

그러나 금지해야 할 것은 이것입니다.

text  
굴진면에서 x방향, 즉 굴진 방향으로  $\lambda$ 를 계산하는 것

이건 실제 face에서 관측 불가능합니다.

---

## 추천 정의

이번 연구에서 Hudson lambda 방법은 우선 다음처럼 정의하는 게 좋습니다.

text  
 $Q_{p\_borehole\_hudson\_lambda}$

즉, 수평시추공에서 얻은 절리 교차 개수로  $\lambda$ 를 계산합니다.

text  
 $\lambda_{bh} = N_{bh} / L_{bh}$

그리고:

text  
 $RQD_{bh\_hudson} = 100 \times \exp(-0.1\lambda_{bh}) \times (0.1\lambda_{bh} + 1)$

그 다음:

text  
 $Q'_{bh\_hudson} = (RQD_{bh\_hudson} / J_{n\_bh}) \times (J_{r\_bh} / J_{a\_bh})$

## 현실성

이 방법도 현실성이 높습니다.

text  
관측 대상: 굴착 전 수평시추공

관측 가능성: 가능

목적: 직접 RQD 대신  $\lambda$  기반 이론 RQD 사용

### Method 1과의 차이

Method 1은 실제 코어 조각 길이로 RQD를 계산합니다.

Method 2는 절리 개수 또는 선형빈도만으로 이론 RQD를 계산합니다.

즉:

text

Method 1:

절리 위치와 intact segment 길이 필요

Method 2:

절리 개수 또는 선형빈도  $\lambda$ 만 필요

그래서 Method 2는 더 단순하지만, segment 길이 분포 정보를 잃습니다.

---

## 4. Method 3: Face scanline method

정의

굴착 후 노출된 굴진면 위에서, 중앙부를 수평으로 가로지르는 하나의 scanline만 사용합니다.

좌표계가 다음과 같다면:

text

x = 터널 축방향

y = 굴진면 수평방향

z = 굴진면 수직방향

face scanline은 다음처럼 정의합니다.

text

x = face\_x 고정

z = 0 고정

y = -R 에서 +R

즉:

python

```
origin = [face_x, -R, 0]
```

```
direction = [0, 1, 0]
length = 2R
```

### 매우 중요한 조건

이 방법에서는 다음을 하면 안 됩니다.

```
text
x방향 scanline 사용 금지
여러 방향 scanline 평균 금지
face 내부를 관통하는 scanline 금지
```

즉, 기존 코드의 이런 방식은 폐기 또는 legacy 처리해야 합니다.

```
python
scan_dirs = [
 [1, 0, 0],
 [0, 1, 0],
 [0, 0, 1],
 [0, 1, 1],
 [0, 1, -1],
]
```

최종 face scanline은 오직 하나입니다.

```
python
scan_dir = [0, 1, 0]
```

### RQD 계산

face scanline 위에서 절리 trace와 교차하는 위치를 구하고, 직접법 RQD를 계산합니다.

```
text
RQD_face_scanline = $\Sigma(\text{length of intact segments} \geq 0.1 \text{ m}) / \text{scanline length} \times 100$
```

### Q' 계산

```
text
Q'_face_scanline = (RQD_face_scanline / Jn_face) * (Jr_face / Ja_face)
```

### 현실성

이 방법은 현장 설명이 매우 좋습니다.

```
text
관측 대상: 굴착 후 노출된 굴진면
관측 가능성: 높음
소요 시간: 비교적 작음
현장성: 좋음
```

논문이나 보고서에서는 이렇게 설명할 수 있습니다.

굴진면 중앙부에 수평 scanline을 설정하고, 이 선분과 교차하는 절리의 위치를 이용해 RQD를 산정하였다.

이건 방어 가능합니다.

---

## 5. Method 4: Jv method

### 정의

Jv는 체적절리계수 또는 volumetric joint count입니다.

Palmström 방식으로 RQD를 추정합니다.

```
text
RQD_Jv = 115 - 3.3 Jv
```

그리고 0~100 범위로 제한합니다.

```
text
RQD_Jv = clip(115 - 3.3 Jv, 0, 100)
```

### Q' 계산

```
text
Q'_face_jv = (RQD_face_jv / Jn_face) * (Jr_face / Ja_face)
```

---

## 6. Jv를 어떻게 정의할 것인가?

여기가 가장 중요합니다.

Jv는 현실성과 코드 구현을 모두 고려해서 단계적으로 정의하는 것이 좋습니다.

---

### 6-1. 1차 구현: joint set 기반 global Jv

가장 단순하고 안정적인 방식입니다.

절리군별 평균 간격이 있다면:

```
text
Jv = Σ 1 / S_i
```

여기서:

text

```
S_i = i번째 절리군의 평균 간격
```

입니다.

하지만 현재 연구에서는 `density_type == "P32"`를 주로 쓰고 있으므로, `mean_spacing`은 실제 DFN 생성에 사용되지 않는 경우가 많습니다.

이 경우에는 다음 proxy를 사용할 수 있습니다.

text

```
Jv_proxy = $\sum P32_i$
```

왜냐하면 평행 절리군의 단순한 경우에는 P32가 대략 1/S와 같은 차원적 의미를 갖기 때문입니다.

다만 이 경우 명칭은 조심해야 합니다.

text

```
Jv
```

라고 단정하기보다는:

text

```
P32-derived Jv proxy
```

또는

text

```
Jv_proxy
```

라고 부르는 것이 좋습니다.

---

## 6-2. 2차 구현: face 주변 local Jv proxy

나중에는 face 주변에 작은 control volume을 잡고, 그 안의 절리 밀도로 local Jv를 계산할 수 있습니다.

예:

text

```
x = face_x $\pm \Delta x/2$
y, z = 터널 단면 범위
```

그 안에 존재하거나 교차하는 절리 disk의 밀도를 이용해:

text

```
Jv_local_proxy ≈ local P32_total
```

을 계산합니다.

이 방식은 face 위치별 변화가 반영되므로 연구적으로 더 좋습니다.

하지만 구현은 더 어렵습니다.

그래서 추천 순서는 다음입니다.

text

```
1단계: global Jv_proxy = $\sum P32_i$
```

```
2단계: local Jv_proxy = face 주변 control volume 기반
```

## 7. 네 방법을 하나의 표로 정리

| 방법              | 산정값                | 관측 위치                        | 방향              | RQD 계산                    | 현실성  | 역할            |
|-----------------|--------------------|------------------------------|-----------------|---------------------------|------|---------------|
| Borehole direct | Qp_borehole_direct | 굴착 전 수평시추공                   | x방향 허용          | Deere 직접법                 | 높음   | 사전 예측         |
| Hudson lambda   | Qp_borehole_hudson | 굴착 전 수평시추공                   | x방향 허용          | Priest-Hudson $\lambda$ 식 | 높음   | 사전 예측         |
| Face scanline   | Qp_face_scanline   | 굴착 후 굴진면                     | y방향만            | 중앙 수평 scanline 직접법        | 높음   | 실제 face 기준    |
| Jv              | Qp_face_jv         | face mapping 또는 joint set 정보 | 방향 비의존 또는 체적 기반 | Palmström형 Jv식            | 중~높음 | face 기준/보조 기준 |

여기서 핵심은 이겁니다.

text

x방향은 borehole에서는 허용된다.

x방향은 face scanline에서는 금지된다.

## 8. 비교 구조 제안

네 방법을 그냥 나열해서 비교하기보다는, 다음 세 가지 비교를 하면 좋습니다.

### 비교 1. Borehole direct vs Face scanline

text

```
Qp_borehole_direct → Qp_face_scanline
```

이게 가장 핵심입니다.



질문:

text

굴착 전 수평시추공에서 직접 계산한 Q'가  
굴착 후 실제 굴진면 중앙 수평 scanline Q'를 예측하는가?

---

### 비교 2. Borehole Hudson vs Face scanline

text

Qp\_borehole\_hudson → Qp\_face\_scanline

질문:

text

시추공에서 직접 RQD를 계산하지 않고,  
 $\lambda$  기반 이론 RQD만 사용해도 face scanline Q'를 예측할 수 있는가?

이 비교는 Method 1과 Method 2의 실무적 차이를 보여줍니다.

---

### 비교 3. Borehole direct vs Face Jv

text

Qp\_borehole\_direct → Qp\_face\_jv

질문:

text

수평시추공 Q'가 face의 체적 절리 발달 정도 기반 Q'와 대응하는가?

---

### 비교 4. Borehole Hudson vs Face Jv

text

Qp\_borehole\_hudson → Qp\_face\_jv

질문:

text

단순  $\lambda$  기반 borehole 방법이 Jv 기반 face condition도 예측할 수 있는가?

---

## 9. decision test에서 사용할 target

Decision usefulness 관점에서는 face 기준을 하나만 고정하지 말고, 여러 기준으로 반복 평가하는 게 좋습니다.

추천 face target은 세 개입니다.

text

```
1. Qp_face_scanline
2. Qp_face_jv
3. Qp_face_conservative
```

여기서 conservative target은 다음입니다.

text

```
Qp_face_conservative = min(Qp_face_scanline, Qp_face_jv)
```

이건 굴착 위험 판단에 유용합니다.

왜냐하면 face scanline은 괜찮게 나오지만 Jv가 나쁘거나, 반대로 Jv는 괜찮지만 중앙 scanline이 나쁘게 나오는 경우가 있을 수 있기 때문입니다.

즉, decision test에서는 이렇게 비교할 수 있습니다.

text

Predictor:

- Qp\_borehole\_direct
- Qp\_borehole\_hudson

Target:

- Qp\_face\_scanline
- Qp\_face\_jv
- Qp\_face\_conservative

총 6개 조합입니다.

text

```
1. BH direct → face scanline
2. BH direct → face Jv
3. BH direct → face conservative
4. BH Hudson → face scanline
5. BH Hudson → face Jv
6. BH Hudson → face conservative
```

이렇게 하면 연구 결과가 훨씬 풍부해집니다.

---

## 10. 공정한 비교를 위한 중요한 원칙

네 방법을 비교할 때 주의해야 할 점이 있습니다.

Q'는 RQD만으로 결정되지 않습니다.

text

$$Q' = (RQD / J_n) \times (J_r / J_a)$$

따라서 방법별 차이가 RQD 때문인지,  $J_n/J_r/J_a$  때문인지 섞일 수 있습니다.

그래서 분석을 두 단계로 나누는 것이 좋습니다.

---

#### 단계 A. RQD-only comparison

먼저 RQD만 비교합니다.

text

```
RQD_borehole_direct
RQD_borehole_hudson
RQD_face_scanline
RQD_face_jv
```

여기서는 각 방법의 RQD 산정 방식 차이만 봅니다.

---

#### 단계 B. $Q'$ comparison

그 다음  $Q'$ 를 비교합니다.

text

```
Qp_borehole_direct
Qp_borehole_hudson
Qp_face_scanline
Qp_face_jv
```

이때는  $J_n, J_r, J_a$ 가 들어가므로 실제 decision value를 평가할 수 있습니다.

---

### 11. 컬럼 이름 제안

코드와 CSV에서는 이름을 명확하게 해야 합니다.

추천 컬럼은 다음입니다.

#### Borehole direct

text

```
RQD_borehole_direct
Qp_borehole_direct
Q_borehole_direct
```

```
borehole_n_intersections
borehole_fracture_frequency
```

### **Borehole Hudson**

```
text
lambda_borehole
RQD_borehole_hudson
Qp_borehole_hudson
Q_borehole_hudson
```

### **Face scanline**

```
text
RQD_face_scanline
Qp_face_scanline
Q_face_scanline
face_scanline_n_intersections
face_scanline_fracture_frequency
```

### **Face Jv**

```
text
Jv_face
Jv_method
RQD_face_jv
Qp_face_jv
Q_face_jv
```

### **Conservative face**

```
text
RQD_face_conservative
Qp_face_conservative
Q_face_conservative
```

그리고 기존 호환성을 위해:

```
text
Qp_borehole_mean = Qp_borehole_direct
Qp_face_mean = Qp_face_scanline
```

처럼 둘 수 있습니다.

다만 내부 주석에 반드시 남겨야 합니다.

```
text
Qp_face_mean now refers to single horizontal face scanline Q', not multidirectional
face average.
```

---

## **12. 현실성 기준으로 본 최종 정의**

최종적으로 네 방법을 이렇게 정의하면 현실적으로 방어 가능합니다.

### 1. 시추공 직접법

text

굴착 전 수평시추공에서 실제 절리 교차 위치를 이용해 Deere 직접법 RQD를 계산한다.

현실성:

text

높음.

실제 사전조사에서 가능.

### 2. Hudson $\lambda$ 법

text

굴착 전 수평시추공에서 관측한 절리 선형빈도  $\lambda$ 를 Priest-Hudson 식에 대입하여 RQD를 계산한다.

현실성:

text

높음.

코어 조각 길이 정보가 불완전할 때도 절리 개수 기반으로 적용 가능.

주의:

text

face에서 x방향  $\lambda$ 를 계산하면 안 됨.

### 3. Face scanline법

text

굴착 후 노출된 굴진면 중앙부를 수평으로 가로지르는 하나의 scanline에서 절리 교차 위치를 기록하고, Deere 직접법 RQD를 계산한다.

현실성:

text

높음.

현장 시간 제약을 고려한 간단한 face mapping 방식.

주의:

text

여러 방향 평균 금지.

x방향 scanline 금지.

---

#### 4. Jv법

text

절리군별 간격 또는 P32 기반 proxy로 Jv를 계산하고, Palmström형 식  $RQD = 115 - 3.3Jv$ 를 이용해 RQD를 추정한다.

현실성:

text

중간~높음.

단, Jv 산정 방법을 명확히 밝혀야 함.

주의:

text

P32 기반이면 Jv가 아니라 Jv\_proxy라고 부르는 것이 안전함.

---

#### 13. 제가 추천하는 최종 플랜

코드 수정 전에 방법론을 이렇게 고정하는 게 좋습니다.

text

Method 1:

- Qp\_borehole\_direct
- source: pre-excavation horizontal borehole
- RQD: Deere direct

Method 2:

- Qp\_borehole\_hudson
- source: same borehole
- $\lambda = N_{\text{intersections}} / \text{borehole interval length}$
- RQD: Priest-Hudson

Method 3:

- Qp\_face\_scanline
- source: exposed tunnel face
- scanline: single horizontal line through face center
- direction: [0, 1, 0]
- RQD: Deere direct

Method 4:

- Qp\_face\_jv
- source: joint set density or face/local DFN proxy
- $Jv = \sum 1/S_i$  or  $\sum P32_i$  proxy
- RQD:  $115 - 3.3Jv$

그리고 분석은 다음 순서로 합니다.

text

1. RQD 네 방법 비교
  2. Q' 네 방법 비교
  3. Borehole direct/Hudson이 face scanline/Jv를 얼마나 예측하는지 평가
  4. threshold decision test
  5. false-safe, false-alarm, expected cost 비교
- 

## 14. 결론

네 가지 방법을 비교하는 것은 가능합니다.

다만 반드시 이렇게 정의해야 합니다.

text

시추공 direct:

굴착 전 수평시추공에서 가능

Hudson  $\lambda$ :

시추공 또는 실제 face scanline에서 얻은  $\lambda$ 만 사용 가능  
face x방향  $\lambda$ 는 금지

Scanline:

굴착면 위 중앙 수평선 하나만 사용

Jv:

절리군 spacing 또는 P32 기반 proxy로 계산하되, 산정 근거를 명확히 표시

이렇게 하면 각 방법의 현실성이 살아납니다.

가장 좋은 최종 비교 구도는 다음입니다.

text

Predictors:

Qp\_borehole\_direct  
Qp\_borehole\_hudson

Face references:

Qp\_face\_scanline  
Qp\_face\_jv  
Qp\_face\_conservative

이 구조로 가면 연구 질문이 매우 명확해집니다.

굴착 전 수평시추공 기반 Q' 산정법은, 실제 굴진면에서 현실적으로 측정 가능한 scanline 및 Jv 기반 face condition을 조건부로 예측할 수 있는가?

GPT-5.5

전반적으로 매우 좋은데 일단 RQD 부터 검증하고 싶어. 연구질문은 시추공 RQD와 굴진면 RQD가 차이가 발생할 수밖에 없나? 그리고 그 원인은 무엇인가? 그렇다면 암반을 평가하는데 무엇이 더 옳다고 할 수 있는가? 차이를 줄일수 있는 방법이 있을까? 등등 할 수 있는 질문이 매우 많은데 그중에서 결국은 시추공을 통해서 전방 지반을 예측하는 것이 얼마나 신뢰성을 줄 수 있는가에 대한 대답을 하는데 도움이 되는 질문과 해답을 찾아 가는 과정이어야 하는 것이라고 할 수 있을 것 같아.



좋습니다. 그러면 지금 단계에서는 **Q'까지 가지 말고 RQD만 먼저 분리해서 검증하는 것**이 맞습니다. 오히려 연구적으로는 이 순서가 더 좋습니다.

왜냐하면 Q'는 다음 요소들이 섞입니다.

text

$$Q' = (RQD / J_n) \times (J_r / J_a)$$

여기서 시추공과 굴진면 Q'가 다르다고 해도, 그 차이가 RQD 때문인지,  $J_n$  때문인지,  $J_r/J_a$  때문인지 바로 알기 어렵습니다.

따라서 지금 연구의 1단계 질문은 이렇게 잡는 게 좋습니다.

수평시추공에서 얻은 RQD는 실제 굴진면에서 관찰되는 RQD와 얼마나 다르며, 그 차이는 어떤 조건에서 커지는가?

이게 먼저 정리되어야 이후에 Q' decision usefulness로 자연스럽게 넘어갈 수 있습니다.

---

## 1. 연구 흐름을 RQD 중심으로 재정리

전체 연구의 큰 질문은 그대로 유지합니다.

text

수평시추공을 통해 전방 암반 상태를 신뢰성 있게 예측할 수 있는가?

다만 이를 바로 Q' decision으로 가지 말고, 다음처럼 계층화하는 것이 좋습니다.

text

Level 1. RQD validation

시추공 RQD와 굴진면 RQD는 얼마나 다른가?

Level 2. 원인 분석



차이는 방향성, 절리 밀도, sampling geometry, 절리군 구조 때문에 발생하는가?

Level 3. 보정 가능성

차이를 줄이는 방법이 있는가?

예: Hudson  $\lambda$ , orientation correction, Jv proxy, calibration model

Level 4. Q' 및 decision usefulness

RQD 차이가 Q' 차이와 굴착 판단 오류로 이어지는가?

지금은 **Level 1~3**에 집중하는 것이 좋습니다.

---

## 2. 핵심 연구질문 후보

RQD 중심으로 보면 좋은 연구질문이 꽤 많이 나옵니다.

그중에서 최종 목적, 즉 “시추공으로 전방 지반을 얼마나 신뢰할 수 있는가”에 직접 연결되는 질문은 아래와 같습니다.

---

**질문 1. 시추공 RQD와 굴진면 RQD는 본질적으로 다를 수밖에 없는가?**

이 질문이 가장 기본입니다.

시추공 RQD는 터널 축방향입니다.

text

RQD\_borehole: x-direction

반면 현실적인 굴진면 scanline RQD는 노출된 face 위의 수평선입니다.

text

RQD\_face\_scanline: y-direction at  $x = \text{face\_x}$ ,  $z = 0$

즉 둘은 애초에 같은 방향의 측정이 아닙니다.

따라서 차이가 발생할 수밖에 없는 이유가 있습니다.

text

1. sampling direction이 다르다.
2. borehole은 1D line sample이다.
3. face scanline도 1D line sample이지만, 다른 방향의 1D sample이다.
4. 절리군의 방향성이 있으면 두 sample이 만나는 절리 수가 달라진다.
5. 절리 disk의 유한 크기와 공간적 불균질성 때문에 같은 face 위치라도 borehole과 face scanline이 보는 절리가 다를 수 있다.

즉, 첫 번째 가설은 이렇게 둘 수 있습니다.

text

H1-RQD:

시추공 RQD와 굴진면 RQD는 일반적으로 동일하지 않으며,  
그 차이는 특히 절리 방향성이 강할수록 커진다.

질문 2. 어떤 조건에서 시추공 RQD가 굴진면 RQD를 과대평가하는가?

이것은 false-safe와 직접 연결됩니다.

text

$RQD_{borehole} > RQD_{face}$

이면 시추공은 암반이 양호하다고 보지만, 실제 굴진면은 더 불량할 수 있습니다.

이것은 위험한 방향의 오류입니다.

가설은 이렇게 세울 수 있습니다.

text

H2-RQD:

시추공 방향  $x$ 와 절리 법선이 이루는 각도 때문에,  
 $borehole$ 이 절리를 적게 만나는 조건에서는  
 $RQD_{borehole}$ 이  $RQD_{face}$ 보다 체계적으로 높아질 수 있다.

즉, 원인은 orientation bias입니다.

절리 법선  $n$ 과 시추공 방향  $d_{bh} = [1, 0, 0]$ 에 대해:

text

$|n \cdot d_{bh}|$ 가 작으면  $borehole$ 은 절리를 잘 만나지 못한다.

반면 face scanline 방향  $d_{face} = [0, 1, 0]$ 에 대해서:

text

$|n \cdot d_{face}|$ 가 크면 face scanline은 절리를 잘 만난다.

그러면 다음 상황이 생깁니다.

text

$borehole$ 은 절리를 적게 만남  $\rightarrow RQD_{borehole}$  높음  
face scanline은 절리를 많이 만남  $\rightarrow RQD_{face}$  낮음

이게 RQD 기준의 false-safe 구조입니다.

---

질문 3. 어떤 조건에서 시추공 RQD가 굴진면 RQD를 과소평가하는가?

반대도 중요합니다.

```
text
RQD_borehole < RQD_face
```

이면 시추공은 암반이 불량하다고 판단하지만, 실제 face는 양호할 수 있습니다.

이것은 false-alarm입니다.

가설은 이렇게 됩니다.

```
text
H3-RQD:
시추공 방향이 특정 절리군을 많이 교차하지만,
굴진면 중앙 수평 scanline은 그 절리군을 적게 교차하는 경우,
RQD_borehole은 RQD_face보다 낮아질 수 있다.
```

이 경우는 시공 보수성 측면에서는 안전하지만, 불필요한 보강 또는 굴착 지연을 유발할 수 있습니다.

---

질문 4. RQD 차이는 절리 밀도 때문에 생기는가, 방향성 때문에 생기는가?

이게 원인 분석의 핵심입니다.

가능한 원인은 크게 네 가지입니다.

```
text
1. 절리 밀도 effect
2. 절리 방향성 effect
3. 절리 크기/유한 disk effect
4. sampling length/window effect
```

각각을 분리해야 합니다.

**절리 밀도 effect**

절리 밀도가 높으면 두 방법 모두 RQD가 낮아질 가능성이 큼니다.

그러나 밀도가 아주 높으면 방향성 차이가 상대적으로 줄어들 수도 있습니다.

왜냐하면 어느 방향으로 재도 절리를 많이 만나기 때문입니다.

반대로 sparse 조건에서는 우연성과 방향성 영향이 커질 수 있습니다.

text

H4-RQD:

sparse DFN에서는 borehole과 face scanline의 RQD 차이와 분산이 커진다.

### 방향성 effect

절리군이 강하게 정렬되어 있으면 scanline 방향에 따라 교차 빈도가 크게 달라집니다.

text

H5-RQD:

fisher\_kappa가 높을수록 RQD\_borehole과 RQD\_face의 차이가 커진다.

### 유한 disk size effect

절리가 무한 평면이면 방향성만으로 교차 여부가 결정됩니다.

하지만 DFN에서는 disk 크기가 유한합니다.

따라서 borehole과 face scanline이 서로 가까운 위치에 있어도 같은 절리 disk를 항상 만나지 않습니다.

text

H6-RQD:

절리 크기가 작고 밀도가 낮을수록,

borehole과 face scanline의 spatial representativeness가 낮아진다.

### sampling length effect

시추공 RQD window 길이와 face scanline 길이가 다르면 RQD 분산이 달라질 수 있습니다.

text

H7-RQD:

짧은 sampling length에서는 RQD의 불확실성이 커지고,

borehole-face 차이도 커진다.

따라서 face scanline 길이와 borehole window 길이를 가능하면 comparable하게 정의하는 것이 중요합니다.

## 3. “무엇이 더 옳은가?”에 대한 답변 방향

이 질문은 매우 중요합니다.

시추공 RQD와 굴진면 RQD가 다르면, 무엇이 더 옳은가?

여기서 조심해야 합니다.

둘 중 하나가 절대적으로 “더 옳다”고 말하면 위험합니다.

대신 이렇게 말하는 것이 좋습니다.

text

각 RQD는 서로 다른 관측 목적과 sampling geometry를 가진다.  
따라서 어느 하나가 절대적으로 더 옳다기보다,  
평가하려는 대상에 따라 reference가 달라진다.

구체적으로는 다음입니다.

---

### 시추공 RQD가 더 적합한 경우

시추공 RQD는 다음 목적에 적합합니다.

text

굴착 전 전방 암반에 대한 사전 예측  
터널 축방향으로 연속적인 암반 변화 탐지  
pre-excavation risk screening

즉, 시추공 RQD는 **사전 정보**로서 가치가 있습니다.

하지만 이것은 “실제 굴진면 상태”의 완전한 관측값은 아닙니다.

---

### 굴진면 RQD가 더 적합한 경우

굴진면 scanline RQD는 다음 목적에 적합합니다.

text

굴착 후 노출된 face condition 평가  
실제 시공 중 support decision  
face mapping record

즉, 굴진면 RQD는 **사후 또는 현장 기준 reference condition**으로 더 적합합니다.

하지만 이것도 face 전체를 완전히 대표하는 것은 아닙니다.

특히 중앙 수평 scanline 하나만 쓰면 국부성 문제가 있습니다.

---

따라서 최종 표현은 이렇게 하는 것이 좋습니다

text

본 연구에서는 굴진면 RQD를 “절대적 진실”로 간주하지 않고,  
현장에서 실제 관찰 가능한 face-based reference condition으로 정의한다.  
시추공 RQD는 굴착 전 predictor로 정의한다.  
따라서 연구의 목적은 두 값의 1:1 동일성을 증명하는 것이 아니라,  
시추공 RQD가 face-based reference condition을 얼마나 신뢰성 있게 예측하는지 평가하는 것이다.

이 문장이 연구 전체의 중심이 될 수 있습니다.

---

#### 4. 차이를 줄일 수 있는 방법

이것도 연구에서 매우 중요한 파트가 됩니다.

RQD\_borehole과 RQD\_face가 다르다면, 그냥 “다르다”에서 끝나면 안 됩니다.

다음 질문으로 가야 합니다.

text

그 차이를 줄이거나 보정할 수 있는가?

가능한 방법은 여러 가지입니다.

---

#### 방법 1. Hudson $\lambda$ 기반으로 단순화

시추공 직접 RQD는 절리 간 spacing distribution에 민감합니다.

Hudson  $\lambda$ 는 절리 개수, 즉 평균 빈도로 RQD를 추정합니다.

따라서 다음 비교가 가능합니다.

text

RQD\_borehole\_direct vs RQD\_face\_scanline  
RQD\_borehole\_hudson vs RQD\_face\_scanline

만약 Hudson 방법이 direct method보다 face와 더 안정적으로 맞는다면:

text

절리 위치의 국부적 우연성보다 평균 선형빈도  $\lambda$ 가 더 robust한 predictor일 수 있다.

반대로 Hudson이 더 나쁘면:

text

절리 간격 분포 정보가 face 예측에 중요하다.

---

## 방법 2. orientation correction

시추공과 face scanline의 방향 차이를 보정할 수 있습니다.

기본 아이디어는 절리 법선과 scanline 방향의 내적입니다.

text

$$\lambda_{app}(d) \propto \sum \text{density}_i \times |n_i \cdot d|$$

따라서 borehole 방향에서 관측한  $\lambda$ 를 face scanline 방향으로 변환할 수 있는지 보는 겁니다.

개념적으로:

text

$$\lambda_{face\_pred} = \lambda_{bh} \times \text{orientation\_factor}$$

여기서:

text

$$\text{orientation\_factor} = \frac{\sum w_i |n_i \cdot d_{face}|}{\sum w_i |n_i \cdot d_{bh}|}$$

이 방식은 꽤 중요합니다.

왜냐하면 시추공 RQD와 굴진면 RQD의 차이가 단순 noise가 아니라 **방향성에 의한 구조적 차이**라면, orientation correction으로 일부 줄일 수 있어야 하기 때문입니다.

가설:

text

H8-RQD:

절리군 방향 정보를 이용해 borehole  $\lambda$ 를 face scanline 방향으로 보정하면, RQD\_face\_scanline 예측 오차가 감소한다.

이건 연구적으로 아주 좋은 질문입니다.

---

## 방법 3. 여러 borehole 또는 offset borehole

현실적으로 하나의 수평시추공은 1D sample이라 대표성이 낮을 수 있습니다.

따라서 차이를 줄이는 가장 직접적인 방법은 sampling을 늘리는 것입니다.

text

1개 borehole → 2개 borehole → 3개 borehole

또는 tunnel center가 아닌 crown/sidewall 방향 offset borehole을 추가할 수도 있습니다.

연구 질문:

text

복수 시추공을 사용하면 face RQD 예측의 false-safe가 줄어드는가?

지금 당장 구현하지 않아도, 후속 연구 제안으로 매우 좋습니다.

---

#### 방법 4. borehole window 길이 조정

borehole RQD를 face\_x 주변 어느 구간에서 계산할지가 중요합니다.

예:

text

face\_x ± 0.5 m  
face\_x ± 1.0 m  
face\_x ± 2.0 m  
face\_x ± 5.0 m

너무 짧으면 random noise가 큼니다.

너무 길면 face의 국부 조건을 흐립니다.

따라서 최적 window가 있을 수 있습니다.

가설:

text

H9-RQD:  
borehole window length는 face RQD 예측 성능에 영향을 미치며,  
너무 짧거나 긴 window는 예측력을 저하시킨다.

---

#### 방법 5. Jv 또는 P32 기반 보조 지표 사용

Jv는 scanline direction에 덜 민감한 체적성 지표입니다.

따라서 다음을 볼 수 있습니다.

text

RQD\_borehole\_direct



```
RQD_borehole_hudson
RQD_face_scanline
RQD_face_jv
```

만약 borehole RQD와 face scanline이 크게 다르지만, borehole-derived  $\lambda$  또는 P32 proxy가 Jv와 더 잘 맞는다면:

text

시추공은 특정 face scanline보다는 전체 절리 발달 정도를 더 잘 반영할 수 있다.

이 해석도 중요합니다.

---

## 5. RQD만 가지고 먼저 비교할 4가지 방법

사용자가 말한 네 방법을 RQD 중심으로 재정리하면 다음입니다.

---

### Method 1. 시추공 직접 RQD

text

```
RQD_BH_direct
```

정의:

text

굴착 전 수평시추공 x방향에서 절리 교차 위치를 이용해 Deere 직접법으로 계산한 RQD

역할:

text

```
pre-excavation predictor
```

---

### Method 2. Hudson $\lambda$ RQD

text

```
RQD_BH_Hudson
```

정의:

text

같은 수평시추공에서 절리 선형빈도  $\lambda_{BH} = N/L$ 를 구하고,  
Priest-Hudson 식으로 계산한 RQD

역할:

text

simplified pre-excavation predictor

중요:

text

face x방향 Hudson은 사용하지 않음.

### Method 3. 굴진면 단일 수평 scanline RQD

text

RQD\_face\_scanline

정의:

text

굴착 후 노출된 굴진면 중앙부의 수평 y방향 선분에서 절리 교차 위치를 이용해 직접법으로 계산한 RQD

역할:

text

field-realistic face reference

### Method 4. Jv 기반 RQD

text

RQD\_face\_Jv

정의:

text

절리군 간격 또는 P32-derived Jv proxy를 이용해 Palmström형 식으로 계산한 RQD

역할:

text

volumetric or joint-set-based reference

## 6. RQD 단계에서의 분석 지표

Q' 이전에는 다음 지표를 보면 됩니다.

### 6-1. 기본 차이

text

```
diff = RQD_BH_direct - RQD_face_scanline
abs_diff = |RQD_BH_direct - RQD_face_scanline|
```

해석:

```
text
diff > 0: borehole이 face보다 좋게 봄
diff < 0: borehole이 face보다 나쁘게 봄
```

---

## 6-2. 상대 오차

RQD는 0~100이므로 log ratio는 Q'보다 덜 적합할 수 있습니다.  
대신 다음이 좋습니다.

```
text
MAE = mean(|BH - face|)
RMSE = sqrt(mean((BH - face)^2))
bias = mean(BH - face)
```

또는 threshold classification도 가능합니다.

---

## 6-3. 상관성

```
text
corr(RQD_BH_direct, RQD_face_scanline)
corr(RQD_BH_Hudson, RQD_face_scanline)
corr(RQD_BH_direct, RQD_face_Jv)
```

다만 RQD가 80~100에 몰리면 상관계수는 불안정할 수 있습니다.

그래서 상관만 보면 안 되고 MAE, bias, classification을 같이 봐야 합니다.

---

## 6-4. RQD threshold 기반 decision

RQD 자체도 threshold로 good/bad를 나눌 수 있습니다.

예:

```
text
RQD ≥ 75: good
RQD < 75: poor/fair 이하
```

또는 연구 목적에 맞춰 여러 threshold를 sweep합니다.

중요한 것은 다음입니다.

```
text
false-safe:
 BH RQD는 good인데 face RQD는 bad

false-alarm:
 BH RQD는 bad인데 face RQD는 good
```

이렇게 하면 Q' decision으로 가기 전에 RQD 수준의 decision usefulness를 평가할 수 있습니다.

---

## 7. 원인 분석에 필요한 **explanatory variables**

RQD 차이의 원인을 찾으려면 각 face row마다 다음 설명변수를 저장해야 합니다.

### 방향성 변수

```
text
orientation_bias_bh
orientation_bias_face
orientation_bias_gap
```

예:

```
text
bias_bh = $\sum P32_i \times |n_i \cdot d_{bh}|$
bias_face = $\sum P32_i \times |n_i \cdot d_{face}|$
orientation_bias_gap = bias_bh - bias_face
```

또는 ratio:

```
text
orientation_bias_ratio = bias_bh / bias_face
```

해석:

```
text
bias_bh > bias_face:
 borehole이 절리를 더 많이 만날 조건

bias_bh < bias_face:
 face scanline이 절리를 더 많이 만날 조건
```

이 변수는 RQD 차이의 가장 중요한 predictor가 될 수 있습니다.

---

### 밀도 변수

```
text
P32_total
Jv_proxy
face_fracture_density
borehole_lambda
```

---

### sampling 변수

```
text
borehole_window_length
face_scanline_length
n_intersections_borehole
n_intersections_face
```

---

### DFN 구조 변수

```
text
fisher_kappa_mean
dominant_joint_set_orientation
joint_set_count
size_r_mean or size parameters
```

이런 변수들을 저장하면 “왜 차이가 났는지”를 설명할 수 있습니다.

---

## 8. 첫 번째 RQD 연구가설 세트

지금 단계에서는 아래 정도로 가설을 세우면 좋습니다.

---

### RQD-H1. Borehole-face RQD discrepancy

```
text
H1:
수평시추공 RQD와 굴진면 중앙 수평 scanline RQD는 일반적으로 1:1로 일치하지 않는다.
```

검정:

```
text
diff = RQD_BH_direct - RQD_face_scanline
mean(diff)
MAE
correlation
paired test
```

---

### RQD-H2. Directional bias

text

H2:

RQD 차이의 주요 원인은 시추공 방향과 굴진면 scanline 방향의 차이에 따른 orientation bias이다.

검정:

text

diff 또는 abs\_diff ~ orientation\_bias\_gap

예상:

text

orientation\_bias\_gap이 클수록 BH와 face RQD 차이가 커짐

---

### RQD-H3. Sparse network uncertainty

text

H3:

희소 절리망에서는 시추공 RQD의 face RQD 예측 오차가 커진다.

검정:

text

density quantile별 MAE 비교

sparse vs dense case 비교

---

### RQD-H4. Hudson $\lambda$ robustness

text

H4:

Hudson  $\lambda$  기반 RQD는 직접 시추공 RQD보다 face RQD와 더 안정적으로 대응할 수 있다.

또는 반대로 나올 수도 있습니다.

검정:

text

MAE(RQD\_BH\_Hudson, RQD\_face\_scanline)

vs

MAE(RQD\_BH\_direct, RQD\_face\_scanline)

---

### RQD-H5. Jv as volumetric reference

text

H5:

Jv 기반 RQD는 단일 face scanline RQD보다 변동성이 작고,

전체 절리 발달 정도를 더 안정적으로 반영한다.

검정:

```
text
RQD_face_scanline vs RQD_face_Jv
case별 분산 비교
density별 trend 비교
```

---

#### RQD-H6. Correctability

```
text
H6:
절리군 방향 정보를 이용한 orientation correction은
시추공 RQD와 굴진면 RQD의 차이를 줄인다.
```

검정:

```
text
uncorrected BH Hudson error
vs
orientation-corrected BH Hudson error
```

이 가설은 연구 결과가 잘 나오면 매우 강한 기여가 됩니다.

---

#### 9. “신뢰성을 줄 수 있는가?”에 대한 RQD 단계의 답변 구조

최종적으로 RQD 단계에서 답할 수 있는 말은 다음 형태가 됩니다.

```
text
수평시추공 RQD는 모든 조건에서 굴진면 RQD와 동일하지는 않다.
그러나 특정 조건에서는 예측력이 있다.
특히 절리 방향성이 시추공 방향과 굴진면 scanline 방향 모두에 대해 균형적이거나,
절리 밀도가 충분히 높고 공간적 대표성이 확보되는 경우에는 오차가 작다.
반면 희소하고 방향성이 강한 절리망에서는 borehole RQD가 face RQD를 과대 또는 과소평가할 수 있다.
이 차이는 orientation bias와 sampling length/window 효과로 일부 설명된다.
따라서 수평시추공 RQD는 단독 결정 기준이라기보다,
orientation correction, Hudson λ , Jv proxy와 함께 사용할 때 신뢰성이 향상된다.
```

이게 Q' decision으로 넘어가기 전의 좋은 결론입니다.

---

#### 10. 지금 당장 해야 할 코드/분석 설계

코딩 전에 설계를 이렇게 고정하면 됩니다.

### 저장할 RQD 컬럼

```
text
RQD_borehole_direct
lambda_borehole
RQD_borehole_hudson

RQD_face_scanline
lambda_face_scanline
RQD_face_hudson_optional

Jv_face_proxy
RQD_face_jv

RQD_face_conservative = min(RQD_face_scanline, RQD_face_jv)
```

여기서 `RQD_face_hudson_optional`은 가능은 합니다.

다만 명확히 해야 합니다.

```
text
face 중앙 수평 scanline에서 얻은 $\lambda_{\text{face_scanline}}$ 으로 Hudson RQD 계산
```

이건 현실적으로 가능합니다.

하지만 네 가지 핵심 방법에서는 일단 제외해도 됩니다.

---

### 저장할 원인 분석 컬럼

```
text
orientation_bias_borehole
orientation_bias_face_scanline
orientation_bias_gap
orientation_bias_ratio

P32_total
Jv_proxy
case_name
seed
face_x

borehole_n_intersections
face_scanline_n_intersections

borehole_window_length
face_scanline_length
```

---



## 11. 분석 순서

### Step 1. Method distribution

각 방법의 RQD 분포를 봅니다.

```
text
RQD_BH_direct
RQD_BH_Hudson
RQD_face_scanline
RQD_face_Jv
```

출력:

```
text
min, p10, p25, median, p75, p90, max
```

---

### Step 2. Pairwise comparison

핵심 비교:

```
text
BH_direct vs face_scanline
BH_Hudson vs face_scanline
BH_direct vs face_Jv
BH_Hudson vs face_Jv
face_scanline vs face_Jv
```

지표:

```
text
bias
MAE
RMSE
corr
overestimation_rate
underestimation_rate
```

---

### Step 3. Regime analysis

조건별 분석:

```
text
sparse / medium / dense
favorable / unfavorable
orientation_bias_gap quantile
Jv_proxy quantile
```

---

### Step 4. Threshold decision

RQD threshold별:

```
text
false-safe
false-alarm
balanced accuracy
expected cost
baseline comparison
```

RQD threshold는 예를 들면:

```
text
50, 75, 90
```

또는 data-driven sweep.

---

### Step 5. Correction test

가능하면 다음을 비교합니다.

```
text
BH_direct
BH_Hudson
BH_Hudson_orientation_corrected
```

face target:

```
text
face_scanline
face_Jv
```

---

## 12. 연구 제목을 RQD 단계로 잡으면

지금 단계만 따로 논문/챕터처럼 잡으면 제목은 이런 식이 좋습니다.

```
text
Reliability of Horizontal Borehole RQD for Predicting Tunnel Face RQD in Discrete Fracture Networks
```

또는 한국어로:

```
text
DFN 기반 수평시추공 RQD의 굴진면 RQD 예측 신뢰성 평가
```

핵심은 “Q”보다 먼저 “RQD”를 정리하는 겁니다.

---

### 13. 결론적으로 지금 방향

제가 보기에는 지금 방향을 이렇게 잡으면 가장 좋습니다.

text

1. Q' 분석은 잠시 보류
2. RQD 네 방법을 명확히 정의
3. face scanline은 중앙 수평 단일선으로 제한
4. borehole direct와 borehole Hudson은 pre-excavation predictor로 정의
5. face scanline과 Jv는 post-excavation reference로 정의
6. 차이의 원인을 orientation, density, sparsity, sampling length로 설명
7. 차이를 줄이는 방법으로 Hudson, orientation correction, Jv combination을 테스트
8. 마지막에 Q' decision usefulness로 확장

이렇게 가면 연구 질문이 매우 탄탄해집니다.

가장 중요한 한 문장으로 요약하면 다음입니다.

본 연구의 첫 단계는 시추공 RQD와 굴진면 RQD가 단순히 일치하는지를 보는 것이 아니라, 두 값이 어떤 관측 기하와 절리 구조 조건에서 달라지는지, 그 차이가 예측 가능한 구조적 bias인지, 그리고 보정을 통해 수평시추공 RQD의 전방 지반 예측 신뢰성을 높일 수 있는지를 검증하는 것이다.

GPT-5.5

먼저 rqd\_calculator.py를 순차적으로 수정해야 해. 필요한 파일부터 먼저 작성하고 rqd를 위한 별도의 분석코드를 작성하는게 좋을 것 같은데. 각 방식의 코드를 순서대로 작성해줘. 그리고 어디를 대체한다고 예기해주면 내가 넣어보고 나중에 한꺼번에 붙여넣어서 확인해주면 될 것 같아.



좋습니다. 그러면 1단계는 **rqd\_calculator.py**를 현실적인 4개 RQD 방식의 기반 코드로 정리하는 것입니다.

이번 단계의 목표는 다음입니다.

text

1. 시추공 직접 RQD
2. 시추공 Hudson  $\lambda$  RQD
3. 굴진면 중앙 수평 scanline RQD
4. Jv 기반 RQD

단, 중요한 원칙은 이것입니다.

text

- x방향 RQD는 시추공에서는 허용.
- x방향 RQD는 굴진면 scanline에서는 금지.
- 굴진면 scanline은  $x = \text{face}_x$  평면 위의 중앙 수평선 하나만 사용.
- 기존 multi-direction face RQD는 폐기 또는 legacy 처리.

아래 순서대로 넣으면 됩니다.

---

## 0. 추천 작업 순서

먼저 파일을 이렇게 나누는 것을 추천합니다.

text

수정:

src/core/rqd\_calculator.py

신규 추가:

src/core/rqd\_analysis.py

이번 답변에서는 두 파일 코드를 먼저 줍니다.

아직 tunnel.py, comparison.py 연결은 다음 단계에서 하면 됩니다.

---

## 1. rqd\_calculator.py 수정 방향

현재 파일 전체를 아래 코드로 **통째로 교체**하는 것을 추천합니다.

이유는 기존 calc\_direct, calc\_theoretical, rqd\_along\_borehole는 유지하면서, 문제가 되는 rqd\_at\_face()의 다방향 scanline 구조를 현실적인 단일 수평 scanline 방식으로 바꾸기 때문입니다.

즉, 대체 위치는 다음입니다.

text

src/core/rqd\_calculator.py 전체 내용 대체

---

## 2. 새 rqd\_calculator.py 전체 코드

python

"""

RQD 계산기

현실적인 RQD 비교를 위한 4가지 방법을 지원한다.

1. Borehole direct RQD
  - 굴착 전 수평시추공에서 절리 교차 위치를 이용한 Deere 직접법 RQD
2. Borehole Hudson/Priest-Hudson  $\lambda$  RQD
  - 같은 시추공에서 얻은 선형 절리빈도  $\lambda = N/L$  기반 이론식 RQD
3. Face single horizontal scanline RQD
  - 굴착 후 노출된 굴진면  $x = \text{face}_x$  평면 위에서  
중앙부를 수평으로 가로지르는 하나의  $y$ 방향 scanline만 사용
4. Jv-based RQD
  - Palmström형 RQD =  $115 - 3.3 * J_v$
  - P32 기반일 경우 엄밀한  $J_v$ 라기보다  $J_v$ \_proxy로 취급하는 것이 안전함

중요한 원칙:

- face RQD에서는  $x$ 방향, 즉 굴진 방향 scanline을 사용하지 않는다.
  - face RQD에서는 multi-direction scanline 평균을 사용하지 않는다.
  - $x$ 방향은 실제 수평시추공을 의미할 때만 허용한다.
- """

```
import numpy as np
from typing import Any, Dict, List, Optional, Tuple
```

```
class RQDCalculator:
 """RQD 계산 정적 메서드 모음"""

 @staticmethod
 def _clean_intersection_distances(
 intersection_distances: List[float],
 total_length: float,
 tol: float = 1e-8,
) -> List[float]:
 """
 scanline 구간 내부의 절리 교차 위치를 정리한다.

 - 0과 total_length 경계점은 제외한다.
 - NaN, inf는 제외한다.
 - 거의 같은 위치의 중복 교차점은 하나로 병합한다.
 """
 if total_length <= 0:
 return []

 vals = []
 for t in intersection_distances:
 try:
 tf = float(t)
 except Exception:
 continue
```

```

 if not np.isfinite(tf):
 continue

 if tol < tf < total_length - tol:
 vals.append(tf)

 vals = sorted(vals)

 unique_vals = []
 for v in vals:
 if len(unique_vals) == 0:
 unique_vals.append(v)
 elif abs(v - unique_vals[-1]) > tol:
 unique_vals.append(v)

 return unique_vals

@staticmethod
def calc_direct(
 intersection_distances: List[float],
 total_length: float,
 threshold: float = 0.1,
) -> float:
 """
 Deere 직접 정의:

$$RQD = \frac{\sum(\text{길이} \geq \text{threshold인 intact segment})}{\text{total_length}} \times 100$$

 Parameters

 intersection_distances:
 scanline 시작점으로부터 절리 교차점까지의 거리 목록.
 total_length:
 scanline 또는 시추공 평가 구간 길이.
 threshold:
 RQD에 포함할 최소 intact segment 길이.
 일반적으로 0.1 m.

 Returns

 float
 0~100 범위의 RQD.
 """
 details = RQDCalculator.calc_direct_details(
 intersection_distances=intersection_distances,
 total_length=total_length,
 threshold=threshold,
)
 return details["rqd"]

@staticmethod
def calc_direct_details(

```

```

intersection_distances: List[float],
total_length: float,
threshold: float = 0.1,
) -> Dict[str, Any]:
 """
 Deere 직접법 RQD와 계산 세부 정보를 함께 반환한다.

 Returns

 dict
 {
 "rqd": ...,
 "total_length": ...,
 "threshold": ...,
 "intersection_positions": ...,
 "n_intersections": ...,
 "points": ...,
 "intact_lengths": ...,
 "counted_lengths": ...,
 "sum_counted_length": ...,
 "fracture_frequency": ...
 }
 """
 if total_length <= 0:
 return {
 "rqd": 0.0,
 "total_length": float(total_length),
 "threshold": float(threshold),
 "intersection_positions": [],
 "n_intersections": 0,
 "points": [],
 "intact_lengths": [],
 "counted_lengths": [],
 "sum_counted_length": 0.0,
 "fracture_frequency": np.nan,
 }

 clean_t = RQDCalculator._clean_intersection_distances(
 intersection_distances=intersection_distances,
 total_length=total_length,
)

 points = [0.0] + clean_t + [float(total_length)]

 intact_lengths = []
 counted_lengths = []

 for i in range(len(points) - 1):
 spacing = points[i + 1] - points[i]
 intact_lengths.append(float(spacing))

 if spacing >= threshold:
 counted_lengths.append(float(spacing))

```

```

sum_counted = float(np.sum(counted_lengths)) if counted_lengths else 0.0
rqd = float(np.clip((sum_counted / total_length) * 100.0, 0.0, 100.0))

fracture_frequency = len(clean_t) / total_length

return {
 "rqd": rqd,
 "total_length": float(total_length),
 "threshold": float(threshold),
 "intersection_positions": clean_t,
 "n_intersections": int(len(clean_t)),
 "points": points,
 "intact_lengths": intact_lengths,
 "counted_lengths": counted_lengths,
 "sum_counted_length": sum_counted,
 "fracture_frequency": float(fracture_frequency),
}

@staticmethod
def calc_theoretical(
 linear_frequency: float,
 threshold: float = 0.1,
) -> float:
 """
 Priest-Hudson 이론식:

$$RQD = 100 \times \exp(-\lambda t) \times (\lambda t + 1)$$

 여기서:
 - λ = linear_frequency = 절리 개수 / scanline 길이
 - t = threshold, 일반적으로 0.1 m

 주의:
 이 함수 자체는 방향을 모른다.
 따라서 face에서 사용할 경우 반드시 실제 굴진면 위에서 관찰 가능한
 scanline의 λ 만 넣어야 한다.
 face에서 x방향, 즉 굴진 방향 λ 를 넣으면 안 된다.
 """
 try:
 lam = float(linear_frequency)
 except Exception:
 return np.nan

 if not np.isfinite(lam):
 return np.nan

 lam = max(lam, 0.0)
 lt = lam * threshold

 return float(np.clip(100.0 * np.exp(-lt) * (lt + 1.0), 0.0, 100.0))

@staticmethod

```



```

def calc_hudson_from_lambda(
 linear_frequency: float,
 threshold: float = 0.1,
) -> float:
 """
 Priest-Hudson RQD 계산 alias.

 calc_theoretical()와 동일하지만, 코드 가독성을 위해 별도 이름 제공.
 """
 return RQDCalculator.calc_theoretical(
 linear_frequency=linear_frequency,
 threshold=threshold,
)

@staticmethod
def calc_lambda_from_count(
 n_intersections: int,
 total_length: float,
) -> float:
 """
 선형 절리빈도 lambda 계산.

 $\lambda = N / L$
 """
 if total_length <= 0:
 return np.nan

 return float(max(int(n_intersections), 0) / total_length)

@staticmethod
def calc_apparent_frequency(
 joint_normal: np.ndarray,
 true_spacing: float,
 scanline_direction: np.ndarray,
) -> float:
 """
 겉보기 선형빈도:

 $\lambda_{app} = |\cos \alpha| / \text{spacing}$

 여기서:
 - α = 절리 법선과 scanline 방향 사이의 각도
 - joint_normal = 절리면 법선
 - scanline_direction = 관찰 또는 시추 방향

 이 함수는 orientation bias 분석용으로 사용할 수 있다.
 """
 if true_spacing <= 0:
 return np.nan

 n = np.asarray(joint_normal, dtype=float)
 d = np.asarray(scanline_direction, dtype=float)

```

```

n_norm = np.linalg.norm(n)
d_norm = np.linalg.norm(d)

if n_norm == 0 or d_norm == 0:
 return np.nan

cos_alpha = abs(np.dot(n / n_norm, d / d_norm))
return float(cos_alpha / true_spacing)

@staticmethod
def calc_from_jv(jv: float) -> float:
 """
 Palmström형 Jv 기반 RQD 계산.

 RQD = 115 - 3.3 * Jv

 최종값은 0~100으로 clipping한다.

 주의:
 P32 기반 proxy를 jv로 넣는 경우, 엄밀한 현장 Jv라기보다는
 Jv_proxy 또는 P32-derived Jv proxy로 명명하는 것이 안전하다.
 """
 try:
 jv_f = float(jv)
 except Exception:
 return np.nan

 if not np.isfinite(jv_f):
 return np.nan

 jv_f = max(jv_f, 0.0)
 rqd = 115.0 - 3.3 * jv_f

 return float(np.clip(rqd, 0.0, 100.0))

@staticmethod
def calc_jv_from_spacings(spacings: List[float]) -> float:
 """
 절리군 평균 간격 기반 Jv 계산.

$$Jv = \sum 1 / S_i$$

 Parameters

 spacings:
 각 절리군 평균 간격 목록.

 Returns

 float
 Jv

```

```

"""
total = 0.0

for s in spacings:
 try:
 sf = float(s)
 except Exception:
 continue

 if np.isfinite(sf) and sf > 0:
 total += 1.0 / sf

return float(total)

@staticmethod
def _get_jointset_value(joint_set: Any, key: str, default: Any = None) -> Any:
 """
 joint_set이 dict이든 object이든 동일하게 값을 꺼내기 위한 helper.
 """
 if isinstance(joint_set, dict):
 return joint_set.get(key, default)

 return getattr(joint_set, key, default)

@staticmethod
def calc_jv_proxy_from_joint_sets(
 joint_sets: List[Any],
 prefer_p32: bool = True,
) -> Dict[str, Any]:
 """
 Joint set 정의로부터 Jv 또는 Jv_proxy를 계산한다.

 기준:
 - density_type == "P32"이고 P32가 있으면 P32를 contribution으로 사용.
 - 그렇지 않고 mean_spacing이 있으면 1 / mean_spacing 사용.
 - prefer_p32=True이면 density_type과 무관하게 P32가 있으면 우선 사용.

 Returns

 dict
 {
 "jv": ...,
 "method": ...,
 "contributions": [
 {
 "index": ...,
 "source": "P32_proxy" or "spacing",
 "value": ...
 },
 ...
]
 }
 """

```

주의:

P32를 사용한 경우 엄밀한 Palmström 현장 Jv라기보다

P32-derived Jv proxy로 해석하는 것이 안전하다.

"""

```
contributions = []
```

```
jv_total = 0.0
```

```
used_sources = set()
```

```
for idx, js in enumerate(joint_sets):
```

```
 density_type = RQDCalculator._get_jointset_value(js, "density_type", None)
```

```
 p32 = RQDCalculator._get_jointset_value(js, "P32", None)
```

```
 mean_spacing = RQDCalculator._get_jointset_value(js, "mean_spacing", None)
```

```
 p32_val = None
```

```
 spacing_val = None
```

```
 try:
```

```
 if p32 is not None:
```

```
 p32_val = float(p32)
```

```
 except Exception:
```

```
 p32_val = None
```

```
 try:
```

```
 if mean_spacing is not None:
```

```
 spacing_val = float(mean_spacing)
```

```
 except Exception:
```

```
 spacing_val = None
```

```
 use_p32 = False
```

```
> 0: if prefer_p32 and p32_val is not None and np.isfinite(p32_val) and p32_val
```

```
 use_p32 = True
```

```
 elif (
```

```
 str(density_type).upper() == "P32"
```

```
 and p32_val is not None
```

```
 and np.isfinite(p32_val)
```

```
 and p32_val > 0
```

```
):
```

```
 use_p32 = True
```

```
 if use_p32:
```

```
 value = float(p32_val)
```

```
 source = "P32_proxy"
```

```
> 0: elif spacing_val is not None and np.isfinite(spacing_val) and spacing_val
```

```
 value = float(1.0 / spacing_val)
```

```
 source = "spacing"
```

```
 elif p32_val is not None and np.isfinite(p32_val) and p32_val > 0:
```

```
 value = float(p32_val)
```

```
 source = "P32_proxy"
```

```
 else:
```

```

 value = 0.0
 source = "missing"

 jv_total += value
 used_sources.add(source)

 contributions.append(
 {
 "index": idx,
 "source": source,
 "value": value,
 "density_type": density_type,
 "P32": p32_val,
 "mean_spacing": spacing_val,
 }
)

 if used_sources == {"spacing"}:
 method = "spacing_based_Jv"
 elif "P32_proxy" in used_sources:
 method = "P32_derived_Jv_proxy"
 else:
 method = "missing_or_zero"

 return {
 "jv": float(jv_total),
 "method": method,
 "contributions": contributions,
 }

```

```
class DirectionalRQDCalculator:
```

```
 """
```

```
 시추공 및 굴진면 RQD 계산.
```

```
 주요 원칙:
```

- borehole은 실제 수평시추공이므로 x방향 line sampling이 가능하다.
- face scanline은 노출된 굴진면 위에서만 가능하므로 x방향을 사용하지 않는다.
- face scanline은 중앙 수평 y방향 단일 선분만 사용한다.

```
 """
```

```
 def __init__(self, dfn):
 self.dfn = dfn
```

```
 @staticmethod
```

```
 def _normalize_direction(direction: np.ndarray) -> np.ndarray:
 d = np.asarray(direction, dtype=float)
 norm = np.linalg.norm(d)
```

```
 if norm == 0:
 raise ValueError("Direction vector must be non-zero.")
```

```
 return d / norm
```

```

def _get_line_intersection_positions(
 self,
 origin: np.ndarray,
 direction: np.ndarray,
 length: float,
) -> List[float]:
 """
 DFN 절리와 line segment의 교차 위치 t 목록을 반환한다.

 t는 origin으로부터 direction 방향으로 측정한 거리다.
 """
 if length <= 0:
 return []

 origin = np.asarray(origin, dtype=float)
 direction = self._normalize_direction(direction)

 intersections = self.dfn.get_joints_intersecting_line(
 origin,
 direction,
 length,
)

 t_values = []
 for item in intersections:
 try:
 t = item[0]
 except Exception:
 continue

 try:
 tf = float(t)
 except Exception:
 continue

 if np.isfinite(tf) and 0.0 <= tf <= length:
 t_values.append(tf)

 return t_values

def rqd_on_line(
 self,
 origin: np.ndarray,
 direction: np.ndarray,
 length: float,
 threshold: float = 0.1,
 label: str = "line",
) -> Dict[str, Any]:
 """
 임의의 line segment에 대해 직접 RQD와 Hudson RQD를 함께 계산한다.

 이 함수는 내부 공통 함수다.
 """

```

주의:

face RQD 계산에서 이 함수를 쓸 때는 direction이 반드시  
굴진면 위의 방향이어야 한다. 즉 face에서 [1,0,0]은 사용하지 않는다.  
"""

```
if length <= 0:
 return {
 "label": label,
 "origin": np.asarray(origin, dtype=float).tolist(),
 "direction": np.asarray(direction, dtype=float).tolist(),
 "length": float(length),
 "threshold": float(threshold),
 "intersection_positions": [],
 "n_intersections": 0,
 "lambda": np.nan,
 "rqd_direct": 0.0,
 "rqd_hudson": np.nan,
 "direct_details": None,
 }

direction_unit = self._normalize_direction(direction)

t_values = self._get_line_intersection_positions(
 origin=origin,
 direction=direction_unit,
 length=length,
)

direct_details = RQDCalculator.calc_direct_details(
 intersection_distances=t_values,
 total_length=length,
 threshold=threshold,
)

n_intersections = direct_details["n_intersections"]
lam = RQDCalculator.calc_lambda_from_count(
 n_intersections=n_intersections,
 total_length=length,
)

rqd_hudson = RQDCalculator.calc_hudson_from_lambda(
 linear_frequency=lam,
 threshold=threshold,
)

return {
 "label": label,
 "origin": np.asarray(origin, dtype=float).tolist(),
 "direction": direction_unit.tolist(),
 "length": float(length),
 "threshold": float(threshold),
 "intersection_positions": direct_details["intersection_positions"],
 "n_intersections": int(n_intersections),
}
```

```

 "lambda": float(lam),
 "rqd_direct": float(direct_details["rqd"]),
 "rqd_hudson": float(rqd_hudson),
 "direct_details": direct_details,
 }

def rqd_borehole_interval(
 self,
 origin: np.ndarray,
 direction: np.ndarray,
 length: float,
 threshold: float = 0.1,
) -> Dict[str, Any]:
 """
 Method 1 & 2:
 시추공 평가 구간에서 직접 RQD와 Hudson lambda RQD를 계산한다.

 사용 예:
 - origin: 평가 구간 시작점
 - direction: 수평시추공 방향. 일반적으로 [1,0,0]
 - length: 평가 구간 길이

 Returns

 dict
 {
 "RQD_borehole_direct": ...,
 "lambda_borehole": ...,
 "RQD_borehole_hudson": ...,
 ...
 }
 """
 result = self.rqd_on_line(
 origin=origin,
 direction=direction,
 length=length,
 threshold=threshold,
 label="borehole_interval",
)

 return {
 "method": "borehole_direct_and_hudson",
 "origin": result["origin"],
 "direction": result["direction"],
 "length": result["length"],
 "threshold": result["threshold"],
 "intersection_positions": result["intersection_positions"],
 "borehole_n_intersections": result["n_intersections"],
 "lambda_borehole": result["lambda"],
 "RQD_borehole_direct": result["rqd_direct"],
 "RQD_borehole_hudson": result["rqd_hudson"],
 "direct_details": result["direct_details"],
 }

```



```

def rqd_along_borehole(
 self,
 origin: np.ndarray,
 direction: np.ndarray,
 length: float,
 window: float = 1.0,
 step: float = 1.0,
 threshold: float = 0.1,
) -> Tuple[np.ndarray, np.ndarray]:
 """
 기존 코드 호환용:
 이동 윈도우 방식 시추공 직접 RQD를 반환한다.

 Returns

 positions, rqd_values

 주의:
 이 함수는 기존 tuple 반환 구조를 유지한다.
 상세 정보가 필요하면 rqd_borehole_interval()을 사용한다.
 """
 if length <= 0:
 return np.array([]), np.array([])

 if window <= 0:
 raise ValueError("window must be positive.")

 if step <= 0:
 raise ValueError("step must be positive.")

 direction = self._normalize_direction(direction)

 all_ix = self.dfn.get_joints_intersecting_line(origin, direction, length)
 t_values = []

 for item in all_ix:
 try:
 t = float(item[0])
 except Exception:
 continue

 if np.isfinite(t) and 0.0 <= t <= length:
 t_values.append(t)

 t_values = np.array(t_values, dtype=float)

 if window > length:
 positions = np.array([0.0])
 effective_window = length
 else:
 positions = np.arange(0.0, length - window + 1e-9, step)
 effective_window = window

```

```

rqd_values = np.zeros(len(positions), dtype=float)

for idx, pos in enumerate(positions):
 mask = (t_values >= pos) & (t_values < pos + effective_window)
 window_t = (t_values[mask] - pos).tolist()

 rqd_values[idx] = RQDCalculator.calc_direct(
 intersection_distances=window_t,
 total_length=effective_window,
 threshold=threshold,
)

return positions, rqd_values

def rqd_at_face_horizontal_scanline(
 self,
 face_x: float,
 tunnel_radius: Optional[float] = None,
 y_range: Optional[Tuple[float, float]] = None,
 y_center: float = 0.0,
 z_center: float = 0.0,
 threshold: float = 0.1,
) -> Dict[str, Any]:
 """
 Method 3:
 굴진면 중앙 수평 단일 scanline RQD.

 scanline 정의:
 - 굴진면 평면: $x = \text{face_x}$
 - 방향: y방향 $[0, 1, 0]$
 - 위치: $z = z_center$
 - 범위:
 1) y_range 가 주어지면 $y_range[0] \sim y_range[1]$
 2) 아니면 $tunnel_radius$ 를 이용해 $y_center - R \sim y_center + R$

 중요한 제한:
 - x방향 scanline을 사용하지 않는다.
 - 여러 방향 scanline 평균을 사용하지 않는다.
 - 실제 노출된 굴진면 위의 중앙 수평선 하나만 사용한다.

 Returns

 dict
 {
 "RQD_face_scanline": ...,
 "lambda_face_scanline": ...,
 "RQD_face_scanline_hudson": ...,
 "face_scanline_n_intersections": ...,
 ...
 }

```

```

"""
if y_range is None:
 if tunnel_radius is None:
 raise ValueError("Either y_range or tunnel_radius must be provided.")

 y_min = y_center - float(tunnel_radius)
 y_max = y_center + float(tunnel_radius)
else:
 y_min = float(y_range[0])
 y_max = float(y_range[1])

if y_max <= y_min:
 raise ValueError("Invalid y_range. y_max must be greater than y_min.")

origin = np.array([float(face_x), y_min, float(z_center)], dtype=float)
direction = np.array([0.0, 1.0, 0.0], dtype=float)
length = y_max - y_min

result = self.rqd_on_line(
 origin=origin,
 direction=direction,
 length=length,
 threshold=threshold,
 label="face_single_horizontal_scanline",
)

return {
 "method": "face_single_horizontal_scanline",
 "face_x": float(face_x),
 "y_range": (float(y_min), float(y_max)),
 "z_center": float(z_center),
 "origin": result["origin"],
 "direction": result["direction"],
 "length": result["length"],
 "threshold": result["threshold"],
 "intersection_positions": result["intersection_positions"],
 "face_scanline_n_intersections": result["n_intersections"],
 "lambda_face_scanline": result["lambda"],
 "RQD_face_scanline": result["rqd_direct"],
 "RQD_face_scanline_hudson": result["rqd_hudson"],
 "direct_details": result["direct_details"],
}

def rqd_at_face(
 self,
 face_x: float,
 y_range: Tuple[float, float],
 z_range: Tuple[float, float],
 grid_spacing: float = 1.0,
 scan_length: float = 3.0,
 threshold: float = 0.1,
) -> Dict[str, Any]:
 """
 기존 rqd_at_face() 대체용 함수.

```

기존 구현은 다음 문제가 있었다.

- face에서 x방향 scanline 사용
- 여러 방향 scanline 평균 사용
- 현장 굴진면 mapping으로 보기 어려움

따라서 이 함수는 더 이상 multi-direction scanline을 수행하지 않고, 중앙 수평 단일 scanline만 계산한다.

#### Parameters

-----

face\_x:

굴진면 x 좌표.

y\_range:

굴진면에서 수평 scanline의 y 범위.

z\_range:

중앙 z 위치를 계산하기 위해 사용.

$z\_center = (z\_min + z\_max) / 2$

grid\_spacing:

기존 signature 호환용. 사용하지 않음.

scan\_length:

기존 signature 호환용. 사용하지 않음.

threshold:

RQD threshold. 기본 0.1 m.

#### Returns

-----

dict

기존 코드와 최소 호환을 위해 rqd\_mean, rqd\_std 등의 key를 포함하되,  
실제 값은 중앙 수평 단일 scanline 기준이다.

"""

$z\_center = 0.5 * (float(z\_range[0]) + float(z\_range[1]))$

scan = self.rqd\_at\_face\_horizontal\_scanline(

face\_x=face\_x,

y\_range=y\_range,

z\_center=z\_center,

threshold=threshold,

)

rqd\_val = scan["RQD\_face\_scanline"]

return {

"method": "face\_single\_horizontal\_scanline",

"face\_x": float(face\_x),

*# 신규 명확한 key*

"RQD\_face\_scanline": rqd\_val,

"lambda\_face\_scanline": scan["lambda\_face\_scanline"],

"RQD\_face\_scanline\_hudson": scan["RQD\_face\_scanline\_hudson"],

```

 "face_scanline_n_intersections": scan["face_scanline_n_intersections"],
 "intersection_positions": scan["intersection_positions"],
 "scanline_origin": scan["origin"],
 "scanline_direction": scan["direction"],
 "scanline_length": scan["length"],
 "z_center": scan["z_center"],
 "y_range": scan["y_range"],

 # 기존 코드 최소 호환용 key
 # 이제 rqd_mean은 multi-direction 평균이 아니라
 # 단일 중앙 수평 scanline RQD를 의미한다.
 "y_points": np.array([0.5 * (float(y_range[0]) + float(y_range[1]))]),
 "z_points": np.array([z_center]),
 "rqd_map": np.array([[rqd_val]], dtype=float),
 "rqd_by_direction": np.array([[rqd_val]], dtype=float),
 "rqd_mean": float(rqd_val),
 "rqd_std": 0.0,
 }

```

### 3. 위 코드에서 각 방식이 어디에 해당하는지

#### 3-1. Method 1: 시추공 직접 RQD

사용 함수:

```
python
DirectionalRQDCalculator.rqd_borehole_interval()
```

반환 key:

```
python
"RQD_borehole_direct"
```

정의:

```
text
시추공 구간의 절리 교차 위치를 직접 사용해 Deere RQD 계산.
```

#### 3-2. Method 2: 시추공 Hudson lambda RQD

같은 함수에서 같이 반환됩니다.

```
python
DirectionalRQDCalculator.rqd_borehole_interval()
```

반환 key:

```
python
```

```
"lambda_borehole"
"RQD_borehole_hudson"
```

정의:

```
text
lambda_borehole = 시추공 절리 교차 개수 / 시추공 평가 길이
RQD_borehole_hudson = $100 * \exp(-\lambda * 0.1) * (\lambda * 0.1 + 1)$
```

---

### 3-3. Method 3: 굴진면 중앙 수평 scanline RQD

사용 함수:

```
python
DirectionalRQDCalculator.rqd_at_face_horizontal_scanline()
```

반환 key:

```
python
"RQD_face_scanline"
"lambda_face_scanline"
"RQD_face_scanline_hudson"
"face_scanline_n_intersections"
```

정의:

```
text
x = face_x 평면 위에서
z = z_center 고정,
y_min에서 y_max까지 중앙 수평선 하나만 사용.
```

방향은 고정입니다.

```
python
direction = [0, 1, 0]
```

x방향은 사용하지 않습니다.

---

### 3-4. Method 4: Jv 기반 RQD

사용 함수:

```
python
RQDCalculator.calc_from_jv(jv)
```

그리고 joint set에서 Jv proxy를 만들 때는:

```
python
RQDCalculator.calc_jv_proxy_from_joint_sets(joint_sets)
```

반환 예:

```
python
jv_info = RQDCalculator.calc_jv_proxy_from_joint_sets(joint_sets)
jv = jv_info["jv"]
rqd_jv = RQDCalculator.calc_from_jv(jv)
```

정의:

```
text
RQD = 115 - 3.3 * Jv
```

주의:

```
text
P32 기반이면 엄밀한 현장 Jv가 아니라 P32-derived Jv proxy로 해석.
```

---

#### 4. 기존 코드 중 대체되는 핵심 부분

기존 `rqd_at_face()`의 이 부분은 더 이상 사용하면 안 됩니다.

```
python
scan_dirs = [
 np.array([1, 0, 0]),
 np.array([0, 1, 0]),
 np.array([0, 0, 1]),
 np.array([0, 1, 1]) / np.sqrt(2),
 np.array([0, 1, -1]) / np.sqrt(2),
]
```

새 코드에서는 `rqd_at_face()`가 내부적으로 단일 수평 scanline만 사용합니다.

즉, 기존:

```
text
face multi-direction average RQD
```

는 새 코드에서:

```
text
face single horizontal scanline RQD
```

로 대체됩니다.

---

## 5. 별도 RQD 분석 코드 추가

이제 RQD만 비교하기 위한 분석 파일을 하나 추가하는 것이 좋습니다.

신규 파일:

```
text
src/core/rqd_analysis.py
```

아래 코드를 새 파일로 넣으면 됩니다.

---

## 6. `src/core/rqd_analysis.py` 신규 코드

```
python
"""
RQD 방법 비교 분석 유틸리티.

비교 대상 예:
1. RQD_borehole_direct
2. RQD_borehole_hudson
3. RQD_face_scanline
4. RQD_face_jv

주요 분석:
- 분포 요약
- pairwise error
- bias, MAE, RMSE, correlation
- overestimation / underestimation
- RQD threshold 기반 false-safe / false-alarm
"""

import numpy as np
import pandas as pd
from typing import Dict, List, Optional, Sequence, Tuple

def _safe_numeric_series(df: pd.DataFrame, col: str) -> pd.Series:
 if col not in df.columns:
 raise KeyError(f"Missing column: {col}")

 return pd.to_numeric(df[col], errors="coerce")

def _safe_corr(x: np.ndarray, y: np.ndarray) -> float:
 mask = np.isfinite(x) & np.isfinite(y)

 if mask.sum() < 3:
```



```

 return np.nan

 x2 = x[mask]
 y2 = y[mask]

 if np.std(x2) == 0 or np.std(y2) == 0:
 return np.nan

 return float(np.corrcoef(x2, y2)[0, 1])

def summarize_rqdcolumns(
 df: pd.DataFrame,
 cols: Sequence[str],
) -> pd.DataFrame:
 """
 RQD 컬럼들의 분포 요약.

 Returns

 pd.DataFrame
 column별 count, mean, std, min, p10, p25, p50, p75, p90, max
 """
 rows = []

 for col in cols:
 if col not in df.columns:
 rows.append(
 {
 "column": col,
 "exists": False,
 "count": 0,
 "mean": np.nan,
 "std": np.nan,
 "min": np.nan,
 "p10": np.nan,
 "p25": np.nan,
 "p50": np.nan,
 "p75": np.nan,
 "p90": np.nan,
 "max": np.nan,
 }
)
 continue

 s = _safe_numeric_series(df, col).replace([np.inf, -np.inf], np.nan).dropna()

 if len(s) == 0:
 rows.append(
 {
 "column": col,
 "exists": True,
 "count": 0,
 "mean": np.nan,

```

```

 "std": np.nan,
 "min": np.nan,
 "p10": np.nan,
 "p25": np.nan,
 "p50": np.nan,
 "p75": np.nan,
 "p90": np.nan,
 "max": np.nan,
 }
)
continue

rows.append(
 {
 "column": col,
 "exists": True,
 "count": int(len(s)),
 "mean": float(s.mean()),
 "std": float(s.std(ddof=1)) if len(s) > 1 else 0.0,
 "min": float(s.min()),
 "p10": float(s.quantile(0.10)),
 "p25": float(s.quantile(0.25)),
 "p50": float(s.quantile(0.50)),
 "p75": float(s.quantile(0.75)),
 "p90": float(s.quantile(0.90)),
 "max": float(s.max()),
 }
)

return pd.DataFrame(rows)

def rqd_pairwise_metrics(
 df: pd.DataFrame,
 predictor_col: str,
 target_col: str,
) -> Dict[str, float]:
 """
 두 RQD 방법 간 기본 비교.

 predictor - target 기준으로 bias를 계산한다.

 diff > 0:
 predictor가 target보다 RQD를 높게 평가.
 borehole predictor라면 잠재적 overestimation.

 diff < 0:
 predictor가 target보다 RQD를 낮게 평가.
 """
 pred = _safe_numeric_series(df, predictor_col)
 target = _safe_numeric_series(df, target_col)

 sub = pd.DataFrame(

```

```

 {
 "pred": pred,
 "target": target,
 }
).replace([np.inf, -np.inf], np.nan).dropna()

 if len(sub) == 0:
 return {
 "predictor": predictor_col,
 "target": target_col,
 "n": 0,
 "pred_mean": np.nan,
 "target_mean": np.nan,
 "bias": np.nan,
 "mae": np.nan,
 "rmse": np.nan,
 "median_abs_error": np.nan,
 "corr": np.nan,
 "overestimation_rate": np.nan,
 "underestimation_rate": np.nan,
 "equal_rate": np.nan,
 }

 x = sub["pred"].values.astype(float)
 y = sub["target"].values.astype(float)

 diff = x - y
 abs_diff = np.abs(diff)

 return {
 "predictor": predictor_col,
 "target": target_col,
 "n": int(len(sub)),
 "pred_mean": float(np.mean(x)),
 "target_mean": float(np.mean(y)),
 "bias": float(np.mean(diff)),
 "mae": float(np.mean(abs_diff)),
 "rmse": float(np.sqrt(np.mean(diff ** 2))),
 "median_abs_error": float(np.median(abs_diff)),
 "corr": _safe_corr(x, y),
 "overestimation_rate": float(np.mean(diff > 0)),
 "underestimation_rate": float(np.mean(diff < 0)),
 "equal_rate": float(np.mean(diff == 0)),
 }

def rqd_threshold_metrics(
 df: pd.DataFrame,
 predictor_col: str,
 target_col: str,
 threshold: float,
 cost_false_safe: float = 10.0,
 cost_false_alarm: float = 1.0,
) -> Dict[str, float]:

```

```

"""
RQD threshold 기반 decision metric.

good 조건:
 RQD >= threshold

false-safe:
 predictor는 good이라고 판단했지만 target은 bad.
 즉, pred_good=True, target_good=False

false-alarm:
 predictor는 bad라고 판단했지만 target은 good.
 즉, pred_good=False, target_good=True
"""
pred = _safe_numeric_series(df, predictor_col)
target = _safe_numeric_series(df, target_col)

sub = pd.DataFrame(
 {
 "pred": pred,
 "target": target,
 }
).replace([np.inf, -np.inf], np.nan).dropna()

if len(sub) == 0:
 return {
 "predictor": predictor_col,
 "target": target_col,
 "threshold": float(threshold),
 "n": 0,
 "prior_target_good_rate": np.nan,
 "TP": 0,
 "FP_false_safe": 0,
 "FN_false_alarm": 0,
 "TN": 0,
 "accuracy": np.nan,
 "balanced_accuracy": np.nan,
 "false_safe_rate": np.nan,
 "false_alarm_rate": np.nan,
 "expected_cost": np.nan,
 }

x = sub["pred"].values.astype(float)
y = sub["target"].values.astype(float)

pred_good = x >= threshold
target_good = y >= threshold

TP = int(np.sum(pred_good & target_good))
FP = int(np.sum(pred_good & ~target_good)) # false-safe
FN = int(np.sum(~pred_good & target_good)) # false-alarm
TN = int(np.sum(~pred_good & ~target_good))

```

```

n = len(sub)
n_good = TP + FN
n_bad = FP + TN

sensitivity = TP / n_good if n_good > 0 else np.nan
specificity = TN / n_bad if n_bad > 0 else np.nan

balanced_accuracy = np.nanmean([sensitivity, specificity])
accuracy = (TP + TN) / n if n > 0 else np.nan

false_safe_rate = FP / n_bad if n_bad > 0 else np.nan
false_alarm_rate = FN / n_good if n_good > 0 else np.nan

expected_cost = (
 cost_false_safe * FP + cost_false_alarm * FN
) / n

return {
 "predictor": predictor_col,
 "target": target_col,
 "threshold": float(threshold),
 "n": int(n),
 "prior_target_good_rate": float(np.mean(target_good)),
 "TP": TP,
 "FP_false_safe": FP,
 "FN_false_alarm": FN,
 "TN": TN,
 "accuracy": float(accuracy),
 "balanced_accuracy": float(balanced_accuracy),
 "false_safe_rate": float(false_safe_rate),
 "false_alarm_rate": float(false_alarm_rate),
 "expected_cost": float(expected_cost),
}

def compare_rqd_methods(
 df: pd.DataFrame,
 predictor_cols: Optional[Sequence[str]] = None,
 target_cols: Optional[Sequence[str]] = None,
 thresholds: Sequence[float] = (50.0, 75.0, 90.0),
 cost_false_safe: float = 10.0,
 cost_false_alarm: float = 1.0,
) -> Dict[str, pd.DataFrame]:
 """
 RQD 방법들을 일괄 비교한다.

 기본 predictor:
 - RQD_borehole_direct
 - RQD_borehole_hudson

 기본 target:
 - RQD_face_scanline

```

- RQD\_face\_jv
- RQD\_face\_conservative, 있으면 사용 가능

Returns

-----

dict

```
{
 "summary": ...,
 "pairwise": ...,
 "threshold": ...
}
```

"""

```
if predictor_cols is None:
```

```
 predictor_cols = [
 "RQD_borehole_direct",
 "RQD_borehole_hudson",
]
```

```
if target_cols is None:
```

```
 target_cols = [
 "RQD_face_scanline",
 "RQD_face_jv",
 "RQD_face_conservative",
]
```

```
all_cols = list(dict.fromkeys(list(predictor_cols) + list(target_cols)))
```

```
summary = summarize_rqd_columns(df, all_cols)
```

```
pairwise_rows = []
```

```
threshold_rows = []
```

```
for pred_col in predictor_cols:
```

```
 if pred_col not in df.columns:
 continue
```

```
 for target_col in target_cols:
```

```
 if target_col not in df.columns:
 continue
```

```
 pairwise_rows.append(
 rqd_pairwise_metrics(
 df=df,
 predictor_col=pred_col,
 target_col=target_col,
)
)
```

```
 for th in thresholds:
```

```
 threshold_rows.append(
 rqd_threshold_metrics(
 df=df,
 predictor_col=pred_col,
 target_col=target_col,
```

```

 threshold=th,
 cost_false_safe=cost_false_safe,
 cost_false_alarm=cost_false_alarm,
)
)

return {
 "summary": pd.DataFrame(summary),
 "pairwise": pd.DataFrame(pairwise_rows),
 "threshold": pd.DataFrame(threshold_rows),
}

def compare_rqd_by_regime(
 df: pd.DataFrame,
 group_cols: Sequence[str],
 predictor_col: str,
 target_col: str,
) -> pd.DataFrame:
 """
 case, density regime, orientation regime 등 그룹별 RQD 오차 비교.

 예:
 compare_rqd_by_regime(
 df,
 group_cols=["case_name"],
 predictor_col="RQD_borehole_direct",
 target_col="RQD_face_scanline",
)
 """
 missing = [c for c in group_cols if c not in df.columns]
 if missing:
 raise KeyError(f"Missing group columns: {missing}")

 rows = []

 for keys, sub in df.groupby(list(group_cols)):
 metrics = rqd_pairwise_metrics(
 df=sub,
 predictor_col=predictor_col,
 target_col=target_col,
)

 if not isinstance(keys, tuple):
 keys = (keys,)

 for col, val in zip(group_cols, keys):
 metrics[col] = val

 rows.append(metrics)

 out = pd.DataFrame(rows)

 ordered_cols = list(group_cols) + [

```

```
 c for c in out.columns if c not in group_cols
]

 return out[ordered_cols]
```

---

## 7. 이 분석 코드가 기대하는 CSV 컬럼

나중에 `tunnel.py`와 `comparison.py`에서 아래 컬럼을 저장하면 이 분석 코드가 바로 작동합니다.

```
text
RQD_borehole_direct
RQD_borehole_hudson
RQD_face_scanline
RQD_face_jv
RQD_face_conservative
```

그리고 있으면 좋은 보조 컬럼은 다음입니다.

```
text
lambda_borehole
lambda_face_scanline
borehole_n_intersections
face_scanline_n_intersections
Jv_face
Jv_method
case_name
seed
face_x
orientation_bias_gap
P32_total
```

---

## 8. 나중에 분석 코드를 호출하는 예시

아직 CLI를 만들 필요는 없지만, 나중에는 이런 식으로 쓸 수 있습니다.

```
python
import pandas as pd
from src.core.rqd_analysis import compare_rqd_methods

df = pd.read_csv("outputs/all_cases_face_rows.csv")

results = compare_rqd_methods(
 df,
 predictor_cols=[
 "RQD_borehole_direct",
 "RQD_borehole_hudson",
],
 target_cols=[
 "RQD_face_scanline",
 "RQD_face_jv",
 "RQD_face_conservative",
```



```
],
 thresholds=[50, 75, 90],
)

print(results["summary"])
print(results["pairwise"])
print(results["threshold"])
```

## 9. 이번 단계에서 아직 연결하지 않은 부분

이번 코드는 `rqd_calculator.py`와 분석 유틸리티만 준비한 것입니다.

아직 다음 파일들은 수정하지 않았습니다.

```
text
src/core/tunnel.py
src/core/comparison.py
src/cli/batch_runner.py
src/cli/multicase_batch_runner.py
```

다음 단계에서는 `Tunnel.sample_borehole()`와 `Tunnel.sample_face()`에서 위 함수를 호출하도록 연결하면 됩니다.

특히 다음 값들을 생성해야 합니다.

### `sample_borehole()` 쪽

```
text
RQD_borehole_direct
lambda_borehole
RQD_borehole_hudson
borehole_n_intersections
```

### `sample_face()` 쪽

```
text
RQD_face_scanline
lambda_face_scanline
RQD_face_scanline_hudson
face_scanline_n_intersections

Jv_face
Jv_method
RQD_face_jv

RQD_face_conservative
```

## 10. 가장 중요한 변경점 요약

이번 `rqd_calculator.py` 수정으로 바뀌는 핵심은 다음입니다.

text

기존:

굴진면에서 x, y, z, 대각선 등 5방향 scanline 평균

수정:

굴진면에서 x방향 제거

multi-direction 평균 제거

중앙 수평 y방향 scanline 하나만 사용

그리고 RQD 비교 방법은 다음 네 개로 정리됩니다.

text

1. RQD\_borehole\_direct

2. RQD\_borehole\_hudson

3. RQD\_face\_scanline

4. RQD\_face\_jv

이제 다음 단계에서 `tunnel.py`에 이 네 값을 실제 batch 결과에 저장하도록 연결하면 됩니다.

GPT-5.5

tunnel.py: """

터널 기하학, 시추공 샘플링, 막장면 샘플링

★ 수정: 격자값 읽기 → DFN 직접 교차 기반 샘플링

- 시추공: 1D 라인(5cm 직경) × DFN → 방향 편향 + 소규모 샘플링
- 막장면: 2D 원형면(10m) × DFN → 다방향 스캔 + 대규모 샘플링

import numpy as np

from typing import Dict, List, Tuple, Optional

from .domain import RockDomain

from .q\_calculator import QCalculator

from .rqd\_calculator import RQDCalculator

from .q\_calculator import QCalculator

class Tunnel:

"""터널 & 시추공 & 막장면 — DFN 직접 교차 기반"""

text

BH\_NAMES = ['Center', 'Top', 'Bottom']

# — 물리 상수 —

BOREHOLE\_DIAMETER = 0.076

FACE\_OBSERVATION\_DEPTH = 1.0

```

—— 스케일 효과 파라미터 ——
JR_SCALE_FACTOR_FACE = 0.85
JR_SCALE_FACTOR_BH = 1.10
JA_UNDERESTIMATE_BH = 0.85
JA_SCALE_FACTOR_FACE = 1.05

—— 코어 손실 모델 ——
CORE_LOSS_THRESHOLD = 15
CORE_LOSS_RATE = 0.15

—— ★ 추가: 현실성 보정 파라미터 ——
[1] 기계적 코어 파쇄 (Mechanical Break)
MECHANICAL_BREAK_PROB = 0.15 # 절리 교차점에서 추가 파쇄 확률
MECHANICAL_BREAK_OFFSET_MEAN = 0.03 # 추가 파쇄 위치 (절리에서 ~3cm)
MECHANICAL_BREAK_OFFSET_STD = 0.02

[2] 미세절리 (Micro-fractures)
DFN에 포함되지 않는 소규모 절리
실측 기반: 대규모 절리 빈도의 0.5~2.0배 추가
MICRO_FRACTURE_MULTIPLIER = 0.8 # 관측 절리 수 × 이 계수 = 추가 미세절리

[3] 코어 회수율 감소 (절리 밀집구간)
CORE_RECOVERY_BASE = 0.98 # 기본 회수율 98%
CORE_RECOVERY_DECAY = 0.03 # 절리 1개당 회수율 감소

def __init__(self, domain: RockDomain, correction_mode: str = "pure"):
 """
 correction_mode:
 - "practice": 기존 보정 모두 사용
 - "neutral_strength": Jr/Ja/face conservatism만 제거
 - "pure": 모든 artificial correction 제거
 """
 self.correction_mode = correction_mode

 valid_modes = ["practice", "neutral_strength", "pure"]
 if correction_mode not in valid_modes:
 raise ValueError(f"Invalid correction_mode={correction_mode}. Use {valid_modes}")
 self.domain = domain
 self.case = domain.case
 self.dfn = domain.dfn
 self.rng = np.random.RandomState(self.case.seed + 200)

 self.center_y = self.case.tunnel_center_y
 self.center_z = self.case.tunnel_center_z
 self.center_y_idx = int(self.center_y / domain.dy)
 self.center_z_idx = int(self.center_z / domain.dz)
 self.radius = self.case.tunnel_radius

```

```

self.borehole_positions = []
self.borehole_coords = []
for dy_off, dz_off in self.case.borehole_offsets:
 y_m = self.center_y + dy_off
 z_m = self.center_z + dz_off
 y_idx = int(np.clip(y_m / domain.dy, 0, domain.ny - 1))
 z_idx = int(np.clip(z_m / domain.dz, 0, domain.nz - 1))
 self.borehole_positions.append((y_idx, z_idx))
 self.borehole_coords.append((y_m, z_m))

def _use_strength_scaling(self) -> bool:
 return self.correction_mode == "practice"

def _use_drilling_damage(self) -> bool:
 return self.correction_mode == "practice"

def _use_micro_fractures(self) -> bool:
 return self.correction_mode == "practice"

def _use_face_conservatism(self) -> bool:
 return self.correction_mode == "practice"

def _use_practical_face_rqf(self) -> bool:
 return self.correction_mode in ["practice", "neutral_strength"]

=====
시추공 샘플링 - DFN 직접 교차 + 현실 보정
=====
def sample_borehole(self, bh_idx: int,
 x_start: int, x_end: int,
 window: float = None) -> Dict:
 """
 시추공 1D 라인 샘플링 - DFN 직접 교차 + 현실 보정

 현실 보정:
 1. 기계적 코어 파쇄 (절리 근처 추가 break)
 2. 미세절리 (DFN 미포함 소규모 절리)
 3. 코어 회수율 감소 (절리 밀집구간)
 4. Jr/Ja 스케일 효과
 """
 y_m, z_m = self.borehole_coords[bh_idx]
 x_start_m = x_start * self.domain.dx
 x_end_m = x_end * self.domain.dx
 bh_length = x_end_m - x_start_m

 if bh_length <= 0:
 return self._empty_borehole_result(bh_idx, x_start, x_end)

 # ★ DFN과 직접 교차 계산
 origin = np.array([x_start_m, y_m, z_m])
 direction = np.array([1.0, 0.0, 0.0])

```

```

intersections = self.dfn.get_joints_intersecting_line(
 origin, direction, bh_length
)

t_values = np.array([t for t, _ in intersections]) if intersections else
np.array([])
hit_joints = [j for _, j in intersections]

— 구간별 계산 —
n_segments = x_end - x_start
segment_length = self.domain.dx

rqd_arr = np.zeros(n_segments)
jr_arr = np.zeros(n_segments)
ja_arr = np.zeros(n_segments)
jn_arr = np.full(n_segments, self.case.joint_config.Jn)
jw_arr = np.zeros(n_segments)
srf_arr = np.zeros(n_segments)
joint_count_arr = np.zeros(n_segments, dtype=int)

y_idx, z_idx = self.borehole_positions[bh_idx]
for i in range(n_segments):
 xi = x_start + i
 if xi < self.domain.nx:
 jw_arr[i] = self.domain.fields['Jw'][xi, y_idx, z_idx]
 srf_arr[i] = self.domain.fields['SRF'][xi, y_idx, z_idx]
 else:
 jw_arr[i] = self.case.joint_config.Jw_mean
 srf_arr[i] = self.case.joint_config.SRF_mean

global_jr = np.mean([js.Jr_mean for js in self.case.joint_config.joint_sets])
global_ja = np.mean([js.Ja_mean for js in self.case.joint_config.joint_sets])

for i in range(n_segments):
 seg_start = i * segment_length
 seg_end = seg_start + segment_length

 # 이 구간 내 DFN 교차점
 if len(t_values) > 0:
 mask = (t_values >= seg_start) & (t_values < seg_end)
 seg_t = t_values[mask] - seg_start
 seg_joints = [hit_joints[idx] for idx in np.where(mask)[0]]
 else:
 seg_t = np.array([])
 seg_joints = []

 n_dfn_hits = len(seg_joints)

 if self._use_drilling_damage():
 mechanical_breaks = self._generate_mechanical_breaks(
 seg_t, segment_length
)
 else:
 mechanical_breaks = np.array([])

```

```

if self._use_micro_fractures():
 micro_fractures = self._generate_micro_fractures(
 n_dfn_hits, segment_length
)
else:
 micro_fractures = np.array([])

모든 파단점 합산
all_breaks = np.concatenate([
 seg_t,
 mechanical_breaks,
 micro_fractures
])
all_breaks = np.sort(all_breaks)
all_breaks = all_breaks[(all_breaks >= 0) & (all_breaks <= segment_length)]

total_breaks = len(all_breaks)
joint_count_arr[i] = total_breaks

★ [3] RQD 계산 (Deere 직접법, 모든 파단점 포함)
rqd_raw = RQDCalculator.calc_direct(
 all_breaks.tolist(), segment_length, threshold=0.1
)

★ [4] 코어 회수율 보정
if self._use_drilling_damage():
 core_recovery = self._calc_core_recovery(total_breaks, segment_length)
else:
 core_recovery = 1.0

코어 손실 구간은 RQD 계산에서 "0cm 조각"으로 처리
rqd_arr[i] = rqd_raw * core_recovery

Jr, Ja: DFN 절리에서만 수집 (미세절리는 Jr/Ja 없음)
if n_dfn_hits > 0:
 jr_raw = np.mean([j.Jr for j in seg_joints])
 ja_raw = np.mean([j.Ja for j in seg_joints])
 if self._use_strength_scaling():
 jr_arr[i] = np.clip(jr_raw * self.JR_SCALE_FACTOR_BH, 0.5, 4.0)
 ja_arr[i] = np.clip(ja_raw * self.JA_UNDERESTIMATE_BH, 0.75, 20.0)
 else:
 jr_arr[i] = np.clip(jr_raw, 0.5, 4.0)
 ja_arr[i] = np.clip(ja_raw, 0.75, 20.0)
else:
 jr_arr[i] = global_jr
 ja_arr[i] = global_ja

Q, Q' 계산
q_arr = np.array([
 QCalculator.calc_Q(rqd_arr[i], jn_arr[i], jr_arr[i],
 ja_arr[i], jw_arr[i], srf_arr[i])

```

```

 for i in range(n_segments)
])
 qp_arr = np.array([
 QCalculator.calc_Qprime(rqd_arr[i], jn_arr[i],
 jr_arr[i], ja_arr[i])
 for i in range(n_segments)
])

 unique_hit_joints = list({id(j): j for _, j in intersections}.values())
 unique_hit_joint_ids = [id(j) for j in unique_hit_joints]

 return {
 'x': np.arange(x_start, x_end) * self.domain.dx,
 'borehole_name': self.BH_NAMES[bh_idx] if bh_idx < 3 else f'BH-{bh_idx}',
 'RQD': rqd_arr,
 'Jn': jn_arr,
 'Jr': jr_arr,
 'Ja': ja_arr,
 'Jw': jw_arr,
 'SRF': srf_arr,
 'Q': q_arr,
 'Qprime': qp_arr,
 '_joint_count': joint_count_arr,
 '_intersections': intersections,
 '_joint_objects': unique_hit_joints,
 '_joint_ids': unique_hit_joint_ids,
 '_joint_ids_set': set(unique_hit_joint_ids),
 }
=====
★ 기계적 코어 파쇄 생성
=====
def _generate_mechanical_breaks(self, dfn_intersections: np.ndarray,
 segment_length: float) -> np.ndarray:
 """
 절리 교차점 근처에서 기계적 파쇄 생성

 현실:
 - 시추 진동으로 절리면 근처 암석이 추가 파손
 - 응력 해방으로 기존 미세균열 활성화
 - 특히 Ja가 높은(약한) 절리 근처에서 빈번
 """
 if len(dfn_intersections) == 0:
 return np.array([])

 breaks = []
 for t in dfn_intersections:
 # 각 DFN 절리 교차점에서 확률적 추가 파쇄
 if self.rng.random() < self.MECHANICAL_BREAK_PROB:
 # 절리 앞뒤로 ~3cm 위치에서 추가 파쇄
 offset = abs(self.rng.normal(
 self.MECHANICAL_BREAK_OFFSET_MEAN,
 self.MECHANICAL_BREAK_OFFSET_STD
))

```

```

))
 # 앞쪽 또는 뒤쪽
 if self.rng.random() < 0.5:
 breaks.append(t - offset)
 else:
 breaks.append(t + offset)

 # 양쪽 다 파쇄될 수도 있음 (심한 경우)
 if self.rng.random() < 0.3:
 offset2 = abs(self.rng.normal(0.05, 0.02))
 breaks.append(t + offset2 if breaks[-1] < t else t - offset2)

 return np.array(breaks) if breaks else np.array([])

=====
★ 미세절리 생성
=====
def _generate_micro_fractures(self, n_dfn_hits: int,
 segment_length: float) -> np.ndarray:
 """
 DFN에 포함되지 않는 미세절리 생성

 현실:
 - DFN 최소 반경 = 0.3m → 이보다 작은 절리 미포함
 - 실제 코어에는 mm~cm 스케일 불연속면 다수
 - 빈도: DFN 절리 수에 비례 (같은 지질환경)
 """
 # DFN 절리가 많은 구간 = 미세절리도 많은 구간
 n_micro = self.rng.poisson(
 max(n_dfn_hits * self.MICRO_FRACTURE_MULTIPLIER, 0.5)
)

 if n_micro == 0:
 return np.array([])

 # 균일 분포로 위치 생성
 return self.rng.uniform(0, segment_length, n_micro)

=====
★ 코어 회수율 계산
=====
def _calc_core_recovery(self, n_breaks: int,
 segment_length: float) -> float:
 """
 절리 밀집도에 따른 코어 회수율

 현실:
 - 절리 밀집 → 코어 붕괴 → 회수율 감소
 - 미회수 구간은 RQD 0으로 처리

```



```

- TCR 80% → RQD에 0.8 곱함
"""
linear_freq = n_breaks / segment_length
recovery = self.CORE_RECOVERY_BASE - self.CORE_RECOVERY_DECAY * linear_freq
return np.clip(recovery, 0.2, 1.0) # 최소 20%

=====
막장면 샘플링 - DFN 직접 교차 (2D) + 현실 보정
=====
def sample_face(self, x_idx: int,
 n_scanlines: int = 8,
 scan_length: float = None) -> Dict:
 """
 터널 막장면 2D 원형 단면 샘플링 - DFN 직접 교차

 막장면의 현실성:
 - 2D 관찰이므로 미세절리도 관찰 가능
 - 다방향 스캔 → 모든 방향의 절리 포착
 - 지질기사의 보수적 평가 경향
 """
 if scan_length is None:
 scan_length = self.radius * 2

 face_x = (x_idx + 0.5) * self.domain.dx

 # ★ 막장면과 교차하는 절리 찾기
 face_joints = self._find_joints_intersecting_face(face_x)

 # ★ 다방향 스캔라인 RQD (미세절리 포함)
 face_rqd_values = self._multi_direction_face_rqd(
 face_x, n_scanlines, scan_length
)

 # ★ 면에 노출된 절리에서 Jr, Ja 수집
 if len(face_joints) > 0:
 jr_raw = np.array([j.Jr for j in face_joints])
 ja_raw = np.array([j.Ja for j in face_joints])
 if self._use_strength_scaling():
 jr_values = np.clip(jr_raw * self.JR_SCALE_FACTOR_FACE, 0.5, 4.0)
 ja_values = np.clip(ja_raw * self.JA_SCALE_FACTOR_FACE, 0.75, 20.0)
 else:
 jr_values = np.clip(jr_raw, 0.5, 4.0)
 ja_values = np.clip(ja_raw, 0.75, 20.0)
 jr_mean = float(np.mean(jr_values))
 ja_mean = float(np.mean(ja_values))
 jr_std = float(np.std(jr_values))
 ja_std = float(np.std(ja_values))
 else:
 global_jr = np.mean([js.Jr_mean for js in self.case.joint_config.joint_sets])
 global_ja = np.mean([js.Ja_mean for js in self.case.joint_config.joint_sets])
 jr_mean, ja_mean = global_jr, global_ja

```

```

 jr_std, ja_std = 0.0, 0.0
 jr_values = np.array([])
 ja_values = np.array([])

Jn, Jw, SRF: 터널 단면 평균
cy, cz = self.center_y_idx, self.center_z_idx
r_cells = self.radius / self.domain.dy

face_points = []
jw_list, srf_list = [], []
for j in range(self.domain.ny):
 for k in range(self.domain.nz):
 if np.sqrt((j - cy) ** 2 + (k - cz) ** 2) <= r_cells:
 face_points.append((j, k))
 if x_idx < self.domain.nx:
 jw_list.append(self.domain.fields['Jw'][x_idx, j, k])
 srf_list.append(self.domain.fields['SRF'][x_idx, j, k])

jw_mean = float(np.mean(jw_list)) if jw_list else self.case.joint_config.Jw_mean
srf_mean = float(np.mean(srf_list)) if srf_list else
self.case.joint_config.SRF_mean
jn_val = self.case.joint_config.Jn

RQD 통계
rqd_mean = float(np.mean(face_rqd_values['all_rqd']))
rqd_std = float(np.std(face_rqd_values['all_rqd']))
rqd_min_dir = float(np.min(face_rqd_values['direction_means']))

"가장 불리한 방향" RQD
if self._use_practical_face_rqd():
 # 기존 practical estimator
 rqd_practical = 0.6 * rqd_mean + 0.4 * rqd_min_dir
else:
 # pure mode에서는 방향 평균 RQD를 face 대표값으로 사용
 rqd_practical = rqd_mean

★ 막장면 관찰 보정: 지질기사 보수적 평가
실무에서 막장면 RQD는 시추공보다 낮게 평가하는 경향
if self._use_face_conservatism():
 FACE_CONSERVATISM = 0.95
 rqd_practical *= FACE_CONSERVATISM

Q 계산
Q_mean = QCalculator.calc_Q(rqd_practical, jn_val, jr_mean, ja_mean, jw_mean,
srf_mean)
Qp_mean = QCalculator.calc_Qprime(rqd_practical, jn_val, jr_mean, ja_mean)

셀별 배열
n_points = len(face_points)
rqd_arr = np.full(n_points, rqd_practical)

```

```

jr_arr = np.full(n_points, jr_mean)
ja_arr = np.full(n_points, ja_mean)
jn_arr = np.full(n_points, jn_val)
jw_arr = np.array(jw_list) if jw_list else np.full(n_points, jw_mean)
srf_arr = np.array(srf_list) if srf_list else np.full(n_points, srf_mean)

q_arr = np.array([
 QCalculator.calc_Q(rqd_arr[i], jn_arr[i], jr_arr[i],
 ja_arr[i], jw_arr[i], srf_arr[i])
 for i in range(n_points)
])
qp_arr = np.array([
 QCalculator.calc_Qprime(rqd_arr[i], jn_arr[i], jr_arr[i], ja_arr[i])
 for i in range(n_points)
])

return {
 'x_idx': x_idx,
 'x_coord': face_x,
 'points': face_points,
 'RQD': rqd_arr,
 'Jn': jn_arr,
 'Jr': jr_arr,
 'Ja': ja_arr,
 'Jw': jw_arr,
 'SRF': srf_arr,
 'Q': q_arr,
 'Qprime': qp_arr,
 'stats': {
 'RQD': {'mean': rqd_practical, 'median': rqd_practical,
 'std': rqd_std, 'min': rqd_min_dir,
 'max': float(np.max(face_rqd_values['all_rqd'])),
 'n_points': n_points},
 'Jn': {'mean': jn_val, 'median': jn_val,
 'std': 0.0, 'min': jn_val, 'max': jn_val,
 'n_points': n_points},
 'Jr': {'mean': jr_mean, 'median': jr_mean,
 'std': jr_std,
 'min': float(np.min(jr_values)) if len(jr_values) > 0 else jr_mean,
 'max': float(np.max(jr_values)) if len(jr_values) > 0 else jr_mean,
 'n_points': n_points},
 'Ja': {'mean': ja_mean, 'median': ja_mean,
 'std': ja_std,
 'min': float(np.min(ja_values)) if len(ja_values) > 0 else ja_mean,
 'max': float(np.max(ja_values)) if len(ja_values) > 0 else ja_mean,
 'n_points': n_points},
 'Jw': {'mean': jw_mean, 'median': float(np.median(jw_arr)),
 'std': float(np.std(jw_arr)), 'min': float(np.min(jw_arr)),
 'max': float(np.max(jw_arr)), 'n_points': n_points},
 'SRF': {'mean': srf_mean, 'median': float(np.median(srf_arr)),
 'std': float(np.std(srf_arr)), 'min': float(np.min(srf_arr)),
 'max': float(np.max(srf_arr)), 'n_points': n_points},
 'Q': {'mean': float(np.mean(q_arr)), 'median': float(np.median(q_arr)),
 'std': float(np.std(q_arr)), 'min': float(np.min(q_arr)),

```

```

 'max': float(np.max(q_arr)), 'n_points': n_points},
 'Qprime': {'mean': float(np.mean(qp_arr)), 'median':
float(np.median(qp_arr)),
 'std': float(np.std(qp_arr)), 'min': float(np.min(qp_arr)),
 'max': float(np.max(qp_arr)), 'n_points': n_points},
 },
 '_face_joints_count': len(face_joints),
 '_rqd_by_direction': face_rqd_values,
 '_rqd_mean': rqd_mean,
 '_rqd_min_direction': rqd_min_dir,
 '_rqd_practical': rqd_practical,
 '_face_joints': face_joints,
 '_face_joint_ids': [id(j) for j in face_joints],
 '_face_joint_ids_set': set(id(j) for j in face_joints),
}

=====
내부: 다방향 스캔라인 RQD (미세절리 포함)
=====
def _multi_direction_face_rqd(self, face_x: float,
 n_scanlines: int = 8,
 scan_length: float = 10.0) -> Dict:
 """
 막장면에서 다방향 스캔라인 RQD (미세절리 포함)

 막장면에서는:
 - 2D 노출면이므로 미세절리도 육안 관찰 가능
 - 시추공보다 더 많은 절리를 관찰
 - 따라서 미세절리 승수를 높게 적용
 """

 FACE_MICRO_MULTIPLIER = 1.2 # 막장면 미세절리 (시추공보다 관찰↑)

 scan_directions = []
 for i in range(n_scanlines):
 angle = np.pi * i / n_scanlines
 d = np.array([0.0, np.cos(angle), np.sin(angle)])
 scan_directions.append(d)
 scan_directions.append(np.array([1.0, 0.0, 0.0]))

 all_rqd = []
 direction_means = []

 for d_idx, scan_dir in enumerate(scan_directions):
 dir_rqd_values = []
 n_grid = max(3, int(self.radius / 2))

 for iy in range(n_grid):
 for iz in range(n_grid):
 y_off = -self.radius + (2 * self.radius * (iy + 0.5) / n_grid)
 z_off = -self.radius + (2 * self.radius * (iz + 0.5) / n_grid)

 if np.sqrt(y_off**2 + z_off**2) > self.radius * 0.9:

```

```

 continue

 origin = np.array([
 face_x - scan_length / 2 * scan_dir[0],
 self.center_y + y_off - scan_length / 2 * scan_dir[1],
 self.center_z + z_off - scan_length / 2 * scan_dir[2],
])

 intersections = self.dfn.get_joints_intersecting_line(
 origin, scan_dir, scan_length
)
 t_vals_dfn = [t for t, _ in intersections]
 n_dfn = len(t_vals_dfn)

 # ★ 막장면 미세절리 추가
 if self._use_micro_fractures():
 n_micro = self.rng.poisson(
 max(n_dfn * FACE_MICRO_MULTIPLIER, 0.5)
)
 micro_t = self.rng.uniform(0, scan_length, n_micro).tolist() if
n_micro > 0 else []
 else:
 micro_t = []

 all_t = sorted(t_vals_dfn + micro_t)
 rqd = RQDCalculator.calc_direct(all_t, scan_length)

 dir_rqd_values.append(rqd)
 all_rqd.append(rqd)

 if dir_rqd_values:
 direction_means.append(float(np.mean(dir_rqd_values)))

 return {
 'all_rqd': np.array(all_rqd) if all_rqd else np.array([100.0]),
 'direction_means': np.array(direction_means) if direction_means else
np.array([100.0]),
 'scan_directions': scan_directions,
 'n_scanlines_total': len(all_rqd),
 }

=====
내부: 막장면-절리 교차 탐색 (기존과 동일)
=====
def _find_joints_intersecting_face(self, face_x: float) -> list:
 """막장면(y-z 평면)과 교차하는 절리 탐색"""
 face_joints = []
 obs_depth = self.FACE_OBSERVATION_DEPTH

 for joint in self.dfn.joints:
 c = joint.center
 n = joint.normal
 r = joint.radius

```

```

 if abs(c[0] - face_x) > r + obs_depth:
 continue

 if abs(n[0]) < 1e-10:
 if abs(c[0] - face_x) <= r:
 dy = c[1] - self.center_y
 dz = c[2] - self.center_z
 dist_to_tunnel = np.sqrt(dy**2 + dz**2)
 if dist_to_tunnel <= self.radius + r:
 face_joints.append(joint)
 else:
 x_dist = abs(c[0] - face_x)
 if x_dist <= r:
 dy = c[1] - self.center_y
 dz = c[2] - self.center_z
 dist_to_tunnel = np.sqrt(dy**2 + dz**2)
 if dist_to_tunnel <= self.radius + r:
 face_joints.append(joint)

 return face_joints

=====
내부 유틸리티
=====
def _generate_mechanical_breaks(self, dfn_intersections, segment_length):
 """절리 교차점 근처에서 기계적 파쇄 생성"""
 if len(dfn_intersections) == 0:
 return np.array([])
 breaks = []
 for t in dfn_intersections:
 if self.rng.random() < self.MECHANICAL_BREAK_PROB:
 offset = abs(self.rng.normal(
 self.MECHANICAL_BREAK_OFFSET_MEAN,
 self.MECHANICAL_BREAK_OFFSET_STD
))
 if self.rng.random() < 0.5:
 breaks.append(t - offset)
 else:
 breaks.append(t + offset)
 if self.rng.random() < 0.3:
 offset2 = abs(self.rng.normal(0.05, 0.02))
 breaks.append(t + offset2 if breaks[-1] < t else t - offset2)
 return np.array(breaks) if breaks else np.array([])

def _generate_micro_fractures(self, n_dfn_hits, segment_length):
 """DFN 미포함 미세절리 생성"""
 n_micro = self.rng.poisson(
 max(n_dfn_hits * self.MICRO_FRACTURE_MULTIPLIER, 0.5)
)
 if n_micro == 0:
 return np.array([])
 return self.rng.uniform(0, segment_length, n_micro)

```

```

def _calc_core_recovery(self, n_breaks, segment_length):
 """절리 밀집도에 따른 코어 회수율"""
 linear_freq = n_breaks / segment_length
 recovery = self.CORE_RECOVERY_BASE - self.CORE_RECOVERY_DECAY * linear_freq
 return np.clip(recovery, 0.2, 1.0)

def _empty_borehole_result(self, bh_idx, x_start, x_end):
 """빈 시추공 결과"""
 return {
 'x': np.array([]),
 'borehole_name': self.BH_NAMES[bh_idx] if bh_idx < 3 else f'BH-{bh_idx}',
 'RQD': np.array([]), 'Jn': np.array([]),
 'Jr': np.array([]), 'Ja': np.array([]),
 'Jw': np.array([]), 'SRF': np.array([]),
 'Q': np.array([]), 'Qprime': np.array([]),
 }

=====
기존 호환 (비교 검증용)
=====
def sample_borehole_grid(self, bh_idx, x_start, x_end):
 """[기존 방식] 격자값 읽기"""
 y_idx, z_idx = self.borehole_positions[bh_idx]
 x_indices = np.arange(x_start, x_end)
 result = {
 'x': x_indices * self.domain.dx,
 'borehole_name': self.BH_NAMES[bh_idx] if bh_idx < 3 else f'BH-{bh_idx}',
 }
 for name in self.domain.fields:
 result[name] = self.domain.fields[name][x_start:x_end, y_idx, z_idx]
 result['Q'] = self.domain.Q_field[x_start:x_end, y_idx, z_idx]
 result['Qprime'] = self.domain.Qprime_field[x_start:x_end, y_idx, z_idx]
 return result

def sample_face_grid(self, x_idx):
 """[기존 방식] 격자값 읽기"""
 cy, cz = self.center_y_idx, self.center_z_idx
 r_cells = self.radius / self.domain.dy
 face_points = []
 for j in range(self.domain.ny):
 for k in range(self.domain.nz):
 if np.sqrt((j - cy)**2 + (k - cz)**2) <= r_cells:
 face_points.append((j, k))
 data = {name: [] for name in self.domain.fields}
 data['Q'], data['Qprime'], data['points'] = [], [], face_points
 for j, k in face_points:
 for name in self.domain.fields:
 data[name].append(self.domain.fields[name][x_idx, j, k])
 data['Q'].append(self.domain.Q_field[x_idx, j, k])
 data['Qprime'].append(self.domain.Qprime_field[x_idx, j, k])
 for key in data:
 if key != 'points':

```

```

 data[key] = np.array(data[key])
 data['stats'] = {}
 for key in ['RQD', 'Jn', 'Jr', 'Ja', 'Jw', 'SRF', 'Q', 'Qprime']:
 arr = data[key]
 data['stats'][key] = {
 'mean': float(np.mean(arr)), 'median': float(np.median(arr)),
 'std': float(np.std(arr)), 'min': float(np.min(arr)),
 'max': float(np.max(arr)), 'n_points': len(arr),
 }
 data['x_idx'] = x_idx
 data['x_coord'] = x_idx * self.domain.dx
 return data

```



좋습니다. 이제 `tunnel.py`는 **RQD 4방식 비교가 가능하도록 연결하는 단계**입니다.

이번 수정의 목표는 다음입니다.

text

1. 시추공 direct RQD 저장
2. 시추공 Hudson lambda RQD 저장
3. 굴진면 중앙 수평 scanline RQD 저장
4. Jv 기반 RQD 저장
5. 기존 다방향 face scanline 사용 중단
6. face에서 x방향 scanline 완전 제거

아래는 **순서대로 어디를 대체할지와 붙여 넣을 코드**입니다.

## 0. 전제

먼저 이전에 제안한 새 `rqd_calculator.py`가 적용되어 있어야 합니다.

특히 아래 두 클래스/함수가 있어야 합니다.

```

python
RQDCalculator.calc_direct()
RQDCalculator.calc_hudson_from_lambda()
RQDCalculator.calc_lambda_from_count()
RQDCalculator.calc_from_jv()
RQDCalculator.calc_jv_proxy_from_joint_sets()

DirectionalRQDCalculator.rqd_at_face_horizontal_scanline()

```

## 1. import 부분 수정



## 기존 부분

현재 상단이 이렇게 되어 있습니다.

```
python
import numpy as np
from typing import Dict, List, Tuple, Optional
from .domain import RockDomain
from .q_calculator import QCalculator
from .rqd_calculator import RQDCalculator
from .q_calculator import QCalculator
```

QCalculator가 중복 import 되어 있고, DirectionalRQDCalculator가 없습니다.

---

## 이 부분으로 대체

```
python
import numpy as np
from typing import Dict, List, Tuple, Optional

from .domain import RockDomain
from .q_calculator import QCalculator
from .rqd_calculator import RQDCalculator, DirectionalRQDCalculator
```

---

## 2. \_\_init\_\_()에 RQD calculator 객체 추가

### 기존 위치

\_\_init\_\_() 내부에 현재 이런 부분이 있습니다.

```
python
self.domain = domain
self.case = domain.case
self.dfn = domain.dfn
self.rng = np.random.RandomState(self.case.seed + 200)
```

---

### 아래처럼 수정

```
python
self.domain = domain
self.case = domain.case
self.dfn = domain.dfn
self.rqd_calculator = DirectionalRQDCalculator(self.dfn)
self.rng = np.random.RandomState(self.case.seed + 200)
```

즉, 한 줄만 추가합니다.

```
python
```

```
self.rqd_calculator = DirectionalRQDCalculator(self.dfn)
```

### 3. `sample_borehole()`에 RQD direct / Hudson 배열 추가

현재 `sample_borehole()` 안에는 기존 RQD 배열이 있습니다.

```
python
rqd_arr = np.zeros(n_segments)
jr_arr = np.zeros(n_segments)
ja_arr = np.zeros(n_segments)
jn_arr = np.full(n_segments, self.case.joint_config.Jn)
jw_arr = np.zeros(n_segments)
srf_arr = np.zeros(n_segments)
joint_count_arr = np.zeros(n_segments, dtype=int)
```

이 부분을 아래로 대체

```
python
rqd_arr = np.zeros(n_segments)
jr_arr = np.zeros(n_segments)
ja_arr = np.zeros(n_segments)
jn_arr = np.full(n_segments, self.case.joint_config.Jn)
jw_arr = np.zeros(n_segments)
srf_arr = np.zeros(n_segments)
joint_count_arr = np.zeros(n_segments, dtype=int)

=====
RQD 방법 비교용 신규 배열

RQD_borehole_direct:
DFN 자연 절리 교차점만 사용한 시추공 Deere 직접 RQD
#
RQD_borehole_hudson:
같은 DFN 자연 절리 교차점 수로부터 $\lambda = N/L$ 계산 후
Priest-Hudson 식으로 계산한 RQD
#
주의:
practice mode에서 기계적 파쇄/미세절리를 추가하더라도,
아래 RQD 비교용 컬럼은 "자연 DFN 절리" 기준으로 유지한다.
그래야 시추공 RQD와 face RQD의 기하학적/방향성 차이를 분리해서 볼 수 있다.
=====
rqd_borehole_direct_arr = np.zeros(n_segments)
lambda_borehole_arr = np.zeros(n_segments)
rqd_borehole_hudson_arr = np.zeros(n_segments)
borehole_n_intersections_arr = np.zeros(n_segments, dtype=int)
```

### 4. `sample_borehole()` segment loop 안에 Hudson 계산 추가

현재 loop 안에 이 부분이 있습니다.

```
python
n_dfn_hits = len(seg_joints)

if self._use_drilling_damage():
 mechanical_breaks = self._generate_mechanical_breaks(
 seg_t, segment_length
)
else:
 mechanical_breaks = np.array([])
```

---

**n\_dfn\_hits = len(seg\_joints)** 바로 아래에 추가

```
python
=====
Method 1: Borehole direct RQD
Method 2: Borehole Hudson lambda RQD

둘 다 자연 DFN 절리 교차점 seg_t만 사용한다.
기계적 파쇄나 미세절리는 RQD 방법 비교용 컬럼에는 넣지 않는다.
=====
borehole_n_intersections_arr[i] = n_dfn_hits

rqd_borehole_direct_arr[i] = RQDCalculator.calc_direct(
 seg_t.tolist(),
 segment_length,
 threshold=0.1,
)

lambda_bh = RQDCalculator.calc_lambda_from_count(
 n_intersections=n_dfn_hits,
 total_length=segment_length,
)

lambda_borehole_arr[i] = lambda_bh

rqd_borehole_hudson_arr[i] = RQDCalculator.calc_hudson_from_lambda(
 linear_frequency=lambda_bh,
 threshold=0.1,
)
```

그러면 이 위치는 최종적으로 다음 흐름이 됩니다.

```
python
n_dfn_hits = len(seg_joints)

신규 RQD 비교용 계산
borehole_n_intersections_arr[i] = n_dfn_hits
rqd_borehole_direct_arr[i] = ...
lambda_borehole_arr[i] = ...
```

```

rqd_borehole_hudson_arr[i] = ...

if self._use_drilling_damage():
 ...

```

## 5. sample\_borehole() return dict에 신규 key 추가

현재 return dict 마지막 쪽이 이런 형태입니다.

```

python
return {
 'x': np.arange(x_start, x_end) * self.domain.dx,
 'borehole_name': self.BH_NAMES[bh_idx] if bh_idx < 3 else f'BH-{bh_idx}',
 'RQD': rqd_arr,
 'Jn': jn_arr,
 'Jr': jr_arr,
 'Ja': ja_arr,
 'Jw': jw_arr,
 'SRF': srf_arr,
 'Q': q_arr,
 'Qprime': qp_arr,
 '_joint_count': joint_count_arr,
 '_intersections': intersections,
 '_joint_objects': unique_hit_joints,
 '_joint_ids': unique_hit_joint_ids,
 '_joint_ids_set': set(unique_hit_joint_ids),
}

```

아래처럼 신규 key를 추가

```

python
return {
 'x': np.arange(x_start, x_end) * self.domain.dx,
 'borehole_name': self.BH_NAMES[bh_idx] if bh_idx < 3 else f'BH-{bh_idx}',

 # 기존 호환용
 # practice mode에서는 기계적 파쇄/미세절리/회수율 보정이 반영될 수 있음.
 # pure mode에서는 RQD_borehole_direct와 사실상 동일해짐.
 'RQD': rqd_arr,

 'Jn': jn_arr,
 'Jr': jr_arr,
 'Ja': ja_arr,
 'Jw': jw_arr,
 'SRF': srf_arr,
 'Q': q_arr,
 'Qprime': qp_arr,

 # =====
 # RQD 방법 비교용 신규 컬럼

```

```
=====
'RQD_borehole_direct': rqd_borehole_direct_arr,
'lambda_borehole': lambda_borehole_arr,
'RQD_borehole_hudson': rqd_borehole_hudson_arr,
'borehole_n_intersections': borehole_n_intersections_arr,

기존/보조 내부 정보
'_joint_count': joint_count_arr,
'_intersections': intersections,
'_joint_objects': unique_hit_joints,
'_joint_ids': unique_hit_joint_ids,
'_joint_ids_set': set(unique_hit_joint_ids),
}
```

## 6. `sample_face()` 전체 대체

여기가 가장 중요합니다.

기존 `sample_face()`는 다음 문제를 갖고 있습니다.

text

1. `_multi_direction_face_rqd()` 호출
2. 여러 방향 scanline 평균
3. x방향 scanline 포함 가능
4. `rqd_mean`, `rqd_min_dir`, `rqd_practical` 구조 사용

이제 이것을 다음 구조로 바꿉니다.

text

1. `face_joints` 찾기
2. 중앙 수평 scanline RQD 계산
3. Jv 기반 RQD 계산
4. 대표 face RQD는 scanline RQD로 설정
5. 기존 Q/Qprime 호환 구조 유지

기존 `sample_face()` 함수 전체를 아래 코드로 대체

python

```
=====
막장면 샘플링 - 현실적 중앙 수평 scanline + Jv 기반 RQD
=====
def sample_face(self, x_idx: int,
 n_scanlines: int = 8,
 scan_length: float = None) -> Dict:
 """
 터널 막장면 샘플링.
```

수정 후 원칙:

- 굴진면 RQD는 노출된 face 위에서만 계산한다.
- x방향, 즉 굴진 방향 scanline은 사용하지 않는다.
- multi-direction scanline 평균은 사용하지 않는다.
- 중앙부 수평 y방향 scanline 하나만 사용한다.
- 별도로 Jv 기반 RQD를 계산해 face volumetric reference로 저장한다.

n\_scanlines, scan\_length는 기존 호출부 호환을 위해 남겨두지만,  
더 이상 multi-direction scanline에는 사용하지 않는다.

```
"""
face_x = (x_idx + 0.5) * self.domain.dx

1. 막장면과 교차 또는 근접 노출되는 절리 찾기

face_joints = self._find_joints_intersecting_face(face_x)

2. Method 3: 굴진면 중앙 수평 scanline RQD

scanline:
x = face_x
y = center_y - R ~ center_y + R
z = center_z
direction = [0, 1, 0]
#
절대 x방향 scanline을 사용하지 않는다.

face_scan = self.rqd_calculator.rqd_at_face_horizontal_scanline(
 face_x=face_x,
 tunnel_radius=self.radius,
 y_center=self.center_y,
 z_center=self.center_z,
 threshold=0.1,
)

rqd_face_scanline = float(face_scan["RQD_face_scanline"])
lambda_face_scanline = float(face_scan["lambda_face_scanline"])
rqd_face_scanline_hudson = float(face_scan["RQD_face_scanline_hudson"])
face_scanline_n_intersections =
int(face_scan["face_scanline_n_intersections"])

3. Method 4: Jv 기반 RQD

현재는 joint set 정의에서 global Jv 또는 P32-derived Jv_proxy를 계산한다.
density_type == P32인 case에서는 엄밀한 현장 Jv라기보다는
P32-derived Jv proxy로 해석해야 한다.

jv_info = RQDCalculator.calc_jv_proxy_from_joint_sets(
```

```

 self.case.joint_config.joint_sets,
 prefer_p32=True,
)

 jv_face = float(jv_info["jv"])
 jv_method = jv_info["method"]
 rqd_face_jv = float(RQDCalculator.calc_from_jv(jv_face))

 # 보수적 face RQD 후보
 rqd_face_conservative = float(min(rqd_face_scanline, rqd_face_jv))

 # -----
 # 4. Face 대표 RQD 설정
 # -----
 # 기존 코드 호환을 위해 sample_face()의 대표 RQD는 scanline RQD로 둔다.
 #
 # 중요:
 # 이 값은 더 이상 multi-direction average가 아니다.
 # 굴진면 중앙 수평 단일 scanline RQD이다.
 # -----
 rqd_reference = rqd_face_scanline

 # -----
 # 5. face에 노출된 절리에서 Jr, Ja 수집
 # -----
 if len(face_joints) > 0:
 jr_raw = np.array([j.Jr for j in face_joints])
 ja_raw = np.array([j.Ja for j in face_joints])

 if self._use_strength_scaling():
 jr_values = np.clip(jr_raw * self.JR_SCALE_FACTOR_FACE, 0.5, 4.0)
 ja_values = np.clip(ja_raw * self.JA_SCALE_FACTOR_FACE, 0.75, 20.0)
 else:
 jr_values = np.clip(jr_raw, 0.5, 4.0)
 ja_values = np.clip(ja_raw, 0.75, 20.0)

 jr_mean = float(np.mean(jr_values))
 ja_mean = float(np.mean(ja_values))
 jr_std = float(np.std(jr_values))
 ja_std = float(np.std(ja_values))
 else:
 global_jr = np.mean([js.Jr_mean for js in
self.case.joint_config.joint_sets])
 global_ja = np.mean([js.Ja_mean for js in
self.case.joint_config.joint_sets])

 jr_mean = float(global_jr)
 ja_mean = float(global_ja)
 jr_std = 0.0
 ja_std = 0.0
 jr_values = np.array([])
 ja_values = np.array([])

```

```

6. Jn, Jw, SRF: 터널 단면 평균

cy, cz = self.center_y_idx, self.center_z_idx
r_cells = self.radius / self.domain.dy

face_points = []
jw_list, srf_list = [], []

for j in range(self.domain.ny):
 for k in range(self.domain.nz):
 if np.sqrt((j - cy) ** 2 + (k - cz) ** 2) <= r_cells:
 face_points.append((j, k))

 if x_idx < self.domain.nx:
 jw_list.append(self.domain.fields['Jw'][x_idx, j, k])
 srf_list.append(self.domain.fields['SRF'][x_idx, j, k])

jw_mean = float(np.mean(jw_list)) if jw_list else
self.case.joint_config.Jw_mean
srf_mean = float(np.mean(srf_list)) if srf_list else
self.case.joint_config.SRF_mean
jn_val = self.case.joint_config.Jn

7. Q, Qprime 계산

기존 호환 대표값은 scanline RQD 기반으로 계산한다.
Jv 기반 Qprime도 별도 key로 저장한다.

Q_mean = QCalculator.calc_Q(
 rqd_reference,
 jn_val,
 jr_mean,
 ja_mean,
 jw_mean,
 srf_mean,
)

Qp_mean = QCalculator.calc_Qprime(
 rqd_reference,
 jn_val,
 jr_mean,
 ja_mean,
)

Q_from_jv = QCalculator.calc_Q(
 rqd_face_jv,
 jn_val,
 jr_mean,
 ja_mean,
 jw_mean,

```



```

 srf_mean,
)

 Qp_from_jv = QCalculator.calc_Qprime(
 rqd_face_jv,
 jn_val,
 jr_mean,
 ja_mean,
)

 Q_from_conservative = QCalculator.calc_Q(
 rqd_face_conservative,
 jn_val,
 jr_mean,
 ja_mean,
 jw_mean,
 srf_mean,
)

 Qp_from_conservative = QCalculator.calc_Qprime(
 rqd_face_conservative,
 jn_val,
 jr_mean,
 ja_mean,
)

8. 셀별 배열 생성

n_points = len(face_points)

rqd_arr = np.full(n_points, rqd_reference)
jr_arr = np.full(n_points, jr_mean)
ja_arr = np.full(n_points, ja_mean)
jn_arr = np.full(n_points, jn_val)
jw_arr = np.array(jw_list) if jw_list else np.full(n_points, jw_mean)
srf_arr = np.array(srf_list) if srf_list else np.full(n_points, srf_mean)

q_arr = np.array([
 QCalculator.calc_Q(
 rqd_arr[i],
 jn_arr[i],
 jr_arr[i],
 ja_arr[i],
 jw_arr[i],
 srf_arr[i],
)
 for i in range(n_points)
])

qp_arr = np.array([
 QCalculator.calc_Qprime(
 rqd_arr[i],
 jn_arr[i],

```

```

 jr_arr[i],
 ja_arr[i],
)
 for i in range(n_points)
])

9. 기존 _rqd_by_direction 호환용 구조

이제 direction은 단 하나, [0,1,0]뿐이다.

rqd_by_direction_compat = {
 'all_rqd': np.array([rqd_face_scanline]),
 'direction_means': np.array([rqd_face_scanline]),
 'scan_directions': [np.array([0.0, 1.0, 0.0])],
 'n_scanlines_total': 1,
 'method': 'face_single_horizontal_scanline',
}

return {
 'x_idx': x_idx,
 'x_coord': face_x,
 'points': face_points,

 # 기존 호환용 대표값
 # 이제 이 RQD는 multi-direction 평균이 아니라
 # 중앙 수평 face scanline RQD이다.
 'RQD': rqd_arr,
 'Jn': jn_arr,
 'Jr': jr_arr,
 'Ja': ja_arr,
 'Jw': jw_arr,
 'SRF': srf_arr,
 'Q': q_arr,
 'Qprime': qp_arr,

 # =====
 # RQD 방법 비교용 신규 key
 # =====
 'RQD_face_scanline': rqd_face_scanline,
 'lambda_face_scanline': lambda_face_scanline,
 'RQD_face_scanline_hudson': rqd_face_scanline_hudson,
 'face_scanline_n_intersections': face_scanline_n_intersections,

 'Jv_face': jv_face,
 'Jv_method': jv_method,
 'Jv_contributions': jv_info.get("contributions", []),
 'RQD_face_jv': rqd_face_jv,

 'RQD_face_conservative': rqd_face_conservative,

 # Q/Qprime의 estimator별 값

```

```

'Q_face_scanline': Q_mean,
'Qp_face_scanline': Qp_mean,
'Q_face_jv': Q_from_jv,
'Qp_face_jv': Qp_from_jv,
'Q_face_conservative': Q_from_conservative,
'Qp_face_conservative': Qp_from_conservative,

scanline geometry 기록
'face_scanline_origin': face_scan["origin"],
'face_scanline_direction': face_scan["direction"],
'face_scanline_length': face_scan["length"],
'face_scanline_intersection_positions':
face_scan["intersection_positions"],

'stats': {
 'RQD': {
 'mean': rqd_reference,
 'median': rqd_reference,
 'std': 0.0,
 'min': rqd_reference,
 'max': rqd_reference,
 'n_points': n_points,
 },
 'Jn': {
 'mean': jn_val,
 'median': jn_val,
 'std': 0.0,
 'min': jn_val,
 'max': jn_val,
 'n_points': n_points,
 },
 'Jr': {
 'mean': jr_mean,
 'median': jr_mean,
 'std': jr_std,
 'min': float(np.min(jr_values)) if len(jr_values) > 0 else
jr_mean,
 'max': float(np.max(jr_values)) if len(jr_values) > 0 else
jr_mean,
 'n_points': n_points,
 },
 'Ja': {
 'mean': ja_mean,
 'median': ja_mean,
 'std': ja_std,
 'min': float(np.min(ja_values)) if len(ja_values) > 0 else
ja_mean,
 'max': float(np.max(ja_values)) if len(ja_values) > 0 else
ja_mean,
 'n_points': n_points,
 },
 'Jw': {
 'mean': jw_mean,

```

```

 'median': float(np.median(jw_arr)) if len(jw_arr) > 0 else
jw_mean,
 'std': float(np.std(jw_arr)) if len(jw_arr) > 0 else 0.0,
 'min': float(np.min(jw_arr)) if len(jw_arr) > 0 else jw_mean,
 'max': float(np.max(jw_arr)) if len(jw_arr) > 0 else jw_mean,
 'n_points': n_points,
 },
 'SRF': {
 'mean': srf_mean,
 'median': float(np.median(srf_arr)) if len(srf_arr) > 0 else
srf_mean,
 'std': float(np.std(srf_arr)) if len(srf_arr) > 0 else 0.0,
 'min': float(np.min(srf_arr)) if len(srf_arr) > 0 else srf_mean,
 'max': float(np.max(srf_arr)) if len(srf_arr) > 0 else srf_mean,
 'n_points': n_points,
 },
 'Q': {
 'mean': float(np.mean(q_arr)) if len(q_arr) > 0 else Q_mean,
 'median': float(np.median(q_arr)) if len(q_arr) > 0 else Q_mean,
 'std': float(np.std(q_arr)) if len(q_arr) > 0 else 0.0,
 'min': float(np.min(q_arr)) if len(q_arr) > 0 else Q_mean,
 'max': float(np.max(q_arr)) if len(q_arr) > 0 else Q_mean,
 'n_points': n_points,
 },
 'Qprime': {
 'mean': float(np.mean(qp_arr)) if len(qp_arr) > 0 else Qp_mean,
 'median': float(np.median(qp_arr)) if len(qp_arr) > 0 else
Qp_mean,
 'std': float(np.std(qp_arr)) if len(qp_arr) > 0 else 0.0,
 'min': float(np.min(qp_arr)) if len(qp_arr) > 0 else Qp_mean,
 'max': float(np.max(qp_arr)) if len(qp_arr) > 0 else Qp_mean,
 'n_points': n_points,
 },
},
},

기존 내부 호환 key
'_face_joints_count': len(face_joints),
'_rqd_by_direction': rqd_by_direction_compat,
'_rqd_mean': rqd_face_scanline,
'_rqd_min_direction': rqd_face_scanline,
'_rqd_practical': rqd_face_scanline,
'_face_joints': face_joints,
'_face_joint_ids': [id(j) for j in face_joints],
'_face_joint_ids_set': set(id(j) for j in face_joints),

신규 내부 key
'_RQD_face_scanline': rqd_face_scanline,
'_lambda_face_scanline': lambda_face_scanline,
'_RQD_face_scanline_hudson': rqd_face_scanline_hudson,
'_face_scanline_n_intersections': face_scanline_n_intersections,
'_Jv_face': jv_face,
'_Jv_method': jv_method,
'_RQD_face_jv': rqd_face_jv,
'_RQD_face_conservative': rqd_face_conservative,

```

```

 '_Qp_face_scanline': Qp_mean,
 '_Qp_face_jv': Qp_from_jv,
 '_Qp_face_conservative': Qp_from_conservative,
 }

```

## 7. `_multi_direction_face_rqd()` 대체

기존 `_multi_direction_face_rqd()`는 반드시 더 이상 x방향을 포함하면 안 됩니다.

현재 기존 함수 안에는 이 부분이 있습니다.

```

python
scan_directions.append(np.array([1.0, 0.0, 0.0]))

```

이것은 제거되어야 합니다.

가장 안전한 방법은 `_multi_direction_face_rqd()` 전체를 **legacy wrapper**로 바꾸는 것입니다. 즉, 누군가 실수로 이 함수를 호출해도 다방향 scan을 하지 않고 중앙 수평 scanline만 반환하게 합니다.

## 기존 `_multi_direction_face_rqd()` 전체를 아래 코드로 대체

```

python
=====
내부: 기존 다방향 face RQD 함수의 legacy wrapper
=====
def _multi_direction_face_rqd(self, face_x: float,
 n_scanlines: int = 8,
 scan_length: float = 10.0) -> Dict:
 """
 Legacy wrapper.

 기존 구현은 막장면에서 여러 방향 scanline을 평균하고,
 심지어 x방향, 즉 굴진 방향 scanline까지 포함했다.

 수정 후 원칙:
 - face에서는 x방향 scanline 금지
 - multi-direction 평균 금지
 - 중앙 수평 y방향 scanline 하나만 사용

 따라서 이 함수가 호출되더라도 중앙 수평 scanline 결과만 반환한다.
 """
 face_scan = self.rqd_calculator.rqd_at_face_horizontal_scanline(
 face_x=face_x,
 tunnel_radius=self.radius,
 y_center=self.center_y,
 z_center=self.center_z,

```

```

 threshold=0.1,
)

 rqd_val = float(face_scan["RQD_face_scanline"])

 return {
 'all_rqd': np.array([rqd_val]),
 'direction_means': np.array([rqd_val]),
 'scan_directions': [np.array([0.0, 1.0, 0.0])],
 'n_scanlines_total': 1,
 'method': 'legacy_wrapper_single_horizontal_face_scanline',
 'lambda_face_scanline': float(face_scan["lambda_face_scanline"]),
 'RQD_face_scanline_hudson': float(face_scan["RQD_face_scanline_hudson"]),
 'face_scanline_n_intersections':
int(face_scan["face_scanline_n_intersections"]),
 'intersection_positions': face_scan["intersection_positions"],
 }

```

이렇게 하면 기존 코드 어딘가에서 `_multi_direction_face_rqd()`를 실수로 호출해도 더 이상 문제가 되는 x방향 scan이 들어가지 않습니다.

## 8. 중복 함수 정리 권장

현재 `tunnel.py`에는 아래 함수들이 두 번 정의되어 있습니다.

```

python
_generate_mechanical_breaks()
_generate_micro_fractures()
_calc_core_recovery()

```

파일 중간에 한 번 있고, 아래쪽 내부 유틸리티 부분에 다시 있습니다.

Python에서는 아래쪽에 정의된 함수가 위쪽 정의를 덮어씁니다.

현재 두 구현이 거의 같아 큰 문제는 없지만, 정리하는 것이 좋습니다.

### 권장

앞쪽 또는 뒤쪽 중 하나만 남기면 됩니다.

가장 간단한 방법:

```

text
앞쪽의 세 함수는 그대로 두고,
아래쪽 "내부 유틸리티"에 중복으로 있는 세 함수는 삭제

```

또는 반대로 해도 됩니다.

다만 지금 당장 실행에 치명적인 문제는 아니므로, 우선은 나중에 정리해도 됩니다.

---

## 9. `_empty_borehole_result()`에 신규 key 추가 권장

현재 빈 결과는 다음과입니다.

```
python
def _empty_borehole_result(self, bh_idx, x_start, x_end):
 """빈 시추공 결과"""
 return {
 'x': np.array([]),
 'borehole_name': self.BH_NAMES[bh_idx] if bh_idx < 3 else f'BH-{bh_idx}',
 'RQD': np.array([]), 'Jn': np.array([]),
 'Jr': np.array([]), 'Ja': np.array([]),
 'Jw': np.array([]), 'SRF': np.array([]),
 'Q': np.array([]), 'Qprime': np.array([]),
 }
```

이 함수도 아래처럼 대체 권장

```
python
def _empty_borehole_result(self, bh_idx, x_start, x_end):
 """빈 시추공 결과"""
 return {
 'x': np.array([]),
 'borehole_name': self.BH_NAMES[bh_idx] if bh_idx < 3 else f'BH-{bh_idx}',

 'RQD': np.array([]),
 'Jn': np.array([]),
 'Jr': np.array([]),
 'Ja': np.array([]),
 'Jw': np.array([]),
 'SRF': np.array([]),
 'Q': np.array([]),
 'Qprime': np.array([]),

 # RQD 방법 비교용 빈 배열
 'RQD_borehole_direct': np.array([]),
 'lambda_borehole': np.array([]),
 'RQD_borehole_hudson': np.array([]),
 'borehole_n_intersections': np.array([], dtype=int),
 }
```

---

## 10. 이번 수정 후 `tunnel.py`에서 생성되는 RQD method

수정 후 `sample_borehole()`는 다음 값을 생성합니다.

text

```
RQD_borehole_direct
lambda_borehole
RQD_borehole_hudson
borehole_n_intersections
```

수정 후 `sample_face()`는 다음 값을 생성합니다.

```
text
RQD_face_scanline
lambda_face_scanline
RQD_face_scanline_hudson
face_scanline_n_intersections

Jv_face
Jv_method
RQD_face_jv

RQD_face_conservative
```

따라서 RQD 4방식 비교가 가능해집니다.

```
text
1. RQD_borehole_direct
2. RQD_borehole_hudson
3. RQD_face_scanline
4. RQD_face_jv
```

---

## 11. 중요한 해석상 변화

기존:

```
text
sample_face()['RQD'] = 다방향 face scanline 평균 기반 RQD
```

수정 후:

```
text
sample_face()['RQD'] = 중앙 수평 단일 face scanline RQD
```

즉, 기존 RQD key는 호환을 위해 유지하지만 의미가 바뀝니다.

따라서 나중에 논문/보고서에서는 이렇게 명시해야 합니다.

```
text
굴진면 RQD는 노출된 굴진면 중앙부를 수평으로 가로지르는 단일 scanline에서 산정하였다.
기존의 다방향 scanline 평균이나 굴진 방향 scanline은 사용하지 않았다.
```

---

## 12. 아직 다음 단계에서 해야 할 것



이번 수정만으로 tunnel.py는 값을 만들 수 있습니다.  
하지만 CSV로 저장되려면 다음 파일도 수정해야 합니다.

```
text
src/core/comparison.py
```

거기에서 아래 key를 face row에 추가해야 합니다.

```
text
RQD_borehole_direct
RQD_borehole_hudson
lambda_borehole
borehole_n_intersections

RQD_face_scanline
lambda_face_scanline
RQD_face_scanline_hudson
face_scanline_n_intersections

Jv_face
Jv_method
RQD_face_jv
RQD_face_conservative
```

즉, 다음 단계는 comparison.py 수정입니다.

---

### 13. 이번 수정의 핵심 한 줄

이번 tunnel.py 수정의 핵심은 이것입니다.

```
text
시추공은 x방향 직접법/Hudson법으로 평가하고,
굴진면은 x방향을 완전히 제거한 중앙 수평 scanline과 Jv 기반 RQD로 평가한다.
```

이제 RQD 수준에서 다음 질문을 검증할 수 있습니다.

```
text
수평시추공 RQD는 실제 굴진면에서 현실적으로 관찰 가능한 RQD를 얼마나 신뢰성 있게 예측하는가?
GPT-5.5
```

11. 중요한 해석상의 변화는 어떻게 수정하라고 하는지 모르겠는데. 수정전 표현은 없는데



맞습니다. 제가 말한 **“11. 중요한 해석상 변화”**는 코드에서 꼭 찾아서 바꿔야 하는 특정 문장이 있다는 뜻이 아니라, **코드 주석/docstring과 결과 해석에서 의미가 바뀐다는 주의사항**이었습니다.

즉, 현재 코드에 정확히 이런 문장이 있어서 바꾸라는 뜻은 아니었습니다.

text

```
sample_face()['RQD'] = 다방향 face scanline 평균 기반 RQD
```

이건 제가 설명을 위해 쓴 표현입니다. 혼란스럽게 말했네요.

실제로 수정해야 할 것은 다음 세 군데 정도입니다.

---

## 1. tunnel.py 맨 위 파일 설명 주석 수정

현재 tunnel.py 맨 위에는 이런 주석이 있습니다.

python

```
"""
```

터널 기하학, 시추공 샘플링, 막장면 샘플링

★ 수정: 격자값 읽기 → DFN 직접 교차 기반 샘플링

- 시추공: 1D 라인(5cm 직경) × DFN → 방향 편향 + 소규모 샘플링
- 막장면: 2D 원형면(10m) × DFN → 다방향 스캔 + 대규모 샘플링

```
"""
```

여기서 마지막 줄이 이제 틀립니다.

text

```
막장면: 2D 원형면(10m) × DFN → 다방향 스캔 + 대규모 샘플링
```

이 부분을 바꿔야 합니다.

---

이걸로 대체하세요

python

```
"""
```

터널 기하학, 시추공 샘플링, 굴진면 샘플링

DFN 직접 교차 기반 샘플링:

- 시추공:
  - 굴착 전 수평시추공을 x방향 1D line으로 샘플링한다.
  - Deere 직접법 RQD와 Hudson/Priest-Hudson lambda 기반 RQD를 계산한다.
- 굴진면:
  - 굴착 후 노출된 face plane(x = face\_x) 위에서만 RQD를 계산한다.

```
현실적인 현장 mapping을 반영하여 중앙 수평 y방향 단일 scanline만 사용한다.
x방향, 즉 굴진 방향 face scanline과 multi-direction 평균은 사용하지 않는다.
```

```
- Jv:
 절리군 spacing 또는 P32-derived Jv proxy를 이용하여 Palmström형 RQD를 계산한다.
"""
```

이게 제가 말한 “해석상 변화”를 코드 주석에 반영하는 가장 직접적인 수정입니다.

---

## 2. `sample_face()` docstring 수정

기존 `sample_face()` docstring에는 이런 설명이 있습니다.

```
python
"""
터널 막장면 2D 원형 단면 샘플링 - DFN 직접 교차

막장면의 현실성:
- 2D 관찰이므로 미세절리도 관찰 가능
- 다방향 스캔 → 모든 방향의 절리 포착
- 지질기사의 보수적 평가 경향
"""
```

이건 이제 연구 방향과 맞지 않습니다.

특히 이 줄은 삭제/수정해야 합니다.

```
text
다방향 스캔 → 모든 방향의 절리 포착
```

왜냐하면 이제 다방향 스캔을 쓰지 않기로 했기 때문입니다.

---

## 새 `sample_face()`로 전체 대체했다면 이미 해결됨

제가 앞에서 드린 새 `sample_face()` 코드에는 docstring이 이렇게 되어 있습니다.

```
python
"""
터널 막장면 샘플링.

수정 후 원칙:
- 굴진면 RQD는 노출된 face 위에서만 계산한다.
- x방향, 즉 굴진 방향 scanline은 사용하지 않는다.
```

- multi-direction scanline 평균은 사용하지 않는다.
- 중앙부 수평 y방향 scanline 하나만 사용한다.
- 별도로 Jv 기반 RQD를 계산해 face volumetric reference로 저장한다.

n\_scanlines, scan\_length는 기존 호출부 호환을 위해 남겨두지만,  
더 이상 multi-direction scanline에는 사용하지 않는다.  
"""

따라서 제가 준 새 `sample_face()` 전체 대체 코드를 넣었다면 이 부분은 추가로 수정할 필요 없습니다.

하지만 아직 전체 대체를 안 했다면, 기존 docstring만이라도 위 내용으로 바꾸면 됩니다.

### 3. `_multi_direction_face_rqd()` docstring 수정 또는 함수 대체

기존 함수 설명은 다음과 같습니다.

```
python
"""
막장면에서 다방향 스캔라인 RQD (미세절리 포함)

막장면에서는:
- 2D 노출면이므로 미세절리도 육안 관찰 가능
- 시추공보다 더 많은 절리를 관찰
- 따라서 미세절리 승수를 높게 적용
"""
```

이것도 이제 맞지 않습니다.

제가 앞에서 제안한 대로 `_multi_direction_face_rqd()` 전체를 legacy wrapper로 바꿨다면 이미 해결됩니다.

새 설명은 이렇습니다.

```
python
"""
Legacy wrapper.

기존 구현은 막장면에서 여러 방향 scanline을 평균하고,
심지어 x방향, 즉 굴진 방향 scanline까지 포함했다.

수정 후 원칙:
- face에서는 x방향 scanline 금지
- multi-direction 평균 금지
- 중앙 수평 y방향 scanline 하나만 사용
"""
```

따라서 이 함수가 호출되더라도 중앙 수평 scanline 결과만 반환한다.

```
"""
```

#### 4. 그러니까 실제로 바뀌어야 하는 곳은?

정리하면 “11번 해석상 변화” 때문에 실제 수정할 곳은 다음입니다.

반드시 수정 권장

##### A. 파일 맨 위 docstring

```
python
"""
터널 기하학...
...
막장면: 2D 원형면(10m) × DFN → 다방향 스캔 + 대규모 샘플링
"""
```

이걸 새 설명으로 교체.

##### B. `sample_face()` 함수 전체

기존 다방향 스캔 기반 `sample_face()`를 제가 준 새 `sample_face()`로 교체.

이렇게 하면 docstring도 같이 수정됩니다.

##### C. `_multi_direction_face_rqd()` 함수 전체

기존 다방향 함수는 제가 준 legacy wrapper로 교체.

#### 5. 수정하지 않아도 되는 것

다음 표현은 코드에 실제로 없으면 찾을 필요 없습니다.

```
text
sample_face()['RQD'] = 다방향 face scanline 평균 기반 RQD
```

이건 제가 설명용으로 쓴 말입니다.

수정 대상 문장이 아닙니다.

---

## 6. 가장 간단한 체크리스트

수정 후 `tunnel.py`에서 아래 표현이 남아 있으면 안 됩니다.

```
text
다방향 스캔 → 모든 방향의 절리 포착
다방향 스캔라인 RQD
x방향 scanline을 face RQD에 포함
scan_directions.append(np.array([1.0, 0.0, 0.0]))
```

특히 코드에서 이 줄은 없어야 합니다.

```
python
scan_directions.append(np.array([1.0, 0.0, 0.0]))
```

만약 `_multi_direction_face_rqd()`를 legacy wrapper로 바꿨다면 이 줄은 사라집니다.

---

## 7. 결론

제가 말한 “중요한 해석상 변화”는 코드 안의 특정 한 줄을 고치라는 의미가 아니라, 다음 사실을 코드 주석과 결과 해석에서 명확히 하라는 뜻이었습니다.

```
text
기존 sample_face의 RQD는 다방향 synthetic face RQD였지만,
수정 후 sample_face의 RQD는 현실적인 중앙 수평 단일 scanline RQD이다.
```

따라서 실제 수정은 다음 세 가지만 하면 됩니다.

```
text
1. tunnel.py 맨 위 docstring 수정
2. sample_face()를 새 버전으로 교체
3. _multi_direction_face_rqd()를 legacy wrapper로 교체
GPT-5.5
```

```
comparison.py : """
시추공 vs 막장면 Q값 비교 분석
```

- DFN 직접 교차 기반 비교
  - missed fracture metrics 포함
- ```
"""
```

```
import numpy as np
from typing import Dict, List, Optional
from .tunnel_old import Tunnel
```

```
class ComparisonEngine:
```

```
    """시추공 데이터 vs 막장면 데이터 비교"""
```

```
    text
```

```
    def __init__(self, tunnel: Tunnel):
```

```
        self.tunnel = tunnel
```

```
        self.domain = tunnel.domain
```

```
    def compare_at_face(self, face_x_idx: int, borehole_window: int = 5) -> Dict:
```

```
        """
```

```
        특정 굴진면 위치에서 시추공 vs 막장면 비교
```

```
        Args:
```

```
            face_x_idx: 굴진면 x 격자 인덱스
```

```
            borehole_window: 시추공에서 굴진면 주변 ±window 범위 평균
```

```
        """
```

```
        # -----
```

```
        # 1) Face sample
```

```
        # -----
```

```
        face = self.tunnel.sample_face(face_x_idx)
```

```
        face_joint_ids = face.get("_face_joint_ids_set", set())
```

```
        face_joint_count = len(face_joint_ids)
```

```
        face_joints = face.get('_face_joints', [])
```

```
        face_orientation_bias = self._calc_orientation_bias(face_joints)
```

```
        # -----
```

```
        # 2) Borehole samples around the face
```

```
        # -----
```

```
        x_start = max(0, face_x_idx - borehole_window)
```

```
        x_end = min(self.domain.nx, face_x_idx + borehole_window + 1)
```

```
        # 그 다음 geometry / density metrics 계산
```

```
        face_area = np.pi * (self.tunnel.radius ** 2)
```

```
        window_length = (x_end - x_start) * self.domain.dx
```

```
        face_fracture_density = face_joint_count / face_area if face_area > 0 else 0.0
```

```
        borehole_results = []
```

```
        borehole_joint_sets = []
```

```
        borehole_joint_objects_all = []
```

```
        for bh_idx in range(len(self.tunnel.borehole_positions)):
```

```
            bh = self.tunnel.sample_borehole(bh_idx, x_start, x_end)
```

```
            bh_joint_ids = bh.get('_joint_ids_set', set())
```

```
            bh_joint_objects = bh.get('_joint_objects', [])
```

```
            borehole_joint_sets.append(bh_joint_ids)
```

```
            borehole_joint_objects_all.extend(bh_joint_objects)
```

```
            common_ids = face_joint_ids & bh_joint_ids
```

```
            missed_ids = face_joint_ids - bh_joint_ids
```

```

    bh_orientation_bias = self._calc_orientation_bias(bh_joint_objects)

    bh_summary = {
        'name': bh['borehole_name'],
        'Q_mean': float(np.mean(bh['Q'])) if len(bh['Q']) > 0 else 0.0,
        'Q_median': float(np.median(bh['Q'])) if len(bh['Q']) > 0 else 0.0,
        'Qprime_mean': float(np.mean(bh['Qprime'])) if len(bh['Qprime']) > 0 else
0.0,
        'Qprime_median': float(np.median(bh['Qprime'])) if len(bh['Qprime']) > 0
else 0.0,

        'face_fracture_count': face_joint_count,
        'bh_fracture_count': len(bh_joint_ids),
        'common_fracture_count': len(common_ids),
        'missed_fracture_count': len(missed_ids),
        'miss_ratio': float(len(missed_ids) / face_joint_count) if
face_joint_count > 0 else 0.0,
        'detection_ratio': float(len(common_ids) / face_joint_count) if
face_joint_count > 0 else 0.0,

        'bh_window_length': window_length,
        'bh_fracture_frequency': float(len(bh_joint_ids) / window_length) if
window_length > 0 else 0.0,
        'orientation_bias_bh': bh_orientation_bias,
    }

    for param in ['RQD', 'Jn', 'Jr', 'Ja', 'Jw', 'SRF']:
        arr = bh[param]
        bh_summary[f'{param}_mean'] = float(np.mean(arr)) if len(arr) > 0 else 0.0

    borehole_results.append(bh_summary)

# loop 후: 3개 시추공 전체가 관측한 unique joint objects
unique_bh_union_joint_objects = list(
    {id(j): j for j in borehole_joint_objects_all}.values()
)

# orientation bias 계산
face_joints = face.get('_face_joints', [])
face_orientation_bias = self._calc_orientation_bias(face_joints)
bh_union_orientation_bias =
self._calc_orientation_bias(unique_bh_union_joint_objects)
orientation_bias_gap = face_orientation_bias - bh_union_orientation_bias

# -----
# 3) Aggregate borehole stats
# -----
bh_Q_means = [b["Q_mean"] for b in borehole_results]
bh_Qp_means = [b["Qprime_mean"] for b in borehole_results]

face_Q_mean = face["stats"]["Q"]["mean"]
face_Qp_mean = face["stats"]["Qprime"]["mean"]

```



```

# 3개 시추공의 절리 검출 합집합
bh_union_ids = (
    set().union(*borehole_joint_sets) if borehole_joint_sets else set()
)
common_union_ids = face_joint_ids & bh_union_ids
missed_union_ids = face_joint_ids - bh_union_ids
bh_union_fracture_frequency = (
    float(len(bh_union_ids) / window_length) if window_length > 0 else 0.0
)
common_fracture_density = (
    float(len(common_union_ids) / face_area) if face_area > 0 else 0.0
)
missed_fracture_density = (
    float(len(missed_union_ids) / face_area) if face_area > 0 else 0.0
)

bh_fracture_frequency_mean = (
    float(np.mean([b["bh_fracture_frequency"] for b in borehole_results]))
    if borehole_results
    else 0.0
)

bh_union_joint_objects = {}
for bh_idx in range(len(self.tunnel.borehole_positions)):
    pass

# -----
# 4) Build comparison dict
# -----
comparison = {
    "face_x": face_x_idx,
    "face_x_coord": face["x_coord"],
    "face_stats": face["stats"],
    "boreholes": borehole_results,
    # Q comparison
    "Q_face_mean": face_Q_mean,
    "Q_face_median": face["stats"]["Q"]["median"],
    "Q_borehole_mean": float(np.mean(bh_Q_means)) if bh_Q_means else 0.0,
    "Q_borehole_each": bh_Q_means,
    "Q_ratio": (
        float(np.mean(bh_Q_means) / max(face_Q_mean, 1e-10))
        if bh_Q_means
        else 0.0
    ),
    "Q_difference": (
        float(np.mean(bh_Q_means) - face_Q_mean) if bh_Q_means else 0.0
    ),
    "Q_log_ratio": (
        float(
            np.log10(max(np.mean(bh_Q_means), 1e-10))
            - np.log10(max(face_Q_mean, 1e-10))
        )
        if bh_Q_means
        else 0.0
    )
}

```

```

    ),
    # Q' comparison
    "Qp_face_mean": face_Qp_mean,
    "Qp_face_median": face["stats"]["Qprime"]["median"],
    "Qp_borehole_mean": float(np.mean(bh_Qp_means)) if bh_Qp_means else 0.0,
    "Qp_borehole_each": bh_Qp_means,
    "Qp_ratio": (
        float(np.mean(bh_Qp_means) / max(face_Qp_mean, 1e-10))
        if bh_Qp_means
        else 0.0
    ),
    "Qp_difference": (
        float(np.mean(bh_Qp_means) - face_Qp_mean) if bh_Qp_means else 0.0
    ),
    # face diagnostic info
    "_face_joints_count": face.get("_face_joints_count", 0),
    "_face_rqd_practical": face.get("_rqd_practical", 0),
    "_face_rqd_mean": face.get("_rqd_mean", 0),
    "_face_rqd_min_dir": face.get("_rqd_min_direction", 0),
    # missed fracture metrics (union across boreholes)
    "_face_joint_count": face_joint_count,
    "_bh_union_fracture_count": len(bh_union_ids),
    "_common_union_fracture_count": len(common_union_ids),
    "_missed_union_fracture_count": len(missed_union_ids),
    "_miss_ratio_union": (
        float(len(missed_union_ids) / face_joint_count)
        if face_joint_count > 0
        else 0.0
    ),
    "_detection_ratio_union": (
        float(len(common_union_ids) / face_joint_count)
        if face_joint_count > 0
        else 0.0
    ),
    # per borehole comparisons
    "per_borehole": [],
    # fracture density metrics
    "_face_area": face_area,
    "_window_length": window_length,
    "_face_fracture_density": face_fracture_density,
    "_bh_fracture_frequency_mean": bh_fracture_frequency_mean,
    "_bh_union_fracture_frequency": bh_union_fracture_frequency,
    "_common_fracture_density": common_fracture_density,
    "_missed_fracture_density": missed_fracture_density,
    # orientation bias metrics
    "_face_orientation_bias": face_orientation_bias,
    "_bh_union_orientation_bias": bh_union_orientation_bias,
    "_orientation_bias_gap": orientation_bias_gap,
}

for bh in borehole_results:
    comparison["per_borehole"].append(
        {
            "name": bh["name"],

```

```

        "Q_ratio": float(bh["Q_mean"] / max(face_Q_mean, 1e-10)),
        "Qp_ratio": float(bh["Qprime_mean"] / max(face_Qp_mean, 1e-10)),
        "miss_ratio": bh["miss_ratio"],
        "detection_ratio": bh["detection_ratio"],
    }
)

return comparison

def progressive_comparison(
    self, face_positions: List[int] = None, borehole_window: int = 3
) -> List[Dict]:
    """
    터널 굴진에 따른 연속 비교
    """
    if face_positions is None:
        step = max(1, int(10 / self.domain.dx))
        face_positions = list(range(step, self.domain.nx - step, step))

    results = []
    for i, x_idx in enumerate(face_positions):
        if i % max(1, len(face_positions) // 10) == 0:
            print(
                f" 비교 진행: {i+1}/{len(face_positions)} "
                f"(x={x_idx * self.domain.dx:.0f}m)"
            )
        comp = self.compare_at_face(x_idx, borehole_window)
        results.append(comp)

    return results

def summary_statistics(self, comparisons: List[Dict]) -> Dict:
    """비교 결과 종합 통계"""
    if not comparisons:
        return {}

    Q_ratios = [c["Q_ratio"] for c in comparisons]
    Qp_ratios = [c["Qp_ratio"] for c in comparisons]
    Q_log_ratios = [c["Q_log_ratio"] for c in comparisons]

    face_Q = [c["Q_face_mean"] for c in comparisons]
    bh_Q = [c["Q_borehole_mean"] for c in comparisons]
    face_Qp = [c["Qp_face_mean"] for c in comparisons]
    bh_Qp = [c["Qp_borehole_mean"] for c in comparisons]

    face_density = [c.get("_face_fracture_density", 0.0) for c in comparisons]
    bh_freq_mean = [c.get("_bh_fracture_frequency_mean", 0.0) for c in comparisons]
    bh_union_freq = [
        c.get("_bh_union_fracture_frequency", 0.0) for c in comparisons
    ]

    corr_Q = float(np.corrcoef(bh_Q, face_Q)[0, 1]) if len(comparisons) > 2 else 0.0
    corr_Qp = (

```

```

        float(np.corrcoef(bh_Qp, face_Qp)[0, 1]) if len(comparisons) > 2 else 0.0
    )

def match_rate(ratios, lo, hi):
    return sum(1 for r in ratios if lo <= r <= hi) / len(ratios) * 100

miss_ratio_union = [c.get("_miss_ratio_union", 0.0) for c in comparisons]
detection_ratio_union = [
    c.get("_detection_ratio_union", 0.0) for c in comparisons
]

# orientation bias metrics
face_obi = [c.get('_face_orientation_bias', 0.0) for c in comparisons]
bh_union_obi = [c.get('_bh_union_orientation_bias', 0.0) for c in comparisons]
obi_gap = [c.get('_orientation_bias_gap', 0.0) for c in comparisons]

return {
    "n_faces": len(comparisons),
    "Q_ratio_mean": float(np.mean(Q_ratios)),
    "Q_ratio_std": float(np.std(Q_ratios)),
    "Q_ratio_median": float(np.median(Q_ratios)),
    "Q_log_ratio_mean": float(np.mean(Q_log_ratios)),
    "Q_log_ratio_std": float(np.std(Q_log_ratios)),
    "Q_correlation": corr_Q,
    "Q_match_strict": match_rate(Q_ratios, 0.8, 1.25),
    "Q_match_normal": match_rate(Q_ratios, 0.67, 1.5),
    "Q_match_loose": match_rate(Q_ratios, 0.5, 2.0),
    "Qp_ratio_mean": float(np.mean(Qp_ratios)),
    "Qp_ratio_std": float(np.std(Qp_ratios)),
    "Qp_ratio_median": float(np.median(Qp_ratios)),
    "Qp_correlation": corr_Qp,
    "Qp_match_strict": match_rate(Qp_ratios, 0.8, 1.25),
    "Qp_match_normal": match_rate(Qp_ratios, 0.67, 1.5),
    "Qp_match_loose": match_rate(Qp_ratios, 0.5, 2.0),
    "Q_log_RMSE": float(np.sqrt(np.mean(np.array(Q_log_ratios) ** 2))),
    "diagnostic": {
        "face_rqd_mean": float(
            np.mean([c.get("_face_rqd_mean", 0) for c in comparisons])
        ),
        "face_rqd_min_dir": float(
            np.mean([c.get("_face_rqd_min_dir", 0) for c in comparisons])
        ),
        "bh_rqd_mean": float(
            np.mean(
                [
                    np.mean([b["RQD_mean"] for b in c["boreholes"]])
                    for c in comparisons
                ]
            )
        ),
        "face_joints_mean": float(
            np.mean([c.get("_face_joints_count", 0) for c in comparisons])
        ),
    },
}

```

```

        # missed fracture metrics
        "miss_ratio_union_mean": float(np.mean(miss_ratio_union)),
        "miss_ratio_union_std": float(np.std(miss_ratio_union)),
        "detection_ratio_union_mean": float(np.mean(detection_ratio_union)),
        "detection_ratio_union_std": float(np.std(detection_ratio_union)),
        # fracture density metrics
        "face_fracture_density_mean": float(np.mean(face_density)),
        "face_fracture_density_std": float(np.std(face_density)),
        "bh_fracture_frequency_mean": float(np.mean(bh_freq_mean)),
        "bh_fracture_frequency_std": float(np.std(bh_freq_mean)),
        "bh_union_fracture_frequency_mean": float(np.mean(bh_union_freq)),
        "bh_union_fracture_frequency_std": float(np.std(bh_union_freq)),
        # orientation bias metrics
        'face_orientation_bias_mean': float(np.mean(face_obi)),
        'face_orientation_bias_std': float(np.std(face_obi)),
        'bh_union_orientation_bias_mean': float(np.mean(bh_union_obi)),
        'bh_union_orientation_bias_std': float(np.std(bh_union_obi)),
        'orientation_bias_gap_mean': float(np.mean(obi_gap)),
        'orientation_bias_gap_std': float(np.std(obi_gap)),
    },
}

def print_report(self, comparisons: List[Dict], summary: Dict = None):
    """텍스트 비교 리포트 출력"""
    if summary is None:
        summary = self.summary_statistics(comparisons)

    print("\n" + "=" * 120)
    print(
        "    시추공 vs 막장면 Q값 비교 분석 보고서 (DFN 직접 교차 + Missed Fractures)"
    )
    print("=" * 120)

    print(
        f"\n{'Chainage':>10} | {'Q_Face':>10} | {'Q_BH':>10} | "
        f"{'Q_Ratio':>10} | {'Qp_Face':>10} | {'Qp_BH':>10} | "
        f"{'Qp_Ratio':>10} | {'Miss_U':>8} | {'Detect_U':>8}"
    )
    print("-" * 120)

    for c in comparisons:
        print(
            f"{c['face_x_coord']:>10.1f} | "
            f"{c['Q_face_mean']:>10.3f} | "
            f"{c['Q_borehole_mean']:>10.3f} | "
            f"{c['Q_ratio']:>10.3f} | "
            f"{c['Qp_face_mean']:>10.3f} | "
            f"{c['Qp_borehole_mean']:>10.3f} | "
            f"{c['Qp_ratio']:>10.3f} | "
            f"{c.get('_miss_ratio_union', 0.0):>8.3f} | "
            f"{c.get('_detection_ratio_union', 0.0):>8.3f}"
        )

    print("\n" + "-" * 80)

```

```

print(" 🇰🇷 종합 통계")
print("-" * 80)
print(f" 비교 단면 수: {summary['n_faces']}")
print(
    f" Q 비율 평균: {summary['Q_ratio_mean']:.3f} ±
{summary['Q_ratio_std']:.3f}"
)
print(
    f" Q' 비율 평균: {summary['Qp_ratio_mean']:.3f} ±
{summary['Qp_ratio_std']:.3f}"
)
print(f" Q 상관계수 (r): {summary['Q_correlation']:.4f}")
print(f" Q' 상관계수 (r): {summary['Qp_correlation']:.4f}")
print(f" Q log-RMSE: {summary['Q_log_RMSE']:.4f}")

print(f"\n 📏 일치율")
print(
    f" Strict (±25%) Q={summary['Q_match_strict']:.1f}%
Q'={summary['Qp_match_strict']:.1f}%"
)
print(
    f" Normal (±50%) Q={summary['Q_match_normal']:.1f}%
Q'={summary['Qp_match_normal']:.1f}%"
)
print(
    f" Loose (±100%) Q={summary['Q_match_loose']:.1f}%
Q'={summary['Qp_match_loose']:.1f}%"
)

diag = summary.get("diagnostic", {})
print(f"\n 📊 진단 정보")
print(f" 막장면 RQD (평균): {diag.get('face_rqd_mean', 0):.1f}")
print(f" 막장면 RQD (최소방향): {diag.get('face_rqd_min_dir', 0):.1f}")
print(f" 시추공 RQD (평균): {diag.get('bh_rqd_mean', 0):.1f}")
print(f" 막장면 교차절리 (평균): {diag.get('face_joints_mean', 0):.1f}")
print(
    f" Miss ratio union (평균): {diag.get('miss_ratio_union_mean', 0):.3f} ±
{diag.get('miss_ratio_union_std', 0):.3f}"
)
print(
    f" Detect ratio union (평균): {diag.get('detection_ratio_union_mean', 0):.3f}
± {diag.get('detection_ratio_union_std', 0):.3f}"
)

print("=" * 120)

def comparisons_to_face_rows(
    self, comparisons: List[Dict], case_name: str, seed: int
) -> List[Dict]:
    """

```

```

comparison 결과 → face-level row list
1 row = 1 face position
"""
rows = []

for c in comparisons:
    row = {
        "case_name": case_name,
        "seed": seed,
        "face_x_idx": c.get("face_x"),
        "face_x_coord": c.get("face_x_coord"),
        # face summary
        "Q_face_mean": c.get("Q_face_mean"),
        "Q_face_median": c.get("Q_face_median"),
        "Qp_face_mean": c.get("Qp_face_mean"),
        "Qp_face_median": c.get("Qp_face_median"),
        # borehole aggregated summary
        "Q_borehole_mean": c.get("Q_borehole_mean"),
        "Qp_borehole_mean": c.get("Qp_borehole_mean"),
        # agreement metrics
        "Q_ratio": c.get("Q_ratio"),
        "Q_difference": c.get("Q_difference"),
        "Q_log_ratio": c.get("Q_log_ratio"),
        "Qp_ratio": c.get("Qp_ratio"),
        "Qp_difference": c.get("Qp_difference"),
        # diagnostics
        "face_joints_count": c.get("_face_joints_count"),
        "face_rq_d_practical": c.get("_face_rq_d_practical"),
        "face_rq_d_mean": c.get("_face_rq_d_mean"),
        "face_rq_d_min_dir": c.get("_face_rq_d_min_dir"),
        # missed fracture metrics
        "face_joint_count": c.get("_face_joint_count"),
        "bh_union_fracture_count": c.get("_bh_union_fracture_count"),
        "common_union_fracture_count": c.get("_common_union_fracture_count"),
        "missed_union_fracture_count": c.get("_missed_union_fracture_count"),
        "miss_ratio_union": c.get("_miss_ratio_union"),
        "detection_ratio_union": c.get("_detection_ratio_union"),
        # fracture density metrics
        "face_area": c.get("_face_area"),
        "window_length": c.get("_window_length"),
        "face_fracture_density": c.get("_face_fracture_density"),
        "bh_fracture_frequency_mean": c.get("_bh_fracture_frequency_mean"),
        "bh_union_fracture_frequency": c.get("_bh_union_fracture_frequency"),
        "common_fracture_density": c.get("_common_fracture_density"),
        "missed_fracture_density": c.get("_missed_fracture_density"),
        "face_orientation_bias": c.get("_face_orientation_bias"),
        "bh_union_orientation_bias": c.get("_bh_union_orientation_bias"),
        "orientation_bias_gap": c.get("_orientation_bias_gap"),
    }

    q_ratio = c.get("Q_ratio", None)
    qp_ratio = c.get("Qp_ratio", None)

    row["Q_match_strict"] = int(q_ratio is not None and 0.8 <= q_ratio <= 1.25)

```

```

row["Q_match_normal"] = int(q_ratio is not None and 0.67 <= q_ratio <= 1.5)
row["Q_match_loose"] = int(q_ratio is not None and 0.5 <= q_ratio <= 2.0)

row["Qp_match_strict"] = int(
    qp_ratio is not None and 0.8 <= qp_ratio <= 1.25
)
row["Qp_match_normal"] = int(
    qp_ratio is not None and 0.67 <= qp_ratio <= 1.5
)
row["Qp_match_loose"] = int(qp_ratio is not None and 0.5 <= qp_ratio <= 2.0)

# face_stats flatten
face_stats = c.get("face_stats", {})
for param, stats in face_stats.items():
    for stat_name, stat_val in stats.items():
        row[f"face_{param}_{stat_name}"] = stat_val

rows.append(row)

return rows

def comparisons_to_borehole_rows(
    self, comparisons: List[Dict], case_name: str, seed: int
) -> List[Dict]:
    """
    comparison 결과 → borehole-level row list
    1 row = 1 face position × 1 borehole
    """
    rows = []

    for c in comparisons:
        face_x = c.get("face_x")
        face_x_coord = c.get("face_x_coord")
        q_face_mean = c.get("Q_face_mean")
        qp_face_mean = c.get("Qp_face_mean")

        boreholes = c.get("boreholes", [])
        per_bh = c.get("per_borehole", [])
        per_bh_map = {b["name"]: b for b in per_bh}

        for bh in boreholes:
            name = bh.get("name")
            bh_ratio_info = per_bh_map.get(name, {})

            row = {
                "case_name": case_name,
                "seed": seed,
                "face_x_idx": face_x,
                "face_x_coord": face_x_coord,
                "borehole_name": name,
                # face reference
                "Q_face_mean": q_face_mean,
                "Qp_face_mean": qp_face_mean,
                # borehole values
            }

```



```

        "Q_bh_mean": bh.get("Q_mean"),
        "Q_bh_median": bh.get("Q_median"),
        "Qp_bh_mean": bh.get("Qprime_mean"),
        "Qp_bh_median": bh.get("Qprime_median"),
        "RQD_bh_mean": bh.get("RQD_mean"),
        "Jn_bh_mean": bh.get("Jn_mean"),
        "Jr_bh_mean": bh.get("Jr_mean"),
        "Ja_bh_mean": bh.get("Ja_mean"),
        "Jw_bh_mean": bh.get("Jw_mean"),
        "SRF_bh_mean": bh.get("SRF_mean"),
        # per-borehole ratios
        "Q_ratio_bh_to_face": bh_ratio_info.get("Q_ratio"),
        "Qp_ratio_bh_to_face": bh_ratio_info.get("Qp_ratio"),
        # missed fracture metrics
        "face_fracture_count": bh.get("face_fracture_count"),
        "bh_fracture_count": bh.get("bh_fracture_count"),
        "common_fracture_count": bh.get("common_fracture_count"),
        "missed_fracture_count": bh.get("missed_fracture_count"),
        "miss_ratio": bh.get("miss_ratio"),
        "detection_ratio": bh.get("detection_ratio"),
        "bh_window_length": bh.get("bh_window_length"),
        "bh_fracture_frequency": bh.get("bh_fracture_frequency"),
        "face_fracture_density": c.get("_face_fracture_density"),
        "bh_union_fracture_frequency": c.get(
            "_bh_union_fracture_frequency"
        ),
        "common_fracture_density": c.get("_common_fracture_density"),
        "missed_fracture_density": c.get("_missed_fracture_density"),
        "orientation_bias_bh": bh.get("orientation_bias_bh"),
        "face_orientation_bias": c.get("_face_orientation_bias"),
        "orientation_bias_gap_to_face": (
            c.get("_face_orientation_bias") - bh.get("orientation_bias_bh")
            if bh.get("orientation_bias_bh") is not None and
c.get("_face_orientation_bias") is not None
            else None
        ),
    }

    q_ratio = row["Q_ratio_bh_to_face"]
    qp_ratio = row["Qp_ratio_bh_to_face"]

    row["Q_match_strict"] = int(
        q_ratio is not None and 0.8 <= q_ratio <= 1.25
    )
    row["Q_match_normal"] = int(
        q_ratio is not None and 0.67 <= q_ratio <= 1.5
    )
    row["Q_match_loose"] = int(
        q_ratio is not None and 0.5 <= q_ratio <= 2.0
    )

    row["Qp_match_strict"] = int(
        qp_ratio is not None and 0.8 <= qp_ratio <= 1.25
    )

```

```

        row["Qp_match_normal"] = int(
            qp_ratio is not None and 0.67 <= qp_ratio <= 1.5
        )
        row["Qp_match_loose"] = int(
            qp_ratio is not None and 0.5 <= qp_ratio <= 2.0
        )

        rows.append(row)

    return rows

def _calc_orientation_bias(self, joints, borehole_dir=np.array([1.0, 0.0, 0.0])) ->
float:
    """
    절리 집합에 대한 borehole orientation bias 계산

    OBI = 1 - mean(|n · d_bh|)
    """
    if not joints:
        return 0.0

    d = borehole_dir / np.linalg.norm(borehole_dir)
    vals = []

    for j in joints:
        n = j.normal / np.linalg.norm(j.normal)
        vals.append(abs(np.dot(n, d)))

    return float(1.0 - np.mean(vals))

```



좋습니다. `comparison.py`는 이제 **새 RQD 4방식 컬럼을 CSV로 뽑아주는 연결부**가 되어야 합니다.

목표는 다음입니다.

text

1. Borehole direct RQD 저장
2. Borehole Hudson RQD 저장
3. Face scanline RQD 저장
4. Face Jv RQD 저장
5. face_rows CSV에 위 컬럼들이 들어가도록 수정
6. borehole_rows CSV에도 per-borehole RQD 컬럼이 들어가도록 수정

아래 순서대로 수정하면 됩니다.

1. import 수정

현재:

```
python
from .tunnel_old import Tunnel
```

이 부분은 현재 수정 중인 `tunnel.py`를 쓰려면 바꾸는 게 맞습니다.

대체

```
python
from .tunnel import Tunnel
```

즉, 상단 import는 이렇게 됩니다.

```
python
import numpy as np
from typing import Dict, List, Optional
from .tunnel import Tunnel
```

만약 프로젝트에서 의도적으로 `tunnel_old.py`를 쓰는 구조가 아니라면, 반드시 `.tunnel`로 바꾸는 게 맞습니다.

2. `compare_at_face()` 안의 `bh_summary`에 RQD method 컬럼 추가

현재 `bh_summary`는 이 부분입니다.

```
python
bh_summary = {
    'name': bh['borehole_name'],
    'Q_mean': float(np.mean(bh['Q'])) if len(bh['Q']) > 0 else 0.0,
    'Q_median': float(np.median(bh['Q'])) if len(bh['Q']) > 0 else 0.0,
    'Qprime_mean': float(np.mean(bh['Qprime'])) if len(bh['Qprime']) > 0 else 0.0,
    'Qprime_median': float(np.median(bh['Qprime'])) if len(bh['Qprime']) > 0 else 0.0,
```

여기에 새 RQD method를 추가해야 합니다.

이 부분으로 대체

```
python
bh_summary = {
    'name': bh['borehole_name'],

    # 기존 Q/Q' 요약
    'Q_mean': float(np.mean(bh['Q'])) if len(bh['Q']) > 0 else 0.0,
    'Q_median': float(np.median(bh['Q'])) if len(bh['Q']) > 0 else 0.0,
    'Qprime_mean': float(np.mean(bh['Qprime'])) if len(bh['Qprime']) > 0 else 0.0,
    'Qprime_median': float(np.median(bh['Qprime'])) if len(bh['Qprime']) > 0 else 0.0,

    # =====
```

```

# RQD 방법 비교용 borehole 요약
# -----
# RQD_borehole_direct:
# 자연 DFN 절리 교차점만 사용한 borehole Deere 직접법 RQD
#
# RQD_borehole_hudson:
# 같은 borehole window에서  $\lambda = N/L$  기반 Hudson RQD
# =====
'RQD_borehole_direct_mean': (
    float(np.mean(bh.get('RQD_borehole_direct', [])))
    if len(bh.get('RQD_borehole_direct', [])) > 0 else 0.0
),
'RQD_borehole_direct_median': (
    float(np.median(bh.get('RQD_borehole_direct', [])))
    if len(bh.get('RQD_borehole_direct', [])) > 0 else 0.0
),
'RQD_borehole_hudson_mean': (
    float(np.mean(bh.get('RQD_borehole_hudson', [])))
    if len(bh.get('RQD_borehole_hudson', [])) > 0 else 0.0
),
'RQD_borehole_hudson_median': (
    float(np.median(bh.get('RQD_borehole_hudson', [])))
    if len(bh.get('RQD_borehole_hudson', [])) > 0 else 0.0
),
'lambda_borehole_mean': (
    float(np.mean(bh.get('lambda_borehole', [])))
    if len(bh.get('lambda_borehole', [])) > 0 else 0.0
),
'borehole_n_intersections_sum': (
    int(np.sum(bh.get('borehole_n_intersections', [])))
    if len(bh.get('borehole_n_intersections', [])) > 0 else 0
),

'face_fracture_count': face_joint_count,
'bh_fracture_count': len(bh_joint_ids),
'common_fracture_count': len(common_ids),
'missed_fracture_count': len(missed_ids),
'miss_ratio': float(len(missed_ids) / face_joint_count) if face_joint_count > 0
else 0.0,
'detection_ratio': float(len(common_ids) / face_joint_count) if face_joint_count >
0 else 0.0,

'bh_window_length': window_length,
'bh_fracture_frequency': float(len(bh_joint_ids) / window_length) if window_length
> 0 else 0.0,
'orientation_bias_bh': bh_orientation_bias,
}

```

그리고 바로 아래 기존 loop는 그대로 둡니다.

```

python
for param in ['RQD', 'Jn', 'Jr', 'Ja', 'Jw', 'SRF']:
    arr = bh[param]

```

```
bh_summary[f'{param}_mean'] = float(np.mean(arr)) if len(arr) > 0 else 0.0
```

이렇게 하면 기존 RQD_mean도 계속 남고, 새 method별 RQD도 추가됩니다.

3. borehole aggregate 변수 추가

현재 이 부분이 있습니다.

```
python
bh_Q_means = [b["Q_mean"] for b in borehole_results]
bh_Qp_means = [b["Qprime_mean"] for b in borehole_results]

face_Q_mean = face["stats"]["Q"]["mean"]
face_Qp_mean = face["stats"]["Qprime"]["mean"]
```

이 바로 아래에 추가하세요.

```
python
# -----
# RQD method aggregate stats
# -----
bh_RQD_direct_means = [
    b["RQD_borehole_direct_mean"] for b in borehole_results
]

bh_RQD_hudson_means = [
    b["RQD_borehole_hudson_mean"] for b in borehole_results
]

bh_lambda_means = [
    b["lambda_borehole_mean"] for b in borehole_results
]

bh_n_intersections_sums = [
    b["borehole_n_intersections_sum"] for b in borehole_results
]

RQD_borehole_direct_mean = (
    float(np.mean(bh_RQD_direct_means)) if bh_RQD_direct_means else 0.0
)

RQD_borehole_hudson_mean = (
    float(np.mean(bh_RQD_hudson_means)) if bh_RQD_hudson_means else 0.0
)

lambda_borehole_mean = (
    float(np.mean(bh_lambda_means)) if bh_lambda_means else 0.0
)

borehole_n_intersections_total = (
    int(np.sum(bh_n_intersections_sums)) if bh_n_intersections_sums else 0
)
```

```

)

# Face-side RQD methods from sample_face()
RQD_face_scanline = face.get("RQD_face_scanline", face.get("_RQD_face_scanline", 0.0))
lambda_face_scanline = face.get("lambda_face_scanline",
face.get("_lambda_face_scanline", 0.0))
RQD_face_scanline_hudson = face.get(
    "RQD_face_scanline_hudson",
    face.get("_RQD_face_scanline_hudson", 0.0),
)
face_scanline_n_intersections = face.get(
    "face_scanline_n_intersections",
    face.get("_face_scanline_n_intersections", 0),
)

Jv_face = face.get("Jv_face", face.get("_Jv_face", 0.0))
Jv_method = face.get("Jv_method", face.get("_Jv_method", None))
RQD_face_jv = face.get("RQD_face_jv", face.get("_RQD_face_jv", 0.0))
RQD_face_conservative = face.get(
    "RQD_face_conservative",
    face.get("_RQD_face_conservative", min(RQD_face_scanline, RQD_face_jv)),
)

```

4. comparison dict에 RQD method 컬럼 추가

현재 `comparison = { ... }` 안에 Q, Q' comparison들이 있습니다.

그 안에, 예를 들어 `Qp_difference` 다음 또는 face diagnostic info 전에 아래 블록을 추가하세요.

추가할 코드

```

python

# =====
# RQD method comparison
# =====
"RQD_borehole_direct_mean": RQD_borehole_direct_mean,
"RQD_borehole_direct_each": bh_RQD_direct_means,

"RQD_borehole_hudson_mean": RQD_borehole_hudson_mean,
"RQD_borehole_hudson_each": bh_RQD_hudson_means,

"lambda_borehole_mean": lambda_borehole_mean,
"lambda_borehole_each": bh_lambda_means,
"borehole_n_intersections_total": borehole_n_intersections_total,

"RQD_face_scanline": float(RQD_face_scanline),
"lambda_face_scanline": float(lambda_face_scanline),
"RQD_face_scanline_hudson": float(RQD_face_scanline_hudson),
"face_scanline_n_intersections": int(face_scanline_n_intersections),

"Jv_face": float(Jv_face),
"Jv_method": Jv_method,

```

```

"RQD_face_jv": float(RQD_face_jv),
"RQD_face_conservative": float(RQD_face_conservative),

# Pairwise RQD differences: predictor - target
"RQD_diff_bh_direct_minus_face_scanline": (
    float(RQD_borehole_direct_mean - RQD_face_scanline)
),
"RQD_diff_bh_hudson_minus_face_scanline": (
    float(RQD_borehole_hudson_mean - RQD_face_scanline)
),
"RQD_diff_bh_direct_minus_face_jv": (
    float(RQD_borehole_direct_mean - RQD_face_jv)
),
"RQD_diff_bh_hudson_minus_face_jv": (
    float(RQD_borehole_hudson_mean - RQD_face_jv)
),

"RQD_absdiff_bh_direct_face_scanline": (
    float(abs(RQD_borehole_direct_mean - RQD_face_scanline))
),
"RQD_absdiff_bh_hudson_face_scanline": (
    float(abs(RQD_borehole_hudson_mean - RQD_face_scanline))
),
"RQD_absdiff_bh_direct_face_jv": (
    float(abs(RQD_borehole_direct_mean - RQD_face_jv))
),
"RQD_absdiff_bh_hudson_face_jv": (
    float(abs(RQD_borehole_hudson_mean - RQD_face_jv))
),

```

즉, `comparison`에는 이제 RQD 분석에 필요한 거의 모든 컬럼이 들어갑니다.

5. 기존 face diagnostic info는 의미를 조금 바꾸기

현재 `comparison dict` 안에 이 부분이 있습니다.

```

python
# face diagnostic info
"_face_joints_count": face.get("_face_joints_count", 0),
"_face_rqd_practical": face.get("_rqd_practical", 0),
"_face_rqd_mean": face.get("_rqd_mean", 0),
"_face_rqd_min_dir": face.get("_rqd_min_direction", 0),

```

이제 `face_rqd_min_dir`는 더 이상 “여러 방향 중 최소 방향”이 아닙니다.
호환성을 위해 남겨도 되지만, 모두 scanline RQD와 같은 값이 됩니다.

이 정도로 바꾸는 걸 추천합니다

```

python
# face diagnostic info

```

```
# 주의:
# 수정 후 _face_rqd_mean, _face_rqd_practical, _face_rqd_min_dir는
# 더 이상 multi-direction face RQD가 아니다.
# 모두 중앙 수평 단일 face scanline RQD와 호환되도록 유지하는 값이다.
_face_joints_count": face.get("_face_joints_count", 0),
_face_rqd_practical": face.get("_rqd_practical", RQD_face_scanline),
_face_rqd_mean": face.get("_rqd_mean", RQD_face_scanline),
_face_rqd_min_dir": face.get("_rqd_min_direction", RQD_face_scanline),
```

6. per_borehole에도 RQD ratio 추가

현재 아래 부분이 있습니다.

```
python
for bh in borehole_results:
    comparison["per_borehole"].append(
        {
            "name": bh["name"],
            "Q_ratio": float(bh["Q_mean"] / max(face_Q_mean, 1e-10)),
            "Qp_ratio": float(bh["Qprime_mean"] / max(face_Qp_mean, 1e-10)),
            "miss_ratio": bh["miss_ratio"],
            "detection_ratio": bh["detection_ratio"],
        }
    )
```

이 부분으로 대체

```
python
for bh in borehole_results:
    comparison["per_borehole"].append(
        {
            "name": bh["name"],

            "Q_ratio": float(bh["Q_mean"] / max(face_Q_mean, 1e-10)),
            "Qp_ratio": float(bh["Qprime_mean"] / max(face_Qp_mean, 1e-10)),

            # RQD ratios against face scanline
            "RQD_direct_ratio_to_face_scanline": float(
                bh["RQD_borehole_direct_mean"] / max(RQD_face_scanline, 1e-10)
            ),
            "RQD_hudson_ratio_to_face_scanline": float(
                bh["RQD_borehole_hudson_mean"] / max(RQD_face_scanline, 1e-10)
            ),

            # RQD differences
            "RQD_direct_diff_to_face_scanline": float(
                bh["RQD_borehole_direct_mean"] - RQD_face_scanline
            ),
            "RQD_hudson_diff_to_face_scanline": float(
                bh["RQD_borehole_hudson_mean"] - RQD_face_scanline
            ),
```



```

        "miss_ratio": bh["miss_ratio"],
        "detection_ratio": bh["detection_ratio"],
    }
)

```

7. comparisons_to_face_rows()에 RQD 컬럼 추가

현재 `row = { ... }` 안에 diagnostics 부분이 있습니다.

```

python
# diagnostics
"face_joints_count": c.get("_face_joints_count"),
"face_rqd_practical": c.get("_face_rqd_practical"),
"face_rqd_mean": c.get("_face_rqd_mean"),
"face_rqd_min_dir": c.get("_face_rqd_min_dir"),

```

이 바로 아래에 새 RQD method 컬럼을 추가하세요.

추가 코드

```

python
# =====
# RQD method comparison columns
# =====
"RQD_borehole_direct": c.get("RQD_borehole_direct_mean"),
"RQD_borehole_hudson": c.get("RQD_borehole_hudson_mean"),
"lambda_borehole": c.get("lambda_borehole_mean"),
"borehole_n_intersections_total":
c.get("borehole_n_intersections_total"),

"RQD_face_scanline": c.get("RQD_face_scanline"),
"lambda_face_scanline": c.get("lambda_face_scanline"),
"RQD_face_scanline_hudson": c.get("RQD_face_scanline_hudson"),
"face_scanline_n_intersections":
c.get("face_scanline_n_intersections"),

"Jv_face": c.get("Jv_face"),
"Jv_method": c.get("Jv_method"),
"RQD_face_jv": c.get("RQD_face_jv"),
"RQD_face_conservative": c.get("RQD_face_conservative"),

"RQD_diff_bh_direct_minus_face_scanline": c.get(
    "RQD_diff_bh_direct_minus_face_scanline"
),
"RQD_diff_bh_hudson_minus_face_scanline": c.get(
    "RQD_diff_bh_hudson_minus_face_scanline"
),
"RQD_diff_bh_direct_minus_face_jv": c.get(
    "RQD_diff_bh_direct_minus_face_jv"
),
"RQD_diff_bh_hudson_minus_face_jv": c.get(

```

```

        "RQD_diff_bh_hudson_minus_face_jv"
    ),
    "RQD_absdiff_bh_direct_face_scanline": c.get(
        "RQD_absdiff_bh_direct_face_scanline"
    ),
    "RQD_absdiff_bh_hudson_face_scanline": c.get(
        "RQD_absdiff_bh_hudson_face_scanline"
    ),
    "RQD_absdiff_bh_direct_face_jv": c.get(
        "RQD_absdiff_bh_direct_face_jv"
    ),
    "RQD_absdiff_bh_hudson_face_jv": c.get(
        "RQD_absdiff_bh_hudson_face_jv"
    ),
),

```

이렇게 하면 `all_cases_face_rows.csv`에 `rqd_analysis.py`가 기대하는 컬럼들이 들어갑니다.

특히 아래 컬럼이 반드시 생깁니다.

```

text
RQD_borehole_direct
RQD_borehole_hudson
RQD_face_scanline
RQD_face_jv
RQD_face_conservative

```

8. `comparisons_to_borehole_rows()`에도 per-borehole RQD 추가

현재 borehole row에는 이런 부분이 있습니다.

```

python
"RQD_bh_mean": bh.get("RQD_mean"),
"Jn_bh_mean": bh.get("Jn_mean"),

```

이 아래에 추가하세요.

추가 코드

```

python
# RQD method columns per borehole
"RQD_borehole_direct_mean": bh.get("RQD_borehole_direct_mean"),
"RQD_borehole_direct_median":
bh.get("RQD_borehole_direct_median"),
"RQD_borehole_hudson_mean": bh.get("RQD_borehole_hudson_mean"),
"RQD_borehole_hudson_median":
bh.get("RQD_borehole_hudson_median"),
"lambda_borehole_mean": bh.get("lambda_borehole_mean"),
"borehole_n_intersections_sum":
bh.get("borehole_n_intersections_sum"),

```

```

# Face RQD references copied to each borehole row
"RQD_face_scanline": c.get("RQD_face_scanline"),
"RQD_face_jv": c.get("RQD_face_jv"),
"RQD_face_conservative": c.get("RQD_face_conservative"),
"lambda_face_scanline": c.get("lambda_face_scanline"),
"face_scanline_n_intersections":
c.get("face_scanline_n_intersections"),
"Jv_face": c.get("Jv_face"),
"Jv_method": c.get("Jv_method"),

```

그리고 per-borehole ratio 쪽에도 추가하면 좋습니다.

현재:

```

python
"Q_ratio_bh_to_face": bh_ratio_info.get("Q_ratio"),
"Qp_ratio_bh_to_face": bh_ratio_info.get("Qp_ratio"),

```

그 아래에 추가:

```

python
"RQD_direct_ratio_to_face_scanline": bh_ratio_info.get(
    "RQD_direct_ratio_to_face_scanline"
),
"RQD_hudson_ratio_to_face_scanline": bh_ratio_info.get(
    "RQD_hudson_ratio_to_face_scanline"
),
"RQD_direct_diff_to_face_scanline": bh_ratio_info.get(
    "RQD_direct_diff_to_face_scanline"
),
"RQD_hudson_diff_to_face_scanline": bh_ratio_info.get(
    "RQD_hudson_diff_to_face_scanline"
),

```

9. `summary_statistics()` diagnostic 부분도 수정 권장

현재 diagnostic에 이 부분이 있습니다.

```

python
"face_rqd_mean": float(
    np.mean([c.get("_face_rqd_mean", 0) for c in comparisons])
),
"face_rqd_min_dir": float(
    np.mean([c.get("_face_rqd_min_dir", 0) for c in comparisons])
),
"bh_rqd_mean": float(
    np.mean(
        [
            np.mean([b["RQD_mean"] for b in c["boreholes"]])
            for c in comparisons
        ]
    )
)

```

```
)  
,
```

여기서 `face_rqd_min_dir`는 이제 의미가 약해졌습니다.

대신 RQD method summary를 추가하는 게 좋습니다.

이 부분으로 대체 추천

```
python  
"face_rqd_scanline_mean": float(  
    np.mean([c.get("RQD_face_scanline", 0) for c in comparisons])  
),  
"face_rqd_jv_mean": float(  
    np.mean([c.get("RQD_face_jv", 0) for c in comparisons])  
),  
"face_rqd_conservative_mean": float(  
    np.mean([c.get("RQD_face_conservative", 0) for c in comparisons])  
),  
"bh_rqd_direct_mean": float(  
    np.mean([c.get("RQD_borehole_direct_mean", 0) for c in comparisons])  
),  
"bh_rqd_hudson_mean": float(  
    np.mean([c.get("RQD_borehole_hudson_mean", 0) for c in comparisons])  
),  
"lambda_borehole_mean": float(  
    np.mean([c.get("lambda_borehole_mean", 0) for c in comparisons])  
),  
"lambda_face_scanline_mean": float(  
    np.mean([c.get("lambda_face_scanline", 0) for c in comparisons])  
),
```

다만 기존 출력 코드에서 `face_rqd_mean`, `face_rqd_min_dir`, `bh_rqd_mean`을 참조하고 있으므로, 호환성을 위해 아래도 같이 남기는 게 안전합니다.

```
python  
"face_rqd_mean": float(  
    np.mean([c.get("_face_rqd_mean", c.get("RQD_face_scanline", 0)) for c in  
comparisons])  
),  
"face_rqd_min_dir": float(  
    np.mean([c.get("_face_rqd_min_dir", c.get("RQD_face_scanline", 0)) for c in  
comparisons])  
),  
"bh_rqd_mean": float(  
    np.mean(  
        [  
            np.mean([b["RQD_mean"] for b in c["boreholes"]])  
            for c in comparisons  
        ]  
    )  
),
```

즉, 새 값도 추가하고 기존 값도 유지하면 됩니다.

10. `print_report()`의 진단 정보 문구 수정

현재 출력은:

```
python
print(f"   막장면 RQD (평균):           {diag.get('face_rqd_mean', 0):.1f}")
print(f"   막장면 RQD (최소방향):       {diag.get('face_rqd_min_dir', 0):.1f}")
print(f"   시추공 RQD (평균):             {diag.get('bh_rqd_mean', 0):.1f}")
```

이제 “최소방향”은 부적절합니다.

이 부분으로 대체 추천

```
python
print(f"   Face RQD scanline 평균:      {diag.get('face_rqd_scanline_mean',
diag.get('face_rqd_mean', 0)):.1f}")
print(f"   Face RQD Jv 평균:              {diag.get('face_rqd_jv_mean', 0):.1f}")
print(f"   BH RQD direct 평균:              {diag.get('bh_rqd_direct_mean',
diag.get('bh_rqd_mean', 0)):.1f}")
print(f"   BH RQD Hudson 평균:             {diag.get('bh_rqd_hudson_mean', 0):.1f}")
```

11. 선택 사항: `orientation bias`를 RQD 방향성 분석에 맞게 확장

현재 `_calc_orientation_bias()`는 기본 방향이 x방향입니다.

```
python
def _calc_orientation_bias(self, joints, borehole_dir=np.array([1.0, 0.0, 0.0])) ->
float:
```

그리고 정의는:

```
text
OBI = 1 - mean(|n · d|)
```

이건 유지해도 되지만, RQD 차이 원인 분석에는 아래 두 방향이 필요합니다.

```
text
borehole direction = [1,0,0]
face scanline direction = [0,1,0]
```

따라서 나중에 다음 helper를 추가하면 좋습니다.

```
python
```

```
def _calc_orientation_intersection_tendency(
    self,
    joints,
    direction=np.array([1.0, 0.0, 0.0]),
) -> float:
    """
    특정 방향 scanline이 절리들을 얼마나 잘 만날 수 있는지의 단순 지표.

    tendency = mean(|n · d|)

    값이 클수록 해당 방향 line이 절리면을 교차할 가능성이 크다.
    """
    if not joints:
        return 0.0

    d = direction / np.linalg.norm(direction)
    vals = []

    for j in joints:
        n = j.normal / np.linalg.norm(j.normal)
        vals.append(abs(np.dot(n, d)))

    return float(np.mean(vals))
```

그리고 `compare_at_face()` 안에서 나중에 이렇게 쓸 수 있습니다.

```
python
face_tendency_x = self._calc_orientation_intersection_tendency(
    face_joints,
    np.array([1.0, 0.0, 0.0]),
)

face_tendency_y = self._calc_orientation_intersection_tendency(
    face_joints,
    np.array([0.0, 1.0, 0.0]),
)

orientation_tendency_gap_y_minus_x = face_tendency_y - face_tendency_x
```

이건 RQD 차이 원인 분석에 좋지만, 지금 당장 필수는 아닙니다.

12. 최종적으로 `face_rows` CSV에 들어가야 하는 핵심 컬럼

수정 후 `all_cases_face_rows.csv`에 아래가 있으면 성공입니다.

```
text
RQD_borehole_direct
RQD_borehole_hudson
lambda_borehole
borehole_n_intersections_total
```

```

RQD_face_scanline
lambda_face_scanline
RQD_face_scanline_hudson
face_scanline_n_intersections

Jv_face
Jv_method
RQD_face_jv
RQD_face_conservative

RQD_diff_bh_direct_minus_face_scanline
RQD_diff_bh_hudson_minus_face_scanline
RQD_diff_bh_direct_minus_face_jv
RQD_diff_bh_hudson_minus_face_jv

RQD_absdiff_bh_direct_face_scanline
RQD_absdiff_bh_hudson_face_scanline
RQD_absdiff_bh_direct_face_jv
RQD_absdiff_bh_hudson_face_jv

```

이 컬럼들이 있으면 이전에 작성한 `rqd_analysis.py`에서 바로 비교할 수 있습니다.

13. 이번 `comparison.py` 수정의 핵심

이번 수정은 계산 자체를 바꾸는 게 아니라, `tunnel.py`에서 계산된 RQD method들을 **comparison 결과와 CSV로 내보내는 작업**입니다.

즉 역할은 다음입니다.

```

text
tunnel.py:
    RQD 계산

comparison.py:
    RQD 결과 집계
    borehole 평균 계산
    face reference와 비교
    CSV row로 export

```

여기까지 수정하면 다음 단계에서 바로 할 수 있습니다.

```

python
from src.core.rqd_analysis import compare_rqd_methods

df = pd.read_csv("outputs/.../all_cases_face_rows.csv")

results = compare_rqd_methods(df)

```

```
print(results["summary"])
print(results["pairwise"])
print(results["threshold"])
```

GPT-5.5

11번 선택사항은 지금은 채 택하지 않았어. 그럼 다음 단계로 진행했으면 좋겠는데, 먼저 RQD 분석을 진행해봤으면 좋겠어.



Thought Process

좋습니다. 이제 다음 단계는 **RQD 전용 분석을 실제로 돌리는 단계**입니다.

지금 목표는 Q'나 decision usefulness로 바로 가지 않고, 먼저 아래 네 RQD 방법을 비교하는 것입니다.

text

```
1. RQD_borehole_direct
2. RQD_borehole_hudson
3. RQD_face_scanline
4. RQD_face_jv
```

분석 질문은 우선 이렇게 잡으면 됩니다.

text

```
Q1. 시추공 direct RQD는 face scanline RQD와 얼마나 다른가?
Q2. 시추공 Hudson RQD는 face scanline RQD와 얼마나 다른가?
Q3. 시추공 RQD는 Jv 기반 face RQD와 더 잘 맞는가?
Q4. direct 방식과 Hudson 방식 중 어느 것이 face reference를 더 안정적으로 예측하는가?
Q5. RQD threshold 기준에서 false-safe / false-alarm은 어느 정도인가?
```

1. 먼저 실행 순서

수정이 완료되었다면 순서는 이렇습니다.

bash

```
python -m src.cli.run_multicase_decision
```

이걸 실행해서 아래 파일이 생성되어야 합니다.

text

```
outputs/.../all_cases_face_rows.csv
```

그 다음 RQD 분석 스크립트를 실행합니다.

bash


```
python -m src.cli.analyze_rqd_methods --csv outputs/.../all_cases_face_rows.csv --out
outputs/rqd_analysis
```

이제 필요한 분석 스크립트를 하나 추가하면 됩니다.

2. 새 파일 추가

신규 파일:

```
text
src/cli/analyze_rqd_methods.py
```

아래 코드를 그대로 넣으면 됩니다.

3. `src/cli/analyze_rqd_methods.py`

```
python
"""
RQD method analysis CLI.

이 스크립트는 all_cases_face_rows.csv를 읽어서
다음 RQD 방법들을 비교한다.

Predictors:
- RQD_borehole_direct
- RQD_borehole_hudson

Face references:
- RQD_face_scanline
- RQD_face_jv
- RQD_face_conservative

Outputs:
- rqd_summary.csv
- rqd_pairwise.csv
- rqd_threshold_metrics.csv
- rqd_threshold_sweep.csv
- rqd_by_case.csv
- rqd_by_density_regime.csv, 가능한 경우
- rqd_by_orientation_regime.csv, 가능한 경우
"""

import argparse
import os
from pathlib import Path

import numpy as np
```

```

import pandas as pd

from src.core.rqd_analysis import (
    compare_rqd_methods,
    compare_rqd_by_regime,
    rqd_threshold_metrics,
)

REQUIRED_COLUMNS = [
    "RQD_borehole_direct",
    "RQD_borehole_hudson",
    "RQD_face_scanline",
    "RQD_face_jv",
]

def _ensure_output_dir(out_dir: str) -> Path:
    out_path = Path(out_dir)
    out_path.mkdir(parents=True, exist_ok=True)
    return out_path

def _check_columns(df: pd.DataFrame):
    missing = [c for c in REQUIRED_COLUMNS if c not in df.columns]

    if missing:
        print("\n[ERROR] Missing required RQD columns:")
        for c in missing:
            print(f"  - {c}")

        print("\n현재 CSV 컬럼:")
        for c in df.columns:
            print(f"  - {c}")

        raise KeyError(
            "Required RQD columns are missing. "
            "Check tunnel.py and comparison.py export logic."
        )

def _prepare_dataframe(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()

    # 숫자형 변환
    numeric_cols = [
        "RQD_borehole_direct",
        "RQD_borehole_hudson",
        "RQD_face_scanline",
        "RQD_face_jv",
        "RQD_face_conservative",
        "lambda_borehole",
        "lambda_face_scanline",
        "Jv_face",
    ]

```

```

        "face_fracture_density",
        "orientation_bias_gap",
    ]

    for col in numeric_cols:
        if col in df.columns:
            df[col] = pd.to_numeric(df[col], errors="coerce")

    # conservative 컬럼이 없으면 생성
    if "RQD_face_conservative" not in df.columns:
        if "RQD_face_scanline" in df.columns and "RQD_face_jv" in df.columns:
            df["RQD_face_conservative"] = np.minimum(
                df["RQD_face_scanline"],
                df["RQD_face_jv"],
            )

    return df

def _make_quantile_regime(
    df: pd.DataFrame,
    source_col: str,
    out_col: str,
    labels=("low", "mid", "high"),
) -> pd.DataFrame:
    """
    source_col을 분위수 기준으로 low/mid/high regime으로 나눈다.
    값이 충분하지 않으면 생성하지 않는다.
    """
    df = df.copy()

    if source_col not in df.columns:
        return df

    s = pd.to_numeric(df[source_col], errors="coerce")
    valid = s.replace([np.inf, -np.inf], np.nan).dropna()

    if len(valid) < 10:
        return df

    if valid.nunique() < 3:
        return df

    try:
        df[out_col] = pd.qcut(
            s,
            q=3,
            labels=labels,
            duplicates="drop",
        )
    except Exception:
        return df

```

```

    return df

def _run_threshold_sweep(
    df: pd.DataFrame,
    predictor_cols,
    target_cols,
    thresholds,
    cost_false_safe=10.0,
    cost_false_alarm=1.0,
) -> pd.DataFrame:
    rows = []

    for pred in predictor_cols:
        if pred not in df.columns:
            continue

        for target in target_cols:
            if target not in df.columns:
                continue

            for th in thresholds:
                rows.append(
                    rqd_threshold_metrics(
                        df=df,
                        predictor_col=pred,
                        target_col=target,
                        threshold=float(th),
                        cost_false_safe=cost_false_safe,
                        cost_false_alarm=cost_false_alarm,
                    )
                )

    return pd.DataFrame(rows)

def _run_by_case_analysis(df: pd.DataFrame, out_path: Path):
    """
    case별 핵심 pairwise 분석.
    """
    if "case_name" not in df.columns:
        print("[WARN] case_name column not found. Skip by-case analysis.")
        return

    pairs = [
        ("RQD_borehole_direct", "RQD_face_scanline"),
        ("RQD_borehole_hudson", "RQD_face_scanline"),
        ("RQD_borehole_direct", "RQD_face_jv"),
        ("RQD_borehole_hudson", "RQD_face_jv"),
        ("RQD_borehole_direct", "RQD_face_conservative"),
        ("RQD_borehole_hudson", "RQD_face_conservative"),
    ]

    all_rows = []

```

```

for pred, target in pairs:
    if pred not in df.columns or target not in df.columns:
        continue

    res = compare_rq_d_by_regime(
        df=df,
        group_cols=["case_name"],
        predictor_col=pred,
        target_col=target,
    )
    all_rows.append(res)

if all_rows:
    out = pd.concat(all_rows, ignore_index=True)
    out.to_csv(out_path / "rqd_by_case.csv", index=False)
    print(f"[OK] Saved: {out_path / 'rqd_by_case.csv'}")

def _run_regime_analysis(df: pd.DataFrame, out_path: Path):
    """
    density/orientation regime별 분석.

    현재 별도 orientation tendency는 채택하지 않았으므로,
    기존에 CSV에 있는 face_fracture_density, orientation_bias_gap만 사용한다.
    """
    df2 = df.copy()

    df2 = _make_quantile_regime(
        df2,
        source_col="face_fracture_density",
        out_col="density_regime",
        labels=("low_density", "mid_density", "high_density"),
    )

    df2 = _make_quantile_regime(
        df2,
        source_col="orientation_bias_gap",
        out_col="orientation_gap_regime",
        labels=("low_gap", "mid_gap", "high_gap"),
    )

    pairs = [
        ("RQD_borehole_direct", "RQD_face_scanline"),
        ("RQD_borehole_hudson", "RQD_face_scanline"),
        ("RQD_borehole_direct", "RQD_face_jv"),
        ("RQD_borehole_hudson", "RQD_face_jv"),
    ]

    if "density_regime" in df2.columns:
        density_rows = []

        for pred, target in pairs:

```

```

        if pred not in df2.columns or target not in df2.columns:
            continue

        res = compare_rqd_by_regime(
            df=df2,
            group_cols=["density_regime"],
            predictor_col=pred,
            target_col=target,
        )
        density_rows.append(res)

    if density_rows:
        out = pd.concat(density_rows, ignore_index=True)
        out.to_csv(out_path / "rqd_by_density_regime.csv", index=False)
        print(f"[OK] Saved: {out_path / 'rqd_by_density_regime.csv'}")

    if "orientation_gap_regime" in df2.columns:
        orientation_rows = []

        for pred, target in pairs:
            if pred not in df2.columns or target not in df2.columns:
                continue

            res = compare_rqd_by_regime(
                df=df2,
                group_cols=["orientation_gap_regime"],
                predictor_col=pred,
                target_col=target,
            )
            orientation_rows.append(res)

        if orientation_rows:
            out = pd.concat(orientation_rows, ignore_index=True)
            out.to_csv(out_path / "rqd_by_orientation_regime.csv", index=False)
            print(f"[OK] Saved: {out_path / 'rqd_by_orientation_regime.csv'}")

def _print_key_results(summary_df: pd.DataFrame, pairwise_df: pd.DataFrame):
    print("\n" + "=" * 100)
    print("RQD METHOD SUMMARY")
    print("=" * 100)

    if len(summary_df) > 0:
        cols = [
            "column",
            "count",
            "mean",
            "std",
            "min",
            "p10",
            "p25",
            "p50",
            "p75",
            "p90",

```

```

        "max",
    ]
    cols = [c for c in cols if c in summary_df.columns]
    print(summary_df[cols].to_string(index=False))

print("\n" + "=" * 100)
print("RQD PAIRWISE COMPARISON")
print("=" * 100)

if len(pairwise_df) > 0:
    cols = [
        "predictor",
        "target",
        "n",
        "pred_mean",
        "target_mean",
        "bias",
        "mae",
        "rmse",
        "corr",
        "overestimation_rate",
        "underestimation_rate",
    ]
    cols = [c for c in cols if c in pairwise_df.columns]
    print(pairwise_df[cols].to_string(index=False))

def main():
    parser = argparse.ArgumentParser()

    parser.add_argument(
        "--csv",
        type=str,
        required=True,
        help="Path to all_cases_face_rows.csv",
    )

    parser.add_argument(
        "--out",
        type=str,
        default="outputs/rqd_analysis",
        help="Output directory for RQD analysis results",
    )

    parser.add_argument(
        "--thresholds",
        type=float,
        nargs="*",
        default=[50.0, 75.0, 90.0],
        help="RQD thresholds for decision metrics",
    )

    parser.add_argument(
        "--cost-false-safe",

```

```

        type=float,
        default=10.0,
        help="Cost assigned to false-safe error",
    )

    parser.add_argument(
        "--cost-false-alarm",
        type=float,
        default=1.0,
        help="Cost assigned to false-alarm error",
    )

    args = parser.parse_args()

    out_path = _ensure_output_dir(args.out)

    print(f"[INFO] Loading CSV: {args.csv}")
    df = pd.read_csv(args.csv)

    df = _prepare_dataframe(df)
    _check_columns(df)

    predictor_cols = [
        "RQD_borehole_direct",
        "RQD_borehole_hudson",
    ]

    target_cols = [
        "RQD_face_scanline",
        "RQD_face_jv",
        "RQD_face_conservative",
    ]

    # 존재하는 target만 사용
    target_cols = [c for c in target_cols if c in df.columns]

    results = compare_rqd_methods(
        df=df,
        predictor_cols=predictor_cols,
        target_cols=target_cols,
        thresholds=args.thresholds,
        cost_false_safe=args.cost_false_safe,
        cost_false_alarm=args.cost_false_alarm,
    )

    summary_df = results["summary"]
    pairwise_df = results["pairwise"]
    threshold_df = results["threshold"]

    summary_df.to_csv(out_path / "rqd_summary.csv", index=False)
    pairwise_df.to_csv(out_path / "rqd_pairwise.csv", index=False)
    threshold_df.to_csv(out_path / "rqd_threshold_metrics.csv", index=False)

    print(f"[OK] Saved: {out_path / 'rqd_summary.csv'}")

```



```

print(f"[OK] Saved: {out_path / 'rqd_pairwise.csv'}")
print(f"[OK] Saved: {out_path / 'rqd_threshold_metrics.csv'}")

# Threshold sweep
sweep_thresholds = np.arange(10.0, 100.1, 5.0)

sweep_df = _run_threshold_sweep(
    df=df,
    predictor_cols=predictor_cols,
    target_cols=target_cols,
    thresholds=sweep_thresholds,
    cost_false_safe=args.cost_false_safe,
    cost_false_alarm=args.cost_false_alarm,
)

sweep_df.to_csv(out_path / "rqd_threshold_sweep.csv", index=False)
print(f"[OK] Saved: {out_path / 'rqd_threshold_sweep.csv'}")

# Group analyses
_run_by_case_analysis(df, out_path)
_run_regime_analysis(df, out_path)

_print_key_results(summary_df, pairwise_df)

print("\n[DONE] RQD analysis completed.")
print(f"Output directory: {out_path}")

if __name__ == "__main__":
    main()

```

4. 실행 명령

예를 들어 결과 csv가 여기에 있다면:

```

text
outputs/multicase_pure/all_cases_face_rows.csv

```

실행은 이렇게 합니다.

```

bash
python -m src.cli.analyze_rqd_methods \
    --csv outputs/multicase_pure/all_cases_face_rows.csv \
    --out outputs/rqd_analysis_pure

```

Windows PowerShell이면 한 줄로 실행해도 됩니다.

```

bash
python -m src.cli.analyze_rqd_methods --csv
outputs/multicase_pure/all_cases_face_rows.csv --out outputs/rqd_analysis_pure

```

만약 기존 `run_multicase_decision.py`가 다른 output 폴더를 쓰면 그 경로를 넣으면 됩니다.

5. 생성되는 결과 파일

실행 후 다음 파일들이 생깁니다.

```
text
outputs/rqd_analysis_pure/rqd_summary.csv
outputs/rqd_analysis_pure/rqd_pairwise.csv
outputs/rqd_analysis_pure/rqd_threshold_metrics.csv
outputs/rqd_analysis_pure/rqd_threshold_sweep.csv
outputs/rqd_analysis_pure/rqd_by_case.csv
outputs/rqd_analysis_pure/rqd_by_density_regime.csv
outputs/rqd_analysis_pure/rqd_by_orientation_regime.csv
```

단, `face_fracture_density`나 `orientation_bias_gap` 컬럼이 없거나 값이 부족하면 regime 파일은 생기지 않을 수 있습니다.

그건 오류가 아니라 정상입니다.

6. 먼저 확인해야 할 핵심 파일 3개

처음에는 아래 3개만 보면 됩니다.

```
text
rqd_summary.csv
rqd_pairwise.csv
rqd_threshold_metrics.csv
```

6-1. `rqd_summary.csv`

이 파일은 네 방법의 RQD 분포를 보여줍니다.

보야 할 컬럼:

```
text
column
mean
std
min
p10
p50
p90
max
```

여기서 먼저 확인할 것은:

text

```
RQD_borehole_direct와 RQD_face_scanline의 분포가 비슷한가?  
RQD_borehole_hudson이 direct보다 더 높거나 낮은가?  
RQD_face_jv가 너무 일정한 값으로만 나오는가?
```

특히 RQD_face_jv가 거의 일정할 가능성이 있습니다.

현재 jv가 global P32 proxy 기반이면 face position별 변화가 작거나 없기 때문입니다.

이건 코드 오류가 아니라 방법론상 자연스러운 현상입니다.

6-2. rqd_pairwise.csv

가장 중요한 파일입니다.

핵심 비교는 이 네 개입니다.

text

```
RQD_borehole_direct → RQD_face_scanline  
RQD_borehole_hudson → RQD_face_scanline  
RQD_borehole_direct → RQD_face_jv  
RQD_borehole_hudson → RQD_face_jv
```

봐야 할 컬럼:

text

```
bias  
mae  
rmse  
corr  
overestimation_rate  
underestimation_rate
```

해석은 다음입니다.

bias

text

```
bias = predictor - target
```

따라서:

text

```
bias > 0:  
시추공 RQD가 face RQD보다 높음  
잠재적 과대평가
```

```
bias < 0:  
시추공 RQD가 face RQD보다 낮음  
잠재적 보수평가
```

mae

```
text  
평균 절대 오차
```

RQD는 0~100이므로, 예를 들어:

```
text  
MAE 5 이하: 상당히 잘 맞음  
MAE 10 정도: 실무적으로 볼 만함  
MAE 20 이상: 차이가 큼
```

정도로 1차 해석할 수 있습니다.

corr

상관성입니다.

다만 RQD가 90~100에 몰려 있으면 상관계수가 낮게 나올 수 있습니다.
그래서 **corr**만 보면 안 되고 **bias**, **mae**, **rmse**를 같이 봐야 합니다.

6-3. rqd_threshold_metrics.csv

RQD threshold 기준으로 false-safe / false-alarm을 봅니다.

기본 threshold는 다음입니다.

```
text  
50, 75, 90
```

중요한 컬럼:

```
text  
threshold  
TP  
FP_false_safe  
FN_false_alarm  
TN  
false_safe_rate  
false_alarm_rate  
balanced_accuracy  
expected_cost
```

여기서 RQD 단계의 false-safe는 다음입니다.

text

```
시추공 RQD는 good이라고 판단했지만,  
face RQD는 bad인 경우
```

즉:

text

```
predictor >= threshold  
target < threshold
```

이 값이 높으면 위험합니다.

7. Threshold sweep 결과 보는 법

rqd_threshold_sweep.csv는 threshold를 10부터 100까지 5 간격으로 바꿔가며 계산한 결과입니다.

이 파일에서 봐야 할 것은:

text

1. 어느 threshold에서 false-safe가 급격히 증가하는가?
2. direct와 Hudson 중 어느 쪽의 expected_cost가 낮은가?
3. face_scanline target과 face_jv target에서 결과가 어떻게 달라지는가?

예를 들어 나중에 pandas로 이렇게 볼 수 있습니다.

python

```
import pandas as pd  
  
df = pd.read_csv("outputs/rqd_analysis_pure/rqd_threshold_sweep.csv")  
  
sub = df[(df["predictor"] == "RQD_borehole_direct") &  
(df["target"] == "RQD_face_scanline")]  
  
print(sub[[  
    "threshold",  
    "prior_target_good_rate",  
    "false_safe_rate",  
    "false_alarm_rate",  
    "balanced_accuracy",  
    "expected_cost",  
]])
```

8. 이번 RQD 분석에서 기대할 수 있는 해석 패턴

패턴 A. Borehole direct가 face scanline보다 지속적으로 높음

결과 예:

```
text
bias > 0
overestimation_rate 높음
false_safe_rate 높음
```

해석:

```
text
시추공 direct RQD가 굴진면 중앙 수평 scanline RQD를 과대평가하는 경향이 있다.
이는 시추공 방향과 face scanline 방향 차이에 따른 orientation/sampling bias 가능성이 있다.
```

패턴 B. Borehole direct와 Hudson이 비슷함

결과 예:

```
text
MAE_direct ~ MAE_hudson
bias_direct ~ bias_hudson
```

해석:

```
text
절리 간격 분포보다 선형 절리빈도  $\lambda$ 가 RQD 차이의 대부분을 설명할 수 있다.
```

패턴 C. Hudson이 direct보다 face와 더 잘 맞음

결과 예:

```
text
MAE_hudson < MAE_direct
RMSE_hudson < RMSE_direct
```

해석:

```
text
직접 RQD는 국부 spacing 변동에 민감하고,
Hudson  $\lambda$  방식은 평균 절리빈도를 이용하므로 더 안정적인 predictor일 수 있다.
```

패턴 D. Direct가 Hudson보다 더 잘 맞음

결과 예:

```
text
MAE_direct < MAE_hudson
corr_direct > corr_hudson
```

해석:

```
text
절리 개수만으로는 부족하고,
절리 간격 분포와 intact segment 길이 정보가 face RQD 예측에 중요하다.
```

패턴 E. Face Jv가 거의 일정함

결과 예:

```
text
RQD_face_jv std ≈ 0
```

해석:

```
text
현재 Jv는 global joint set 또는 P32 proxy 기반이므로 face 위치별 국부 변동성을 반영하지 않는다.
따라서 Jv는 local face reference라기보다 case-level volumetric reference로 해석해야 한다.
```

이 경우 나중에 개선 방향은:

```
text
local Jv proxy
face 주변 control volume 기반 Jv
```

입니다.

9. 지금은 채택하지 않은 11번 선택사항에 대해

사용자가 말한 대로 현재는 orientation tendency 확장은 하지 않아도 됩니다.

그래도 기존 CSV에 이미 있는:

```
text
orientation_bias_gap
face_fracture_density
```

정도는 regime 분석에 쓸 수 있습니다.

다만 이 `orientation_bias_gap`은 아직 완전히 RQD 방향성 분석용으로 정교화된 지표는 아닙니다.
따라서 결과 해석에서는 조심스럽게 쓰면 됩니다.

표현은 이렇게 하는 게 좋습니다.

```
text
기존 orientation_bias_gap 기준의 보조적 regime 분석을 수행하였다.
다만 face scanline y방향에 대한 명시적 intersection tendency 지표는 후속 단계에서 추가할
예정이다.
```

10. 실행 후 나에게 보여주면 좋은 결과

분석을 돌린 뒤에는 아래 두 파일의 내용을 붙여주면 됩니다.

1순위

```
text
rqd_summary.csv
```

전체 또는 주요 행:

```
text
RQD_borehole_direct
RQD_borehole_hudson
RQD_face_scanline
RQD_face_jv
RQD_face_conservative
```

2순위

```
text
rqd_pairwise.csv
```

특히 이 컬럼들:

```
text
predictor
target
n
pred_mean
target_mean
bias
mae
rmse
corr
overestimation_rate
underestimation_rate
```

3순위


```
text
rqd_threshold_metrics.csv
```

특히 threshold 75 기준:

```
text
predictor
target
threshold
prior_target_good_rate
TP
FP_false_safe
FN_false_alarm
TN
false_safe_rate
false_alarm_rate
balanced_accuracy
expected_cost
```

그러면 제가 바로 해석해드릴 수 있습니다.

11. 다음 단계 요약

지금 할 일은 딱 세 가지입니다.

- ```
text
```
1. 수정된 코드로 multicase 실행해서 all\_cases\_face\_rows.csv 생성
  2. analyze\_rqd\_methods.py 추가
  3. RQD 분석 실행

명령은 예를 들면:

```
bash
python -m src.cli.run_multicase_decision
```

그 다음:

```
bash
python -m src.cli.analyze_rqd_methods --csv
outputs/multicase_pure/all_cases_face_rows.csv --out outputs/rqd_analysis_pure
```

그리고 결과 파일 중 먼저 이 두 개를 확인하면 됩니다.

```
text
rqd_summary.csv
rqd_pairwise.csv
GPT-5.5
```

```

rqd_summary.csv : column,exists,count,mean,std,min,p10,p25,p50,p75,p90,max
RQD_borehole_direct,True,480,99.47825188910619,0.9060139189291068,93.84375382746568,98.31759503732377,9
9.12649842853337,100.0,100.0,100.0,100.0
RQD_borehole_hudson,True,480,99.70087845487859,0.4244071054821265,97.413719545283,99.1358548650228,99.6
1598890532412,99.86631885255872,100.0,100.0,100.0
RQD_face_scanline,True,480,99.61270440687571,0.9070548819996909,92.09472088093489,98.44711826524183,99.8
5820203344483,100.0,100.0,100.0,100.0
RQD_face_jv,True,480,100.0,0.0,100.0,100.0,100.0,100.0,100.0,100.0
RQD_face_conservative,True,480,99.61270440687571,0.9070548819996909,92.09472088093489,98.44711826524183,
99.85820203344483,100.0,100.0,100.0,100.0

```



Thought Process

좋습니다. 이 `rqd_summary.csv`만 봐도 중요한 사실이 꽤 나옵니다.

결론부터 말하면:

현재 6개 case 조건에서는 네 RQD 방법 모두 거의 100에 포화되어 있어서, RQD만으로는 시추공-  
굴진면 차이를 충분히 분리하기 어렵습니다.  
특히 `RQD_face_jv`는 완전히 100으로 clipping되어 있어, 현재 설정에서는 Jv 기반 RQD가 변별력을  
제공하지 못합니다.

## 1. 결과 요약

표를 핵심만 정리하면 다음입니다.

| 방법                                 | 평균     | 표준편차 | 최소    | 중앙값   | 해석                     |
|------------------------------------|--------|------|-------|-------|------------------------|
| <code>RQD_borehole_direct</code>   | 99.48  | 0.91 | 93.84 | 100   | 거의 전부 excellent        |
| <code>RQD_borehole_hudson</code>   | 99.70  | 0.42 | 97.41 | 99.87 | direct보다 더 포화됨         |
| <code>RQD_face_scanline</code>     | 99.61  | 0.91 | 92.09 | 100   | 거의 전부 excellent        |
| <code>RQD_face_jv</code>           | 100.00 | 0.00 | 100   | 100   | 완전 포화                  |
| <code>RQD_face_conservative</code> | 99.61  | 0.91 | 92.09 | 100   | Jv가 100이라 scanline과 동일 |

즉 현재 데이터에서는:

```

text
RQD_borehole_direct ~ RQD_face_scanline ~ 100
RQD_borehole_hudson ~ 100
RQD_face_jv = 100

```

입니다.

## 2. 이전 코드 오류인가?

일단 이 결과만으로는 코드 오류라고 보기는 어렵습니다.

오히려 RQD라는 지표의 특성이 잘 드러난 결과일 가능성이 큼니다.

RQD는 다음 방식입니다.

text

길이 10 cm 이상인 intact segment만 합산

따라서 절리가 몇 개 있어도, 절리 간격이 대부분 10 cm보다 크면 RQD는 쉽게 100이 됩니다.

예를 들어 1 m 구간에 절리가 2개 있어도 간격이 다음과 같으면:

text

0.3 m, 0.4 m, 0.3 m

모든 조각이 10 cm 이상이므로:

text

RQD = 100

입니다.

즉, RQD는 절리 개수에 민감한 지표가 아니라, 10 cm 미만 조각이 얼마나 생기는데 민감한 지표입니다.

그래서 현재 DFN이 “절리 수는 있지만 절리 간격이 대부분 10 cm 이상”인 상태라면 RQD가 거의 100으로 나오는 것이 자연스럽습니다.

---

## 3. Hudson RQD가 더 100에 가까운 이유

RQD\_borehole\_hudson 평균이 99.70이고 표준편차가 0.42입니다.

이는 direct보다 더 포화되어 있습니다.

Hudson/Priest-Hudson 식은:

text

$$RQD = 100 \times \exp(-\lambda t) \times (\lambda t + 1)$$

여기서  $t = 0.1$  m입니다.

이 식은  $\lambda$ 가 작으면 거의 100을 줍니다.

예를 들어:

```
text
λ = 1 /m
λt = 0.1
RQD ≈ 99.5
text
λ = 2 /m
λt = 0.2
RQD ≈ 98.2
```

즉 선형 절리빈도가 아주 높지 않으면 Hudson RQD도 거의 100입니다.

따라서 현재 결과는 다음을 의미합니다.

```
text
현재 borehole window 안의 절리 선형빈도 λ가 RQD를 크게 낮출 정도로 높지 않다.
```

---

#### 4. Face scanline RQD도 거의 100인 이유

RQD\_face\_scanline은 평균 99.61, 최소 92.09입니다.

이건 face scanline에서도 대부분 10 cm 미만 조각이 거의 없다는 뜻입니다.

중요한 점은, face scanline 방식이 이제 현실적으로 바뀌었기 때문에:

```
text
x방향 scanline 때문이 아니라,
실제 중앙 수평 y방향 scanline에서도 RQD가 매우 높게 나온 것
```

입니다.

따라서 현재 case들은 RQD 관점에서는 대부분 매우 양호한 암반으로 분류됩니다.

---

#### 5. Jv 기반 RQD가 전부 100인 이유

RQD\_face\_jv가 완전히 100입니다.

이건 매우 중요한 결과입니다.

Palmström형 식은:

```
text
RQD = 115 - 3.3 Jv
```

그리고 100을 초과하면 100으로 clipping합니다.

즉,

```
text
115 - 3.3 Jv >= 100
```

이면 전부 100이 됩니다.

이를 풀면:

```
text
3.3 Jv <= 15
Jv <= 4.545
```

즉 현재 Jv\_face 또는 P32-derived Jv\_proxy가 대부분 또는 전부 4.545 이하라는 뜻입니다.

그래서:

```
text
RQD_face_jv_raw = 115 - 3.3 Jv > 100
```

가 되고, 최종값은 모두 100으로 clipping된 것입니다.

따라서 현재 Jv 방법은 변별력이 없습니다.

---

## 6. 현재 단계에서의 첫 번째 결론

현재 결과만 놓고 보면 RQD 단계의 1차 결론은 다음입니다.

현재 사용한 granite 계열 6개 case에서는 시추공 direct RQD, 시추공 Hudson RQD, 굴진면 중앙 수평 scanline RQD가 모두 100에 가까운 값을 보였다. 따라서 이 조건에서는 RQD가 시추공-굴진면 차이를 민감하게 드러내지 못하며, 대부분의 단면이 RQD 기준 excellent rock으로 포화된다.

조금 더 연구적으로 쓰면:

```
text
RQD is nearly saturated under the current DFN parameter ranges.
Therefore, the discrepancy between borehole and face conditions cannot be effectively
diagnosed using RQD alone in these cases.
```

한국어로는:

text

현재 DFN 조건에서는 RQD가 상한값 100 부근에 포화되어, 시추공과 굴진면 사이의 방향성 또는 sampling 차이를 식별하는 민감도가 낮다.

---

## 7. 이것이 연구적으로 나쁜 결과인가?

아닙니다. 오히려 중요한 결과입니다.

왜냐하면 실제 연구 질문 중 하나가 다음이었기 때문입니다.

text

시추공 RQD와 굴진면 RQD가 차이가 발생할 수밖에 없나?

현재 결과는 이렇게 답할 수 있습니다.

text

현재 암반 조건에서는 RQD 자체가 거의 100에 포화되어 있기 때문에, 시추공 RQD와 굴진면 RQD의 차이는 크게 나타나지 않았다. 하지만 이것은 두 방법이 본질적으로 동일하다는 의미가 아니라, RQD 지표가 양호한 암반 조건에서 차이를 드러낼 만큼 민감하지 않다는 의미일 수 있다.

즉, 지금 결과는 \*\*\*“차이가 없다”\*\*\*라기보다는:

text

RQD가 너무 둔감해서 차이를 보기 어렵다

에 가깝습니다.

---

## 8. 지금 꼭 봐야 할 다음 파일

이제 반드시 `rqd_pairwise.csv`를 봐야 합니다.

특히 아래 비교가 중요합니다.

text

RQD\_borehole\_direct → RQD\_face\_scanline  
RQD\_borehole\_hudson → RQD\_face\_scanline

봐야 할 컬럼은:

text

bias  
mae  
rmse  
corr

```
overestimation_rate
underestimation_rate
```

하지만 현재 summary상으로 예상하면:

```
text
MAE는 매우 작을 가능성이 높음
corr는 낮거나 불안정할 수 있음
```

왜냐하면 값들이 거의 100에 몰려 있기 때문입니다.

상관계수는 값의 분산이 작으면 의미가 약해집니다.

---

## 9. 현재 threshold 50, 75, 90은 거의 의미 없을 가능성이 큼

현재 최소값이 다음 정도입니다.

```
text
RQD_borehole_direct min = 93.84
RQD_borehole_hudson min = 97.41
RQD_face_scanline min = 92.09
RQD_face_jv min = 100
```

따라서 threshold를 50, 75, 90으로 두면 대부분이 good입니다.

즉:

```
text
RQD >= 90
```

기준으로도 거의 전부 good일 가능성이 높습니다.

따라서 decision metric에서 false-safe / false-alarm이 거의 안 나올 수 있습니다.

현재 데이터에서는 threshold를 더 높게 봐야 합니다.

추천 threshold:

```
text
95
98
99
99.5
100
```

특히 RQD 포화 조건에서는 90보다 95, 98, 99가 더 informative합니다.

---

## 10. 다음 분석에서 추가로 봐야 할 값

현재 RQD가 전부 100에 가까우므로, RQD만 보면 정보가 적습니다.

따라서 다음 값들을 반드시 같이 봐야 합니다.

```
text
lambda_borehole
lambda_face_scanline
borehole_n_intersections_total
face_scanline_n_intersections
Jv_face
```

이 값들이 더 중요할 수 있습니다.

왜냐하면 RQD는 100에 포화되어도, 실제 절리 교차 개수나  $\lambda$ 는 차이를 보일 수 있기 때문입니다.

예를 들어:

```
text
RQD_borehole_direct = 100
RQD_face_scanline = 100
```

이어도,

```
text
lambda_borehole = 0.2 /m
lambda_face_scanline = 1.5 /m
```

라면 실제로는 face가 더 절리화되어 있는 것입니다.

단지 RQD가 그 차이를 못 보여준 것뿐입니다.

---

## 11. 분석 코드에 하나 추가하면 좋은 것

`rqd_analysis.py` 또는 별도 분석에서 다음 컬럼도 요약하는 것이 좋습니다.

```
text
lambda_borehole
lambda_face_scanline
borehole_n_intersections_total
face_scanline_n_intersections
Jv_face
```

예를 들어 `rqd_summary.csv`에 RQD뿐 아니라 이들도 포함되게 하면 좋습니다.



분석 스크립트에서 summary 대상 컬럼에 아래를 추가하면 됩니다.

```
python
extra_cols = [
 "lambda_borehole",
 "lambda_face_scanline",
 "borehole_n_intersections_total",
 "face_scanline_n_intersections",
 "Jv_face",
]
```

그러면 RQD는 포화되어도  $\lambda$ 와 교차 개수 차이를 볼 수 있습니다.

---

## 12. Jv 방법 개선 필요

현재 RQD\_face\_jv = 100으로 전부 포화되었으므로, Jv 방법을 연구에 쓰려면 다음 중 하나가 필요합니다.

### 방법 A. RQD\_face\_jv\_raw도 저장

현재는 clipping 후 값만 보고 있습니다.

추가로 다음 값을 저장하면 좋습니다.

```
text
RQD_face_jv_raw = 115 - 3.3 * Jv_face
```

예를 들어 raw 값이:

```
text
108
104
101
```

이면 모두 clipping 후 100이지만, raw에서는 차이가 있습니다.

물론 공식상 RQD는 100을 넘을 수 없으므로 최종 RQD는 100이 맞습니다.  
하지만 연구 분석용으로는 raw value를 보면 Jv 차이를 확인할 수 있습니다.

추천 컬럼:

```
text
RQD_face_jv_raw
RQD_face_jv_clipped
```

현재 RQD\_face\_jv는 clipped 값으로 두면 됩니다.

---

### 방법 B. 더 높은 절리 밀도 case 추가

Jv 기반 RQD가 100 아래로 내려가려면 대략:

```
text
Jv > 4.545
```

이어야 합니다.

현재 P32-derived Jv proxy가 그보다 작다면, Jv RQD는 항상 100입니다.

따라서 Jv 방법을 테스트하려면 다음 case가 필요합니다.

```
text
higher P32 case
very_dense case
poor rock case
highly fractured case
```

---

### 방법 C. local Jv proxy로 바꾸기

현재 Jv가 global joint set 기반이면 face 위치별 변동성이 없습니다.

그래서 RQD\_face\_jv가 case 내부에서 일정하거나 완전히 100이 될 수 있습니다.

나중에는 face 주변 control volume에서 local Jv proxy를 계산하는 게 좋습니다.

하지만 이건 다음 단계입니다.

---

## 13. 현재 결과에서 조심해야 할 해석

이 결과를 보고 이렇게 말하면 안 됩니다.

```
text
시추공 RQD와 굴진면 RQD는 거의 완전히 일치한다.
따라서 시추공 RQD는 굴진면을 신뢰성 있게 예측한다.
```

이건 과한 해석입니다.

왜냐하면 지금은 RQD가 모두 100에 포화되어 있기 때문입니다.

더 정확한 해석은 다음입니다.

text

현재 case 범위에서는 RQD가 상한값에 포화되어 시추공-굴진면 차이를 민감하게 드러내지 못했다. 따라서 RQD만으로는 전방 지반 예측 신뢰성을 평가하기에 정보량이 부족할 수 있다. 절리 선형빈도  $\lambda$ , 절리 교차 개수, Jv proxy, Q' 구성요소 등을 함께 검토해야 한다.

이게 훨씬 안전합니다.

---

#### 14. 지금 바로 할 다음 작업

우선 다음 두 파일을 봐야 합니다.

text

```
rqd_pairwise.csv
rqd_threshold_metrics.csv
```

특히 `rqd_pairwise.csv`를 먼저 보여주시면 됩니다.

또한 가능하면 `all_cases_face_rows.csv`에서 아래 컬럼들의 summary도 보고 싶습니다.

text

```
lambda_borehole
lambda_face_scanline
borehole_n_intersections_total
face_scanline_n_intersections
Jv_face
```

간단히 pandas로 이렇게 확인할 수 있습니다.

python

```
import pandas as pd

df = pd.read_csv("outputs/rqd_analysis_pure/./all_cases_face_rows.csv")

cols = [
 "lambda_borehole",
 "lambda_face_scanline",
 "borehole_n_intersections_total",
 "face_scanline_n_intersections",
 "Jv_face",
]

print(df[cols].describe(percentiles=[0.1, 0.25, 0.5, 0.75, 0.9]))
```

---

#### 15. 현재 결과의 한 줄 결론

현재 `rqd_summary.csv`의 핵심 결론은 이것입니다.

현재 DFN case들에서는 RQD가 거의 100에 포화되어, 시추공 direct RQD, Hudson RQD, 굴진면 scanline RQD 간 차이가 매우 작게 나타났다. 이는 시추공과 굴진면이 본질적으로 동일하다는 뜻이라기보다, 양호한 암반 조건에서 RQD 지표가 절리 방향성 및 sampling 차이를 식별하는 민감도가 낮다는 것을 시사한다. Jv 기반 RQD 역시 현재 Jv proxy 범위에서는 100으로 clipping되어 변별력을 제공하지 못했다.

#### GPT-5.5

```
rqd_pairwise.csv:
predictor,target,n,pred_mean,target_mean,bias,mae,rmse,median_abs_error,corr,overestimation_rate,underestimation_rate,
RQD_borehole_direct,RQD_face_scanline,480,99.47825188910619,99.61270440687571,-
0.13445251776948994,0.7891260031341113,1.3480738418814533,0.27716701264969146,-
0.09696470687872287,0.225,0.34791666666666665,0.4270833333333333
RQD_borehole_direct,RQD_face_jv,480,99.47825188910619,100.0,-
0.5217481108938001,0.5217481108938001,1.0446876016159885,0.0,0.0,0.39375,0.60625
RQD_borehole_direct,RQD_face_conservative,480,99.47825188910619,99.61270440687571,-
0.13445251776948994,0.7891260031341113,1.3480738418814533,0.27716701264969146,-
0.09696470687872287,0.225,0.34791666666666665,0.4270833333333333
RQD_borehole_hudson,RQD_face_scanline,480,99.70087845487859,99.61270440687571,0.08817404800288828,0.5973
680558834846,1.0497144760495605,0.25032994723460433,-
0.12149304522121111,0.24583333333333332,0.5395833333333333,0.21458333333333332
RQD_borehole_hudson,RQD_face_jv,480,99.70087845487859,100.0,-
0.29912154512142203,0.29912154512142203,0.51886398703441,0.1336811474412798,,0.0,0.71875,0.28125
RQD_borehole_hudson,RQD_face_conservative,480,99.70087845487859,99.61270440687571,0.08817404800288828,0.
5973680558834846,1.0497144760495605,0.25032994723460433,-
0.12149304522121111,0.24583333333333332,0.5395833333333333,0.21458333333333332
```



좋습니다. 이 `rqd_pairwise.csv`는 상당히 중요한 메시지를 줍니다.

요약하면:

현재 case들에서는 시추공 RQD와 굴진면 scanline RQD의 평균 차이는 매우 작습니다.  
Hudson  $\lambda$  방식이 direct 방식보다 face scanline RQD에 대해 약간 더 낮은 MAE/RMSE를 보입니다.  
하지만 전체 RQD가 100 근처에 포화되어 있어서 상관계수는 거의 의미가 없고, “잘 예측한다”기보다는 “RQD가 포화되어 차이가 작게 보인다”는 해석이 더 안전합니다.

---

## 1. 핵심 결과 정리

가장 중요한 비교는 이 두 개입니다.

```
text
RQD_borehole_direct → RQD_face_scanline
RQD_borehole_hudson → RQD_face_scanline
```

| predictor | target        | bias   | MAE   | RMSE  | corr   | over  | under | equal |
|-----------|---------------|--------|-------|-------|--------|-------|-------|-------|
| BH direct | face scanline | -0.134 | 0.789 | 1.348 | -0.097 | 0.225 | 0.348 | 0.427 |
| BH Hudson | face scanline | +0.088 | 0.597 | 1.050 | -0.121 | 0.246 | 0.540 | 0.215 |

해석:

text

BH direct는 face scanline보다 평균 0.13 낮다.  
 BH Hudson은 face scanline보다 평균 0.09 높다.  
 둘 다 bias는 매우 작다.  
 Hudson의 MAE/RMSE가 direct보다 약간 작다.

즉, 평균적으로는 거의 차이가 없습니다.

## 2. Direct vs face scanline 해석

text

RQD\_borehole\_direct → RQD\_face\_scanline  
 bias = -0.134  
 MAE = 0.789  
 RMSE = 1.348

이건 굉장히 작은 차이입니다.

RQD scale이 0~100이라는 점을 생각하면:

text

평균 절대 차이 < 1 RQD point  
 RMSE ≈ 1.35 RQD point

입니다.

즉 현재 case 조건에서는 시추공 direct RQD와 face scanline RQD가 수치적으로 매우 가깝습니다.

하지만 이것을 그대로 “예측력이 높다”고 말하면 위험합니다.

이유는 `rqd_summary.csv`에서 이미 확인했듯이:

text

둘 다 대부분 100에 포화

되어 있기 때문입니다.

### 3. Hudson이 direct보다 약간 좋아 보임

text

```
BH direct MAE = 0.789
```

```
BH Hudson MAE = 0.597
```

text

```
BH direct RMSE = 1.348
```

```
BH Hudson RMSE = 1.050
```

따라서 현재 자료에서는 Hudson 방식이 face scanline RQD와 약간 더 가깝습니다.

이유는 아마 다음입니다.

text

direct RQD는 segment 내 절리 위치와 10 cm 미만 조각 발생 여부에 민감하다.

Hudson RQD는  $\lambda$  기반의 smooth한 이론값이어서 극단적 국부 변동이 완화된다.

하지만 차이가 매우 작습니다.

따라서 지금 단계에서는 이렇게 표현하는 것이 좋습니다.

text

Hudson  $\lambda$  기반 RQD는 direct RQD보다 face scanline RQD에 대해 약간 낮은 MAE와 RMSE를 보였으나, 전체 RQD가 100 부근에 포화되어 있어 이 차이의 실질적 의미는 제한적이다.

### 4. 상관계수가 음수인 이유

corr가 다음처럼 나왔습니다.

text

```
BH direct vs face scanline corr = -0.097
```

```
BH Hudson vs face scanline corr = -0.121
```

이걸 보고 “역상관이다”라고 해석하면 안 됩니다.

이 결과는 RQD 값들이 거의 100에 몰려 있기 때문에 생긴 것입니다.

즉:

text

분산이 너무 작다.

많은 값이 100으로 tied 되어 있다.

상관계수가 불안정하다.

따라서 현재 데이터에서는 상관계수보다 다음 지표가 더 중요합니다.

```
text
bias
MAE
RMSE
overestimation_rate
underestimation_rate
equal_rate
```

특히 RQD 포화 조건에서는 `corr`는 거의 무시해도 됩니다.

표현은 이렇게 하는 게 안전합니다.

```
text
Due to the near-saturation of RQD values, correlation coefficients were not considered
reliable indicators of predictive performance.
```

한국어로:

```
text
RQD 값이 상한값 100 부근에 집중되어 있어, 상관계수는 예측 성능 지표로 해석하기 어렵다.
```

---

## 5. overestimation / underestimation 해석

### Direct vs face scanline

```
text
overestimation_rate = 0.225
underestimation_rate = 0.348
equal_rate = 0.427
```

의미:

```
text
22.5%: BH direct > face scanline
34.8%: BH direct < face scanline
42.7%: 동일
```

즉 direct 방식은 face보다 약간 낮게 나오는 경우가 더 많습니다.

하지만 차이 크기는 매우 작습니다.

---

### Hudson vs face scanline

```
text
overestimation_rate = 0.246
underestimation_rate = 0.540
equal_rate = 0.215
```

여기서 조금 이상해 보일 수 있습니다.

평균 bias는 +0.088인데, underestimation\_rate는 54.0%입니다.

이건 다음 상황을 의미합니다.

text

대부분의 경우 Hudson이 face보다 아주 조금 낮지만,  
일부 경우에는 Hudson이 face보다 더 크게 높아서 평균 bias가 양수로 바뀜.

또는 face scanline에 100이 많은 상태에서 Hudson은 99.8~100 사이 값이 많아 under로 잡히는 경우가 많을 수 있습니다.

즉, 이 rate도 포화 상황에서는 조심해서 봐야 합니다.

---

## 6. Jv target 비교

### Direct vs Jv

text

```
target_mean = 100
bias = -0.522
MAE = 0.522
RMSE = 1.045
corr = NaN
```

### Hudson vs Jv

text

```
target_mean = 100
bias = -0.299
MAE = 0.299
RMSE = 0.519
corr = NaN
```

여기서 corr가 비어 있는 것은 정상입니다.

왜냐하면:

text

```
RQD_face_jv = 100.0 constant
```

이기 때문입니다.

상대가 상수이면 상관계수는 정의되지 않습니다.

이 결과는 Jv 방법이 현재 데이터에서는 reference로서 변별력이 없다는 뜻입니다.



즉:

text

Jv 기반 RQD는 모든 face에서 100으로 clipping되어,  
현재 case 조건에서는 시추공 RQD와 비교 가능한 독립적 target 역할을 하지 못한다.

---

## 7. Conservative target이 face scanline과 같은 이유

결과에서:

text

```
RQD_face_conservative = RQD_face_scanline
```

입니다.

왜냐하면:

text

```
RQD_face_jv = 100
RQD_face_conservative = min(RQD_face_scanline, RQD_face_jv)
```

이므로:

text

```
min(RQD_face_scanline, 100) = RQD_face_scanline
```

입니다.

따라서 conservative target은 현재 별도 의미가 없습니다.

---

## 8. 현재 RQD 단계의 결론

지금까지 summary + pairwise만 보면 1차 결론은 다음입니다.

text

현재 DFN case들에서 RQD는 전반적으로 100 부근에 포화되어 있다.  
시추공 direct RQD와 굴진면 중앙 수평 scanline RQD의 평균 차이는 매우 작고, MAE도 1 RQD point 미만이다.  
Hudson  $\lambda$  기반 시추공 RQD는 direct 방식보다 약간 낮은 MAE/RMSE를 보였지만, 이 역시 RQD 포화 조건에서의 작은 차이이다.  
Jv 기반 RQD는 모든 값이 100으로 clipping되어 현재 조건에서는 변별력을 제공하지 못했다.  
따라서 현재 case 범위에서는 RQD만으로 시추공-굴진면 차이 또는 예측 신뢰성을 평가하기에는 정보량이 부족하다.

이 결론이 매우 중요합니다.

---

## 9. 이 결과가 말해주는 연구적 의미

이 결과는 다음 질문에 대한 좋은 답을 줍니다.

**질문:** 시추공 RQD와 굴진면 RQD는 차이가 발생할 수밖에 없는가?

현재 case에서는:

text

차이가 크게 발생하지 않았다.

하지만 그 이유는:

text

둘이 본질적으로 같은 정보를 보기 때문

이라기보다:

text

RQD가 100에 포화되어 차이를 드러내지 못했기 때문

일 가능성이 큼니다.

즉, 더 정확한 답은:

text

현재 양호한 암반 조건에서는 시추공 RQD와 굴진면 RQD의 차이가 RQD 지표상으로는 작게 나타난다. 그러나 이는 RQD의 포화 특성 때문일 수 있으며, 절리 빈도나 방향성 차이가 실제로 없는 것을 의미하지 않는다.

---

## 10. 다음에 반드시 봐야 할 것: $\lambda$ 와 교차 개수

이제 RQD 자체가 포화되었으므로, 다음 분석은 RQD가 아니라 그 원재료를 봐야 합니다.

즉:

text

lambda\_borehole  
lambda\_face\_scanline  
borehole\_n\_intersections\_total  
face\_scanline\_n\_intersections  
Jv\_face

입니다.

특히 핵심은 이 두 개입니다.

```
text
lambda_borehole vs lambda_face_scanline
borehole_n_intersections_total vs face_scanline_n_intersections
```

가능한 해석은 다음입니다.

---

#### 경우 A. $\lambda$ 도 비슷하다

```
text
lambda_borehole $\approx$ lambda_face_scanline
```

그러면 현재 case에서는 실제 절리 교차 빈도도 비슷한 것입니다.

이 경우:

```
text
현재 절리 조건에서는 borehole RQD가 face scanline RQD를 잘 대표할 수 있다.
```

라고 더 강하게 말할 수 있습니다.

---

#### 경우 B. $\lambda$ 는 다른데 RQD만 비슷하다

```
text
lambda_borehole $\neq$ lambda_face_scanline
RQD_borehole $\approx$ RQD_face_scanline
```

이 경우가 더 흥미롭습니다.

해석:

```
text
시추공과 굴진면은 실제 절리 교차 빈도는 다르지만,
RQD가 100 부근에 포화되어 그 차이를 숨기고 있다.
```

이게 나오면 연구적으로 매우 중요한 결론입니다.

```
text
RQD alone may be insensitive to directional sampling bias under good rock conditions.
```

---

## 11. 분석 스크립트에 `lambda summary` 추가 권장

지금 결과를 보면 `rqd_summary.csv`에  $\lambda$ 와 교차 개수를 포함하는 게 필요합니다.

간단히 별도 파일을 하나 만들어도 됩니다.

예:

```
text
rqd_input_diagnostics.csv
```

분석 대상 컬럼:

```
python
diagnostic_cols = [
 "lambda_borehole",
 "lambda_face_scanline",
 "borehole_n_intersections_total",
 "face_scanline_n_intersections",
 "Jv_face",
]
```

요약은 기존 `summarize_rqd_columns()`를 그대로 써도 됩니다. 이름만 RQD가 아니지만 숫자 요약에는 문제 없습니다.

---

## 12. 지금 당장 볼 수 있는 간단한 `pandas` 코드

아래를 실행해서 결과를 보여주시면 다음 해석이 가능합니다.

```
python
import pandas as pd

df = pd.read_csv("outputs/multicase_pure/all_cases_face_rows.csv")

cols = [
 "lambda_borehole",
 "lambda_face_scanline",
 "borehole_n_intersections_total",
 "face_scanline_n_intersections",
 "Jv_face",
]

print(df[cols].describe(percentiles=[0.1, 0.25, 0.5, 0.75, 0.9]))

df["lambda_diff_bh_minus_face"] = df["lambda_borehole"] - df["lambda_face_scanline"]

print(df["lambda_diff_bh_minus_face"].describe(percentiles=[0.1, 0.25, 0.5, 0.75, 0.9]))
```

추가로:

```
python
print(df[["lambda_borehole", "lambda_face_scanline"]].corr())
```

도 보면 좋습니다.

---

### 13. Threshold metrics는 큰 기대를 하지 않아도 됨

현재 RQD 최소값이 대략 92 이상이므로:

```
text
threshold 50
threshold 75
threshold 90
```

에서는 대부분 good입니다.

그래서 `rqd_threshold_metrics.csv`는 정보가 적을 가능성이 큼니다.

만약 RQD threshold decision을 계속 보고 싶다면 threshold를 이렇게 바꾸세요.

```
text
95, 98, 99, 99.5, 100
```

실행 예:

```
bash
python -m src.cli.analyze_rqd_methods \
 --csv outputs/multicase_pure/all_cases_face_rows.csv \
 --out outputs/rqd_analysis_pure_high_threshold \
 --thresholds 95 98 99 99.5 100
```

이렇게 하면 포화 영역에서 조금 더 차이를 볼 수 있습니다.

다만 RQD=100이 많아서 threshold=100은 해석에 주의해야 합니다.

---

### 14. 다음 연구 방향

현재 결과는 다음 방향을 제안합니다.

**방향 1. RQD는 양호 암반에서 너무 둔감하다**

이건 매우 중요한 결론입니다.

즉, 전방 지반 예측에서 RQD만 보면:

text

거의 다 excellent로 나온다.

그래서:

text

RQD는 좋은 암반 조건에서 시추공-굴진면 차이를 평가하는 데 한계가 있다.

라는 결론으로 갈 수 있습니다.

---

**방향 2.  $\lambda$  또는 절리 개수 기반 분석이 필요하다**

RQD가 100이어도  $\lambda$ 는 차이가 있을 수 있습니다.

따라서 다음 단계는:

text

borehole lambda vs face scanline lambda

입니다.

이건 RQD보다 방향성/sampling 차이를 더 잘 보여줄 가능성이 큼니다.

---

**방향 3. 더 불량한 암반 case 추가 필요**

RQD 차이를 실제로 보려면 절리 간격이 10 cm 미만으로 쪼개지는 구간이 생겨야 합니다.

이를 위해서는:

text

P32를 크게 증가

절리 수 증가

미세절리 포함

절리군 추가

더 작은 spacing 조건 추가

poor rock case 추가

가 필요합니다.

특히 RQD 검증용 case를 따로 만들어야 할 가능성이 높습니다.

예:

text

```
granite_very_dense.yaml
granite_heavily_fractured.yaml
granite_faulted.yaml
```

## 15. 지금 결과의 최종 해석 문장

현재 결과는 이렇게 정리하면 좋습니다.

시추공 direct RQD와 굴진면 중앙 수평 scanline RQD의 평균 차이는 -0.13, MAE는 0.79로 매우 작았다. Hudson  $\lambda$  기반 RQD는 face scanline RQD에 대해 MAE 0.60으로 direct 방식보다 약간 작은 오차를 보였다. 그러나 모든 RQD 값이 100에 근접하여 분포가 강하게 포화되었고, Jv 기반 RQD는 모든 단면에서 100으로 clipping되었다. 따라서 현재 case 조건에서는 RQD가 시추공-굴진면 sampling 차이를 식별하기에 충분히 민감하지 않으며,  $\lambda$ , 절리 교차 개수, Jv proxy 등 RQD 구성 원자료를 함께 분석할 필요가 있다.

GPT-5.5

지금 모든 암반 케이스에서 절리의 평균 간격이 정말 10cm 이상이라는 예기인가? 이것부터 확인해보고 절리의 간격을 줄여봐야 할 것 같은데. 만약 평균 절리 간격이 10cm 이하인데도 RQD가 높다면 뭔가 단단히 잘못된거 같아~



맞습니다. **이걸 먼저 확인해야 합니다.**

현재 RQD가 거의 100이라는 결과를 보고 바로 “절리 평균 간격이 10cm 이상이다”라고 결론내리면 안 됩니다.

정확히는 이렇게 말해야 합니다.

현재 샘플링된 시추공 line과 face scanline 위에서는, RQD 계산상 10cm 미만 intact segment가 거의 발생하지 않았다.

이 말은 다음과 다릅니다.

암반 전체의 절리 평균 간격이 10cm 이상이다.

둘은 다릅니다.

---

## 1. 가장 먼저 구분해야 할 것

## A. YAML의 mean\_spacing

이건 절리군 정의에 들어 있는 입력값입니다.

하지만 당신 코드에서는 이미 정리했듯이:

```
python
if density_type == 'P32':
 n_expected = P32 * domain_volume / mean_area
else:
 n_expected = domain_volume / (mean_spacing * mean_area)
```

즉:

```
text
density_type == "P32"이면 mean_spacing은 DFN 생성에 사용되지 않음
```

입니다.

따라서 YAML에 `mean_spacing = 0.05` 또는 `0.1`이 들어 있어도, `density_type: P32`이면 실제 절리 개수에는 영향을 주지 않습니다.

그러므로 먼저 확인할 것은:

```
text
각 case의 density_type이 P32인가?
각 joint set의 P32가 얼마인가?
mean_spacing은 실제로 쓰이고 있는가?
```

입니다.

---

## B. 실제 DFN에서 scanline이 만난 절리 간격

RQD는 3D 평균 절리 간격을 직접 쓰지 않습니다.

RQD는 특정 line 위에서 절리 교차점 사이의 간격을 봅니다.

즉, RQD가 보는 것은:

```
text
시추공 또는 scanline 위의 apparent spacing
```

입니다.

이 값은 절리군의 실제 간격과 다를 수 있습니다.



특히 다음 요소 때문에 달라집니다.

text

1. 절리 방향성
2. scanline 방향
3. 절리 disk의 유한 반경
4. 절리 중심의 공간 분포
5. P32 값
6. sampling line 위치

따라서 평균 절리 간격이 작더라도, 특정 scanline이 절리를 잘 못 만나면 RQD는 높게 나올 수 있습니다.

하지만 당신 말대로:

실제 scanline 위에서 절리 간격이 10cm 이하인데도 RQD가 높다면 그건 코드 문제입니다.

이건 맞습니다.

---

## 2. 현재 결과로부터 추정되는 것

RQD\_borehole\_hudson 평균이 다음이었습니다.

text

```
RQD_borehole_hudson mean = 99.70
```

Hudson 식은:

text

```
RQD = 100 * exp(-λt) * (λt + 1)
t = 0.1 m
```

입니다.

이 값이 99.7이라는 것은  $\lambda$ 가 매우 작다는 뜻입니다.

대략적으로:

text

```
 $\lambda \approx 0.7 \sim 0.8 \text{ /m 수준}$
```

일 가능성이 큼니다.

즉, 시추공 line 위에서는 평균적으로:

```
text
1 / λ $\approx$ 1.2~1.4 m
```

정도의 apparent spacing이 나왔을 가능성이 있습니다.

이러면 RQD가 100 근처로 나오는 게 자연스럽습니다.

따라서 현재 상태는 아마:

```
text
scanline 위 절리 간격이 10cm 이하가 아니라,
대부분 1m 이상 수준으로 넓게 잡히고 있음
```

일 가능성이 큼니다.

하지만 추정만 하면 안 되고, 실제로 확인해야 합니다.

---

### 3. 지금 바로 확인할 1차 진단: $\lambda$ 와 apparent spacing

먼저 all\_cases\_face\_rows.csv에서 다음을 확인하세요.

```
text
lambda_borehole
lambda_face_scanline
borehole_n_intersections_total
face_scanline_n_intersections
Jv_face
```

아래 코드를 아무 분석용 파일이나 Python 콘솔에서 실행하면 됩니다.

```
python
import numpy as np
import pandas as pd

csv_path = "outputs/multicase_pure/all_cases_face_rows.csv"
df = pd.read_csv(csv_path)

cols = [
 "lambda_borehole",
 "lambda_face_scanline",
 "borehole_n_intersections_total",
 "face_scanline_n_intersections",
 "Jv_face",
]

print("\n=== Available columns check ===")
for c in cols:
 print(c, c in df.columns)
```

```

print("\n== Lambda / count summary ==")
print(
 df[cols].describe(
 percentiles=[0.1, 0.25, 0.5, 0.75, 0.9]
)
)

apparent spacing = 1 / lambda
lambda = 0이면 spacing은 infinity로 둔다.
df["spacing_borehole_apparent"] = np.where(
 df["lambda_borehole"] > 0,
 1.0 / df["lambda_borehole"],
 np.inf,
)

df["spacing_face_scanline_apparent"] = np.where(
 df["lambda_face_scanline"] > 0,
 1.0 / df["lambda_face_scanline"],
 np.inf,
)

print("\n== Apparent spacing summary ==")
spacing_cols = [
 "spacing_borehole_apparent",
 "spacing_face_scanline_apparent",
]

print(
 df[spacing_cols].replace([np.inf, -np.inf], np.nan).describe(
 percentiles=[0.1, 0.25, 0.5, 0.75, 0.9]
)
)

print("\n== Proportion of apparent spacing below 0.1 m ==")
print(
 "BH spacing < 0.1 m:",
 np.mean(df["spacing_borehole_apparent"] < 0.1),
)

print(
 "Face scanline spacing < 0.1 m:",
 np.mean(df["spacing_face_scanline_apparent"] < 0.1),
)

print("\n== Proportion of lambda above 10 /m ==")
print(
 "BH lambda > 10 /m:",
 np.mean(df["lambda_borehole"] > 10.0),
)

print(
 "Face lambda > 10 /m:",
 np.mean(df["lambda_face_scanline"] > 10.0),
)

```

)

이 결과가 중요합니다.

---

#### 4. 해석 기준

경우 1. **lambda\_borehole, lambda\_face\_scanline**이 대부분 10보다 훨씬 작다

예:

```
text
lambda_borehole median ≈ 0.5~2 /m
lambda_face_scanline median ≈ 0.5~2 /m
```

그러면 apparent spacing은:

```
text
1 / lambda ≈ 0.5~2 m
```

입니다.

이 경우 RQD가 100 근처인 것은 정상입니다.

즉:

```
text
현재 DFN이 RQD 기준으로 너무 양호하거나,
scanline이 절리를 충분히 자주 만나지 못하고 있음
```

입니다.

이 경우 해야 할 일은:

```
text
P32를 증가시켜야 함
절리군을 더 조밀하게 해야 함
더 불량한 RQD 검증용 case를 추가해야 함
```

입니다.

---

경우 2. **lambda**가 10 이상인데 RQD가 계속 100에 가깝다

예:

```
text
lambda_face_scanline median > 10 /m
그런데 RQD_face_scanline median = 100
```

이 경우는 의심해야 합니다.

왜냐하면  $\lambda$ 가 10/m이면 평균 apparent spacing은:

```
text
1 / 10 = 0.1 m
```

입니다.

이 정도면 RQD가 꽤 낮아져야 합니다.

Hudson 식 기준으로:

```
text
λ = 10 /m
λt = 1
RQD = 100 × exp(-1) × 2 ≈ 73.6
```

입니다.

따라서  $\lambda$ 가 10/m 이상인데 RQD가 100이면 다음을 의심해야 합니다.

```
text
1. intersection distance가 제대로 RQD 함수로 들어가지 않음
2. t 값 단위가 잘못됨
3. threshold 0.1 m가 실제 좌표 단위와 맞지 않음
4. 중복 교차점 제거 또는 filtering이 과도하게 작동
5. scanline 길이가 생각보다 매우 길게 잡힘
6. lambda 컬럼 계산 방식과 RQD 계산 방식이 다른 절리 집합을 사용
```

---

경우 3. YAML의 **mean\_spacing**은 0.1 이하인데 **lambda**는 작다

이건 충분히 가능합니다.

특히 **density\_type == "P32"**이면:

```
text
mean_spacing은 실제 DFN 생성에 사용되지 않기 때문
```

입니다.

또한 절리 방향성 때문에 scanline이 절리를 잘 못 만나도 lambda는 작아질 수 있습니다.

따라서 이 경우는 코드 버그라기보다 설정 해석 문제일 가능성이 큽니다.

---

## 5. RQD가 낮아지려면 어느 정도 lambda가 필요할까?

Hudson 식 기준으로 감을 잡으면 다음과 같습니다.

**$\lambda$ , intersections/m    평균    apparent spacing    Hudson RQD**

|       |         |        |
|-------|---------|--------|
| 1 /m  | 1.0 m   | 약 99.5 |
| 2 /m  | 0.5 m   | 약 98.2 |
| 5 /m  | 0.2 m   | 약 91.0 |
| 10 /m | 0.1 m   | 약 73.6 |
| 15 /m | 0.067 m | 약 55.8 |
| 20 /m | 0.05 m  | 약 40.6 |

즉, RQD가 의미 있게 낮아지려면:

text

lambda가 적어도 5~10 /m 이상

이어야 합니다.

현재 RQD가 99~100이라는 것은 대체로:

text

lambda가 0~2 /m 수준

일 가능성이 큽니다.

---

## 6. P32를 얼마나 키워야 하나?

현재 density\_type == P32라면 절리 밀도는 P32가 좌우합니다.

대략적인 방향성 효과를 단순화하면:

text

$\lambda_{app} \approx \sum P32_i \times |n_i \cdot d|$

입니다.

여기서:

```
text
d = scanline direction
n_i = 절리군 법선
```

입니다.

만약 평균적으로  $|n \cdot d| \approx 0.5$ 라고 보면:

```
text
lambda_app $\approx 0.5 \times P32_total$
```

입니다.

그러면 목표 lambda에 필요한 P32\_total은 대략:

```
text
lambda 5 /m $\rightarrow P32_total \approx 10$
lambda 10 /m $\rightarrow P32_total \approx 20$
lambda 15 /m $\rightarrow P32_total \approx 30$
```

입니다.

현재 Jv proxy 결과에서  $RQD\_face\_jv = 100$ 이었으므로:

```
text
Jv_proxy = $\sum P32_i \leq$ 약 4.5
```

일 가능성이 큼니다.

그러면 현재 P32\_total은 RQD를 낮추기에는 너무 작습니다.

따라서 RQD 검증용 case를 만들려면 P32를 꽤 크게 올려야 합니다.

---

## 7. 우선 만들어볼 RQD 검증용 case 수준

기존 6개 case는 Q' 검증용으로는 의미가 있었지만, RQD 검증용으로는 너무 양호한 것 같습니다.

따라서 RQD 전용 case를 따로 추가하는 게 좋습니다.

예:

```
text
granite_rqd_dense.yaml
granite_rqd_very_dense.yaml
granite_rqd_heavily_fractured.yaml
```

P32\_total 기준으로는 대략:

text

```
mild: ΣP32 ≈ 5
dense: ΣP32 ≈ 10
very_dense: ΣP32 ≈ 20
extreme: ΣP32 ≈ 30
```

정도를 테스트해보면 좋습니다.

단, 너무 갑자기 30으로 올리면 DFN 생성 절리 수가 폭증할 수 있으니 계산 시간이 늘 수 있습니다.

추천 순서:

text

```
1차: 기존 P32의 2배
2차: 기존 P32의 5배
3차: 기존 P32의 10배
```

또는 절리군별로 예를 들면:

yaml

```
joint_sets:
- P32: 3.0
- P32: 3.0
- P32: 3.0
```

이면:

text

```
ΣP32 = 9
```

입니다.

더 강하게는:

yaml

```
joint_sets:
- P32: 6.0
- P32: 6.0
- P32: 6.0
```

이면:

text

```
ΣP32 = 18
```

입니다.



이 정도면 RQD가 내려가기 시작할 가능성이 있습니다.

---

## 8. 단, P32만 올리면 안 될 수도 있음

RQD는 10cm 미만 조각이 얼마나 생기느냐가 중요합니다.

P32를 올려도 다음 조건이면 line이 절리를 충분히 못 만날 수 있습니다.

text

1. 절리 방향이 scanline에 불리함
2. disk radius가 작음
3. domain 내 절리 위치가 sparse함
4. scanline이 중앙 하나라서 운 좋게 절리를 피함

따라서 RQD 검증용 case에서는 다음도 조정할 수 있습니다.

text

1. P32 증가
2. 절리군 방향을 face scanline 또는 borehole 방향에 잘 걸리도록 설정
3. fisher\_kappa를 높여 방향성 contrast 강화
4. 절리 radius를 충분히 크게 설정
5. seed 수 증가

하지만 우선은 P32부터 올리는 게 맞습니다.

---

## 9. 먼저 확인해야 할 두 가지

다음 순서로 가면 됩니다.

### Step 1. 현재 결과에서 lambda 확인

아래 값을 먼저 보여주세요.

text

```
lambda_borehole describe
lambda_face_scanline describe
spacing_borehole_apparent describe
spacing_face_scanline_apparent describe
Jv_face describe
```

이걸 보면 현재 RQD가 높은 이유가 바로 확인됩니다.

---

## Step 2. 현재 YAML의 P32 확인

각 case별로 joint set의 다음 값을 확인해야 합니다.

```
text
density_type
P32
mean_spacing
```

특히:

```
text
density_type == P32이면 mean_spacing은 무시됨
```

입니다.

즉, `mean_spacing`이 작게 되어 있어도 아무 의미 없을 수 있습니다.

---

## 10. 간단한 YAML 점검 스크립트

YAML 파일들이 예를 들어 `configs/cases/`에 있다면, 이런 식으로 확인할 수 있습니다.

```
python
import glob
import yaml

case_files = glob.glob("configs/cases/*.yaml")

for path in case_files:
 print("\n" + "=" * 80)
 print(path)

 with open(path, "r", encoding="utf-8") as f:
 cfg = yaml.safe_load(f)

 # 실제 구조에 따라 joint_config 위치는 다를 수 있음
 # 아래는 예시이므로, YAML 구조에 맞춰 key를 조정해야 함.
 joint_sets = None

 if "joint_sets" in cfg:
 joint_sets = cfg["joint_sets"]
 elif "joint_config" in cfg and "joint_sets" in cfg["joint_config"]:
 joint_sets = cfg["joint_config"]["joint_sets"]

 if joint_sets is None:
 print("Cannot find joint_sets")
 continue

 p32_total = 0.0
```

```

inv_spacing_total = 0.0

for i, js in enumerate(joint_sets):
 density_type = js.get("density_type")
 p32 = js.get("P32")
 mean_spacing = js.get("mean_spacing")

 print(f"\nJoint set {i+1}")
 print(f" density_type: {density_type}")
 print(f" P32: {p32}")
 print(f" mean_spacing: {mean_spacing}")

 if p32 is not None:
 p32_total += float(p32)

 if mean_spacing is not None and float(mean_spacing) > 0:
 inv_spacing_total += 1.0 / float(mean_spacing)

print("\nTotals")
print(f" sum(P32): {p32_total}")
print(f" sum(1/spacing): {inv_spacing_total}")
print(f" RQD_Jv_from_P32: {max(0, min(100, 115 - 3.3 * p32_total))}")
print(f" RQD_Jv_from_spacing: {max(0, min(100, 115 - 3.3 * inv_spacing_total))}")

```

이 스크립트에서 핵심은:

```

text
sum(P32)가 4.5 이하이면 Jv RQD는 100으로 clipping됨
sum(P32)가 10이면 Jv RQD ≈ 82
sum(P32)가 20이면 Jv RQD ≈ 49
sum(P32)가 30이면 Jv RQD ≈ 16

```

입니다.

## 11. 코드가 잘못되었는지 확인하는 직접 테스트

`RQDCalculator.calc_direct()` 자체가 정상인지 간단히 확인해볼 수 있습니다.

예를 들어 1m line에서 절리 간격이 5cm마다 있으면 RQD는 0에 가까워야 합니다.

```

python
from src.core.rqd_calculator import RQDCalculator

1m 구간에서 5cm마다 절리
t_vals = [0.05 * i for i in range(1, 20)] # 0.05, 0.10, ..., 0.95

rqd = RQDCalculator.calc_direct(t_vals, 1.0, threshold=0.1)

```

```
print(rqd)
```

예상:

```
text
```

```
거의 0 또는 매우 낮은 값
```

왜냐하면 segment가 대부분 0.05m라서 0.1m 미만이기 때문입니다.

반대로 20cm마다 있으면:

```
python
```

```
t_vals = [0.2, 0.4, 0.6, 0.8]
```

```
rqd = RQDCalculator.calc_direct(t_vals, 1.0, threshold=0.1)
```

```
print(rqd)
```

예상:

```
text
```

```
100
```

왜냐하면 모든 segment가 0.2m 이상이기 때문입니다.

이 테스트가 정상이라면 RQD 계산 함수 자체는 괜찮습니다.

---

## 12. 결론

지금 상황에서 가장 중요한 판단은 이것입니다.

```
text
```

```
현재 RQD가 높은 이유가
```

```
A. 실제 scanline apparent spacing이 10cm보다 훨씬 크기 때문인지,
```

```
B. 평균 절리 간격은 작는데 RQD 계산 또는 DFN 교차 계산이 잘못되었기 때문인지
```

를 구분해야 합니다.

현재 결과만 보면 저는 A일 가능성이 높다고 봅니다.

특히:

```
text
```

```
RQD_borehole_hudson mean = 99.7
```

```
RQD_face_jv = 100
```

이라는 점은 현재 P32/λ가 RQD를 낮출 만큼 크지 않다는 신호입니다.

하지만 확정하려면 다음을 확인해야 합니다.

```
text
lambda_borehole
lambda_face_scanline
apparent spacing = 1 / lambda
Jv_face
각 YAML의 P32_total
```

이 네 가지를 보면 바로 판단할 수 있습니다.

다음으로는 그 결과를 보고:

```
text
1. P32를 몇 배 올릴지
2. RQD 검증용 dense case를 어떻게 만들지
3. Jv가 100에 포화되지 않도록 할지
```

를 정하면 됩니다.

GPT-5.5

script 실행결과야.

=== Available columns check ===

```
lambda_borehole True
lambda_face_scanline True
borehole_n_intersections_total True
face_scanline_n_intersections True
Jv_face True
```

=== Lambda / count summary ===

```
lambda_borehole lambda_face_scanline borehole_n_intersections_total face_scanline_n_intersections Jv_face
count 480.000000 480.000000 480.000000 480.000000 480.000000
mean 0.392857 0.540417 2.750000 5.404167 0.925000
std 0.437086 0.717505 3.059603 7.175054 0.721718
min 0.000000 0.000000 0.000000 0.000000 0.250000
10% 0.000000 0.000000 0.000000 0.000000 0.250000
25% 0.000000 0.100000 0.000000 1.000000 0.300000
50% 0.285714 0.200000 2.000000 2.000000 0.700000
75% 0.571429 0.600000 4.000000 6.000000 1.400000
90% 1.142857 1.900000 8.000000 19.000000 2.200000
max 2.000000 3.300000 14.000000 33.000000 2.200000
```

/Users/myungkyusong/Documents/Develop/12. QSimEx/temp\_ana\_rqd.py:28: PerformanceWarning: DataFrame is highly fragmented. This is usually the result of calling `frame.insert` many times, which has poor performance.

Consider joining all columns at once using `pd.concat(axis=1)` instead. To get a de-fragmented frame, use `newframe = frame.copy()`

```
df["spacing_borehole_apparent"] = np.where(
/Users/myungkyusong/Documents/Develop/12. QSimEx/temp_ana_rqd.py:34: PerformanceWarning: DataFrame is highly fragmented. This is usually the result of calling frame.insert many times, which has poor performance.
```

Consider joining all columns at once using `pd.concat(axis=1)` instead. To get a de-fragmented frame, use `newframe = frame.copy()`

```
df["spacing_face_scanline_apparent"] = np.where(
```

```
=== Apparent spacing summary ===
spacing_borehole_apparent spacing_face_scanline_apparent
count 345.000000 392.000000
mean 3.329473 4.150006
std 2.410515 3.459787
min 0.500000 0.303030
10% 0.777778 0.478571
25% 1.400000 1.250000
50% 2.333333 3.333333
75% 7.000000 5.000000
90% 7.000000 10.000000
max 7.000000 10.000000
```

```
=== Proportion of apparent spacing below 0.1 m ===
BH spacing < 0.1 m: 0.0
Face scanline spacing < 0.1 m: 0.0
```

```
=== Proportion of lambda above 10 /m ===
BH lambda > 10 /m: 0.0
Face lambda > 10 /m: 0.0
```



좋습니다. 이 결과로는 거의 확정적으로 말할 수 있습니다.

현재 RQD가 100 근처로 나온 것은 코드 오류 때문이라기보다, 실제로 시추공/굴진면 scanline에서 만난 절리 간격이 RQD를 낮출 만큼 조밀하지 않기 때문입니다.

즉, 지금 상황은:

```
text
평균 절리 간격이 10cm 이하인데도 RQD가 높게 나온 것
```

이 아니라,

```
text
scanline apparent spacing이 대부분 0.3m~10m 수준이라서 RQD가 높게 나온 것
```

입니다.

---

## 1. 핵심 진단 결과

### 1-1. Borehole lambda

```
text
lambda_borehole mean = 0.393 /m
median = 0.286 /m
90% = 1.143 /m
```

max = 2.000 /m

시추공에서 절리 교차 빈도가 매우 낮습니다.

평균적으로:

text  
0.393 intersections/m

이므로 apparent spacing은 대략:

text  
1 / 0.393 ≈ 2.54 m

입니다.

즉, 시추공 line 위에서는 평균적으로 절리를 2.5m에 한 번 만나는 정도입니다.

이러면 RQD가 100에 가까운 게 정상입니다.

---

## 1-2. Face scanline lambda

text  
lambda\_face\_scanline mean = 0.540 /m  
median = 0.200 /m  
90% = 1.900 /m  
max = 3.300 /m

face scanline도 마찬가지로입니다.

평균적으로:

text  
0.540 intersections/m

이므로 apparent spacing은 대략:

text  
1 / 0.540 ≈ 1.85 m

입니다.

최대 λ도 3.3/m입니다.

즉 가장 조밀한 경우에도 평균 apparent spacing은:

text

```
1 / 3.3 ≈ 0.303 m
```

입니다.

30cm 정도입니다.

RQD threshold는 10cm이므로, 이 정도로는 RQD가 크게 낮아지지 않습니다.

---

## 2. 가장 중요한 확인 결과

당신이 걱정한 핵심은 이것이었죠.

평균 절리 간격이 10cm 이하인데도 RQD가 높다면 코드가 잘못된 것 아닌가?

이번 결과에서 확인된 것은:

text

```
BH apparent spacing < 0.1m 비율 = 0.0
Face apparent spacing < 0.1m 비율 = 0.0
BH lambda > 10/m 비율 = 0.0
Face lambda > 10/m 비율 = 0.0
```

즉, 어느 경우에도 apparent spacing이 10cm 이하로 내려가지 않았습니다.

따라서 현재 RQD가 높은 것은 정상입니다.

---

## 3. RQD가 높을 수밖에 없는 수치적 이유

RQD는 10cm 미만 조각이 생겨야 떨어집니다.

그런데 현재 apparent spacing은:

text

```
BH min spacing = 0.50 m
Face min spacing = 0.303 m
```

입니다.

즉, 관측된 apparent spacing의 최소값도 10cm보다 훨씬 큼니다.

text

```
0.303 m = 30.3 cm
```



```
0.500 m = 50.0 cm
```

따라서 scanline 위 절리 간격 기준으로는 RQD를 낮출 조건이 거의 없습니다.

그래서:

```
text
RQD ≈ 100
```

이 되는 것이 맞습니다.

---

#### 4. Jv도 너무 낮음

```
text
Jv_face mean = 0.925
max = 2.2
```

Palmström형 식:

```
text
RQD = 115 - 3.3 Jv
```

Jv가 최대 2.2이면:

```
text
RQD_raw = 115 - 3.3 × 2.2
 = 115 - 7.26
 = 107.74
```

그런데 RQD는 100을 넘을 수 없으므로 clipping하면:

```
text
RQD = 100
```

입니다.

따라서 RQD\_face\_jv가 전부 100인 것도 정상입니다.

Jv 기반 RQD가 100 아래로 내려가려면:

```
text
115 - 3.3 Jv < 100
Jv > 4.545
```

이어야 합니다.

현재 max가 2.2이므로 한참 낮습니다.

---

## 5. 현재 암반 case의 실제 의미

지금 6개 case는 이름은 dense, medium, sparse가 섞여 있지만, RQD 관점에서는 모두 매우 양호한 암반입니다.

정확히는:

text  
절리 교차 빈도가 RQD를 낮출 정도로 높지 않다.

입니다.

즉, Q'에서는 Jr/Ja, 방향성, Jn, 보정 모델 때문에 차이가 보일 수 있지만, RQD만 놓고 보면:

text  
대부분 excellent rock mass

입니다.

따라서 현재 case들은 RQD 검증용으로는 너무 sparse합니다.

---

## 6. 결론: 절리 간격을 줄여야 함

네, 맞습니다. 다음 단계는 절리 밀도를 올리는 것입니다.

현재 목표는:

text  
RQD가 100에 포화되지 않는 case 만들기

입니다.

이를 위해서는 scanline  $\lambda$ 가 적어도 다음 수준까지 올라가야 합니다.

| 목표       | $\lambda$ 수준 | apparent spacing | 예상 Hudson RQD |
|----------|--------------|------------------|---------------|
| 약간 저하    | 5/m          | 0.20 m           | 약 91          |
| 중간 저하    | 10/m         | 0.10 m           | 약 74          |
| 강한 저하    | 15/m         | 0.067 m          | 약 56          |
| 매우 강한 저하 | 20/m         | 0.05 m           | 약 41          |

현재는:

text

BH mean  $\lambda = 0.39/m$   
Face mean  $\lambda = 0.54/m$

이므로 목표치와 비교하면 매우 낮습니다.

대략:

text

RQD 저하 시작 목표  $\lambda = 5/m$   
현재  $\lambda \approx 0.5/m$

즉 최소 10배 정도는 올려야 RQD가 움직이기 시작합니다.

---

## 7. P32를 얼마나 올려야 할까?

현재  $J_v\_face\ mean = 0.925$ ,  $max = 2.2$ 입니다.

만약  $J_v\_face$ 가  $\Sigma P32$  기반 proxy라면, 현재 총 P32도 대략 이 정도입니다.

text

$\Sigma P32 \approx 0.25 \sim 2.2$

이건 RQD를 낮추기에는 너무 작습니다.

$J_v$  기반 RQD를 100 아래로 내리려면:

text

$\Sigma P32 > 4.545$

필요합니다.

그리고 scanline RQD를 의미 있게 낮추려면 대략:

text

$\Sigma P32 \approx 10 \sim 20$  이상

은 필요할 가능성이 큼니다.

---

## 8. 추천: RQD 전용 교밀도 case 추가

기존 6개 case를 억지로 바꾸기보다는, **RQD 검증용 case**를 별도로 추가하는 게 좋습니다.

예를 들어:

```
text
granite_rqd_p32_05.yaml
granite_rqd_p32_10.yaml
granite_rqd_p32_20.yaml
granite_rqd_p32_30.yaml
```

여기서 이름의 숫자는  $\Sigma P32$  총합을 의미하게 하면 됩니다.

예:

#### Case A: RQD mild fractured

```
text
 $\Sigma P32 \approx 5$
```

예상:

```
text
Jv RQD $\approx 115 - 3.3 \times 5 = 98.5$
```

Jv는 아직 높지만 clipping은 피하기 시작합니다.

scanline RQD는 조금만 낮아질 수 있습니다.

---

#### Case B: RQD dense fractured

```
text
 $\Sigma P32 \approx 10$
```

예상:

```
text
Jv RQD ≈ 82
```

scanline RQD도 90대 초반 또는 조건에 따라 더 낮아질 수 있습니다.

---

#### Case C: RQD very dense fractured

```
text
 $\Sigma P32 \approx 20$
```

예상:

text

```
Jv RQD ≈ 49
```

이 정도면 RQD 차이가 뚜렷하게 나올 가능성이 큼니다.

---

#### Case D: RQD heavily fractured

text

```
ΣP32 ≈ 30
```

예상:

text

```
Jv RQD ≈ 16
```

이건 매우 불량한 조건입니다. 계산량이 많이 늘 수 있으므로 주의해야 합니다.

---

#### 9. 절리군별 P32 설정 예

절리군 3개를 쓴다면:

**ΣP32 = 5**

yaml

```
P32:
 set1: 1.7
 set2: 1.7
 set3: 1.6
```

**ΣP32 = 10**

yaml

```
P32:
 set1: 3.3
 set2: 3.3
 set3: 3.4
```

**ΣP32 = 20**

yaml

```
P32:
 set1: 6.7
 set2: 6.7
 set3: 6.6
```

**$\Sigma P32 = 30$**

```
yaml
P32:
 set1: 10.0
 set2: 10.0
 set3: 10.0
```

다만 P32를 이렇게 올리면 DFN 생성 절리 수가 크게 늘어납니다.

따라서 먼저:

```
text
ΣP32 = 5, 10, 15
```

정도부터 테스트하는 것을 추천합니다.

---

## 10. 계산량 폭증을 피하는 방법

P32를 올리면 절리 개수가 증가합니다.

DFN 생성식이:

```
python
n_expected = P32 * domain_volume / mean_area
```

이므로 P32가 10배가 되면 절리 개수도 대략 10배가 됩니다.

계산량이 부담되면 다음을 조정할 수 있습니다.

### 방법 1. RQD 검증용 domain을 작게 만들기

예:

```
text
x length 줄이기
face positions 줄이기
seed 수 줄이기
```

초기 테스트는:

```
text
3 cases * 3 seeds * 5 faces
```

정도만 해도 충분합니다.

---

## 방법 2. 절리 radius를 키우기

절리 반경이 커지면 동일 P32에서 필요한 절리 개수는 줄어들 수 있습니다.

생성식에서:

```
python
n_expected = P32 * domain_volume / mean_area
```

이므로 mean\_area가 커지면 절리 개수는 줄어듭니다.

단, 절리 크기를 너무 키우면 face/borehole 교차 특성이 바뀝니다.

---

## 방법 3. 먼저 한 case만 테스트

우선 하나만 만드세요.

```
text
granite_rqd_dense_p32_10.yaml
```

그리고:

```
text
1 case * 3 seeds * 5 faces
```

로 RQD가 내려가지만 확인합니다.

---

## 11. 좋은 RQD 검증용 case의 조건

RQD 검증용 case는 기존 Q' 검증용 case와 목적이 다릅니다.

목표는 RQD가 100에 포화되지 않게 만드는 것입니다.

따라서 다음 조건이 필요합니다.

- ```
text
1. ΣP32가 충분히 큼
2. face scanline과 borehole이 절리를 충분히 만남
3. 절리 방향성 contrast가 있음
4. Jv가 4.5 이상이 되도록 설정
```

추천 구성은 다음입니다.

RQD case set

```
text
rqd_low_density:
  ΣP32 ≈ 5

rqd_medium_density:
  ΣP32 ≈ 10

rqd_high_density:
  ΣP32 ≈ 15

rqd_very_high_density:
  ΣP32 ≈ 20
```

그리고 orientation은 두 가지:

```
text
rqd_high_bh_favorable:
  borehole이 절리를 잘 만나는 방향

rqd_high_face_favorable:
  face scanline이 절리를 잘 만나는 방향
```

하지만 처음부터 너무 늘리지 말고, 우선 density sweep만 하면 됩니다.

12. 지금의 기존 6개 case는 버릴 필요 없음

기존 case들은 다음 결론에 쓸 수 있습니다.

```
text
양호한 암반 조건에서는 RQD가 100에 포화되어,
시추공-굴진면 RQD 차이를 평가하기 어렵다.
```

이건 의미 있는 결과입니다.

그리고 새 RQD-dense case에서 다음을 보게 됩니다.

```
text
절리 밀도가 높아져 RQD가 포화되지 않을 때,
시추공 RQD와 굴진면 RQD 차이가 어떻게 나타나는가?
```

이렇게 하면 연구가 훨씬 좋아집니다.

13. 다음 작업 제안

지금 바로 할 일은 이것입니다.

Step 1. 현재 YAML에서 P32 총합 확인

각 case 별:

```
text
ΣP32
Jv_proxy
RQD_Jv_raw
```

을 확인합니다.

이미 Jv_face 결과를 보면 대략 알 수 있지만 YAML에서도 확인해야 합니다.

Step 2. RQD 전용 case 3개 추가

추천:

```
text
granite_rqd_p32_05.yaml
granite_rqd_p32_10.yaml
granite_rqd_p32_20.yaml
```

각각:

```
text
ΣP32 = 5
ΣP32 = 10
ΣP32 = 20
```

Step 3. 작은 batch로 시험

예:

```
text
3 cases × 3 seeds × 5 faces = 45 rows
```

먼저 RQD summary만 확인합니다.

목표는:

```
text
RQD_face_scanline p50이 100에서 내려오는지
```

RQD_borehole_direct p50이 100에서 내려오는지
lambda가 5/m 이상으로 올라가는지
Jv가 4.5 이상인지

14. 이번 진단의 결론

이번 결과는 매우 명확합니다.

text

현재 RQD가 높은 이유는 scanline apparent spacing이 10cm 이하가 아니기 때문이다.
BH apparent spacing 최소값은 0.5m, face apparent spacing 최소값은 약 0.303m이다.
따라서 RQD 계산이 잘못되었다기보다는, 현재 DFN의 P32/Jv/scanline 교차 빈도가 RQD를 낮추기에는 너무 낮다.

따라서 다음 조치는:

text

RQD 검증용으로 더 높은 P32_total을 갖는 case를 별도로 추가한다.

입니다.

GPT-5.5

모델의 크기를 어디까지 낮출 수 있을까? 너무 낮추면 결과에 영향을 미칠 것 같아서 안전 마진을 고려한 크기를 대략적으로 생각해볼 수 있지?



좋은 질문입니다. **모델 크기를 줄일 수는 있지만**, DFN/RQD에서는 너무 줄이면 바로 **경계효과 boundary effect**가 생깁니다. 특히 지금처럼 RQD가 거의 100으로 포화되는 상황에서는 모델을 작게 줄였을 때 절리 교차 수가 더 줄어들어, 결과가 더 양호하게 왜곡될 수 있습니다.

그래서 모델 크기는 단순히 “터널 지름만 포함”하면 안 되고, 최소한 다음을 포함해야 합니다.

text

1. 터널 단면
2. face scanline
3. 시추공 window
4. 최대 절리 반경에 대한 buffer
5. face position 앞뒤 여유

1. 핵심 원칙

DFN에서 절리는 disk입니다.

어떤 scanline과 절리 disk가 교차하려면, 절리 중심이 scanline 바로 위에 있지 않아도 됩니다. 절리 반경이 크면 중심이 scanline 밖에 있어도 scanline을 자를 수 있습니다.

따라서 domain을 너무 작게 만들면, 원래는 scanline을 교차해야 할 절리들이 **domain 밖에서 생성되지 않아서 사라집니다.**

그 결과:

```
text
절리 교차 수 감소
lambda 감소
apparent spacing 증가
RQD 증가
```

즉, 모델을 너무 줄이면 RQD가 인위적으로 높아질 수 있습니다.

2. 최소 domain 크기를 정하는 기준

좌표계를 다음처럼 두겠습니다.

```
text
x = 터널 축 / 시추공 방향
y = 굴진면 수평 방향
z = 굴진면 수직 방향
R = 터널 반경
r_max = 절리 최대 반경
W_bh = borehole_window 실제 길이
```

현재 face scanline은:

```
text
x = face_x
z = center_z
y = center_y - R ~ center_y + R
```

즉 길이는:

```
text
L_face_scanline = 2R
```

입니다.

3. y, z 방향 최소 크기

터널 단면 전체와 절리 disk buffer를 포함해야 합니다.

가장 기본적인 안전식은:

```
text
Ly_min = 2R + 2M
Lz_min = 2R + 2M
```

여기서 M 은 boundary margin입니다.

권장 margin은:

```
text
M = max(2 * r_max, R)
```

조금 공격적으로 줄이면:

```
text
M = r_max
```

도 가능하지만, RQD 검증에서는 저는 추천하지 않습니다.

예시

터널 지름이 10m라면:

```
text
R = 5m
```

절리 최대 반경이 예를 들어:

```
text
r_max = 3m
```

이면 안전 margin:

```
text
M = max(2*3, 5) = 6m
```

따라서:

```
text
Ly_min = 2*5 + 2*6 = 22m
Lz_min = 22m
```

즉 y-z 단면은 최소 약:

```
text
22m × 22m
```

정도는 두는 게 안전합니다.

조금 더 줄이는 공격적 설정은:

```
text
M = r_max = 3m
Ly_min = 10 + 6 = 16m
Lz_min = 16m
```

하지만 이걸 경계효과 검증을 반드시 해야 합니다.

4. x 방향 최소 크기

x 방향은 다음을 포함해야 합니다.

```
text
1. 분석할 face positions 범위
2. 각 face 주변 borehole_window
3. 절리 반경 buffer
```

만약 분석할 face 위치 범위가:

```
text
x_face_min ~ x_face_max
```

라면 최소 x 길이는:

```
text
Lx_min = (x_face_max - x_face_min) + 2 * W_bh + 2 * Mx
```

여기서:

```
text
W_bh = borehole_window * dx
Mx = max(2 * r_max, W_bh)
```

정도를 추천합니다.

만약 RQD 테스트용으로 face를 몇 개만 쓸 거라면, 전체 긴 터널 domain이 필요 없습니다.

예를 들어:

text

```
face positions를 중앙부 5개만 사용  
face 간격 = 5m  
face span = 20m  
borehole_window = ±5 cells  
dx = 1m → W_bh = 5m  
r_max = 3m
```

이면:

text

```
Mx = max(6m, 5m) = 6m  
Lx_min = 20 + 2*5 + 2*6 = 42m
```

즉 x 방향은 약 40~50m 정도면 1차 RQD 검증에는 충분할 수 있습니다.

5. 현실적인 추천 크기

현재 목적이 **RQD 고밀도 case**를 **빠르게 시험**하는 것이라면, 저는 세 단계로 추천합니다.

5-1. 안전한 축소 모델

터널 지름 10m, 즉 R=5m 기준으로:

text

```
Lx = 50m  
Ly = 25m  
Lz = 25m
```

추천 용도:

text

```
RQD dense case 1차 검증  
P32 증가 테스트  
시추공 vs face scanline 비교
```

이 정도면 기존 대형 모델보다 훨씬 가볍지만, 경계효과는 비교적 작을 가능성이 큼니다.

5-2. 중간 축소 모델

text

```
Lx = 40m  
Ly = 20m
```

Lz = 20m

추천 용도:

text

여러 P32 case 빠른 screening

3 seeds 정도 테스트

다만 절리 최대 반경이 3~5m 이상이면 약간 위험할 수 있습니다.

5-3. 공격적 최소 모델

text

Lx = 30m

Ly = 16m

Lz = 16m

추천 용도:

text

코드 작동 확인

P32를 올렸을 때 RQD가 내려가는지만 확인

하지만 이 크기에서는 최종 결과로 쓰지 않는 게 좋습니다.

경계효과가 꽤 클 수 있습니다.

6. 제가 추천하는 실제 진행안

처음부터 너무 줄이지 말고 이렇게 하세요.

Step 1. RQD dense case 빠른 screening

text

domain size:

Lx = 40m

Ly = 20m

Lz = 20m

seeds:

3개

faces:

중앙부 5개

P32 total:

5, 10, 20

목적:

text

RQD가 100에서 내려가는지 확인

lambda가 5/m 이상으로 올라가는지 확인

Jv가 4.5 이상이 되는지 확인

Step 2. 경계효과 확인

같은 case 하나를 골라서 두 domain으로 비교합니다.

text

small:

40 × 20 × 20

safe:

60 × 30 × 30

비교할 값:

text

lambda_borehole

lambda_face_scanline

RQD_borehole_direct

RQD_face_scanline

Jv_face

만약 차이가 작으면 small model을 계속 써도 됩니다.

예를 들어:

text

RQD 평균 차이 < 2 points

lambda 평균 차이 < 10~15%

정도면 screening 용도로는 충분합니다.

Step 3. 최종용 모델

최종 분석에는 조금 더 안전하게:

text

Lx = 60m

Ly = 30m


```
Lz = 30m
```

정도를 추천합니다.

특히 고밀도 P32에서 절리 수가 많아져도 통계적으로 안정적입니다.

7. 모델을 줄일 때 반드시 지켜야 하는 조건

조건 1. face는 domain 중앙부에서만 뽑기

작은 모델에서는 face를 경계 근처에서 뽑으면 안 됩니다.

예:

```
text
face_x < Mx
face_x > Lx - Mx
```

인 위치는 제외해야 합니다.

추천:

```
text
face_x는 x 중앙 50% 구간에서만 선택
```

예를 들어 $Lx = 40m$ 이면:

```
text
face_x = 15, 20, 25m
```

정도가 안전합니다.

조건 2. borehole window가 domain 밖으로 나가지 않게 하기

현재 코드에서는:

```
python
x_start = max(0, face_x_idx - borehole_window)
x_end = min(domain.nx, face_x_idx + borehole_window + 1)
```

로 잘라냅니다.

하지만 이렇게 잘리면 window 길이가 face 위치에 따라 달라지고, 경계 근처에서 bias가 생깁니다.

따라서 작은 domain에서는 face 위치를 정할 때:

```
text
face_x_idx - borehole_window >= margin
face_x_idx + borehole_window <= nx - margin
```

이 되도록 해야 합니다.

조건 3. y/z 방향에서 터널 단면이 domain 중앙에 있어야 함

터널 중심이 domain 중앙에 있어야 합니다.

```
text
center_y = Ly / 2
center_z = Lz / 2
```

그리고:

```
text
center_y - R - M >= 0
center_y + R + M <= Ly
center_z - R - M >= 0
center_z + R + M <= Lz
```

을 만족해야 합니다.

8. 왜 너무 작은 모델은 위험한가?

예를 들어 y 방향 domain을 터널 지름 10m와 거의 같게:

```
text
Ly = 10m
```

로 만들면 face scanline은 $y=-R \sim +R$ 을 꽉 채웁니다.

그러면 scanline을 가로지를 수 있는 절리 disk 중에서 중심이 domain 밖에 있어야 하는 절리들이 생성되지 않습니다.

결과:

```
text
face_scanline_n_intersections 감소
lambda_face_scanline 감소
RQD_face_scanline 증가
```

즉, 작은 모델이 암반을 인위적으로 좋게 만듭니다.

이게 가장 위험합니다.

9. 절리 최대 반경 기준의 안전 마진

실무적으로는 다음 기준을 쓰면 됩니다.

```
text
최소 margin:
  M = r_max

권장 margin:
  M = 2 * r_max

보수적 margin:
  M = 3 * r_max
```

단, 터널 반경이 5m라면 너무 작은 margin은 피하고:

```
text
M >= 5m
```

를 권장합니다.

그래서 최종 추천식은:

```
text
M = max(2 * r_max, R)
```

입니다.

10. 계산량과 모델 크기의 관계

DFN 절리 개수는 대략:

```
text
N ∝ P32 * domain_volume / mean_area
```

입니다.

즉 domain 부피를 절반으로 줄이면 절리 개수도 대략 절반으로 줄어듭니다.

하지만 P32를 10배 올리면 절리 개수는 10배 증가합니다.

따라서 RQD dense test에서는:

text

```
domain을 1/5로 줄이고  
P32를 10배 올리면  
전체 절리 수는 대략 2배
```

같은 식으로 계산량을 조절할 수 있습니다.

그래서 작은 domain을 쓰는 것은 필요합니다.

다만 boundary margin만 확보해야 합니다.

11. 최종 추천

현재 RQD 전용 dense case 테스트용으로는 이 정도가 가장 적절해 보입니다.

1차 테스트용

text

```
Lx = 40m  
Ly = 20m  
Lz = 20m
```

조건:

text

```
R = 5m 기준  
터널 중심 = (Ly/2, Lz/2)  
face positions = 중앙부만 사용  
seeds = 3  
faces = 5  
P32_total = 5, 10, 20
```

안전 검증용

text

```
Lx = 60m  
Ly = 30m  
Lz = 30m
```

조건:

text

같은 P32 case 하나를 small model과 비교
lambda/RQD 차이 확인

최종 분석용

```
text
Lx = 60m 이상
Ly = 30m 이상
Lz = 30m 이상
```

또는 절리 최대 반경이 큰 경우:

```
text
Ly, Lz >= 2R + 4r_max
```

를 만족하도록 조정합니다.

12. 한 줄 결론

모델은 줄일 수 있지만, 최소한 다음 조건은 지켜야 합니다.

```
text
Ly, Lz >= 2R + 2M
Lx >= face_span + 2*borehole_window + 2M
M = max(2*r_max, R)
```

터널 지름 10m, R=5m 기준으로는:

```
text
빠른 테스트: 40m × 20m × 20m
안전 테스트: 60m × 30m × 30m
최종 권장:   60m × 30m × 30m 이상
```

정도가 좋습니다.

GPT-5.5

rqd 전용으로 만들어서 돌린 결과야.

```
=== Available columns check ===
lambda_borehole True
lambda_face_scanline True
borehole_n_intersections_total True
face_scanline_n_intersections True
Jv_face True
```

```

=== Lambda / count summary ===
lambda_borehole lambda_face_scanline borehole_n_intersections_total face_scanline_n_intersections Jv_face
count 72.000000 72.000000 72.000000 72.000000 72.000000
mean 0.775132 7.073611 4.888889 70.736111 11.666667
std 0.803295 6.027589 5.330105 60.275890 6.279858
min 0.000000 0.000000 0.000000 0.000000 5.000000
10% 0.000000 0.000000 0.000000 0.000000 5.000000
25% 0.000000 2.275000 0.000000 22.750000 5.000000
50% 0.500000 5.900000 3.000000 59.000000 10.000000
75% 1.178571 10.650000 7.000000 106.500000 20.000000
90% 2.142857 16.090000 14.900000 160.900000 20.000000
max 3.000000 18.600000 21.000000 186.000000 20.000000
/Users/myungkyusong/Documents/Develop/12. QSimEx/temp_ana_rqd.py:28: PerformanceWarning: DataFrame is
highly fragmented. This is usually the result of calling frame.insert many times, which has poor performance.
Consider joining all columns at once using pd.concat(axis=1) instead. To get a de-fragmented frame, use newframe =
frame.copy()
df["spacing_borehole_apparent"] = np.where(
/Users/myungkyusong/Documents/Develop/12. QSimEx/temp_ana_rqd.py:34: PerformanceWarning: DataFrame is
highly fragmented. This is usually the result of calling frame.insert many times, which has poor performance.
Consider joining all columns at once using pd.concat(axis=1) instead. To get a de-fragmented frame, use newframe =
frame.copy()
df["spacing_face_scanline_apparent"] = np.where(

=== Apparent spacing summary ===
spacing_borehole_apparent spacing_face_scanline_apparent
count 52.000000 55.000000
mean 1.628448 0.324559
std 1.297739 1.331254
min 0.333333 0.053763
10% 0.466667 0.061277
25% 0.595833 0.065371
50% 1.166667 0.126582
75% 2.333333 0.222332
90% 3.500000 0.274775
max 7.000000 10.000000

=== Proportion of apparent spacing below 0.1 m ===
BH spacing < 0.1 m: 0.0
Face scanline spacing < 0.1 m: 0.25

=== Proportion of lambda above 10 /m ===
BH lambda > 10 /m: 0.0
Face lambda > 10 /m: 0.25 <- 그리고 에러가 난 것 같은데 원인이 뭘까?

```



결론부터 말하면, 보여준 로그에서 **치명적인 에러는 아닙니다.**

보이는 것은 `PerformanceWarning`이고, 계산은 정상적으로 끝난 것으로 보입니다.

다만 결과 자체는 매우 중요합니다.

RQD 전용 case에서 **face scanline**은 충분히 조밀해졌는데, **borehole** 쪽은 여전히 조밀하지 않습니다.

즉, face RQD는 내려갈 가능성이 생겼지만, borehole RQD는 아직 높게 유지될 가능성이 큽니다.

이건 코드 에러라기보다, 오히려 **시추공과 굴진면 scanline의 방향성/샘플링 차이가 강하게 드러난 결과**일 수 있습니다.

1. 먼저 “에러”처럼 보이는 부분

로그에 나온 것은 이것입니다.

```
text
PerformanceWarning: DataFrame is highly fragmented.
```

이건 에러가 아니라 pandas 경고입니다.

원인은 분석 스크립트에서 DataFrame에 새 컬럼을 반복해서 추가했기 때문입니다.

예:

```
python
df["spacing_borehole_apparent"] = ...
df["spacing_face_scanline_apparent"] = ...
```

이미 컬럼이 많은 DataFrame에 이런 식으로 계속 컬럼을 추가하면 pandas가 내부적으로 조각난 DataFrame이라고 경고합니다.

의미

```
text
계산 결과가 틀렸다는 뜻은 아님.
프로그램이 멈춘 것도 아님.
성능이 느려질 수 있다는 경고.
```

간단한 해결

경고를 줄이려면 새 컬럼 추가 전에 다음을 넣으면 됩니다.

```
python
df = df.copy()
```

또는 새 컬럼을 한 번에 붙이면 됩니다.

```
python
new_cols = pd.DataFrame({
    "spacing_borehole_apparent": np.where(
        df["lambda_borehole"] > 0,
        1.0 / df["lambda_borehole"],
        np.inf,
    ),
    "spacing_face_scanline_apparent": np.where(
```

```

        df["lambda_face_scanline"] > 0,
        1.0 / df["lambda_face_scanline"],
        np.inf,
    ),
})

df = pd.concat([df.copy(), new_cols], axis=1)

```

하지만 지금 단계에서는 무시해도 됩니다.

2. 이번 결과의 핵심

새 RQD 전용 case 결과는 기존 case와 완전히 달라졌습니다.

기존 case

```

text
lambda_face_scanline mean ≈ 0.54 /m
max ≈ 3.3 /m

```

RQD 전용 case

```

text
lambda_face_scanline mean = 7.07 /m
median = 5.90 /m
75% = 10.65 /m
90% = 16.09 /m
max = 18.60 /m

```

이건 아주 큰 변화입니다.

즉, 굴진면 중앙 수평 scanline은 이제 RQD가 내려갈 만큼 절리를 많이 만나고 있습니다.

3. Face scanline은 이제 RQD 검증에 적합해졌음

현재 face apparent spacing은 다음입니다.

```

text
face apparent spacing median = 0.1266 m
25% = 0.0654 m
min = 0.0538 m

```

그리고:

```

text
Face spacing < 0.1 m 비율 = 0.25

```



```
Face lambda > 10/m 비율 = 0.25
```

즉 전체 72개 face row 중 약 25%는:

```
text
```

```
평균 apparent spacing이 10cm 이하
```

수준입니다.

이제 이 구간에서는 RQD가 실제로 낮아져야 정상입니다.

따라서 다음 확인은 반드시 해야 합니다.

```
text
```

```
RQD_face_scanline이 실제로 내려갔는가?
```

즉, 새 RQD 전용 case의 `rqd_summary.csv`를 봐야 합니다.

4. 그런데 borehole 쪽은 아직 조밀하지 않음

반면 borehole 쪽은 다음입니다.

```
text
```

```
lambda_borehole mean = 0.775 /m  
median = 0.5 /m  
90% = 2.14 /m  
max = 3.0 /m
```

그리고:

```
text
```

```
BH spacing < 0.1 m 비율 = 0.0  
BH lambda > 10/m 비율 = 0.0
```

즉 borehole은 여전히 RQD를 낮출 만큼 절리를 많이 만나지 못하고 있습니다.

borehole apparent spacing은:

```
text
```

```
median = 1.1667 m  
min = 0.3333 m
```

입니다.

최소 apparent spacing도 33cm이므로, borehole direct RQD는 여전히 높게 나올 가능성이 큼니다.

5. 이게 의미하는 것

현재 RQD 전용 case는 매우 흥미로운 상태입니다.

```
text
Face scanline: 절리를 많이 만남 → RQD 낮아질 가능성 큼
Borehole: 절리를 적게 만남 → RQD 높게 유지될 가능성 큼
```

즉, 이제 연구 질문에 맞는 상황이 만들어졌을 수 있습니다.

수평시추공 RQD가 굴진면 RQD를 과대평가하는가?

특히 결과상으로는 다음 가능성이 큼니다.

```
text
RQD_borehole_direct > RQD_face_scanline
RQD_borehole_hudson > RQD_face_scanline
```

즉 RQD 수준의 **false-safe 위험**이 나타날 수 있습니다.

6. 왜 face는 조밀한데 borehole은 조밀하지 않을까?

가능한 원인은 세 가지입니다.

원인 1. 절리 방향성이 face scanline에 유리하고 borehole에는 불리함

face scanline 방향은:

```
text
d_face = [0, 1, 0]
```

borehole 방향은:

```
text
d_bh = [1, 0, 0]
```

절리 법선 $\mathbf{n}$ 에 대해 scanline이 절리를 잘 만나는 정도는 대략:

```
text
|n · d|
```

에 비례합니다.

만약 현재 RQD 전용 case의 절리군이:

text

```
|n · [0,1,0]| 큼  
|n · [1,0,0]| 작음
```

이면:

text

```
face scanline은 절리를 많이 만남  
borehole은 절리를 적게 만남
```

이 됩니다.

이건 코드 오류가 아니라 **방향성 sampling bias**입니다.

원인 2. borehole은 3개 line뿐이고 face scanline은 중앙 수평선 하나지만 길이와 위치 조건이 다름

borehole은 보통:

text

```
Center, Top, Bottom
```

세 개의 x방향 line입니다.

반면 face scanline은:

text

```
x = face_x에서 y방향으로 터널 직경을 가로지르는 선
```

입니다.

같은 DFN이라도 두 sample line이 보는 절리 집합이 다를 수 있습니다.

특히 절리 disk가 유한 크기이면 더 그렇습니다.

원인 3. borehole window와 face scanline 길이/집계 방식 차이

현재 borehole 쪽 컬럼은 다음입니다.

text

```
borehole_n_intersections_total  
lambda_borehole
```

여기서 주의해야 합니다.

borehole_n_intersections_total은 3개 borehole의 교차 개수 합계일 가능성이 높고, lambda_borehole은 per-borehole lambda 평균일 가능성이 있습니다.

따라서 둘의 관계가 단순히:

```
text
lambda_borehole = total_count / total_length
```

은 아닐 수 있습니다.

이건 에러는 아니지만, 해석할 때 명확히 해야 합니다.

예를 들어:

```
text
mean borehole_n_intersections_total = 4.89
mean lambda_borehole = 0.775 /m
```

이면 borehole window 길이와 3개 borehole 평균 방식 때문에 그렇게 나올 수 있습니다.

7. 이번 결과에서 진짜 확인해야 할 것

이제 가장 중요한 것은 새 RQD 전용 case의 rqd_summary.csv와 rqd_pairwise.csv입니다.

특히 rqd_summary.csv에서 보고 싶은 것은:

```
text
RQD_borehole_direct
RQD_borehole_hudson
RQD_face_scanline
RQD_face_jv
RQD_face_conservative
```

예상은 다음과 같습니다.

```
text
RQD_borehole_direct: 여전히 높을 가능성 큼
RQD_borehole_hudson: 여전히 높을 가능성 큼
RQD_face_scanline: 꽤 내려갔을 가능성 있음
RQD_face_jv: Jv=5,10,20이면 98.5,82,49 수준으로 내려가야 함
```

특히 Jv는 이제 잘 작동할 가능성이 높습니다.

8. Jv 결과는 이제 좋아 보임

이번 결과:

```
text
Jv_face mean = 11.67
min = 5
median = 10
max = 20
```

Palmström 식으로 보면:

Jv = 5

```
text
RQD = 115 - 3.3*5 = 98.5
```

Jv = 10

```
text
RQD = 115 - 33 = 82
```

Jv = 20

```
text
RQD = 115 - 66 = 49
```

즉 이제 RQD_face_jv는 더 이상 100에 포화되지 않을 것입니다.

이건 좋은 변화입니다.

9. 현재 결과가 시사하는 매우 중요한 연구 포인트

이번 RQD 전용 case는 오히려 연구적으로 좋은 방향입니다.

왜냐하면 다음 구조가 보이기 때문입니다.

```
text
절리 밀도를 높였더니 face scanline에서는 절리빈도가 크게 증가했다.
하지만 borehole에서는 절리빈도가 상대적으로 낮게 유지되었다.
```

이 말은 곧:

```
text
수평시추공 RQD는 특정 방향성 조건에서 굴진면 RQD를 과대평가할 수 있다.
```

는 가설을 검증할 수 있는 상황입니다.

이건 기존의 Q' false-safe 문제와 연결됩니다.

단, 이번에는 artificial correction 문제가 아니라:

```
text
sampling geometry + joint orientation
```

에 의한 차이일 가능성이 큼니다.

10. 단, 한 가지 확인해야 할 것

이번 RQD 전용 case에서 face lambda가 매우 높습니다.

```
text
face_scanline_n_intersections max = 186
lambda_face_scanline max = 18.6 /m
```

face scanline 길이가 10m라면:

```
text
186 / 10 = 18.6 /m
```

으로 맞습니다.

이 자체는 문제 없습니다.

하지만 절리 교차점이 너무 많을 때 `RQDCalculator.calc_direct()`가 제대로 낮은 값을 내는지 확인해야 합니다.

즉, 다음을 확인하세요.

```
text
face_scanline_n_intersections가 큰 row에서 RQD_face_scanline이 낮게 나오는가?
```

간단한 확인 코드:

```
python
import pandas as pd

df = pd.read_csv("outputs/your_rqd_case/all_cases_face_rows.csv")

cols = [
    "case_name",
    "seed",
    "face_x_coord",
```

```

    "lambda_face_scanline",
    "face_scanline_n_intersections",
    "RQD_face_scanline",
    "RQD_borehole_direct",
    "RQD_borehole_hudson",
    "RQD_face_jv",
]

print(
    df.sort_values("lambda_face_scanline", ascending=False)[cols].head(20)
)

```

정상이라면:

```

text
lambda_face_scanline이 높은 row에서 RQD_face_scanline이 낮아져야 함

```

입니다.

만약:

```

text
lambda_face_scanline = 18.6인데 RQD_face_scanline = 100

```

이면 그건 문제입니다.

그 경우에는 교차점 위치가 중복되어 있거나, RQD 계산에 들어가는 intersection positions가 이상할 수 있습니다.

11. PerformanceWarning을 없애고 싶으면

분석 스크립트에서 이 부분을:

```

python
df["spacing_borehole_apparent"] = np.where(...)
df["spacing_face_scanline_apparent"] = np.where(...)

```

아래처럼 바꾸면 됩니다.

```

python
spacing_borehole_apparent = np.where(
    df["lambda_borehole"] > 0,
    1.0 / df["lambda_borehole"],
    np.inf,
)

spacing_face_scanline_apparent = np.where(
    df["lambda_face_scanline"] > 0,
    1.0 / df["lambda_face_scanline"],
)

```

```

    np.inf,
)
df = pd.concat(
    [
        df.copy(),
        pd.DataFrame(
            {
                "spacing_borehole_apparent": spacing_borehole_apparent,
                "spacing_face_scanline_apparent": spacing_face_scanline_apparent,
            },
            index=df.index,
        ),
    ],
    axis=1,
)

```

하지만 분석 결과에는 영향 없습니다.

12. 지금 단계에서 할 일

다음 순서로 가면 됩니다.

Step 1. 새 RQD 전용 case의 `rqd_summary.csv` 확인

특히:

```

text
RQD_borehole_direct
RQD_borehole_hudson
RQD_face_scanline
RQD_face_jv

```

Step 2. `rqd_pairwise.csv` 확인

특히:

```

text
RQD_borehole_direct → RQD_face_scanline
RQD_borehole_hudson → RQD_face_scanline
RQD_borehole_direct → RQD_face_jv
RQD_borehole_hudson → RQD_face_jv

```

여기서 아마 중요한 값은:

```

text
bias
MAE

```



```
overestimation_rate
```

입니다.

만약 예상대로라면:

```
text
bias > 0
overestimation_rate 높음
```

즉 borehole RQD가 face RQD를 과대평가할 수 있습니다.

Step 3. 높은 face lambda row 확인

아래 row를 확인하세요.

```
text
lambda_face_scanline 높은 순서 상위 20개
```

그때 RQD_face_scanline이 낮게 나오는지 보면 됩니다.

13. 현재 결과의 1차 해석 문장

이번 결과는 이렇게 정리할 수 있습니다.

RQD 전용 고밀도 case에서는 face scanline의 평균 절리빈도가 7.07/m, 75분위가 10.65/m까지 증가하여 RQD 저하가 발생할 수 있는 조건이 형성되었다. 반면 borehole의 평균 절리빈도는 0.78/m, 최대 3.0/m에 머물러, 같은 DFN 조건에서도 borehole과 face scanline이 매우 다른 절리 교차 빈도를 보였다. 이는 단순한 코드 오류라기보다 scanline 방향과 절리군 방향성의 차이에 따른 sampling bias를 시사한다. 다만 실제 RQD가 이에 맞게 감소하는지는 RQD_face_scanline 분포를 추가 확인해야 한다.

한 줄로는:

```
text
에러가 아니라, face는 조밀해졌고 borehole은 여전히 절리를 적게 만나는 방향성 차이가 드러난 결과로 보입니다.
```

GPT-5.5

Show Claude Sonnet 4.6's response

자 계속해보자고...잠깐 리마인드 하면 시추공과 굴진면의 Q'의 상관성 내지는 굴착 여부에 대한 판정 기준을 삼을 수 있는지, 삼으려면 어떤 경우, 어떤 조건일 경우에 그런지 등에 대한 연구가 있어야 해. 그래서 다른 것 보다 앞서서 RQD

분석을 하고 있고, 이전에 검토에서는 절리 빈도가 낮으니 모든 RQD가 100 언저리 였었고, 절리 빈도를 높이니 유의미한 값의 차이를 보이더라 하는 부분이야. 그렇지만 RQD는 절리의 방향성과 시추공의 방향성에 따라 매우 민감한 측정방법이고 이러한 문제점, 혹은 민감성을 잘 이해하고 궁극적인 Q' 인자를 이용한 판정의 신뢰성을 갖추수 있는지 검증할 수 있을 것 같아~



맞습니다. 지금 연구 흐름은 아주 잘 잡혀 있습니다.
현재 단계의 의미를 다시 정리하면 이렇습니다.

최종 목적은 수평시추공 Q'가 굴진면 Q' 또는 굴착 가능성 판단의 신뢰성 있는 사전 지표가 될 수 있는지를 검증하는 것이다.
그런데 Q'의 핵심 구성요소 중 하나인 RQD가 방향성과 sampling geometry에 매우 민감하므로, 먼저 RQD 수준에서 시추공-굴진면 차이의 발생 조건과 원인을 분리해서 이해해야 한다.

즉, 지금 하는 RQD 분석은 부수적인 분석이 아니라, 최종 Q' decision 연구의 기반입니다.

1. 지금까지 확인된 핵심 사실

1-1. 기존 6개 granite case

기존 6개 case에서는:

```
text
lambda_borehole mean ~ 0.39 /m
lambda_face_scanline mean ~ 0.54 /m
Jv_face mean ~ 0.93
```

이었고, apparent spacing은 대부분 10cm보다 훨씬 컸습니다.

결과:

```
text
RQD_borehole_direct ~ 99.5
RQD_borehole_hudson ~ 99.7
RQD_face_scanline ~ 99.6
RQD_face_jv = 100
```

따라서 결론은:

```
text
절리빈도가 낮은 조건에서는 RQD가 상한값 100 부근에 포화된다.
이 경우 시추공 RQD와 굴진면 RQD의 차이는 작아 보이지만,
```

이는 두 측정이 본질적으로 동일하다는 뜻이라기보다 RQD 지표의 민감도가 부족한 상태일 수 있다.

1-2. RQD 전용 고밀도 case

절리빈도를 높인 RQD 전용 case에서는:

```
text
lambda_face_scanline mean = 7.07 /m
lambda_face_scanline median = 5.90 /m
lambda_face_scanline 75% = 10.65 /m
lambda_face_scanline max = 18.60 /m
```

즉 face scanline은 RQD가 실제로 낮아질 수 있는 영역으로 들어갔습니다.

반면 borehole은:

```
text
lambda_borehole mean = 0.78 /m
lambda_borehole median = 0.50 /m
lambda_borehole max = 3.00 /m
```

으로 여전히 낮았습니다.

따라서 이제 보이는 핵심은:

```
text
같은 DFN에서도 face scanline은 절리를 많이 만나고,
borehole은 절리를 적게 만나는 상황이 발생할 수 있다.
```

이는 곧:

```
text
방향성 sampling bias
```

입니다.

2. 이제 최종 Q' 연구와 어떻게 연결되는가

Q'는 다음입니다.

```
text
Q' = (RQD / Jn) × (Jr / Ja)
```

여기서 현재 우리가 보고 있는 것은 첫 번째 항입니다.

text
RQD / Jn

특히 RQD입니다.

만약 시추공 RQD가 굴진면 RQD를 과대평가하면:

text
RQD_borehole > RQD_face

이 됩니다.

다른 요소가 같거나 유사하다면:

text
Q'_borehole > Q'_face

가 됩니다.

이것이 바로 위험한 경우입니다.

text
시추공 Q' 기준으로는 굴착 가능 또는 양호하다고 판단
하지만 실제 굴진면 Q'는 불량

즉, RQD 수준의 bias가 Q' decision의 false-safe로 이어질 수 있습니다.

3. 현재 RQD 분석의 핵심 연구 질문

지금 단계에서 연구 질문은 이렇게 잡으면 좋습니다.

RQ1. RQD는 어떤 조건에서 포화되는가?

text
절리빈도가 낮은 조건에서는 RQD가 100 부근에 포화되어
시추공과 굴진면의 차이를 식별하지 못하는가?

이 질문에 대해 기존 6개 case가 답을 줍니다.

답:

text
그렇다. 낮은 λ 조건에서는 시추공과 굴진면 모두 RQD가 100 부근에 포화되었다.

RQ2. 절리빈도를 높이면 시추공 RQD와 굴진면 RQD는 다르게 반응하는가?

RQD 전용 case가 답을 줍니다.

현재 관찰:

```
text
face λ는 크게 증가했지만,
borehole λ는 상대적으로 낮게 유지되었다.
```

즉:

```
text
절리 밀도를 높여도 방향성에 따라 특정 sampling line은 절리를 충분히 만나지 못할 수 있다.
```

이게 매우 중요합니다.

RQ3. 그 차이는 방향성 때문인가?

아직 이 부분은 정교하게 검증하지 않았습니다.

현재는 간접 증거가 있습니다.

```
text
face direction = [0,1,0]
borehole direction = [1,0,0]
같은 DFN에서 face λ >> borehole λ
```

이것은 방향성 때문일 가능성이 큼니다.

하지만 이것을 정량적으로 검증하려면 나중에 다음 지표가 필요합니다.

```
text
orientation tendency_bh =  $\sum P_{32\_i} |n\_i \cdot d\_bh|$ 
orientation tendency_face =  $\sum P_{32\_i} |n\_i \cdot d\_face|$ 
orientation tendency ratio = tendency_face / tendency_bh
```

이건 지금 당장 안 해도 됩니다.

하지만 최종 논리에서는 꼭 들어가야 할 가능성이 높습니다.

RQ4. RQD 차이가 Q' 차이와 decision error로 전파되는가?

이게 다음 단계입니다.

RQD 수준에서 다음이 확인되면:

```
text
RQD_borehole > RQD_face
```

그 다음은:

```
text
Q'_borehole > Q'_face
```

가 되는지 확인해야 합니다.

그리고 threshold 기준으로:

```
text
false-safe
false-alarm
expected cost
```

를 봐야 합니다.

4. RQD 분석에서 지금 반드시 확인할 것

현재 RQD 전용 case의 λ 결과는 매우 좋습니다.
이제 다음 3개를 확인해야 합니다.

4-1. rqd_summary.csv

봐야 할 것:

```
text
RQD_borehole_direct
RQD_borehole_hudson
RQD_face_scanline
RQD_face_jv
RQD_face_conservative
```

예상되는 중요한 패턴:

```
text
RQD_borehole_direct는 여전히 높다.
RQD_face_scanline은 낮아졌다.
RQD_face_jv도 Jv=5,10,20에 따라 낮아졌다.
```

이게 나오면 RQD 수준에서 시추공 과대평가가 발생한 것입니다.

4-2. rqd_pairwise.csv

가장 중요한 비교:

```
text
RQD_borehole_direct → RQD_face_scanline
RQD_borehole_hudson → RQD_face_scanline
RQD_borehole_direct → RQD_face_jv
RQD_borehole_hudson → RQD_face_jv
```

여기서 특히:

```
text
bias
MAE
overestimation_rate
```

가 중요합니다.

예상:

```
text
bias > 0
overestimation_rate 높음
```

이면:

```
text
시추공 RQD가 굴진면 RQD를 체계적으로 과대평가한다.
```

입니다.

4-3. threshold metrics

RQD threshold는 이제 기존 50,75,90도 의미가 있을 수 있습니다.

특히 Jv=20 case에서는 RQD_face_jv가 49 정도이므로 threshold 75에서 bad로 나올 수 있습니다.

보야 할 false-safe 정의:

```
text
BH RQD >= threshold
```

Face RQD < threshold

이 값이 높으면:

text

시추공 RQD 기준 굴착 가능 판단이 위험할 수 있다.

5. RQD 단계에서 가능한 결론 구조

지금까지 흐름을 논문식으로 쓰면 다음과 같습니다.

결론 1. RQD saturation under low fracture frequency

text

낮은 절리빈도 조건에서는 시추공 및 굴진면 RQD가 모두 100에 가깝게 포화되었다.
이 조건에서는 RQD가 sampling direction 차이를 거의 드러내지 못했다.

의미:

text

좋은 암반 조건에서는 RQD만으로 시추공과 굴진면의 차이를 평가하기 어렵다.

결론 2. Direction-sensitive divergence under higher fracture density

text

절리빈도를 증가시킨 RQD 전용 case에서는 face scanline λ 가 크게 증가한 반면,
borehole λ 는 상대적으로 낮게 유지되었다.

의미:

text

RQD는 절리 밀도뿐 아니라 측정 방향에 강하게 의존한다.
같은 암반이라도 시추공 방향과 굴진면 scanline 방향이 다르면
절리 교차 빈도와 RQD가 크게 달라질 수 있다.

결론 3. Borehole RQD may be conditionally reliable, not universally reliable

이게 최종 목적과 연결됩니다.

text

수평시추공 RQD는 모든 조건에서 굴진면 RQD를 안정적으로 대표하지 않는다.
특히 절리군 방향이 시추공 방향에는 불리하고 face scanline 방향에는 유리한 경우,

시추공 RQD는 굴진면 RQD를 과대평가할 수 있다.

반대로:

text

절리빈도가 낮거나 RQD가 포화되는 조건에서는 두 값이 유사해 보일 수 있으나,
이는 예측력이 높다기보다 지표가 둔감한 결과일 수 있다.

6. 여기서 Q'로 넘어가는 논리

RQD 분석 후 Q' 분석은 다음 질문으로 자연스럽게 이어집니다.

text

RQD 수준에서 발생한 방향성 bias가 Q' 및 굴착 가능성 판단에 얼마나 영향을 미치는가?

Q'는 RQD 외에도 J_r/J_a 가 들어갑니다.

따라서 가능한 경우는 세 가지입니다.

경우 A. RQD bias가 Q' bias로 그대로 전파됨

text

RQD_bh > RQD_face
Q'_bh > Q'_face
false-safe 증가

해석:

text

시추공 Q'는 해당 조건에서 위험하게 낙관적이다.

경우 B. J_r/J_a 차이가 RQD bias를 완화함

예:

text

RQD_bh > RQD_face
하지만 $J_r/J_{a_bh} < J_r/J_{a_face}$
결과적으로 Q' 차이는 작음

해석:

text

RQD 단독으로는 위험해 보이지만 Q' 전체에서는 일부 보정됨.

경우 C. Jr/Ja 차이가 RQD bias를 더 키움

예:

```
text
RQD_bh > RQD_face
Jr/Ja_bh > Jr/Ja_face
Q'_bh는 훨씬 높아짐
```

해석:

```
text
false-safe 위험이 매우 큼.
```

이렇게 Q'로 넘어가야 합니다.

7. 지금 시점의 분석 우선순위

이제 바로 해야 할 것은 다음입니다.

Step 1. RQD 전용 case의 `rqd_summary.csv` 확인

이걸 먼저 봐야 합니다.

Step 2. `rqd_pairwise.csv` 확인

특히 bias와 overestimation rate.

Step 3. `rqd_threshold_metrics.csv` 확인

threshold 75, 90 중심.

Step 4. 이후 Q' 분석

RQD 전용 case에서 Q'도 같이 출력되어 있을 것이므로:

```
text
```

```
Qp_borehole_mean
Qp_face_mean
Qp_face_scanline
Qp_face_jv
Qp_face_conservative
```

를 비교합니다.

다만 기존 decision tester가 Qp_face_mean만 보고 있다면, face target을 명확히 해야 합니다.

추천 target:

```
text
Qp_face_scanline
Qp_face_jv
Qp_face_conservative
```

그리고 predictor:

```
text
Qp_borehole_mean
```

또는 나중에:

```
text
Qp_borehole_direct
Qp_borehole_hudson
```

도 분리하면 더 좋습니다.

8. RQD 분석에서 꼭 남겨야 할 메시지

아주 중요한 문장입니다.

RQD는 단순한 암반 품질 지표가 아니라, 특정 방향의 line sampling 결과이다. 따라서 RQD 값은 절리 밀도뿐 아니라 절리군 방향성과 측정 방향의 관계에 강하게 의존한다. 수평시추공 RQD가 높게 나온다는 것은 전방 암반 전체가 양호하다는 뜻이 아니라, 수평시추공 방향에서 절리를 적게 만났다는 뜻일 수 있다.

이 문장이 최종 연구의 핵심 중 하나가 될 수 있습니다.

9. 앞으로의 연구 구조 제안

최종 연구를 다음 구조로 만들면 좋습니다.

text

Part 1. RQD method validation

- low-density case: RQD saturation
- high-density case: direction-sensitive divergence
- borehole vs face scanline vs J_v

Part 2. Q' propagation

- RQD 차이가 Q' 차이로 전파되는지
- J_r/J_a 가 이를 완화/증폭하는지

Part 3. Decision usefulness

- threshold 기반 굴착 가능/보류 판단
- false-safe, false-alarm
- expected cost
- baseline comparison

Part 4. Conditional reliability

- 어떤 조건에서 borehole Q' 가 유용한가?
- 어떤 조건에서 위험한가?
- orientation, density, sparsity regime별 분석

이 흐름이 가장 자연스럽습니다.

10. 지금 다음으로 보내주면 좋은 것

다음 결과를 보내주시면 바로 해석할 수 있습니다.

1순위

text

RQD 전용 case의 rqd_summary.csv

2순위

text

RQD 전용 case의 rqd_pairwise.csv

3순위

text

RQD 전용 case의 rqd_threshold_metrics.csv

가능하면 threshold는:

text

50, 75, 90

에 더해서:

text
95, 98

도 같이 보면 좋습니다.

11. 현재까지의 한 줄 리마인드

지금까지의 핵심은 이겁니다.

text
낮은 절리빈도에서는 RQD가 100에 포화되어 차이가 숨겨졌고,
절리빈도를 높이자 face scanline RQD가 크게 민감해졌지만 borehole RQD는 여전히 높게 남을 가능성이
나타났다.
이 차이는 수평시추공 기반 Q' 판단의 조건부 신뢰성을 검증하는 핵심 단서다.

이제 RQD summary/pairwise를 보면, 이 흐름이 수치적으로 확정될 것입니다.

GPT-5.5
Show GPT-5.4's response

rqd_summary.csv : column,exists,count,mean,std,min,p10,p25,p50,p75,p90,max
RQD_borehole_direct,True,480,99.47825188910619,0.9060139189291068,93.84375382746568,98.31759503732377,9
9.12649842853337,100.0,100.0,100.0,100.0
RQD_borehole_hudson,True,480,99.70087845487859,0.4244071054821265,97.413719545283,99.1358548650228,99.6
1598890532412,99.86631885255872,100.0,100.0,100.0
RQD_face_scanline,True,480,99.61270440687571,0.9070548819996909,92.09472088093489,98.44711826524183,99.8
5820203344483,100.0,100.0,100.0,100.0
RQD_face_jv,True,480,100.0,0.0,100.0,100.0,100.0,100.0,100.0,100.0
RQD_face_conservative,True,480,99.61270440687571,0.9070548819996909,92.09472088093489,98.44711826524183,
99.85820203344483,100.0,100.0,100.0,100.0
rqd_pairwise.csv :
predictor,target,n,pred_mean,target_mean,bias,mae,rmse,median_abs_error,corr,overestimation_rate,underestimation_rat
e,equal_rate
RQD_borehole_direct,RQD_face_scanline,480,99.47825188910619,99.61270440687571,-
0.13445251776948994,0.7891260031341113,1.3480738418814533,0.27716701264969146,-
0.09696470687872287,0.225,0.3479166666666665,0.4270833333333333
RQD_borehole_direct,RQD_face_jv,480,99.47825188910619,100.0,-
0.5217481108938001,0.5217481108938001,1.0446876016159885,0.0,0.0,0.39375,0.60625
RQD_borehole_direct,RQD_face_conservative,480,99.47825188910619,99.61270440687571,-
0.13445251776948994,0.7891260031341113,1.3480738418814533,0.27716701264969146,-
0.09696470687872287,0.225,0.3479166666666665,0.4270833333333333
RQD_borehole_hudson,RQD_face_scanline,480,99.70087845487859,99.61270440687571,0.08817404800288828,0.5973
680558834846,1.0497144760495605,0.25032994723460433,-
0.1214930452212111,0.2458333333333333,0.5395833333333333,0.2145833333333333
RQD_borehole_hudson,RQD_face_jv,480,99.70087845487859,100.0,-
0.29912154512142203,0.29912154512142203,0.51886398703441,0.1336811474412798,0.0,0.71875,0.28125
RQD_borehole_hudson,RQD_face_conservative,480,99.70087845487859,99.61270440687571,0.08817404800288828,0.
5973680558834846,1.0497144760495605,0.25032994723460433,-
0.1214930452212111,0.2458333333333333,0.5395833333333333,0.2145833333333333



이건 RQD 전용 case 결과가 아니라 이전 6개 granite case 결과로 보입니다.

가장 명확한 증거는 count입니다.

방금 RQD 전용 case의 lambda 진단 결과는:

```
text
count = 72
Jv_face mean = 11.6667
Jv_face min = 5
Jv_face max = 20
```

였죠.

그런데 지금 붙여준 rqd_summary.csv는:

```
text
count = 480
RQD_face_jv = 100.0 constant
```

입니다.

이건 이전 low-density / original 6 cases 결과와 정확히 일치합니다.

즉, 지금 열어본 rqd_summary.csv와 rqd_pairwise.csv는 새 RQD 전용 72-row CSV를 분석한 결과가 아니라, 예전 480-row CSV를 분석한 결과입니다.

1. 현재 상황 판단

지금 붙여준 파일

```
text
rqd_summary.csv count = 480
```

이 뜻은:

```
text
6 cases × 10 seeds × 8 faces = 480 rows
```

즉 예전 실험입니다.

RQD 전용 case 진단 결과

text

```
lambda/count summary count = 72
```

이 뜻은 아마:

```
text
RQD 전용 case들 × seeds × faces = 72 rows
```

입니다.

따라서 두 결과는 같은 데이터셋이 아닙니다.

2. 제일 먼저 확인할 것

분석에 사용한 csv 경로가 서로 다른 것 같습니다.

아래 코드를 실행해서 지금 보고 있는 `rqd_summary.csv`가 어떤 원본 csv에서 나온 것인지 확인해야 합니다.

현재 RQD 전용 csv 경로가 예를 들어 이런 것이라고 합시다.

```
text
outputs/rqd_multicase/all_cases_face_rows.csv
```

그러면 먼저 이것 확인하세요.

```
python
import pandas as pd

csv_path = "outputs/rqd_multicase/all_cases_face_rows.csv"

df = pd.read_csv(csv_path)

print("n rows:", len(df))
print("cases:", df["case_name"].unique() if "case_name" in df.columns else "no case_name")
print(df[["Jv_face", "lambda_borehole", "lambda_face_scanline"]].describe())
```

여기서 반드시 다음처럼 나와야 합니다.

```
text
n rows: 72
Jv_face min: 5
Jv_face median: 10
Jv_face max: 20
lambda_face_scanline mean: 약 7.07
```

만약 여기서 480이 나오면 RQD 전용 csv가 아니라 예전 csv를 보고 있는 것입니다.

3. RQD 분석 스크립트를 다시 실행해야 함

아마 `analyze_rqd_methods.py`를 실행할 때 `--csv` 경로가 예전 csv로 되어 있었거나, `--out` 폴더가 예전 결과 폴더여서 헷갈린 것 같습니다.

아래처럼 **완전히 다른 output 폴더**로 다시 돌리는 것을 추천합니다.

예시:

```
bash
python -m src.cli.analyze_rqd_methods \
  --csv outputs/rqd_multicase/all_cases_face_rows.csv \
  --out outputs/rqd_analysis_dense_72 \
  --thresholds 50 75 90 95 98
```

PowerShell이나 한 줄 실행이면:

```
bash
python -m src.cli.analyze_rqd_methods --csv
outputs/rqd_multicase/all_cases_face_rows.csv --out outputs/rqd_analysis_dense_72 --
thresholds 50 75 90 95 98
```

여기서 중요한 건 두 가지입니다.

```
text
--csv 는 72-row RQD 전용 all_cases_face_rows.csv
--out 은 새 폴더 outputs/rqd_analysis_dense_72
```

4. 분석 후 반드시 확인할 것

새로 생성된:

```
text
outputs/rqd_analysis_dense_72/rqd_summary.csv
```

를 열었을 때 `count`가 반드시 72여야 합니다.

예상:

```
text
column,count
RQD_borehole_direct,72
RQD_borehole_hudson,72
RQD_face_scanline,72
RQD_face_jv,72
RQD_face_conservative,72
```


만약 여전히 480이면, 분석 CSV 경로가 잘못된 것입니다.

5. 지금 기대되는 RQD 전용 summary 패턴

방금 lambda 결과를 보면 새 RQD 전용 case에서는 대략 이런 패턴이 나와야 정상입니다.

Borehole 쪽

```
text
lambda_borehole mean = 0.775 /m
max = 3.0 /m
```

따라서:

```
text
RQD_borehole_direct는 여전히 높을 가능성 큼
RQD_borehole_hudson도 높을 가능성 큼
```

예상:

```
text
RQD_borehole_direct 평균: 95~100 근처
RQD_borehole_hudson 평균: 95~100 근처
```

물론 실제 값은 봐야 합니다.

Face scanline 쪽

```
text
lambda_face_scanline mean = 7.07 /m
median = 5.9 /m
75% = 10.65 /m
90% = 16.09 /m
```

따라서:

```
text
RQD_face_scanline은 기존보다 확실히 낮아져야 정상
```

특히 λ 가 10/m 이상인 row에서는 RQD가 70대 이하로 내려갈 수 있습니다.

Jv 쪽

```
text
Jv_face = 5, 10, 20
```

이면 Palmström 식상:

```
text
Jv = 5 → RQD = 98.5
Jv = 10 → RQD = 82.0
Jv = 20 → RQD = 49.0
```

따라서 새 summary에서 RQD_face_jv가 전부 100이면 안 됩니다.

새 결과에서 정상이라면:

```
text
RQD_face_jv mean ≈ 76.5
```

정도일 수 있습니다.

왜냐하면 Jv가 5, 10, 20이 비슷한 비율이라면:

```
text
(98.5 + 82 + 49) / 3 = 76.5
```

입니다.

6. 간단한 파일 혼동 방지 방법

앞으로는 분석할 때 output 폴더 이름에 dataset 정보를 넣는 게 좋습니다.

예:

```
text
outputs/rqd_analysis_original_480
outputs/rqd_analysis_dense_72
outputs/rqd_analysis_p32_05_10_20
```

그리고 가능하면 분석 폴더 안에 manifest 파일을 하나 저장하면 좋습니다.

예를 들어 analyze_rqd_methods.py 마지막에 이런 걸 추가할 수 있습니다.

```
python
with open(out_path / "analysis_manifest.txt", "w", encoding="utf-8") as f:
    f.write(f"input_csv: {args.csv}\n")
    f.write(f"n_rows: {len(df)}\n")
    if "case_name" in df.columns:
        f.write(f"case_names: {list(df['case_name'].unique())}\n")
```

```
f.write(f"thresholds: {args.thresholds}\n")
```

그러면 나중에 어떤 CSV를 분석한 결과인지 헛갈리지 않습니다.

7. 지금 해야 할 정확한 다음 작업

Step 1. RQD 전용 CSV row 수 확인

```
python
import pandas as pd

csv_path = "여기에_RQD전용_all_cases_face_rows.csv_경로"

df = pd.read_csv(csv_path)

print(len(df))
print(df["case_name"].unique())
print(df["Jv_face"].describe())
print(df["lambda_face_scanline"].describe())
```

여기서 `len(df) = 72`가 나와야 합니다.

Step 2. 분석 재실행

```
bash
python -m src.cli.analyze_rqd_methods --csv 여기에_RQD전용_all_cases_face_rows.csv_경로 -
-out outputs/rqd_analysis_dense_72 --thresholds 50 75 90 95 98
```

Step 3. 새 결과 확인

```
text
outputs/rqd_analysis_dense_72/rqd_summary.csv
outputs/rqd_analysis_dense_72/rqd_pairwise.csv
```

여기서 `count = 72`인지 먼저 확인하세요.

8. 지금 붙여준 결과의 해석은 보류

지금 붙여준 `rqd_summary.csv`, `rqd_pairwise.csv`는 예전 480-row 결과이므로, RQD 전용 case 해석에는 사용하면 안 됩니다.

즉, 지금 이 결과로는:

text

절리 빈도를 높였을 때 RQD가 어떻게 변했는지

를 판단할 수 없습니다.

새 72-row 결과를 다시 뽑아야 합니다.

9. 한 줄 결론

지금 붙여준 summary/pairwise는 **RQD 전용 case 결과가 아니라 이전 480개 low-density case 결과**입니다.

--csv 입력 경로 또는 --out 결과 폴더가 섞인 것으로 보입니다.

RQD 전용 72-row CSV를 대상으로 analyze_rqd_methods.py를 다시 실행하고, 새 rqd_summary.csv의 count가 72인지 먼저 확인해야 합니다.

GPT-5.5

솔직히 혼란스럽네... 다스 프로세스를 정리해야 할 것 같아. granite_rqd_p32_05.yaml, granite_rqd_p32_10.yaml, granite_rqd_p32_20.yaml 세개의 입력 파일로 분석을 다시 시작했어. 여기서부터 어떤 과정으로 rqd_summary, rqd_pairwise를 구하게 되는지 생각나는대로 정리해줘



좋습니다. 지금은 한 번 프로세스를 고정해야 합니다.

혼란의 핵심은 아마 다음입니다.

text

YAML case 실행 결과 CSV와

RQD 분석 결과 CSV가 서로 다른 폴더에 있고,

이전 480-row 결과와 새 RQD 전용 결과가 섞여 보였음

그래서 granite_rqd_p32_05.yaml, granite_rqd_p32_10.yaml, granite_rqd_p32_20.yaml 세 개만으로 새로 시작하는 절차를 아래처럼 정리하면 됩니다.

전체 흐름 한 줄 요약

text

3개 YAML case

→ MultiCaseBatchRunner 실행

→ all_cases_face_rows.csv 생성

→ analyze_rqd_methods.py 실행

```
→ rqd_summary.csv / rqd_pairwise.csv 생성
→ count와 case_name으로 올바른 결과인지 확인
```

1. 입력 파일

이번 RQD 전용 분석의 입력 case는 딱 세 개입니다.

```
text
granite_rqd_p32_05.yaml
granite_rqd_p32_10.yaml
granite_rqd_p32_20.yaml
```

각 case의 의미는 대략 다음과 같습니다.

```
text
granite_rqd_p32_05:
  ΣP32 ≈ 5
  Jv 기반 RQD 예상값 ≈ 98.5

granite_rqd_p32_10:
  ΣP32 ≈ 10
  Jv 기반 RQD 예상값 ≈ 82

granite_rqd_p32_20:
  ΣP32 ≈ 20
  Jv 기반 RQD 예상값 ≈ 49
```

즉 이 세 개는 기존 granite case와 다르게 **RQD가 100에 포화되지 않도록 만든 RQD 전용 고밀도 case**입니다.

2. 1단계: MultiCase 실행

먼저 이 세 YAML을 대상으로 batch를 실행해야 합니다.

이 단계에서 하는 일은:

```
text
각 YAML case
× seed 반복
× face position 반복
→ tunnel.sample_borehole()
→ tunnel.sample_face()
→ comparison.compare_at_face()
→ face-level 결과 누적
```

입니다.

결과로 생성되어야 하는 핵심 파일은:

```
text
all_cases_face_rows.csv
```

입니다.

예를 들어 output 폴더를 명확하게 이렇게 잡는 것이 좋습니다.

```
text
outputs/rqd_p32_05_10_20/
```

그러면 최종 원본 결과 csv는:

```
text
outputs/rqd_p32_05_10_20/all_cases_face_rows.csv
```

가 됩니다.

이 파일이 **RQD 분석의 원본 데이터**입니다.

3. **all_cases_face_rows.csv** 안에 들어가야 하는 핵심 컬럼

`comparison.py`가 제대로 수정되어 있다면, `all_cases_face_rows.csv`에는 최소한 아래 컬럼이 있어야 합니다.

Borehole RQD predictor

```
text
RQD_borehole_direct
RQD_borehole_hudson
lambda_borehole
borehole_n_intersections_total
```

Face RQD reference

```
text
RQD_face_scanline
lambda_face_scanline
face_scanline_n_intersections
```

Jv 기반 reference

```
text
Jv_face
Jv_method
RQD_face_jv
RQD_face_conservative
```

식별자

```
text
case_name
seed
face_x_idx
face_x_coord
```

이 파일이 없거나, 이 컬럼들이 없으면 `rqd_summary.csv`, `rqd_pairwise.csv`는 제대로 만들 수 없습니다.

4. row 수 계산

여기서 제일 먼저 확인해야 하는 것은 row 수입니다.

row 수는 다음 공식입니다.

```
text
n_rows = n_cases * n_seeds * n_faces
```

이번 case는 3개입니다.

```
text
n_cases = 3
```

예를 들어:

```
text
n_seeds = 3
n_faces = 8
```

이면:

```
text
3 * 3 * 8 = 72 rows
```

입니다.

즉 이전에 네가 보여준 `count = 72`는 아마:

```
text
3 cases * 3 seeds * 8 faces
```

결과였을 가능성이 큼니다.

반대로 `count = 480`이면 그건:

```
text
```

6 cases × 10 seeds × 8 faces

기존 실험 결과입니다.

따라서 이번 RQD 전용 분석에서 `rqd_summary.csv`의 `count`가 480이면 잘못된 파일을 분석한 것입니다.

5. 2단계: 원본 CSV 확인

MultiCase 실행 후 바로 아래를 확인해야 합니다.

```
python
import pandas as pd

csv_path = "outputs/rqd_p32_05_10_20/all_cases_face_rows.csv"

df = pd.read_csv(csv_path)

print("n rows:", len(df))
print("case names:", df["case_name"].unique())

cols = [
    "RQD_borehole_direct",
    "RQD_borehole_hudson",
    "RQD_face_scanline",
    "RQD_face_jv",
    "RQD_face_conservative",
    "lambda_borehole",
    "lambda_face_scanline",
    "Jv_face",
]

print(df[cols].describe(percentiles=[0.1, 0.25, 0.5, 0.75, 0.9]))
```

여기서 기대되는 것은:

```
text
n rows = 72 또는 네가 설정한 3 × seed × face 수
case names = granite_rqd_p32_05, granite_rqd_p32_10, granite_rqd_p32_20
Jv_face = 5, 10, 20 근처
```

입니다.

6. 3단계: RQD 분석 스크립트 실행

그 다음 `analyze_rqd_methods.py`를 실행합니다.

중요한 점은:

text

```
--csv에는 반드시 방금 생성한 RQD 전용 all_cases_face_rows.csv를 넣어야 함
--out에는 기존 결과와 다른 새 폴더를 넣어야 함
```

예:

bash

```
python -m src.cli.analyze_rqd_methods \
  --csv outputs/rqd_p32_05_10_20/all_cases_face_rows.csv \
  --out outputs/rqd_analysis_p32_05_10_20 \
  --thresholds 50 75 90 95 98
```

한 줄로 쓰면:

bash

```
python -m src.cli.analyze_rqd_methods --csv
outputs/rqd_p32_05_10_20/all_cases_face_rows.csv --out
outputs/rqd_analysis_p32_05_10_20 --thresholds 50 75 90 95 98
```

7. 이 스크립트가 하는 일

analyze_rqd_methods.py는 all_cases_face_rows.csv를 읽어서 아래 세 가지 기본 분석을 합니다.

7-1. rqd_summary.csv

각 RQD 방법의 분포 요약입니다.

대상 컬럼:

text

```
RQD_borehole_direct
RQD_borehole_hudson
RQD_face_scanline
RQD_face_jv
RQD_face_conservative
```

출력 컬럼:

text

```
column
exists
count
mean
std
min
```

```
p10  
p25  
p50  
p75  
p90  
max
```

즉 `rqd_summary.csv`는 이런 질문에 답합니다.

```
text
```

각 방법의 RQD 분포가 어떻게 생겼는가?

RQD가 100에 포화되어 있는가?

face_scanline RQD가 실제로 내려갔는가?

Jv 기반 RQD가 100에서 벗어났는가?

이번 RQD 전용 case에서 정상적이라면:

```
text
```

RQD_face_jv는 더 이상 100만 나오면 안 됨

Jv=5,10,20에 따라 대략 98.5, 82, 49 수준이 섞여야 함

7-2. `rqd_pairwise.csv`

시추공 predictor와 face target 사이의 pairwise 비교입니다.

기본 predictor:

```
text
```

```
RQD_borehole_direct  
RQD_borehole_hudson
```

기본 target:

```
text
```

```
RQD_face_scanline  
RQD_face_jv  
RQD_face_conservative
```

따라서 비교 조합은 보통 6개입니다.

```
text
```

```
RQD_borehole_direct  → RQD_face_scanline  
RQD_borehole_direct  → RQD_face_jv  
RQD_borehole_direct  → RQD_face_conservative  
  
RQD_borehole_hudson  → RQD_face_scanline  
RQD_borehole_hudson  → RQD_face_jv  
RQD_borehole_hudson  → RQD_face_conservative
```

출력 컬럼:

```
text
predictor
target
n
pred_mean
target_mean
bias
mae
rmse
median_abs_error
corr
overestimation_rate
underestimation_rate
equal_rate
```

해석 핵심:

```
text
bias = predictor - target
```

즉:

```
text
bias > 0:
    시추공 RQD가 face RQD보다 높음
    잠재적 과대평가

bias < 0:
    시추공 RQD가 face RQD보다 낮음
    보수적 평가 가능성
```

이번 RQD 전용 case에서는 특히 이 두 줄이 중요합니다.

```
text
RQD_borehole_direct → RQD_face_scanline
RQD_borehole_hudson → RQD_face_scanline
```

만약 여기서:

```
text
bias > 0
overestimation_rate 높음
```

이면:

```
text
시추공 RQD가 굴진면 RQD를 체계적으로 과대평가하는 조건이 발생했다
```

고 볼 수 있습니다.

7-3. `rqd_threshold_metrics.csv`

RQD threshold 기준의 classification 결과입니다.

예를 들어 threshold 75라면:

```
text
RQD >= 75 → good
RQD < 75 → bad
```

false-safe 정의:

```
text
시추공 RQD는 good인데,
face RQD는 bad인 경우
```

즉:

```
text
predictor >= threshold
target < threshold
```

출력 컬럼:

```
text
predictor
target
threshold
n
prior_target_good_rate
TP
FP_false_safe
FN_false_alarm
TN
accuracy
balanced_accuracy
false_safe_rate
false_alarm_rate
expected_cost
```

이건 나중에 Q' decision usefulness로 넘어가기 전의 RQD-level decision test입니다.

8. 파일 관계를 그림으로 정리

text

```
[granite_rqd_p32_05.yaml]  
[granite_rqd_p32_10.yaml]  
[granite_rqd_p32_20.yaml]
```

↓ MultiCaseBatchRunner

outputs/rqd_p32_05_10_20/all_cases_face_rows.csv

↓ analyze_rqd_methods.py

```
outputs/rqd_analysis_p32_05_10_20/rqd_summary.csv  
outputs/rqd_analysis_p32_05_10_20/rqd_pairwise.csv  
outputs/rqd_analysis_p32_05_10_20/rqd_threshold_metrics.csv  
outputs/rqd_analysis_p32_05_10_20/rqd_threshold_sweep.csv
```

중요한 점:

text

rqd_summary.csv와 rqd_pairwise.csv는 직접 생성되는 원본이 아님.
항상 all_cases_face_rows.csv를 분석해서 만들어지는 2차 결과물임.

9. 혼동 방지용 체크리스트

분석할 때마다 이 순서로 확인하세요.

체크 1. 원본 CSV row 수

python

```
df = pd.read_csv("outputs/rqd_p32_05_10_20/all_cases_face_rows.csv")  
print(len(df))
```

기대:

text

72 또는 설정한 3 × seeds × faces

체크 2. case name

python

```
print(df["case_name"].unique())
```

기대:

text

```
granite_rqd_p32_05  
granite_rqd_p32_10  
granite_rqd_p32_20
```

기존 case 이름이 나오면 잘못된 파일입니다.

예를 들어 아래가 나오면 잘못된 것입니다.

```
text
granite_good_dense
granite_medium
granite_sparse
...
```

체크 3. Jv_face

```
python
print(df["Jv_face"].describe())
```

기대:

```
text
min ≈ 5
median ≈ 10
max ≈ 20
```

만약:

```
text
mean ≈ 0.925
max ≈ 2.2
```

이면 기존 low-density case입니다.

체크 4. RQD 분석 결과 count

rqd_summary.csv 열었을 때:

```
text
count = 72
```

여야 합니다.

count = 480이면 기존 결과를 보고 있는 겁니다.

10. 추천 폴더 이름

앞으로는 폴더를 이렇게 명확하게 나누면 좋습니다.

```
text
outputs/original_6cases_pure/
outputs/rqd_p32_05_10_20/
outputs/rqd_analysis_p32_05_10_20/
```

그리고 절대 같은 `outputs/rqd_analysis_pure` 같은 폴더를 재사용하지 않는 게 좋습니다.

11. 가능하면 분석 manifest 추가

혼동을 막으려면 `analyze_rqd_methods.py`에 결과 폴더 안에 manifest를 저장하게 하는 게 좋습니다.

추가 위치는 `main()`에서 CSV 읽고 `_check_columns(df)` 이후 정도입니다.

```
python
with open(out_path / "analysis_manifest.txt", "w", encoding="utf-8") as f:
    f.write(f"input_csv: {args.csv}\n")
    f.write(f"n_rows: {len(df)}\n")
    if "case_name" in df.columns:
        f.write("case_names:\n")
        for name in df["case_name"].unique():
            f.write(f"  - {name}\n")
    f.write(f"thresholds: {args.thresholds}\n")
```

그러면 나중에 결과 폴더를 열었을 때:

```
text
이 분석이 어떤 CSV에서 만들어졌는지
몇 row였는지
무슨 case였는지
```

바로 확인할 수 있습니다.

12. 지금부터 다시 시작하는 정확한 절차

정리하면, 지금부터는 아래 순서대로 하면 됩니다.

Step 1. 세 YAML만 포함해서 batch 실행

입력:

```
text
granite_rqd_p32_05.yaml
granite_rqd_p32_10.yaml
```

```
granite_rqd_p32_20.yaml
```

출력:

```
text
outputs/rqd_p32_05_10_20/all_cases_face_rows.csv
```

Step 2. 원본 CSV 확인

```
python
import pandas as pd

df = pd.read_csv("outputs/rqd_p32_05_10_20/all_cases_face_rows.csv")

print(len(df))
print(df["case_name"].unique())
print(df["Jv_face"].describe())
print(df["lambda_face_scanline"].describe())
```

기대:

```
text
len(df) = 72 또는 설정한 값
case_name은 rqd_p32 세 개만
Jv_face는 5, 10, 20 범위
```

Step 3. RQD 분석 실행

```
bash
python -m src.cli.analyze_rqd_methods --csv
outputs/rqd_p32_05_10_20/all_cases_face_rows.csv --out
outputs/rqd_analysis_p32_05_10_20 --thresholds 50 75 90 95 98
```

Step 4. 결과 확인

```
text
outputs/rqd_analysis_p32_05_10_20/rqd_summary.csv
outputs/rqd_analysis_p32_05_10_20/rqd_pairwise.csv
```

반드시:

```
text
rqd_summary.csv의 count = 72 또는 설정한 row 수
```

확인.

13. 이번 분석에서 기대하는 핵심 결과

이번 RQD 전용 분석에서는 기존과 달리 다음을 확인하려는 것입니다.

text

1. 낮은 P32=5에서는 RQD가 거의 높게 유지되는가?
2. P32=10,20으로 갈수록 face_scanline RQD와 Jv RQD가 낮아지는가?
3. 같은 고밀도 조건에서 borehole RQD는 face RQD보다 높게 남는가?
4. 그렇다면 그 차이는 borehole direction과 face scanline direction의 sampling bias 때문인가?

이것이 나중에 Q' 연구로 이어집니다.

14. 최종적으로 이 단계의 목적

이번 RQD 분석의 목적은 단순히 RQD 값을 구하는 게 아닙니다.

목적은 이것입니다.

text

RQD라는 구성요소가 시추공 방향과 굴진면 방향에 따라 얼마나 달라지는지 확인하고,
그 차이가 Q' 기반 굴착 가능성 판단의 신뢰성에 어떤 위험을 줄 수 있는지 평가하기 위한 기반을 만드는 것

즉:

text

RQD_borehole이 RQD_face보다 높게 나오는 조건
→ Qp_borehole이 Qp_face보다 높게 나올 가능성
→ false-safe 위험

으로 이어지는지 보는 출발점입니다.

GPT-5.5

오케이 이제 생각났어. analyze_rqd_method를 아직 실행하지 않았었네. rqd_summary :

```
column,exists,count,mean,std,min,p10,p25,p50,p75,p90,max
RQD_borehole_direct,True,72,73.9662788121388,43.02267572812834,0.0,0.0,70.97214414544388,98.0020896302494
7,100.0,100.0,100.0
RQD_borehole_hudson,True,72,74.22005081131172,43.161267666448495,0.0,0.0,71.8730510231626,98.64792949070
986,99.68414143994283,99.8640908334347,100.0
RQD_face_scanline,True,72,81.34071415909696,18.78353885932161,42.38119555734482,52.36482919426372,71.157
58501555108,87.23572284314346,98.81802670833474,100.0,100.0
RQD_face_jv,True,72,76.5,20.723532245551056,49.0,49.0,49.0,82.0,98.5,98.5
RQD_face_conservative,True,72,74.14845498420449,19.67204450250904,42.38119555734482,49.0,49.0,82.0,92.27785
95477217,96.8915620232743,98.5
```

rqd_pairwise :
 predictor,target,n,pred_mean,target_mean,bias,mae,rmse,median_abs_error,corr,overestimation_rate,underestimation_rate,equal_rate
 RQD_borehole_direct,RQD_face_scanline,72,73.9662788121388,81.34071415909696,-7.374435346958165,42.625564653041835,55.87721852563104,28.201929948506262,-0.5613776551763084,0.75,0.25,0.0
 RQD_borehole_direct,RQD_face_jv,72,73.9662788121388,76.5,-2.5337211878612056,35.7715722154278,47.180172444547416,31.814762763629247,0.016602645951684102,0.722222222222222,0.2777777777777778,0.0
 RQD_borehole_direct,RQD_face_conservative,72,73.9662788121388,74.14845498420449,-0.1821761720656999,38.0678238279343,47.91513803644389,36.8520876165363,-0.0533015531269794,0.75,0.25,0.0
 RQD_borehole_hudson,RQD_face_scanline,72,74.22005081131172,81.34071415909696,-7.1206633477852375,42.87933665221477,56.012255947664215,28.59112248321287,-0.5639587657293739,0.75,0.25,0.0
 RQD_borehole_hudson,RQD_face_jv,72,74.22005081131172,76.5,-2.279949188688281,35.97005081131172,47.33130194444071,32.38194706191475,0.014436856810075118,0.75,0.25,0.0
 RQD_borehole_hudson,RQD_face_conservative,72,74.22005081131172,74.14845498420449,0.07159582710722469,38.321595827107224,48.076572227516856,37.419271914821806,-0.05536735292647115,0.75,0.25,0.0



좋습니다. 이제 드디어 **RQD 전용 case의 진짜 결과**가 나왔습니다.

이 결과는 매우 중요합니다. 그리고 기존 low-density case와 완전히 다른 메시지를 줍니다.

요약하면:

절리빈도를 높인 RQD 전용 case에서는 RQD가 더 이상 100에 포화되지 않았고, 시추공 RQD와 굴진면 RQD 사이에 큰 차이와 강한 불일치가 나타났습니다.

다만 단순 평균 bias만 보면 시추공 RQD가 face보다 낮아 보이지만, overestimation rate가 75%로 매우 높습니다. 이는 일부 극단적인 저값이 평균을 끌어내리는 구조로 보이며, median/quantile 및 case별 분석이 반드시 필요합니다.

1. RQD summary 해석

1-1. Borehole direct

```
text
RQD_borehole_direct
mean = 73.97
std = 43.02
min = 0.0
p10 = 0.0
p25 = 70.97
p50 = 98.00
p75 = 100.0
p90 = 100.0
max = 100.0
```

가장 눈에 띄는 점은 분포가 **양극화**되어 있다는 것입니다.

text

```
p10 = 0
p50 = 98
p75 = 100
```

즉, 대부분은 매우 높지만 일부는 완전히 낮습니다.

이건 평균만 보면 안 되는 분포입니다.

가능한 구조:

text

```
대부분의 borehole segment/window는 절리를 적게 만나서 RQD가 100 근처
일부 case/face/seed에서는 절리가 매우 조밀하게 걸려 RQD가 0까지 떨어짐
```

1-2. Borehole Hudson

text

```
RQD_borehole_hudson
mean = 74.22
std = 43.16
min = 0.0
p10 = 0.0
p50 = 98.65
p75 = 99.68
```

direct와 거의 비슷합니다.

즉, 이번 RQD 전용 case에서는 Hudson도 direct와 비슷한 양극화 양상을 보입니다.

1-3. Face scanline

text

```
RQD_face_scanline
mean = 81.34
std = 18.78
min = 42.38
p10 = 52.36
p25 = 71.16
p50 = 87.24
p75 = 98.82
p90 = 100.0
```

Face scanline은 훨씬 더 연속적이고 안정적인 분포입니다.

중요한 점:

```
text
min = 42.38
p10 = 52.36
p50 = 87.24
```

즉, face RQD가 실제로 크게 내려갔습니다.

이건 RQD 전용 case가 제대로 작동했다는 뜻입니다.

1-4. Face Jv

```
text
RQD_face_jv
mean = 76.5
std = 20.72
min = 49.0
p50 = 82.0
p75 = 98.5
```

정상입니다.

왜냐하면 Jv가 5, 10, 20이면:

```
text
Jv=5   → RQD=98.5
Jv=10  → RQD=82.0
Jv=20  → RQD=49.0
```

이 세 값이 섞인 결과가 정확히 나왔습니다.

즉 Jv 기반 RQD는 이번에는 정상적으로 변별력을 가집니다.

1-5. Conservative face

```
text
RQD_face_conservative
mean = 74.15
std = 19.67
min = 42.38
p50 = 82.0
p75 = 92.28
```

`min(face_scanline, face_jv)`이므로 face reference 중 더 보수적인 기준입니다.

나중에 decision target으로 매우 유용합니다.

2. Pairwise 결과의 이상해 보이는 점

가장 먼저 눈에 띄는 것은 이것입니다.

```
text
RQD_borehole_direct → RQD_face_scanline
bias = -7.37
overestimation_rate = 0.75
underestimation_rate = 0.25
```

처음 보면 모순처럼 보입니다.

```
text
bias가 음수면 평균적으로 borehole이 face보다 낮다.
그런데 overestimation_rate는 75%다.
```

이건 모순이 아닙니다.

의미는 다음입니다.

72개 중 75%에서는 borehole RQD가 face RQD보다 높지만, 나머지 25%에서 borehole RQD가 face보다 훨씬 낮게 나와 평균 bias를 음수로 끌어내렸다.

즉, 분포가 매우 비대칭입니다.

예상 구조:

```
text
많은 경우:
  BH RQD ≈ 100
  Face RQD ≈ 50~90
  → BH overestimation

일부 경우:
  BH RQD ≈ 0
  Face RQD ≈ 70~100
  → 매우 큰 underestimation
```

이 때문에:

```
text
overestimation_rate = 75%
mean bias = -7.37
MAE = 42.63
```

가 동시에 나타납니다.

3. 이 결과의 가장 중요한 메시지

3-1. 평균 bias는 신뢰하기 어려움

현재 분포에서는 평균 bias만 보면 안 됩니다.

왜냐하면:

text

극단적 RQD=0 값들이 평균을 크게 왜곡

하기 때문입니다.

따라서 다음 지표가 더 중요합니다.

text

overestimation_rate

MAE

RMSE

median_abs_error

case별/밀도별/방향별 분포

3-2. Borehole RQD는 face RQD와 상관성이 낮거나 음수

text

corr $\approx$ -0.56 for face scanline

이건 이번에는 이전 포화 case와 다르게 의미가 있을 가능성이 있습니다.

특히 face scanline과 비교할 때 direct와 Hudson 모두:

text

corr $\approx$ -0.56

입니다.

즉, 이번 RQD 전용 case에서는:

text

시추공 RQD가 높을수록 face scanline RQD가 낮아지는 경향

이 나타났을 수 있습니다.

이건 매우 흥미롭습니다.

가능한 이유는 방향성입니다.

text

```
borehole 방향이 절리를 적게 만나는 조건에서는  
face scanline 방향이 절리를 많이 만나는 구조
```

즉, 두 방향이 서로 반대 sampling tendency를 보이는 절리 방향 설정일 수 있습니다.

이건 최종 연구에 아주 중요합니다.

4. Direct와 Hudson 비교

Face scanline target

text

```
Direct:  
MAE = 42.63  
RMSE = 55.88  
corr = -0.561
```

```
Hudson:  
MAE = 42.88  
RMSE = 56.01  
corr = -0.564
```

거의 같습니다.

즉:

text

```
이번 case에서는 Hudson  $\lambda$  방식이 direct RQD를 개선하지 못했다.
```

오히려 거의 동일하게 실패합니다.

이유는:

text

```
문제의 핵심이 RQD 계산식 direct vs Hudson 차이가 아니라,  
borehole sampling direction 자체가 face scanline과 다른 절리 집합을 보는 데 있기 때문
```

입니다.

이건 매우 중요한 결론입니다.

5. Face Jv target 비교

```
text
BH direct → face Jv
bias = -2.53
MAE = 35.77
corr ≈ 0.017
overestimation_rate = 0.722
text
BH Hudson → face Jv
bias = -2.28
MAE = 35.97
corr ≈ 0.014
overestimation_rate = 0.75
```

여기도 비슷합니다.

상관성은 거의 0입니다.

즉:

```
text
시추공 RQD는 Jv 기반 face RQD를 거의 설명하지 못한다.
```

그러나 overestimation_rate는 여전히 높습니다.

즉:

```
text
많은 경우 borehole은 face Jv 기준보다 더 좋게 평가한다.
```

6. Conservative target 비교

```
text
BH direct → face conservative
bias = -0.18
MAE = 38.07
overestimation_rate = 0.75
text
BH Hudson → face conservative
bias = +0.07
MAE = 38.32
overestimation_rate = 0.75
```

mean bias만 보면 거의 0입니다.

하지만 MAE가 매우 큼니다.

즉:

text

평균적으로는 맞는 것처럼 보이지만, 개별 face에서는 매우 크게 틀린다.

이건 decision 관점에서 매우 위험합니다.

특히 overestimation_rate가 75%이므로:

text

시추공 RQD가 conservative face condition보다 높게 나오는 경우가 많다.

7. 이 결과가 연구 질문에 주는 답

질문:

시추공 RQD와 굴진면 RQD가 차이가 발생할 수밖에 없나? 원인은 무엇인가?

이번 결과는 이렇게 답할 수 있습니다.

text

절리빈도가 낮은 경우에는 RQD가 포화되어 차이가 작게 보였다.
하지만 절리빈도를 높여 RQD가 민감하게 반응하는 조건을 만들자,
시추공 RQD와 굴진면 RQD는 크게 달라졌다.
특히 시추공 RQD는 face scanline RQD와 음의 상관을 보였고,
75%의 경우 face RQD보다 높게 평가했다.
이는 시추공과 굴진면 scanline이 서로 다른 방향의 line sampling이며,
절리군 방향성에 따라 절리 교차 빈도가 크게 달라지기 때문이다.

이건 매우 강한 결과입니다.

8. 그런데 반드시 추가 확인해야 할 점

현재 borehole RQD에 0이 나옵니다.

text

```
RQD_borehole_direct min = 0  
p10 = 0
```

이건 가능하긴 하지만, 반드시 확인해야 합니다.

왜냐하면 borehole lambda 진단에서는:

```
text
lambda_borehole max = 3.0 /m
apparent spacing min = 0.333m
```

이었습니다.

만약 borehole lambda가 정말 max 3/m라면, Hudson RQD가 0이 되는 것은 이상합니다.

Hudson 식에서 $\lambda=3/m$ 이면:

```
text
λt = 0.3
RQD = 100 * exp(-0.3) * 1.3 ≈ 96.3
```

인데, 지금 RQD_borehole_hudson min = 0입니다.

이건 매우 중요한 이상 신호입니다.

즉, 다음 중 하나일 가능성이 있습니다.

가능성 A. **rqd_summary**의 **RQD_borehole_hudson**이 새 **lambda** 진단과 다른 CSV에서 나온 값

하지만 count=72이고 face_jv가 정상이라면 같은 CSV일 가능성이 큼니다.

가능성 B. borehole segment별 Hudson RQD 중 빈 구간/잘린 구간이 0으로 들어감

`sample_borehole()`에서 segment 배열에 0이 들어간 상태로 평균이 계산되었을 수 있습니다.

예를 들어:

```
text
n_segments가 x_end - x_start인데,
일부 segment가 계산되지 않거나 length mismatch로 0 유지
```

일 수 있습니다.

가능성 C. **RQD_borehole_hudson** 배열에 NaN/empty 대신 0이 들어감

빈 시추공 결과 또는 잘못된 segment에서 0이 들어가고, 평균에 포함되었을 수 있습니다.

9. 이걸 꼭 디버깅해야 함

현재 가장 이상한 점은 이것입니다.

```
text
lambda_borehole max = 3.0 /m
그런데 RQD_borehole_hudson min = 0
```

수학적으로 맞지 않습니다.

Hudson RQD가 0이 되려면 λ 가 엄청 커야 합니다.

예를 들어 $\lambda=100/\text{m}$ 이면:

```
text
λt=10
RQD≈0.05
```

정도입니다.

그런데 $\lambda \max 3$ 이면 0이 될 수 없습니다.

따라서 `RQD_borehole_hudson` 계산 또는 집계에 버그가 있을 가능성이 있습니다.

10. 바로 확인할 디버그 코드

`all_cases_face_rows.csv`에서 Hudson RQD가 0인 row를 찾아야 합니다.

```
python
import pandas as pd

df = pd.read_csv("outputs/rqd_p32_05_10_20/all_cases_face_rows.csv")

cols = [
    "case_name",
    "seed",
    "face_x_idx",
    "face_x_coord",
    "RQD_borehole_direct",
    "RQD_borehole_hudson",
    "lambda_borehole",
    "borehole_n_intersections_total",
    "RQD_face_scanline",
    "lambda_face_scanline",
    "face_scanline_n_intersections",
    "Jv_face",
    "RQD_face_jv",
]
```

```
bad = df[df["RQD_borehole_hudson"] <= 1.0]

print("n bad:", len(bad))
print(bad[cols].to_string(index=False))
```

이 결과를 보면 됩니다.

특히 확인할 것:

```
text
RQD_borehole_hudson = 0인데 lambda_borehole이 얼마인가?
```

만약 lambda가 0~3이면 계산/집계 버그입니다.

11. per-borehole row도 확인해야 함

all_cases_borehole_rows.csv가 있다면 더 좋습니다.

```
python
bh = pd.read_csv("outputs/rqd_p32_05_10_20/all_cases_borehole_rows.csv")

cols = [
    "case_name",
    "seed",
    "face_x_idx",
    "face_x_coord",
    "borehole_name",
    "RQD_borehole_direct_mean",
    "RQD_borehole_hudson_mean",
    "lambda_borehole_mean",
    "borehole_n_intersections_sum",
    "RQD_bh_mean",
]

bad = bh[bh["RQD_borehole_hudson_mean"] <= 1.0]

print("n bad borehole rows:", len(bad))
print(bad[cols].to_string(index=False))
```

여기서 특정 borehole의 값이 0인지, 아니면 집계 과정에서 0이 생겼는지 확인할 수 있습니다.

12. 왜 이 디버깅이 중요한가

현재 결과는 연구적으로 매우 좋아 보이지만, RQD_borehole_hudson = 0이 수학적으로 말이 안 되면 결과 신뢰성이 흔들립니다.

특히 `RQD_borehole_direct`도 0이 나왔는데, 이것도 $\lambda \max 3/m$ 와 함께 보면 이상합니다.

Direct RQD가 0이 되려면 1m segment 안에서 모든 intact piece가 10cm 미만이어야 합니다.

그렇다면 segment 내 절리 개수가 적어도 10개 이상이어야 합니다.

하지만 λ 가 3/m이면 그 정도가 아닙니다.

따라서 `RQD_borehole_direct = 0`도 의심됩니다.

13. 가장 가능성 높은 원인

제가 보기에는 `comparison.py`의 borehole 집계에서 문제가 있을 가능성이 있습니다.

현재 borehole method는 segment별 배열을 평균합니다.

```
python
RQD_borehole_direct_mean = np.mean(bh.get('RQD_borehole_direct', []))
```

그런데 `sample_borehole()`에서 `n_segments = x_end - x_start`입니다.

만약 `x_start`, `x_end`가 작거나 window가 잘못되어 일부 구간이 이상하게 계산되면 0이 들어갈 수 있습니다.

또는 `sample_borehole()`에서 새 배열을 `np.zeros(n_segments)`로 만들고, loop가 어떤 이유로 충분히 채워지지 못하면 0이 남습니다.

확인할 부분:

```
python
rqd_borehole_direct_arr = np.zeros(n_segments)
lambda_borehole_arr = np.zeros(n_segments)
rqd_borehole_hudson_arr = np.zeros(n_segments)
```

이렇게 0으로 초기화하면, 계산되지 않은 segment가 있으면 0으로 남습니다.

더 안전한 방식은:

```
python
np.full(n_segments, np.nan)
```

으로 초기화하고 평균에서는 `np.nanmean()`을 쓰는 것입니다.

하지만 loop가 정상적으로 모든 segment를 돈다면 0이 남지 않아야 합니다.

14. 우선 디버그 후 판단

지금 결과의 큰 방향은 흥미롭습니다.

하지만 다음 의심점 때문에 바로 결론을 확정하면 안 됩니다.

```
text
lambda_borehole_max = 3/m인데
RQD_borehole_direct/udson_min = 0
```

따라서 다음 순서가 좋습니다.

```
text
1. RQD_borehole_udson <= 1 row 찾기
2. 해당 row의 lambda_borehole 확인
3. all_cases_borehole_rows.csv에서 per-borehole 값 확인
4. sample_borehole 배열에 0이 왜 들어갔는지 확인
5. 필요하면 배열 초기값을 np.nan으로 바꾸고 nanmean 적용
```

15. 현재 결과의 잠정 결론

디버깅 전제하에, 현재 결과의 잠정 결론은 다음과입니다.

```
text
RQD 전용 고밀도 case에서 face scanline 및 Jv 기반 RQD는 의미 있게 낮아졌다.
시추공 RQD와 face RQD의 차이는 매우 커졌고, 특히 75%의 경우 시추공 RQD가 face RQD보다 높게
평가되었다.
이는 수평시추공 RQD가 특정 방향성 조건에서 굴진면 상태를 과대평가할 수 있음을 시사한다.
다만 borehole RQD에 0 값이 나타나는 것은 lambda 통계와 일관되지 않으므로, 집계 또는 계산 과정의
버그 여부를 먼저 확인해야 한다.
```

16. 다음에 보내주면 좋은 것

아래 디버그 결과를 보내주세요.

```
python
bad = df[df["RQD_borehole_udson"] <= 1.0]
print(bad[cols].to_string(index=False))
```

그리고 가능하면:

```
python
bad_direct = df[df["RQD_borehole_direct"] <= 1.0]
```

```
print(bad_direct[cols].to_string(index=False))
```

이걸 보면 버그인지, 실제 극단 case인지 바로 판단할 수 있습니다.

GPT-5.5

icon

The conversation length limit has been exceeded.

Start a new chat



Gemma 3n

Sources